

—— // ELISPRS // ——

elisprs

v0.1.4

THE COMPANION

the reader · the heap · the machine underneath

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	4
Chapter 1 — Emacs Lisp Without the Editor	6
Chapter 2 — Hello, elisprs	9
Chapter 3 — The Reader	12
Chapter 4 — Values and the Object Heap	17
Chapter 5 — The Obarray: A Lisp-2 in Rust	21
Chapter 6 — Binding: Lexical and Dynamic	25
Chapter 7 — The Special Forms	28
Chapter 8 — Macros and Backquote	33
Chapter 9 — The Compiler: Lowering elisp to fusevm::Chunk	37
Chapter 10 — The Shared Machine: fusevm and the Extension Protocol	41
Chapter 11 — The Bytecode Cache	45
Chapter 12 — The Subr Library	48
Chapter 13 — The Derived Prelude	52
Chapter 14 — Control Flow, Errors, and Non-Local Exit	56
Chapter 15 — Strings, Characters, and Regular Expressions	60
Chapter 16 — Buffers and Text Processing	64
Chapter 17 — Tooling: The Language Server and the Debugger	68
Chapter 18 — Ahead-of-Time Compilation	71
Chapter 19 — The Numeric Tower	74
Chapter 20 — Lists, Sequences, and the Mapping Family	78
Chapter 21 — Tables and Records: Hash Tables, Property Lists, and the map Library	82
Chapter 22 — Structures and Generic Functions	85
Chapter 23 — Reading, Printing, and Equality	88
Chapter 24 — The Operating-System Surface	91

Chapter 25 — Self-Contained Sub-Languages: JSON, rx, and pcase	94
Chapter 26 — The cl-loop Facility	97
Chapter 27 — A Tour of Real Programs	100
Chapter 28 — The Catalogue: Numbers and the Math Library	109
Chapter 29 — The Catalogue: Predicates, Equality, and Types	112
Chapter 30 — The Catalogue: Cons Cells, Lists, and Sequences	114
Chapter 31 — The Catalogue: Symbols, Binding, and Evaluation	116
Chapter 32 — The Catalogue: Strings, Characters, and Format	118
Chapter 33 — The Catalogue: Vectors, Hash Tables, and Records	120
Chapter 34 — The Catalogue: Printing, Reading, and Output	122
Chapter 35 — The Catalogue: Regular Expressions and Match Data	123
Chapter 36 — The Catalogue: Buffers and Text Motion	125
Chapter 37 — The Catalogue: Files, Processes, Time, and Crypto	127
Chapter 38 — The Catalogue: Control, Errors, and Higher-Order Functions	129
Chapter 39 — The Prelude Library: cl-lib in Full	130
Chapter 40 — The Prelude Library: seq, map, subr-x, and the Rest	133
Chapter 41 — The Compiler, Form by Form	136
Chapter 42 — The Extension Handler, Op by Op	139
Chapter 43 — Worked Example: script-demo.el, End to End	142
Chapter 44 — Worked Example: data-model.el, End to End	145
Chapter 45 — Worked Example: text-utils.el and stats.el	148
Chapter 46 — The Test Suite as Specification: Values and Control	151
Chapter 47 — The Test Suite as Specification: Closures, Errors, and Objects	153
Chapter 48 — The Reader Syntax Reference	156
Chapter 49 — The Condition System and Error Reference	158
Chapter 50 — Minor Libraries: kbd, Time, Completion, and Display	160
Chapter 51 — Region, Word, and the subr.el Grab-Bag	162
Chapter 52 — Limitations, Edges, and the Moving Frontier	164
Chapter 53 — The Command Line and Tooling Reference	166
Chapter 54 — The ERT Framework, From the Inside	168

Chapter 55 — Inside cl-loop: A Compiler in the Prelude	170
Chapter 56 — Inside pcase and rx: Two More Prelude Compilers	172
Chapter 57 — The fusevm Engine and the Frontend Family	174
Chapter 58 — From Source to Result, One Form Traced	176
Chapter 59 — Inside cl-defstruct and setf	178
Chapter 60 — The Heap Image and Determinism	180
Chapter 61 — What Runs Fast: The JIT, the Fast Path, and the Cache	182
Chapter 62 — The Backquote Expander, Traced	184
Chapter 63 — Truthiness and the TRUTHY Op	186
Chapter 64 — Binding, Traced: Lexical and Dynamic Side by Side	188
Chapter 65 — Interning, Traced	190
Chapter 66 — Closures and Nested VMs, Traced	192
Chapter 67 — The Macro Expander and the Top-Level Loop, Traced	194
Chapter 68 — Characters, Unicode, and Counting in Characters	196
Chapter 69 — The Frontend Thesis, Reviewed	198
Chapter 70 — Sequence Polymorphism: One Vocabulary, Three Types	200
Chapter 71 — The Output Streams: stdout, stderr, and Capture	202
Chapter 72 — The Special-Form Reference, Entry by Entry	204
Chapter 73 — The Test Files, One by One	207
Chapter 74 — Generalized Variables: gv, setf, and First-Class Places	209
Chapter 75 — OClosures: A Closure With a Type and Slots	211
Chapter 76 — Customization Without a UI: defcustom and the Theme Machinery	213
Chapter 77 — Keymaps as Data: The Editor’s Binding Model, Without the Editor	215
Chapter 78 — First-Class Obarrays: Namespaces You Can Hold	217
Chapter 79 — The Printer’s Depth Guard: Circular Structure and PRINT_CIRCLE	219
Chapter 80 — Adversarial Robustness: The Frontend That Must Not Panic	221
Appendix A — The Special Forms	223
Appendix B — Glossary	224
Appendix C — The Reference Shelf	228
Appendix D — A Compact Index of the Standard Library	230

Before You Begin

This is not the reference manual. The reference enumerates every one of elisprs's special forms, primitive subrs, and derived prelude functions — the categories run from arithmetic through hash tables, over three hundred entries — and you reach for it when you need to know exactly what `replace-regexp-in-string` does with a `\&` in its replacement template. This book is the other thing: the one you read in a chair, front to back, to understand not just *what* elisprs does but *why* it is shaped the way it is.

elisprs is Emacs Lisp that runs outside Emacs. That sentence is true and almost completely fails to describe it. A fuller description: elisprs is what you get when you take a language that has only ever existed as the extension layer of one particular text editor — a language inseparable from its host for forty years — and ask whether its *semantics* and its *execution* are really the same thing. They are not. The semantics of Emacs Lisp are a Lisp-2 with separate value and function cells, dynamic and lexical binding, real mutable cons cells, an elisp-correct reader, and a thousand small behavioral commitments that make `.el` code mean what it means. The execution is a bytecode machine. elisprs keeps the first and throws away the second — not to replace it with another bespoke interpreter, but to *borrow* one that already exists. Every form you write is lowered to a `fusevm::Chunk` and run on fusevm, the same stack VM, three-tier Cranelift JIT, and ahead-of-time native compiler that already power four sibling languages. elisprs has no virtual machine of its own. It is a pure frontend.

That single architectural decision — own the language, rent the engine — is the spine of this book, and almost everything interesting about elisprs follows from it. The object heap of cons cells and symbols rides through the borrowed VM as opaque handles. The reader builds those handles directly. The compiler emits a handful of custom opcodes the VM does not understand and hands them back to elisprs through an extension callback. Hot arithmetic lowers to the VM's native add and compare so the JIT can swallow a tight loop whole. A `.el` file's compiled bytecode and a snapshot of its heap are cached in an rkyv shard so the second run skips the reader, the macro expander, and the compiler entirely. And the whole thing can be frozen into a standalone native binary. None of that is plausible if you write your own VM; all of it is close to free if you lower onto someone else's. The frontend posture is not a shortcut. It is the thesis.

The book is in six movements. The first — Chapters 1 through 8 — is the language as a reader and a value model sees it: the pitch, the command line, the reader, the object heap, the obarray, binding, the special forms, and macros. The second — Chapters 9 through 11 — is the machine underneath: the compiler that lowers elisp to bytecode, the shared fusevm engine and the extension protocol that lets an alien object heap ride a language-agnostic VM, and the bytecode cache. The third — Chapters 12 through 16 — is the library and the runtime surface in outline: the primitive subrs written in Rust, the large derived prelude written in elisp on top of them, control flow and the error system, strings and regular expressions, and the minimal buffer model that makes standalone text-processing scripts possible. The fourth — Chapters 17 and 18 — is the tooling: the language server and the debug adapter, and ahead-of-time compilation to native code. The fifth — Chapters 19 through 26 — goes back through the library in depth, one subsystem at a time: the numeric tower, lists and the mapping family, hash tables and the map interface, structures and generic functions, the read/print/equality loop, the operating-system surface, the self-contained sub-languages (JSON, `rx`, `pcase`), and the Common Lisp loop facility. The sixth — Chapter 27 — is a long tour through real, complete example programs from the `examples` directory, walked line by line.

That is the first part of the book: the argument, the *why*, read front to back. The second part — Chapters 28 through 72 — is a reference companion, the *what*, organized for lookup rather than narrative. It opens with a catalogue of the primitive layer walked family by family (Chapters 28 through 38), then the derived prelude function by function (Chapters 39 and 40), then a long sequence of deep references and traced walkthroughs: the compiler form by form and the extension handler op by op, the four complete example programs read end

to end, the test suite as a behavioral specification, the reader syntax, the condition system, the minor libraries, the limitations, the command line, and a set of focused traces — backquote, truthiness, binding, interning, closures, the macro expander, the heap image, performance, characters, sequence polymorphism, and the output streams — closing with a review of the frontend thesis and an entry-by-entry special-form reference. The four reference appendices follow. Read the first part in a chair; keep the second by the keyboard.

Everything here is grounded in the actual repository. Every code snippet is either copied from a real example, test, or source file, or is faithful elisp that the runtime accepts — there are no invented builtins and no aspirational behavior dressed up as fact. Where a number, a phase, or a limitation is still provisional, this book marks it as such rather than rounding up. `elisprs` is the work of MenkeTechnologies, one author building a family of languages on a shared machine, and this book is grounded in his code. The version described throughout is 0.1.2.

Let's start.

Chapter 1 – Emacs Lisp Without the Editor

Emacs Lisp has a peculiar status among programming languages: it is enormously used and almost never run on purpose. Tens of thousands of people write it every week, and essentially all of them write it to configure or extend one program. The language came into the world bolted to its host, and for four decades the bolt has held. There is no `elisp` on anyone's `PATH` the way there is `python` or `perl` or `ruby`. To run a line of Emacs Lisp you start Emacs — or you start Emacs in batch mode, which is still starting Emacs, dragging in the redisplay engine and the buffer subsystem and the keymaps to evaluate `(+ 1 2)`. The language is real, the standard library is vast, the community is deep, and the runtime is an editor.

`elisprs` asks a question that sounds naive until you take it seriously: does it have to be? Is Emacs Lisp *the language* genuinely inseparable from Emacs *the editor*, or is that just an accident of how it was implemented? The answer this project gives is that the inseparability is an accident, and a removable one. You can take the reader, the Lisp-2 object model, the special forms, the macro system, the binding rules — the parts that make `.el` code mean what it means — and you can run them as ordinary programs from a shell prompt, with a REPL, with no Emacs process anywhere in sight. What you give up is the editor: the buffers-as-windows, the point-as-cursor, the interactive command loop, the thousand subrs that exist only to manipulate a live display. What you keep is the language.

The positioning

The README states the position in one line: “The editor’s language — without the editor.” `elisprs` runs `.el` files as standalone programs. You hand the `elisp` binary a file and it evaluates it; you hand it `-e` and an expression and it prints the result; you run it with no arguments and you get a REPL with balanced-paren continuation. It is built to be the fifth language hosted on `fusevm`, after `zshrs` (a Rust rewrite of the Z shell), `stryke` (a Perl-compatible scripting language), `awkrs` (an AWK implementation), and `vimlrs` (Vimscript). Those five share an engine. That sharing is the entire economic argument for the project, and we will spend a good part of this book on exactly how it works, but the headline is simple: `elisprs` did not have to build a virtual machine, a JIT, an ahead-of-time compiler, or an on-disk code cache, because four other languages had already built them and the build was reusable.

Why a frontend, and not another interpreter

The obvious way to run Emacs Lisp outside Emacs is to write a tree-walking interpreter. You parse the `.el` source into an abstract syntax tree of cons cells — which, this being Lisp, is the same thing as the program’s data — and then you write an `eval` function that recurses over that tree, dispatching on the head of each form, looking variables up by name, and reconstructing intermediate values on every pass. This is how Emacs Lisp itself began, how most teaching Lisps work, and how you would knock together a weekend `elisp` in any language with a hashmap and a recursion budget. It is correct, it is direct, and it is the architecture `elisprs` deliberately does not use.

The cost of tree-walking is the cost of re-deciding everything on every execution. A loop that runs a million times walks the same subtree a million times, and on each walk it pays again to ask “is this form a function call or a special form?”, to resolve a variable reference from a name to a location, and to allocate the machinery of

a stack frame. A bytecode compiler pays those costs once. It flattens the tree into a linear instruction stream where a variable reference has already become a slot, a conditional has already become a jump, and the dispatch on form type has already happened at compile time. The inner loop of the program becomes the inner loop of a tight bytecode interpreter, and from there the door is open to a JIT that turns the hottest stretches of bytecode into native machine code. Emacs itself learned this lesson long ago — modern Emacs has a bytecode compiler and, as of Emacs 28, a native-compilation backend through libgccjit — precisely because the tree-walker was too slow for the load real packages put on it.

elisprs takes the bytecode-and-JIT path, but it takes it by a route no single-language project would choose: it does not write the bytecode machine. It writes a *compiler that targets someone else's*. The reader produces elisp forms; the compiler lowers each top-level form to a `fusevm::Chunk`; fusevm executes the chunk with its stack VM and its three-tier Cranelift JIT, calling back into elisprs whenever it hits an instruction that is elisp-specific. The engineering report puts it plainly: “elisprs is a pure fusevm frontend with no bespoke VM or JIT.” The Cargo manifest's own description is the same claim compressed to a subtitle: “a reader and compiler to `fusevm::Chunk` on an `ElispHost` object heap, with no bespoke VM or JIT.”

This is a real architectural commitment with real consequences, and it is worth stating the trade honestly before we admire it. The cost is that elisprs does not control its own execution engine. If fusevm's calling convention changes, elisprs changes with it. If the JIT has a bug, elisprs inherits the bug. The language can only express what it can lower to fusevm ops plus its own extension callbacks, and a few elisp forms — the buffer-scoping forms `save-excursion`, `save-current-buffer`, and `save-restriction` — are not lowered at all yet and fail loudly at compile time rather than silently misbehaving. The benefit is everything else: a stack VM that already works, a Cranelift JIT that already tiers up hot loops, an AOT object emitter, a persistent on-disk cache of compiled native code, and a maintenance burden shared across five languages instead of carried by one. Every improvement to fusevm's JIT is an improvement to elisprs for free. That is the bargain, and the rest of this book is in large part an account of how the bargain is structured so that the benefits land and the costs stay contained.

The reuse-own split

The README draws the line between what elisprs owns and what it rents as a table, and the table is the cleanest one-paragraph summary of the whole design. The value model — interned symbols, real dotted cons cells, vectors, hash tables, closures — is elisprs's own, living in an `ElispHost` object heap. The reader, with its elisp-specific quirks like `1+` as a symbol and `?c` as a character literal and `#'foo` as a function quote, is elisprs's own. The Lisp-2 obarray, the lexical and dynamic binding, the special forms, the macros, the subr library — all elisprs's own. And then one row: bytecode execution, JIT, and AOT are fusevm's, and “elisprs has no VM/JIT of its own.”

The principle behind the split is that elisprs owns *the parts that are the personality of Emacs Lisp* and rents *the parts that are generic*. A Lisp-2 with dynamic scope is not generic; it is the specific, load-bearing weirdness that makes elisp elisp, and it could not be delegated to a language-agnostic VM without teaching that VM about symbols and value cells and special variables, which would defeat the purpose of the VM being language-agnostic. A stack machine that executes `Add` and `Jump` and `LoadConst`, on the other hand, is the same for every language that has ever been compiled, and there is no reason for five languages to write it five times. So elisprs draws the line exactly there: the object heap and the semantics are bespoke, the execution substrate is shared, and a narrow, well-defined protocol — fusevm's extension handler — connects the two. Chapter 10 is entirely about that protocol, because it is the hinge the whole architecture turns on.

Five frontends, one engine

The economic case for renting the engine is not hypothetical, because elisprs is not the first tenant. fusevm is shared across five languages — zshrs (a Rust rewrite of the Z shell), stryke (a Perl-compatible scripting language), awkrs (an AWK implementation), vimlrs (Vimscript), and elisprs — and the report names elisprs as “the fifth consumer of fusevm.” Those five do not agree on anything that matters at the language level: zsh

and elisp do not share a value model, AWK and Vimscript do not share a binding discipline, and none of them share a syntax. What they share is the part that is genuinely generic — a stack machine that pushes and pops values, jumps on a boolean, and loads constants — and that is exactly the part fusevm is. The dividend is concrete and ongoing: a JIT improvement landed for stryke's hot loops improves elisprs's hot loops too, an AOT backend written once serves all five, and a bug fixed in the shared dispatch loop is fixed everywhere at once. The maintenance burden of a VM, a three-tier JIT, an AOT emitter, and an on-disk native-code cache is carried by one shared crate rather than by five separate language teams.

The version that elisprs pins is exact — `fusevm = "0.14.5"` with the `jit-disk-cache` and `aot` features turned on — and the pin is meaningful, because it is the line where the rented engine's capabilities are fixed. When fusevm's calling convention or its `Value` type changes, elisprs changes with it at the next bump; when the JIT gets faster, elisprs gets faster at the next bump. That coupling is the cost side of the bargain Chapter 1 has been describing, and the feature flags are the itemized bill: `jit-disk-cache` buys the three-tier Cranelift JIT and the persistent cache of compiled native code, and `aot` buys the native-object emitter. elisprs pays a version pin and inherits a code-generation pipeline it did not write — which is the whole thesis, stated as a line in a manifest.

What you get, concretely

Strip away the architecture talk and what a user gets is small and sharp. A binary called `elisp`. You can point it at a file:

```
elisp examples/demo.el
```

You can hand it an expression:

```
elisp -e "(+ 1 2)"
```

You can start a REPL:

```
elisp
```

You can run a language server for your editor:

```
elisp --lsp
```

You can run a debug adapter:

```
elisp --dap
```

And you can compile a `.el` file to a standalone native executable that needs no `elisp` binary to run:

```
elisp --aot-exe program.el -o program
```

Everything in this book is in service of explaining what happens between those command lines and the result they produce. The first stop is the command line itself.

Chapter 2 — Hello, elisprs

The first program in most language books prints a greeting and explains nothing. The first program in `elisprs` prints a greeting and, if you read it closely, tells you most of what makes the system what it is. So we will start small and read closely. Here is the opening of `examples/demo.el`, which the project ships as the smoke test of the whole pipeline:

```
(message "== elisprs demo ==")
(message "sum = %d" (+ 1 2 3 4 5))
(message "list = %S" (list 1 2 3))
(message "vec[1] = %d" (aref (vector 10 20 30) 1))
```

That is real Emacs Lisp, and it is also a complete, runnable `elisprs` program. The file's own header comment says what is happening to it, and the sentence is worth internalizing because it is the thesis restated at the smallest scale: “Everything here is read by `elisprs`, lowered to a `fusevm` Chunk, and executed on `fusevm` itself — there is no elisp interpreter. The `ElispHost` (reached from `fusevm`'s extension handler) supplies cons cells, symbols, and the subsr.” Four `message` calls, and already the full machine is in motion: the reader turned the text into cons cells on the heap, the compiler lowered each top-level form to a chunk, `fusevm` ran the chunk, and every time the chunk reached a call like `(+ 1 2 3 4 5)` or `(aref ...)` it handed control back to `elisprs` to do the elisp-specific work.

The binary and its modes

The reference binary is named `elisp`. Its `main` is a dispatcher, and reading the dispatch order tells you the shape of the tool. It checks, in order, for `--version/-V`, for `--help/-h`, for `--lsp`, for `--dap`, for `--aot-exe`, for `--aot`, for `-e/--eval` followed by an expression, and finally for the first argument that is not a flag, which it treats as a file to evaluate. If none of those match — no flags, no file — it drops into the REPL. The whole of `main` is a sequence of `if` checks against the argument vector, which means the modes are mutually exclusive and the precedence is fixed: a `--version` anywhere on the line wins over everything, a `--lsp` wins over a file path, and so on.

The simplest invocation evaluates a file. `elisp examples/demo.el` calls `elisprs::eval_file`, which we will see in Chapter 11 does something more interesting than read-and-run — it consults a bytecode cache first. The result of the last form is discarded for a file (the program runs for its effects), and the process exits zero on success or prints `error: ...` to `stderr` and exits non-zero on failure.

The `-e` form evaluates a single expression and prints its value:

```
elisp -e "(mapcar #'1+ '(10 20 30))"
```

This routes through `eval_str`, and unlike the file path it prints the result — specifically, it prints it in *readable* (`prin1`) form, so a list comes back looking like a list and a string comes back with its quotes. The printing is done by `elisprs::print(&v, true)`, where the `true` is the readable flag; we will meet the printer properly in Chapter 4. If the expression signals an error, `elisprs` formats it the way Emacs's `error-message-string` would — a raw internal condition like `void-variable: foo` is rendered as the familiar “Symbol's value as variable is void: foo” — by actually constructing an error object and calling the interned `error-message-string` subr on it. The error formatting is not a special-case string table in Rust; it is the real elisp error machinery turned on its own diagnostics.

Exit codes and the second binary

The dispatch in `main` settles which mode runs; the exit code reports how it went. A file run that completes returns zero; one that signals prints `error: ...` to stderr and exits non-zero. This is what makes a `.el` file usable as a build step or a CI gate: a script that ends in `ert-run-tests-batch-and-exit` turns any failed `should` into a non-zero exit, so a broken assertion breaks the build, which is precisely the mechanism every example file in this book relies on. The `-e` path differs in that it *prints* its result in readable form before exiting, so a one-liner is a calculator; the file path discards the last value and runs for effect.

There is also a second binary in the crate, easy to miss because it never runs at elisp-evaluation time: `gen-docs`. Its job is offline documentation generation — it renders the language server’s hover-and-completion corpus as the static `docs/reference.html`, and the Cargo manifest is explicit that it is meant to be “run before pushing Pages.” That `gen-docs` is a separate binary from `elisp`, sharing the corpus rather than the runtime, is the single-source-of-truth arrangement Chapter 17 develops: one hand-maintained table of signatures and docstrings feeds both the live editor and the published reference, written once and consumed twice. The `Cargo.toml` lists exactly two binaries — `elisp` for everything a user does and `gen-docs` for the docs build — and a `lib` crate compiled as both an `rlib` and a `staticlib`, the latter so an AOT object can link against the runtime, as Chapter 18 describes.

The REPL and balanced parens

Run `elisp` with no arguments and you get a read-eval-print loop. It prints a banner, then a prompt — `elisp>` when it is waiting for a fresh form, `...>` when it is waiting for you to finish one — reads a line, and decides whether to evaluate yet. The decision is made by a small function called `parens_balanced`, and it is worth looking at because it is the REPL’s entire theory of “is this input complete?”

```
;; conceptually, the REPL accumulates lines until the parens balance:
elisp> (defun fact (n)
...>   (if (<= n 1) 1
...>       (* n (fact (1- n)))))
fact
elisp> (fact 6)
720
```

The continuation logic counts open and close parens, but it is careful about the two places a paren does not count: inside a string and inside a comment. It tracks a string state with backslash escaping, so a `(` inside `"a ( b"` does not increase the depth, and it tracks a `;` comment to end-of-line, so a `)` in a trailing comment does not decrease it. Only when the running depth is back to zero or below does it consider the input a complete form and evaluate it. This is the same discipline a Lisp editor’s electric-paren mode uses, reduced to the minimum a REPL needs: accumulate until balanced, then evaluate, then reset. Ctrl-D — an end-of-file on stdin — exits cleanly.

Notice that the REPL evaluates through exactly the same `eval_str` entry point that `-e` uses, which means every form you type at the prompt goes through the full read, macro-expand, lower-to-chunk, run-on-fusevm pipeline. There is no special “interactive evaluator” that cuts corners. The REPL is fast not because it does less but because the machine underneath is a compiler and a JIT, and a one-line form compiles to a tiny chunk that runs in microseconds.

Output: message, princ, prin1, print

The demo’s workhorse is `message`. In Emacs, `message` writes to the echo area and the `*Messages*` buffer; in `elispr`, with no echo area to write to, it writes a formatted line to standard error. That detail matters more than it sounds, because it means `message` output and *program* output go to different file descriptors, which is exactly what you want when a script’s real result is going to stdout and its progress chatter is going to

the terminal. The functions that write a program's actual output — `princ`, `prin1`, `print`, `terpri` — go to standard output. The split shows up again in the debugger, where `stdout` is captured through a pipe so that program output does not corrupt the debug protocol while `message` chatter passes through untouched.

`message` also takes us to `format`, because its first argument is a format string and the rest are arguments to splice in. The demo uses three directives: `%d` for a decimal integer, `%S` for a value in readable form (so `(list 1 2 3)` prints as `(1 2 3)`), and the difference between `%s` and `%S` — the lowercase one is `princ` style, raw, and the uppercase one is `prin1` style, re-readable — is a distinction you will reach for constantly. `(format "%s" "x")` is `x`; `(format "%S" "x")` is `"x"` with the quotes. The full set of directives is rich — `%o`, `%x`, `%X`, `%c`, `%e`, `%f`, `%g`, with flags for width, zero-padding, left-justification, sign, and precision — and we will exercise them in the strings chapter. For now the point is just that `format` is a primitive subr, implemented host-side in Rust, and `message` is a thin wrapper over it that chooses the output stream.

Truthiness, the smallest surprise

`examples/demo.el` includes a test whose docstring is a one-line lesson in elisp semantics: “elisp truthiness: only nil is false.”

```
(ert-deftest demo-conditionals ()
  "elisp truthiness: only nil is false."
  (should (equal (if (eq 1 1) "yes" "no") "yes")))
(should (eq (when (consp (cons 1 2)) 'ok) 'ok))
(should-not (when nil 'ok)))
```

In elisp, the only false value is `nil`. The number zero is true. The empty string is true. The symbol `'false` — there is no such thing; it would just be a truthy symbol named `false`. Every conditional in the language tests for non-`nil`, and this single rule, that only `nil` is false, propagates through `if`, `when`, `unless`, `cond`, `and`, `or`, and `while`. It also creates a small but real impedance mismatch with the host VM, because `fusevm` has its own idea of truthiness, and `elisp`s cannot simply let the VM's `JumpIfFalse` decide an elisp conditional. The fix is a custom opcode — the compiler emits a `TRUTHY` extension op before every conditional jump that converts the elisp value on the stack into a VM boolean using the elisp rule. We will see exactly that opcode in Chapters 9 and 10. Mention it here only so that the phrase “only nil is false” lands as a fact with teeth: it is not a documentation nicety, it is a custom instruction in the compiled code.

The self-testing example

One more thing about `demo.el` deserves attention, because the pattern recurs in every example file and it is how the project keeps itself honest. The demo does not just print; it tests. After the four `message` calls come a series of `ert-deftest` forms and a final `(ert-run-tests-batch-and-exit)`. ERT is Emacs's own regression-testing framework — `ert-deftest` to name a test, `should` and `should-not` and `should-error` to assert inside it — and `elisp`s ports a subset of it, written in elisp in the prelude. The batch runner exits non-zero if any test fails, so every example file is simultaneously a demo you can read and a test the continuous-integration system runs. When this book quotes an example, it is quoting something that is executed on every commit and asserted to behave as shown. That is the strongest grounding a technical book can have, and it is why the code in these pages can be trusted: the repository runs it.

Chapter 3 – The Reader

Before a single form can be compiled or run, it has to be *read* — turned from a flat string of characters into structured Lisp data. In most languages the equivalent stage is called parsing and it produces an abstract syntax tree that is distinct from the language’s runtime values. Lisp collapses that distinction: the reader produces ordinary Lisp data — cons cells, symbols, numbers, strings, vectors — and the compiler then walks that data as data. This is the deep reason Lisp has macros, and it is why the reader is the right place to start a tour of how elisprs actually works. The reader is `src/reader.rs`, and its job, in the words of its own header, is to build “forms directly as `ElispHost` heap objects (cons cells, symbols) so the compiler can walk them and quoted literals can be emitted as `Value::Obj` constants.”

That last clause is the architectural hook. The reader does not produce a private AST type; it produces the very same heap objects the running program will manipulate. When the reader encounters `(a b c)` it allocates three cons cells in the `ElispHost` arena and returns a `Value::Obj` handle to the first. When the compiler later needs to emit a quoted literal, it can just bake that same handle into the chunk’s constant pool. The reader and the runtime share one representation of data, end to end.

The shape of the reader

The reader is a hand-written recursive descent reader over a vector of characters. It is not table-driven and it is not generated; it is a `Reader` struct holding the character vector and a position, with a `read_form` method that dispatches on the next character and recurses. The top-level entry point, `read_all`, loops: skip whitespace, and if there is input left, read one form and push it. The result is a `Vec<Value>` of top-level forms, which is exactly what the file evaluator iterates over.

The dispatch in `read_form` is a single match on the leading character, and reading it top to bottom is a tour of elisp’s surface syntax. An open paren begins a list. An open bracket begins a vector. A double quote begins a string. A question mark begins a character literal. A single quote is `quote`. A `#` is `function`. A backtick is backquote. A comma is unquote, or unquote-splicing if followed by `@`. A `#` followed by various characters is a reader macro — `#s(...)` for a hash table or record, `#(...)` for a propertized string, `#x/#o/#b/#NNr` for radix literals. And anything else falls through to `read_atom`, which reads a token and classifies it as a number, `nil`, `t`, or a symbol. We will take these in turn.

Whitespace, comments, and where a token ends

Before the dispatch can run, the reader has to skip the inter-token noise, and the two kinds of noise are whitespace and comments. The `skip_ws` routine advances over any whitespace and, when it sees a `;`, consumes everything through the end of the line — the elisp line comment. This is why a `)` in a trailing comment does not close a list and a `(` in a comment does not open one: the reader never sees them as structure, because `skip_ws` ate the whole comment before the dispatch looked at the next character. The REPL’s balanced-paren logic from Chapter 2 reimplements this same comment-and-string awareness for its own “is this input complete?” question, but the reader proper handles it in `skip_ws`.

Equally important is where a *token* ends, because an atom is read by accumulating characters until a delimiter, and the delimiter set defines the boundaries of every symbol and number. The delimiters are whitespace plus the eight structural characters `() [] " ' `` , ;`. Anything that is not one of those extends the current token, which is why a symbol name can contain characters that would be punctuation in most languages — `1+`,

`<=`, `cl-defstruct`, and `string-match-p` are all single tokens because none of `+`, `=`, `-`, or `?` is a delimiter. It is also why escaping exists: to put an actual delimiter *into* a symbol name you backslash it, and the backslash both forces the next character in literally and forces the whole token to be a symbol rather than a number. The delimiter set is small and fixed, and it is the quiet reason elisp can spell operators as ordinary names — there is no separate operator lexer, only atoms that happen to be punctuation.

Atoms: numbers, nil, t, and symbols

The atom reader accumulates characters until it hits a delimiter — whitespace, a paren, a bracket, a quote, a backtick, a comma, or a semicolon — and then hands the token to `classify`. Classification is where elisp's particular rules about what counts as a number live, and they are more subtle than they first appear.

`nil` reads as fusevm's `Value::Undef`, and `t` reads as `Value::Bool(true)`. This is the concrete form of the truthiness mapping from the last chapter: the canonical false value of elisp, `nil`, is represented as the VM's `Undef`, and the canonical true value, `t`, as the VM's boolean true. Everything else that is not a number is interned as a symbol.

For numbers, `classify` tries `i64` first, so `42` is an integer. Then it handles a quirk that trips up everyone who has not read the Emacs manual carefully: a trailing decimal point with no fractional digits is an *integer*, not a float. `1.` is the integer `1`; `-3.` is the integer `-3`. But `1.5` and `1.e3` are floats. The reader implements exactly this: if the `i64` parse fails, it strips a trailing `.` and tries `i64` again before falling back to a float parse. Only a token that looks numeric — digits with optional sign, decimal point, and exponent — and parses as `f64` becomes a float. There is even handling for Emacs's read syntax for the non-finite floats, `1.0e+INF` and `0.0e+NaN`, with sign preserved, because Emacs prints infinities and NaNs that way and a faithful reader has to read back what the printer writes.

Keywords are symbols whose name starts with a colon, and the reader is careful to never classify a colon-led token as a number — `:1` is the keyword `:1`, not a number. Keywords matter because they self-evaluate: in the compiler, a symbol whose name starts with `:` is loaded as a constant rather than looked up as a variable, which is what makes `:greeting` usable as a literal marker in a `cl-defun` argument list without first binding it. We will see that in the compiler chapter; the reader's only job is to intern it as a symbol and let the colon ride along in the name.

There is one more subtlety in the atom reader: backslash escaping inside a symbol. A backslash forces the next character into the symbol name literally, so `foo\ bar` is a single symbol whose name contains a space, and `\,` is a one-character symbol named comma. Crucially, any escape also forces the token to be a symbol — never a number and never `nil` or `t`. This is how you would name a symbol `1` (you would write `\1`) or a symbol literally called `nil` (`\nil`). It is rarely needed, but the reader implements it because Emacs does, and fidelity to the corner cases is how a frontend earns the right to call itself an elisp reader rather than an elisp-like reader.

Numbers in other bases

elisp inherits a small family of radix reader macros from Common Lisp, and `elisprs` reads all of them. `#x1f` is hex, `#o17` is octal, `#b101` is binary, and the general form `#NNrDIGITS` reads `DIGITS` in base `NN` for any base from 2 to 36, so `#16rFF` is 255 and `#36rZ` is 35. Each form accepts an optional sign after the prefix, so `#x-10` is negative sixteen. The implementation reads the prefix, decides the base, consumes an optional sign, gathers characters up to a delimiter, and parses with `i64::from_str_radix`. It is a small, exact reproduction of the Emacs behavior, and it exists because real `.el` files — particularly anything dealing with bytes, characters, or bitmasks — use hex and octal literals freely.

Character literals

A question mark introduces a character literal, and in elisp a character *is* an integer — `?A` is just the integer 65. The simple cases are simple: `?A` is 65, `?0` is 48, `?` (question mark space) is 32. The interesting part is the modifier and escape machinery, which the reader implements in full because elisp's character syntax is genuinely baroque and real code relies on it.

A character can carry control, meta, shift, hyper, super, and alt modifiers, spelled `\C-`, `\M-`, `\S-`, `\H-`, `\s-`, and `\A-`, and they nest: `?\C-\M-a` is control-meta-a. Each modifier is a high bit in the integer — meta is bit 27, control is bit 26, shift is 25, and so on — and the reader sets the appropriate bit on the character it reads recursively. Control is special-cased to do the ASCII fold that everyone expects: `?\C-a` is 1, `?\C-A` is also 1, `?\C-?` is 127 (delete), and control of a letter is the letter's position in the alphabet. The reader has a dedicated `apply_control` function that does this fold while preserving any other modifier bits already present, so `?\M-\C-a` and `?\C-\M-a` land on the same integer.

On top of modifiers there are the numeric and named escapes: `?\x41` is hex 65, `?é` is the four-hex-digit Unicode escape for é, `?\U00001F600` is the eight-digit form, `?\101` is octal 65, `?\N{U+1F600}` is the codepoint-by-name form, and the single-letter escapes `\n`, `\t`, `\r`, `\e`, `\a`, `\b`, `\v`, `\f`, `\d`, `\0` map to their usual control codes. The same escape vocabulary is reused inside string literals, which is why the reader factors it into a shared `read_escape_scalar`. The `regexp.el` example exercises the practical payoff — `(string-match "é" "aébé")` works, and the match positions count characters, not bytes — and that only works because the reader speaks Unicode escapes and the rest of the system counts in characters.

Quote, function-quote, and self-quoting forms

A leading single quote is the reader macro for `quote`: `'x` reads as `(quote x)`. A leading `#'` is the reader macro for `function`: `#'foo` reads as `(function foo)`. Both are implemented the same way — consume the prefix, read the following form, and wrap it in a two-element list headed by the appropriate symbol. There is nothing elisp-specific about quote, but `function` is where elisp's Lisp-2 nature first peeks through: `#'foo` means “the function named foo,” as distinct from `foo` meaning “the value of the variable foo,” and the two can refer to entirely different things because a symbol has two cells. We will develop that fully in Chapter 5. For now, the reader's contribution is just to turn `#'foo` into `(function foo)` so the compiler can decide what it means.

Strings

The string reader consumes characters until a closing quote, processing escapes along the way. Most of it is the shared escape vocabulary — `\n`, `\t`, the hex and Unicode and octal forms — but strings have two wrinkles worth noting. First, a backslash followed by an actual newline is a line continuation: the newline is elided, so a long string literal can be wrapped across source lines. Second, `\s` without a following dash is the space character (whereas `\s-` is the whitespace syntax class, which is a regexp concept, not a string one). These are small things, but they are exactly the small things that make a string literal copied out of a real `.el` file read correctly.

Vectors

Square brackets read as a vector, and elisp vectors are self-evaluating with *unevaluated* elements. That is, `[1 2 3]` reads as a vector of the integers 1, 2, 3, and `[a b c]` reads as a vector of the three symbols `a`, `b`, `c` — not their values. The reader reflects this by reading each element as a form and storing it verbatim in an `Obj::Vector`, with no evaluation. The compiler then treats a bare vector as a self-evaluating constant. This is different from a list, which is a call unless quoted, and the difference is exactly elisp's: `(a b c)` is a call to `a`,

but `[a b c]` is a literal vector of symbols. Real code leans on this for lookup tables and, as we will see, for `pcase`'s vector patterns.

Dotted pairs

A list need not be proper. The reader handles the dotted-tail syntax `(a b . c)`, where the final cdr is not `nil` but some other value. When it is reading a list and encounters a `.` that is followed by a delimiter — so it is a lone dot, not part of a symbol name or number — it reads one more form as the tail and then expects the closing paren. It then folds the accumulated elements onto that tail with `cons`, right to left, producing a chain whose last cdr is the dotted value rather than `nil`. This is how `(cons 1 2)` and the literal `(1 . 2)` produce the same structure, and it is the foundation of association lists, where each entry is often a `(key . value)` pair rather than a two-element list. The reader's willingness to build improper lists is a direct consequence of the object heap having real cons cells with an arbitrary cdr, a point we develop in the next chapter.

Backquote: expansion at read time

Backquote is where the reader does its most interesting work, because `elisp` expands backquote *eagerly, at read time*, rather than carrying an unexpanded backquote form into the compiler. A backquoted template ``(a ,b ,@c)` is, in the manual's standard decomposition, shorthand for a tree of `cons`, `append`, and `quote` calls that builds the templated structure at run time, substituting the unquoted parts. `elisp`'s reader performs that decomposition itself, in a function called `bq_expand`, the moment it reads the backtick.

The algorithm is the textbook one. A top-level ``x` is just `x`. A backquoted list is walked element by element: a plain element `e` becomes `(cons 'e rest)`, an unquoted element `,x` becomes `(cons x rest)`, and a splice `,@x` becomes `(append x rest)`, folding right so the structure builds correctly. A dotted tail and a vector template are handled too — a vector folds like a list and then wraps the result in `vconcat` to rebuild a vector. The output is ordinary `elisp`: `cons`, `append`, `vconcat`, and `quote` calls that the compiler lowers like any other.

Doing this at read time has a consequence that ripples forward into `pcase`. Because backquote is already decomposed into `cons` and `append` calls by the time anyone looks at a form, the `pcase` implementation can recognize backquote *patterns* — ``(,a ,b)` and ``(,a . ,rest)` and nested forms, plus the `vconcat`-headed vector patterns — by recognizing the shape the reader left behind. The reader's eager expansion is not just a convenience; it is what makes structural patterns over backquoted templates implementable. The `data-model.el` and `meta-eval.el` examples both lean on backquote heavily, in macro bodies like:

```
(defmacro swap! (a b)
  (let ((tmp (gensym)))
    `(let ((,tmp ,a)) (setq ,a ,b) (setq ,b ,tmp))))
```

Every comma in that template is an unquote the reader turned into a `cons` of the evaluated subform onto the rest, and the whole backquoted body became a tree of `cons` and `quote` calls before the macro was ever defined.

Records and hash-table literals

The last family of reader macros is `#s(...)`. `#s(hash-table ...)` reads a literal hash table, parsing an optional `test` (one of `eq`, `eq1`, `equal`) and a `data` list of alternating keys and values into an `Obj::HashTable`. `#s(NAME slot...)` reads a record — the print syntax of a `cl-defstruct` instance — into a vector tagged with a `cl-struct-NAME` symbol in slot zero, which is exactly how the struct system represents instances internally. This closes a loop: when you print a struct instance it comes out as `#s(NAME ...)`, and when you read that back you get an equivalent vector-tagged structure. The printer and the reader agree, which is the property that makes `prin1/read` round-tripping work, and round-tripping is the foundation of the `script-demo.el` example's trick of serializing data to disk and reading it back.

The reader as a runtime function

The reader is the front of the compile pipeline, but it is also a function a running program can call, because reading is a first-class elisp operation. The public entry `read_all` reads every top-level form from a string and is what the file evaluator iterates; the companion `read_one` reads a *single* form starting at a given character index and returns both the form and the index just past it. That second return value is the whole reason `read-from-string` can be called in a loop: a program reads one form, gets back where the reader stopped, and reads the next form from there. `meta-eval.el` uses the run-time reader to turn text into structure and then evaluate it — `(read "(a (b c) . d)")` produces a structure `equal` to the quoted literal, dotted tail and all, and `(eval (read "(* 6 7)") t)` reads and runs in one breath. The `Reader` struct itself is minimal — a `Vec<char>` of the source and a position index — and `read_form` is a single `match` on the next character, so “the reader” and “the run-time `read` subr” are the same recursive-descent code reached two different ways.

The character-modifier bits

The character-literal machinery from earlier in this chapter rests on a small table of bit positions that is worth making explicit, because real keybinding code in `.el` files depends on the exact values. A character is an integer, and its modifier state lives in high bits above the codepoint: meta is bit 27, control is bit 26, shift is bit 25, hyper is bit 24, super is bit 23, and alt is bit 22.

```
;; the modifier bits (reader.rs), transcribed:
;; meta  = 1 << 27      hyper = 1 << 24
;; ctrl  = 1 << 26      super = 1 << 23
;; shift = 1 << 25      alt   = 1 << 22
```

The prefixes `\M-`, `\C-`, `\S-`, `\H-`, `\s-`, and `\A-` set the corresponding bit, and they compose by OR-ing, so `?\M-\C-a` carries both the meta and control bits. Control is the one prefix that does not merely set a bit but folds the base character: a dedicated `apply_control` maps `?` to 127 (delete), maps `a` through `z` to 1 through 26, and folds the range `@` through `_` with an exclusive-or against 64, while preserving any other modifier bits already present. That fold is why `?\C-a` and `?\C-A` are both 1, why `?\C-?` is 127, and why `?\M-\C-a` and `?\C-\M-a` land on the same integer regardless of the order the prefixes were written. The `reader.rs` tests pin the simple end of this — `(list ?\n ?\t ?\e ?\0)` is `(10 9 27 0)`, and `?\` (backslash-space) is 32 — and the modifier fold is what makes the full keyboard-event vocabulary read back correctly.

What the reader does not do

A faithful book marks the edges, and the reader has one worth naming: it does not implement the circular-and-shared-structure read syntax, `#N=` and `#N#`. In Emacs those let a printed structure label a sub-object with `#1=` and refer back to it with `#1#`, which is how a self-referential or DAG-shaped value prints and reads without looping. elisprs’s reader has no label table and no placeholder mechanism; a `#` followed by a digit is routed to the radix reader, which expects an `r/R` after the digits and errors otherwise, so `#1=` produces a “malformed radix literal” error rather than a back-reference. This is a deliberate scoping decision — shared-structure syntax is rare in hand-written `.el` and the print side does not emit it — and it is the kind of corner the project documents rather than half-implements. The `#'`, `#s`, `#(`, and radix `#`-macros are the complete set of `#`-dispatched forms; the circular-structure family is simply out of scope.

With the reader understood, we have forms — cons cells, symbols, numbers, strings, vectors — sitting on a heap. The next question is what that heap actually is.

Chapter 4 — Values and the Object Heap

A running `elisprs` program manipulates two very different kinds of value, and the distinction between them is the most important single fact about how the system fits onto a borrowed VM. Some values are *immediates*: integers, floats, strings, booleans, and `nil`. These are `fusevm`'s own `Value` type, and the VM understands them natively — it can add two integers, compare two floats, and branch on a boolean without ever consulting `elisprs`. Other values are *compound*: cons cells, symbols, vectors, closures, hash tables. These are elisp-specific objects the VM knows nothing about, and they live in an `ElispHost` arena and travel through the VM as opaque handles. The handle is a `Value::Obj(u32)` — a tagged index into the arena. The VM can carry a `Value::Obj` around on its stack, copy it, store it in a local, and pass it to a call, but it cannot look inside it. To look inside, it has to hand the handle back to `elisprs`.

This is the representational core of the frontend design, and the engineering report states it as a principle: “elisp cells are not new `fusevm::Value` variants. They live in the `ElispHost` arena and ride through the VM as `Value::Obj(u32)` handles, resolved by the extension handler — `fusevm` stays language-agnostic.” The VM's value type was not forked to add a cons variant. It did not need to be, because one variant — an opaque object handle — is enough to carry anything `elisprs` wants, as long as `elisprs` holds the table that maps handles to objects. `fusevm` already had `Value::Obj` as its extension point for exactly this purpose, and `elisprs` slots its entire object model into that one door.

The Value type

`elisprs` re-exports `fusevm`'s `Value` as its own — `pub use fusevm::Value` is the first public line of the library. So when this book says a value is a `Value::Int` or a `Value::Obj`, those are `fusevm`'s variants, used directly. The immediates that matter to elisp are `Value::Int(i64)` for integers, `Value::Float(f64)` for floats, `Value::Str` for strings, `Value::Bool(bool)` for booleans, `Value::Undef` for `nil`, and `Value::Obj(u32)` for every heap object. There is also a `Value::Undef` doing double duty: it is elisp's `nil`. The mapping is worth stating precisely because it governs everything downstream:

- elisp `nil` is `fusevm Value::Undef`.
- elisp `t` is `fusevm Value::Bool(true)`.
- elisp integers and floats are `fusevm Value::Int` and `Value::Float`.
- elisp strings are `fusevm Value::Str`.
- every elisp cons, symbol, vector, closure, and hash table is a `Value::Obj(u32)` handle into the host arena.

That a string is an immediate and not a heap object is a small efficiency: strings flow through the VM as values, and string builtins like `concat` and `format` run host-side without an arena round-trip for the string data itself. Cons cells, though, must be heap objects, because elisp cons cells have *identity* — `(setcar p 99)` mutates the cell `p` points at, and any other reference to that same cell sees the change. Identity requires a shared mutable location, and that is exactly what an arena slot is.

The Obj enum

The compound objects are the variants of an `Obj` enum in `host.rs`. It is compact — ten variants — and every one of them earns its place:

```
;; The Rust Obj enum, transcribed for reference (host.rs):  
;; Cons(Value, Value)  
;; Symbol(SymbolData)  
;; Vector(Vec<Value>)  
;; Subr { name, min, max, f }  
;; Closure { params, body, is_macro, env }  
;; HashTable { test, entries }  
;; CharTable(CharTable)  
;; Buffer(usize)  
;; Marker(Rc<RefCell<MarkerData>>)  
;; Obarray(ObarrayData)
```

`Cons(Value, Value)` is the car and the cdr, each an arbitrary value. Because both fields are full `Values`, a cons can hold anything in either slot, and a cdr that is not another cons and not nil is precisely the dotted pair the reader knows how to build. There is no separate “list” type; a list is just a chain of conses whose last cdr is nil, and an improper list is the same chain ending in something else. This is the genuine Lisp cons cell, not a vector pretending to be a list, and the README is pointed about the difference: the object model is “an `Obj::Cons(Value, Value)` heap cell, not `rust_lisp`’s list-only cdr.” An earlier bootstrap of the project built on the `rust_lisp` crate, whose value model could not represent a dotted pair; that dependency was removed and replaced with this own model precisely so that `(cons 1 2)` could be a real `(1 . 2)`.

`Symbol(SymbolData)` is a symbol, and its internal structure is important enough that the next chapter is devoted to it. `Vector(Vec<Value>)` is a flat vector of values, the backing for `[1 2 3]` literals, `make-vector`, `aref/aset`, and — tagged in slot zero — `cl-defstruct` instances and records. `Subr` is a primitive function implemented in Rust: a name, a minimum and optional maximum arity, and a function pointer. `Closure` is a user-defined function or macro: compiled parameters, a compiled `fusevm::Chunk` body, an `is_macro` flag, and a captured lexical environment. And `HashTable` is, interestingly, an association vector — a `Vec` of key-value pairs scanned linearly, with a `test` discriminant selecting `eq`, `eqL`, or `equal` comparison. Linear scan sounds slow until you remember the table sizes elisp configuration code actually uses, which are small; the comment in the source says as much, calling it “fine for the table sizes elisp config uses.”

The last four variants are the editor object model, added as the buffer surface grew from a scratch-text model into something closer to Emacs’s own. `CharTable(CharTable)` is an Emacs char-table — `make-char-table`’s sparse map from a character code to a value, carrying a subtype symbol, a default slot, and an optional parent table to fall through — and it also backs the syntax tables the buffer layer consults. `Buffer(usize)` is a named editing buffer, the payload a stable index into the host’s buffer registry so a killed buffer keeps its slot and its handle stays `eq`. `Marker(Rc<RefCell<MarkerData>>)` is a position that moves with edits: its cell is shared with the owning buffer’s live-marker list, so a single insertion or deletion updates every reference at once. And `Obarray(ObarrayData)` is a first-class symbol namespace built by `obarray-make`, a private interning table whose symbols never collide with the global ones. These four are runtime-only — unlike cons cells and symbols they are never written to the heap image or the bytecode cache, because a buffer or a marker has no meaning across a save and a restore.

Two things are notable by their absence. There is no distinct `Macro` variant: a macro is a `Closure` with `is_macro: true`, sharing the closure’s entire shape and machinery, which is why macro expansion and function application can reuse the same code path with the single difference of whether arguments are evaluated first. And there is no `Float` or `Int` or `String` variant in `Obj`, because those are immediates, not heap objects — the arena holds only the things that genuinely need identity or unbounded structure.

Handles, the arena, and dangling references

The arena is a `Vec<Obj>`, and a handle is an index into it wrapped as `Value::Obj(u32)`. Allocation appends; a handle, once issued, is stable for the life of the host. This is a deliberately simple memory model: there is no garbage collector. Objects accumulate in the arena for the duration of a run. For the workloads `elisprs` targets — running a `.el` script to completion, evaluating a REPL form, serving a language-server request — this is entirely adequate, and it avoids an enormous amount of complexity. A long-lived process that allocated unboundedly would eventually want collection, but a script that runs and exits does not, and the arena's simplicity buys determinism: the same program produces the same handles in the same order every time, which is a property the bytecode cache will exploit in Chapter 11.

The printer has a fallback for a handle that does not resolve to a live object — it prints `#<dangling>` — which tells you the model's one sharp edge: a handle is only meaningful against the arena that issued it. This matters for serialization. When `elisprs` writes a heap image to the bytecode cache or embeds one in an AOT object, it is writing the *objects*, and on reload it rebuilds them so that the same handles resolve to equivalent objects. Get that wrong and a baked-in handle in a compiled chunk would resolve to the wrong object or to nothing. The whole apparatus of “builtin count,” “heap image,” and “schema key” that we will meet later exists to guarantee that handles line up across a save and a restore.

Allocation is append-only, and that buys determinism

The allocation rule is as simple as a memory model gets: `alloc` pushes the new object onto the end of the arena vector and returns its index as a `Value::Obj`. Handles are never reused, never moved, never compacted — once issued, an index points at the same slot for the life of the host. There is no free list and no relocation, which is what makes a handle stable enough to bake into a chunk's constant pool in the first place. The cost is that the arena only grows, which for a run-to-completion script or a single REPL form is a non-issue, and the benefit is a property the rest of the system leans on hard: *determinism*. Because allocation is append-only and the prelude loads the same way every time, the same program allocates the same objects in the same order and gets the same handles on every run. That reproducibility is exactly what lets the bytecode cache of Chapter 11 record a chunk whose constant pool holds handle `Value::Obj(412)` and trust that re-running against a rebuilt heap image will put the intended object back at slot 412. A model with a garbage collector that moved objects, or an allocator that reused freed slots, could not make that promise — the handle in the cached chunk might resolve to a different object on the second run. The absence of a collector, which looks at first like a simplification the project got away with, turns out to be a positive enabling property: a deterministic, append-only arena is what makes a serializable heap image *possible*.

Reaching the heap from the VM

Here is the mechanism that makes the whole thing run. The VM executes a chunk. Most instructions it handles itself — load a constant, add two numbers, jump. But some instructions are `elisp`-specific, and for those the VM calls back into `elisprs` through a registered *extension handler*. The handler is a plain Rust function, and it reaches the object heap through a thread-local host: a function called `with_host` borrows the thread-local `ElispHost` mutably and runs a closure against it. So when the VM hits a “get the car of this handle” situation — which actually arrives as a `CALL` of the `car` subr — the handler pops the handle off the VM stack, calls `with_host` to borrow the arena, indexes the handle, reads the `Cons`'s car, and pushes it back onto the VM stack. The VM never saw a cons cell; it saw a handle go in and a value come out.

There is a discipline around that thread-local borrow that is worth flagging now and will matter in Chapter 10: a subr that needs to call *back* into `elisp` — like `mapcar`, which applies a function to every element — must not be holding the host borrow while the callee runs, or the callee's own `with_host` would deadlock on the already-borrowed cell. The system handles this by routing all re-entrant calls through a single function, `call_function`, which is careful to take and release the borrow around the actual call. The higher-order builtins are intercepted before the borrow is taken. It is the kind of constraint that only exists because the object heap

and the VM are separate systems connected by a borrow, and it is the price of the clean separation that buys everything else.

The printer

The last piece of the value model is how a value becomes text, because the REPL, `-e`, `message`, `princ`, `prin1`, and `format`'s `%s` and `%S` all funnel into one printer. The printer takes a value and a `readable` flag — `true` for `prin1` (re-readable syntax), `false` for `princ` (raw, human-facing) — and produces a string.

The immediates are easy: `nil` and `Bool(false)` print as `nil`, `Bool(true)` prints as `t`, integers print as themselves. Floats are printed in the shortest form that reads back to the same float, with an exponent when that is shorter, and an integer-valued float keeps its `.0` so that `6.0` does not collapse to `6` and lose its floatness — `(number-to-string (* 2.0 3))` is `"6.0"`, a behavior the `stats.el` example asserts. NaN and the infinities print in Emacs's read syntax, `0.0e+NaN` and `1.0e+INF`, closing the loop with the reader. A string under `readable` gets its quotes and backslash escapes; under `princ` it comes out raw.

The compound objects are where the printer earns its keep. A symbol under `readable` escapes any characters that would not read back as the same symbol — a symbol whose name happens to look like a number, or starts with a `?`, gets a leading backslash — and an empty-named symbol prints as `##`. A cons prints as a list, walking the spine; if it hits a non-nil, non-cons cdr it prints the dotted-pair `" . "` and the final cdr, so `(1 . 2)` prints exactly that way. The printer also honors the quote-abbreviation controls: `(quote x)` prints as `'x`, `(function x)` as `#'x`, and a backquote form abbreviates too, all under the `print-quoted` control which defaults on. It respects `print-level` and `print-length` — the dynamic variables that cap how deep and how long a structure prints before it is truncated with `...` — which is how Emacs keeps a self-referential or enormous structure from printing forever. A vector prints as `[...]`, except a struct-tagged vector prints as `#s(NAME ...)` record syntax. A subr prints as `#<subr NAME>`, a closure as `#<closure>` or `#<macro>`, and a hash table in Emacs 30's `#s(hash-table ...)` syntax. Every one of these behaviors is a deliberate match to what Emacs's own printer produces, because the examples — and any real `.el` code — depend on a value printing the way Emacs prints it.

With values and their printing understood, we can look closely at the one Obj variant that carries the most semantic weight: the symbol.

Chapter 5 – The Obarray: A Lisp-2 in Rust

Emacs Lisp is a Lisp-2, and that single fact is, as the README puts it, “the whole personality of the language.” Understanding what a Lisp-2 is, and why it matters, and how `elisprs` represents it, is the difference between understanding `elisp` and merely reading it. This chapter is about the symbol — the object that makes `elisp` a Lisp-2 — and the obarray that holds all of them.

Two cells, one symbol

In a Lisp-1 — Scheme, Clojure, most modern functional languages — a name has one meaning. The symbol `list` refers to one thing, whether you write `(list 1 2)` or `(mapcar f list)`. If you bind `list` to a number, you have shadowed the function; you can no longer call it by that name in that scope. There is one namespace, and a name resolves to one value.

In a Lisp-2 — Common Lisp, Emacs Lisp — a name has two independent meanings, selected by *position*. In `(list 1 2)`, `list` is in function position, so it means the function named `list`. In `(mapcar f list)`, `list` is in argument position, so it means the value of the variable named `list`. These can be entirely different objects. You can have a variable called `list` holding the number 5 and a function called `list` that builds lists, simultaneously, with no conflict, because the symbol `list` carries two separate cells: a *value cell* and a *function cell*. The reader does not know or care which you mean; position decides, at compile time.

This is why `#'foo` exists. When you want to pass a function as a value — to `mapcar`, say — you cannot just write `foo`, because in argument position `foo` means the *variable* `foo`, which may be unbound or may hold something unrelated. You write `#'foo`, the function quote, which means “the contents of `foo`’s function cell.” The `meta-eval.el` example shows the two cells diverging deliberately:

```
(ert-deftest meta-symbols ()
  (should (eq (intern "foo") 'foo))
  (should (equal (symbol-name 'bar) "bar")))
(should (not (eq (make-symbol "g") (make-symbol "g"))))
(put 'my-sym 'color 'blue)
(should (eq (get 'my-sym 'color) 'blue)))
```

`intern` returns the canonical symbol for a name; `make-symbol` makes a fresh, uninterned symbol that is *not* the same object even if you make two with the same name; and `put/get` hang properties off a symbol’s property list, a third thing a symbol carries beyond its two cells.

SymbolData

In `elisprs` a symbol is an `Obj::Symbol` wrapping a `SymbolData` struct, and the struct is the Lisp-2 made concrete:

```
;; SymbolData (host.rs), transcribed:
;;   name: String           ; the print name
;;   value: Option<Value>    ; the value cell (symbol-value); None = void
;;   function: Option<Value> ; the function cell (symbol-function)
;;   special: bool          ; dynamic-scope flag (set by defvar/defconst)
```

The two `Option<Value>` cells are the heart of it. The value cell holds the variable binding, reached by `symbol-value` and set by `set`; when it is `None` the variable is void, and reading it signals the `void-variable` error whose message is “Symbol’s value as variable is void.” The function cell holds the function binding, reached by `symbol-function` and set by `fset`; when it is `None` the function is void and calling it signals `void-function`. The two are wholly independent: `set` and `setq` touch only the value cell, `fset` and `defun` touch only the function cell, and a symbol can have either, both, or neither.

The function cell holds a `Value`, and that value can point at three different kinds of thing. It can point at an `Obj::Subr` — a primitive. It can point at an `Obj::Closure` — a user function or macro. Or it can point at *another symbol*, in which case it is an alias, and resolution follows the chain. `elisp` follows function-cell aliases up to 64 hops before giving up, which is generous enough for any real alias chain and bounded enough to never loop forever on a cycle. This is how `defalias` works and how `cl-first` can be an alias for `car`: the function cell of `cl-first` simply points at the symbol `car`, and a call through it resolves one more hop to `car`’s `subr`.

The `special` flag is the fourth field, and it is the switch between dynamic and lexical binding for that symbol. We will develop binding fully in the next chapter, but the one-sentence version is: a symbol marked `special` — which is what `defvar` and `defconst` do — always binds dynamically, through a stack of saved values, rather than lexically, through a captured scope. The flag lives on the symbol because specialness is a property of the variable, not of any particular binding of it.

The obarray and intern

The obarray is the table of all interned symbols, keyed by name. When the reader encounters a token that is not a number, it *interns* it: it looks the name up in the obarray, and if a symbol with that name already exists it returns that exact symbol, and if not it allocates a fresh `Obj::Symbol`, records it in the obarray, and returns it. The consequence is that two reads of `foo` anywhere in a program produce the same symbol object — the same arena handle — which is what makes `eq` work on symbols. Symbol identity is the foundation of nearly everything: `(eq 'foo 'foo)` is true because both `'foos` interned to the same handle, and that in turn is why symbols are the natural keys for property lists, `alist`s, and `pcase`’s literal patterns.

`make-symbol`, by contrast, deliberately does *not* intern. It builds a symbol with a name but never records it in the obarray, so each call yields a distinct object, and `(eq (make-symbol "g") (make-symbol "g"))` is false. This is the mechanism behind `gensym` and macro hygiene: when a macro needs a temporary variable that cannot possibly collide with a variable in the code it is expanding into, it makes an uninterned symbol, and because that symbol is not in the obarray, no source text can ever name it. The `swap!` macro from the last chapter used exactly this — `(gensym)` produced an uninterned symbol for the temporary, so swapping `x` and `y` could never be broken by the user happening to have a variable named `tmp`.

Functions live in the function cell

Because functions live in symbols’ function cells, defining a function is, mechanically, setting a cell. When you write `(defun square (x) (* x x))`, the compiler — as we will see in detail in Chapter 9 — builds a closure object for the body and then emits the extension op that sets the `square` symbol’s function cell to point at it. Calling `(square 6)` resolves `square`’s function cell to the closure and runs it. There is no separate “function table”; the obarray of symbols *is* the function table, just as it is the variable table, because each symbol carries both cells.

This unification is elegant and it is also exactly Emacs’s design. `symbol-function` and `fset` are first-class

operations a program can call, which means a program can inspect and redefine functions at run time — advice, mocking, dynamic dispatch tables built by hand — all by reading and writing function cells. The `data-model.el` example's custom error definitions and the prelude's `defalias`-based aliasing both work by manipulating cells directly. The Lisp-2 is not a restriction; it is a richer namespace model that real elisp code uses constantly, and elisprs implements it faithfully because a Lisp-2 that quietly collapsed to a Lisp-1 would mis-run a large fraction of existing `.el` code.

The third cell: property lists

A symbol carries two cells that make it a Lisp-2 — the value cell and the function cell — and a third store that is just as load-bearing: its property list. Reached with `get` and `put`, the property list hangs arbitrary key-value metadata off a name, and `meta-eval.el` shows the bare operation, `(put 'my-sym 'color 'blue)` followed by `(get 'my-sym 'color)` returning `blue`. What makes the property list more than a convenience is how much of the system rides on it. The condition system of Chapter 14 records each error symbol's place in the hierarchy as its `error-conditions` property and its human message as `error-message`; `define-error` is two `put` calls. The struct system of Chapter 22 records slot indices and tag parentage the same way. So the obarray is three tables at once: the value table (read by `symbol-value`), the function table (read by `symbol-function`), and the metadata table (read by `get`), all keyed by the same interned symbol. The property list is `symbol-plist`, exposed whole, and it is the substrate a great deal of elisp's reflective machinery — advice, font-lock, custom declarations — is built on.

Aliases and indirect-function

Because the function cell can hold *another symbol* rather than a callable, a symbol can be an alias, and resolving a call follows the chain. `defalias` installs such an alias — `(defalias 'cl-first 'car)` makes `cl-first`'s function cell point at the symbol `car` — and a call through `cl-first` resolves one more hop to `car`'s subr. The resolver, `resolve_function`, walks this chain up to 64 hops before giving up with “function indirection too deep,” which is generous enough for any real alias chain and bounded enough never to loop forever on an accidental cycle. The elisp-visible window onto this is `indirect-function`, which follows the chain and returns the ultimate callable, and `symbol-function`, which reads one cell without following. The 64-hop bound is the same one that distinguishes a `defalias` chain (one hop per link) from the built-in aliases of Chapter 12 (a shared subr pointer, zero hops): both present the same calling behavior, but one costs indirection and the other does not.

Walking the obarray

The obarray is not only a lookup table; it is enumerable. `mapatoms` applies a function to every interned symbol, which is how a program inspects or audits the whole namespace — finding every symbol whose name matches a prefix, say, or every function with a docstring. `intern-soft` is the non-creating lookup — it returns the symbol for a name if one is already interned and `nil` otherwise, where plain `intern` would create one — and `unintern` removes a symbol from the obarray. These matter because interning has a side effect (it adds to the table), and a program that only wants to *test* whether a name is known must use `intern-soft` to avoid polluting the obarray with a symbol it created merely by asking. The obarray itself is a plain `HashMap<String, u32>` from name to arena handle inside the host, which is what makes interning a single hashtable lookup and `eq` on two reads of the same name a guaranteed hit on the same handle.

That global table is no longer the only one. `obarray-make` builds a *first-class* obarray — a private `Obj::Obarray` whose own name-to-symbol table is separate from the global one, so symbols interned into it never collide with the global namespace — and `obarrayp` tests for it. The interning primitives take an optional obarray argument that selects which table to intern into: `intern`, `intern-soft`, `unintern`, and `mapatoms` all accept it and default to the global obarray when it is omitted. A private obarray is exactly the

mechanism a reader or a completion table wants when it needs a scratch namespace it can populate and enumerate without leaving a trace in the symbols every other form can see.

Chapter 6 – Binding: Lexical and Dynamic

Emacs Lisp binds variables two different ways, and which way a given variable binds is one of the genuinely hard things about the language. `elisp` implements both — the engineering report lists “lexical and dynamic binding” as working, and the component grid marks “obarray + lexical & dynamic binding” as done — and this chapter is about how. It is the most intricate chapter in the language half of the book, because binding is where `elisp`’s history and its semantics collide, and because getting it right requires two completely different runtime mechanisms cooperating.

The two disciplines

A *lexical* binding is visible only within the textual region of the program where it is written. When you write `(let ((x 10)) ...)` under lexical binding, the `x` refers to that binding everywhere in the body and *nowhere else* — a function called from the body does not see `x` unless the function was defined inside the body and closed over it. Lexical scope is the discipline of nearly every modern language; it is what makes closures work and what makes a function’s behavior depend only on its arguments and its lexical environment, not on who called it.

A *dynamic* binding is visible to everything that runs during the *time* the binding is in effect, regardless of where in the program text that code is written. When you dynamically bind `x` and then call a function, that function sees your binding of `x`, even if it was defined in a completely different file and knows nothing about your code. The binding lasts for the dynamic extent of the form — until the `let` returns — and then the old value is restored. Dynamic scope is older, rarer in modern languages, and still central to `elisp` because so much of Emacs’s configurability depends on it: a variable like `case-fold-search` is dynamically bound around a search so that the search sees your setting without anyone having to thread it through as an argument.

Emacs Lisp historically defaulted to dynamic binding and now defaults to lexical in files that opt in with a `lexical-binding: t` file-local variable. The two coexist: even in a lexical-binding file, variables declared `special` with `defvar` still bind dynamically. So a correct `elisp` runtime cannot pick one discipline; it must run both at once and route each binding to the right one based on whether the variable is special.

How `elisp` represents the two

`elisp` keeps two entirely separate runtime structures, one for each discipline.

For lexical binding there is a `scope`: a mutable list of `(symbol-handle, value)` pairs plus a link to a parent scope. Scopes form a chain, and the chain captured at the moment a closure is created *is* the closure’s environment — that is what the `env` field of `Obj::Closure` holds. Looking up a lexical variable walks the scope chain from innermost out, and because each scope’s bindings live behind a mutable cell, `setq` on a lexical variable mutates the slot in place, so a closure that shares a scope with the code that created it sees later mutations. This is the standard closure machinery: an environment is a linked list of frames, a closure captures a pointer into it, and the indefinite extent of the captured frames is what keeps a closed-over variable alive after the `let` that created it has returned.

For dynamic binding there is a `specstack`: a stack of `(symbol-handle, previous-value)` entries. To dynamically bind a special variable, `elisp` pushes the symbol’s current value cell onto the `specstack` and

overwrites the cell with the new value. To unbind — which happens when the binding form exits — it pops the entry and restores the saved value into the cell. The variable's *current* value always lives in the symbol's value cell, exactly where `symbol-value` reads it; the specstack just remembers what to put back. This is precisely the “shallow binding” technique Emacs itself uses, and its great virtue is that reading a dynamic variable is just reading the value cell, with no search — the cost is paid at bind and unbind time, not at every reference.

The routing decision

The crux is a single function, `bind_here`, that decides for each binding which mechanism to use. Its logic is short enough to state in full: if the symbol is special, bind it on the specstack; otherwise, if there is a current lexical scope, push the binding into that scope; otherwise — at top level, with no scope open — fall back to writing the global value cell directly. That three-way decision is the whole of the lexical-versus-dynamic routing, and it runs on every variable a `let` or a function call binds.

```
;; the routing, in elisp terms:
(defvar *log-level* 'info)      ; marks *log-level* special → dynamic
(let ((*log-level* 'debug)      ; special var → pushed on the specstack
      (tmp (compute)))         ; ordinary var → pushed into the lexical scope
    (run-handlers))             ; handlers see *log-level* = debug dynamically
;; on exit: specstack restores *log-level*; the lexical scope is discarded
```

A `defvar` or `defconst` sets the symbol's `special` flag, and from that point every binding of that symbol routes to the specstack. An ordinary variable, never declared special, routes to the lexical scope. The same `let` can bind one of each, as the sketch shows, and elisprs threads them to the two different mechanisms transparently.

let and let*

The two `let` forms differ in when their initializers are evaluated relative to the bindings taking effect, and elisprs lowers them differently to capture the difference exactly. `let` evaluates *all* of its initializers first, in the surrounding scope, and then installs all the bindings at once. So in `(let ((a 1) (b a)) ...)`, the `a` in `b`'s initializer refers to whatever `a` meant *outside* the `let`, because `b`'s initializer is evaluated before the new `a` binding exists. The compiler lowers this by evaluating every initializer onto the stack first, then emitting a single wide “bind *n* at once” op that opens one scope and installs all *n* bindings together.

`let*` evaluates each initializer and installs its binding *before* moving to the next, so each initializer sees the bindings above it. In `(let* ((a 1) (b a)) ...)`, `b` is 1, because `a`'s binding is in effect when `b`'s initializer runs. The compiler lowers this by opening one scope and then, for each binding in turn, evaluating the initializer and immediately binding it. The difference between the two lowerings — bind-all-at-once versus bind-one-at-a-time — is the difference between the two forms, made mechanical. Both forms, on exit, emit an unbind op that closes the scope and unwinds any dynamic bindings made inside it.

setq

`setq` assigns to an existing binding, and it respects the same lexical-then-global search. Assigning to a variable tries the lexical scope first: if the variable is bound in some enclosing lexical scope, `setq` mutates that slot. If it is not found lexically, `setq` writes the global value cell. This is why a `setq` inside a `let` body updates the `let`'s binding rather than creating a global, and why a `setq` at top level creates or updates a global. The `control-flow.el` example's loop accumulators rely on exactly this:

```
(let ((i 0) (total 0))
  (while (< i 5) (setq total (+ total i)) (setq i (1+ i)))
  (should (= total 10)))
```

Each `setq` finds `i` and `total` in the enclosing lexical scope and mutates them in place; the closure machinery's mutable cells are what make the in-place update visible across loop iterations.

Unwinding on non-local exit

There is one more thing the binding system has to get right, and it is subtle: what happens to the bindings when control leaves a scope *abnormally*. If a `throw` or an error unwinds out of the middle of a `let` body, the `let`'s unbind op never runs in the normal flow, and yet the dynamic bindings it made must still be restored — leaving a special variable stuck at its inner value would corrupt every later computation. The host tracks, for each open scope, the specstack depth at which it was entered, and an exit-recovery function unwinds the specstack and closes scopes back to a target depth when a non-local exit jumps over the normal unbind. This is the same bookkeeping Emacs does with its specpdl unwind-protect machinery, and it is what makes `condition-case` and `catch` safe to use around code that binds dynamic variables. The `errors.el` example even documents a known rough edge here — a comment notes that loop-break via `throw` inside a nested funcall can over-unwind the specstack — which is exactly the kind of honest, specific limitation the project surfaces rather than hides, and a reminder that unwinding across the lexical/dynamic boundary is genuinely hard.

The frame stack and scope recovery

The lexical scope chain and the dynamic specstack are the two binding mechanisms, but there is a third bookkeeping structure that ties them together at the boundaries of a function call: the frame stack. Each entry records a saved lexical environment and the specstack depth at which a scope was entered, so the host can answer the question “what was the binding state before this call, and how do I get back to it?” When a closure runs, `run_closure` notes the current scope depth on entry, opens a child scope of the closure's captured environment, binds the parameters, runs the body, and on the way out calls `unwind_scopes_to` to return to exactly the entry depth. In the normal case this is tidy: the body opened some scopes, the body closed them, and the recovery is a no-op.

The structure earns its keep on the abnormal case, the one the error chapter will return to. If a `throw` or an error unwinds out of the middle of a `let` inside the body, that `let`'s `UNBIND` op never runs in the normal flow, so its lexical scope leaks and its dynamic bindings sit unrestored. The frame stack is what makes this recoverable: because each scope recorded the specstack depth it was entered at, `unwind_scopes_to` can pop scopes and restore dynamic value cells back to the target depth as the non-local exit passes through, so the caller's environment is not corrupted by the callee's abandoned bindings. The source is candid that this cross-boundary recovery is the hard part — a comment notes that a `throw`/error out of an inner `let` “leaks scopes that this restores,” and the CHANGELOG records a fix where exactly this leak was “the root cause of the ‘void variable’ failures” that showed up when an ERT `should` wrapped a macro expanding to `catch` and nested `let`, which is to say `pcase` and `cl-loop`. That single fix — unwind the scope stack to the entry depth on non-local exit — is why structural pattern matching and the loop facility work inside a test assertion, and it is the most concrete example in the book of how genuinely intricate the lexical/dynamic boundary is to get exactly right.

With reading, values, symbols, and binding in hand, we have everything the special forms need. They are next.

Chapter 7 – The Special Forms

Most of what looks like a function call in elisp is one. `(car x)`, `(+ 1 2)`, `(mapcar f xs)` — these are ordinary calls: every argument is evaluated, and the results are handed to a function. But a handful of forms are not calls, because they do *not* evaluate their arguments in the ordinary way. `(if c a b)` must not evaluate both `a` and `b`; that is the whole point of `if`. `(setq x 5)` must not evaluate `x`, because `x` names the place to assign, not a value to read. `(quote x)` must not evaluate `x` at all. These are the *special forms*, and they are special precisely because they control evaluation rather than submit to it. They are the irreducible core of the language — everything else, including the entire macro and prelude surface, is built on top of them.

elisp recognizes twenty-two special forms in the compiler, and the engineering report counts them at exactly that: “22 compiler-recognized special forms with macros.” They are recognized by name in the compiler’s dispatch, before the general call-lowering path, because each one needs its own custom lowering to bytecode. This chapter walks them by family. The lowering details — which bytecode each one emits — are Chapter 9’s subject; here the focus is what each form *means* and why it has to be special.

The quoting forms: quote and function

`quote` is the simplest special form and the most fundamental. `(quote x)` returns `x` unevaluated. The reader’s `'x` abbreviation expands to it, so `'foo` is `(quote foo)` and evaluates to the symbol `foo` itself rather than the value of the variable `foo`. Without `quote` there would be no way to write a literal symbol or a literal list, because every symbol would be read as a variable reference and every list as a call. `quote` is the escape hatch from evaluation, and the compiler lowers it to nothing more than loading its argument as a constant.

`function` is `quote`’s Lisp-2 sibling. `(function foo)`, abbreviated `#'foo`, means “the function named `foo`,” and for a bare symbol it behaves like `quote` — it loads the symbol as a constant, and the call machinery resolves the symbol’s function cell when the value is later funcalled. But `function` has a second job: when its argument is a lambda form, `#'(lambda (x) ...)`, it builds a *closure*, capturing the current lexical environment, exactly as a bare `lambda` would. This distinction matters because a literal lambda form is not callable — `funcall` rejects it as an *invalid-function* — whereas a closure is. The CHANGELOG records the fix that made this correct: “`#'(lambda ...)` now compiles to a closure instead of loading the literal lambda form (which `funcall` rejected as *invalid-function*).” So `#'foo` on a symbol is a quote, and `#'(lambda ...)` on a lambda is a closure constructor, and the compiler decides which by inspecting the argument.

lambda: the anonymous function

`lambda` builds an anonymous function. `(lambda (a b) (+ a b))` evaluates to a closure that takes two arguments and returns their sum. The body is compiled — *compiled*, not stored as a tree — into its own `fusevm::Chunk`, and the closure object carries that chunk plus the parsed parameter list and a capture of the current lexical environment. Calling the closure runs its chunk on a nested VM with the arguments bound. The `functions.el` example puts this directly:

```
(ert-deftest fn-lambda ()
  "Anonymous functions are first-class values."
  (should (= (funcall (lambda (a b) (+ a b)) 3 4) 7))
  (let ((adder (lambda (x) (+ x 100))))
    (should (= (funcall adder 5) 105))))
```

The header comment of `functions.el` describes the mechanism exactly: “A user closure carries a precompiled `fusevm::chunk`; calling it runs that chunk on a nested fusevm VM.” The lambda is special because its parameter names must not be evaluated — they are names to bind, not expressions — and its body must be compiled to a chunk rather than evaluated immediately. A lambda that appears in the head position of a call, `((lambda (x) x) 5)`, is also compiled to a closure and then immediately called, so the immediately-invoked-function idiom works.

Parameter lists may use `optional` and `rest`. A required parameter must be supplied; an `optional` parameter defaults to `nil` when absent; an `rest` parameter collects all remaining arguments into a freshly-consed list. The parameter parser walks the list with a small state machine: it pushes symbols into the required list until it sees `optional`, then into the optional list until it sees `rest`, then sets the rest parameter. Calling with too few required arguments, or too many when there is no rest parameter, signals `wrong-number-of-arguments`. The richer Common-Lisp lambda-list features — `key`, per-argument defaults, `aux` — are *not* special-form features; they are provided by `cl-defun` in the prelude, which we will see desugars to `cl-structuring-bind` over a plain `rest` list.

The sequencing forms: `progn` and `prog1`

`progn` evaluates its body forms in order and returns the value of the last one. It is the canonical way to do several things where one expression is expected — the body of a `when`, the consequent of an `if`. `prog1` is its mirror: it evaluates its body in order but returns the value of the *first* form, discarding the rest. `prog1` is the idiom for “do this, then clean up, but return the original value,” and the `functions.el` example pins down the difference:

```
(ert-deftest fn-sequencing ()
  (should (= (prog1 1 2 3) 1))
  (should (= (progn 1 2 3) 3)))
```

These are special forms rather than functions for a subtle reason: a function would evaluate all its arguments and then receive the values, at which point it could only return one of them but could not control the *order* or the discarding in the way the sequencing semantics require — and more importantly, `progn` is the substrate that macros expand into when they need to splice multiple forms into one position, so it has to be a primitive the compiler understands directly. The compiler lowers both by compiling each form and inserting a stack `Pop` after every form whose value is not kept.

The conditionals: `if`, `when`, `unless`, `cond`

`if` is the root conditional: `(if cond then else...)` evaluates `cond`, and if it is non-`nil` evaluates `then`, otherwise evaluates the `else` forms as an implicit `progn`. The “non-`nil`” is the truthiness rule from Chapter 2 — only `nil` is false — and it is the reason the compiler cannot use the VM’s native branch directly. It emits a `TRUTHY` op to convert the elisp condition value to a VM boolean under the elisp rule, and only then a conditional jump.

`when` and `unless` are the one-armed conditionals: `(when c body...)` runs the body if `c` is non-`nil` and returns `nil` otherwise; `(unless c body...)` is the negation. In Emacs these are macros over `if`, but elispr makes them special forms recognized directly by the compiler, lowering them with the same `TRUTHY`-then-jump shape but with the jump polarity flipped for `unless`. The result is identical behavior with a slightly more direct lowering.

`cond` is the multi-way conditional: a sequence of clauses, each a test followed by a body, where the first clause whose test is non-`nil` has its body evaluated and returned. A clause with a test but no body returns the test value itself when it is `truthy` — the `(cond (x) ...)` idiom that returns `x` if it is non-`nil`. A final `(t ...)` clause is the else. The `control-flow.el` example’s `classify` is the canonical shape:

```
(defun classify (n)
  (cond ((< n 0) 'neg)
        ((= n 0) 'zero)
        (t 'pos)))
```

The compiler lowers `cond` to a chain of test-and-jump sequences, each clause jumping past the rest on a match, with the test-only clause handled specially so it can keep the test value on the stack.

The boolean connectives: `and`, `or`

`and` and `or` are special forms because they short-circuit, and short-circuiting is a control-flow property a function cannot have. `(and a b c)` evaluates `a`, `b`, `c` left to right and stops at the first `nil`, returning it; if none is `nil` it returns the last value. `(or a b c)` stops at the first non-`nil` and returns it; if all are `nil` it returns `nil`. The `control-flow.el` example shows that these return *values*, not just booleans:

```
(ert-deftest cf-logic ()
  "and / or short-circuit."
  (should (= (and 1 2 3) 3))
  (should-not (and 1 nil 3))
  (should (= (or nil nil 5) 5))
  (should-not (or nil nil)))
```

`(and 1 2 3)` is 3, the last value, not `t`; `(or nil nil 5)` is 5, the first non-`nil`. This value-returning behavior is what makes `(or x default)` the idiom for “x, or a default if x is nil,” and `(and x (foo x))` the idiom for “call foo on x only if x is non-nil.” The compiler lowers them by duplicating each intermediate value, testing it with `TRUTHY`, and jumping out early on the short-circuit condition while leaving the deciding value on the stack.

The loop: `while`

`while` is elisp’s only primitive loop. `(while cond body...)` evaluates `cond`, and while it is non-`nil`, evaluates the body and tests again, returning `nil` when the condition finally fails. Every other looping construct in the language — `dotimes`, `dolist`, `cl-loop` — is a macro that expands into a `while`. The `control-flow.el` example uses all three, but `while` is the one underneath:

```
(let ((i 0) (total 0))
  (while (< i 5) (setq total (+ total i)) (setq i (1+ i)))
  (should (= total 10)))
```

The compiler lowers `while` to a backward jump: a loop top, the condition with a `TRUTHY` test, a conditional exit jump, the body, a `Pop` to discard each iteration’s value, and an unconditional jump back to the top. It returns `nil` because a `while` has no meaningful accumulated value — accumulation is the body’s job, through `setq` on a `let`-bound variable, as the example shows.

The assignment form: `setq`

`setq` assigns to variables. `(setq x 5)` sets `x` to 5; `(setq x 1 y 2)` sets both, left to right, and returns the last value assigned. It is special because its odd arguments are variable *names*, which must not be evaluated, while its even arguments *are* evaluated. The lowering, as we saw in the binding chapter, writes through the lexical-then-global search: the assignment finds the variable in an enclosing lexical scope if it is there, and otherwise writes the global value cell. `setq` is the workhorse of imperative elisp, and because the value cell it targets may be lexical or global depending on context, it is the form where the binding chapter’s machinery becomes visible in everyday code.

The definition forms: `defun`, `defmacro`, `defvar`, `defconst`

These four forms define things at the top level. `defun` defines a function: `(defun square (x) (* x x))` builds a closure for the body and installs it in the `square` symbol's function cell. `defmacro` defines a macro the same way, but the resulting closure is flagged `is_macro`, so it is expanded rather than called. The difference between the two lowerings is exactly one thing: a `defun` captures its defining lexical environment (so a function defined inside a `let` can close over the `let`'s variables), while a `defmacro` does not (a macro runs at expansion time, in no particular lexical environment). The `functions.el` example defines both a function and a macro and uses each:

```
(defun fib (n) (if (< n 2) n (+ (fib (- n 1)) (fib (- n 2)))))
(defmacro my-incr (place) (list 'setq place (list '1+ place)))
```

`defvar` and `defconst` declare *variables*, and their crucial side effect is the one from the binding chapter: they mark the symbol `special`, so it binds dynamically from then on. `(defvar x 10)` sets `x`'s value to 10 if it is unbound and, more importantly, flags `x` as a special variable so that every later `let` of `x` routes to the specstack. `defconst` is the same with the connotation of a constant. Both are special forms because the variable name must not be evaluated and because their effect on the symbol's `special` flag has to happen at the right time relative to the forms that bind the variable. The `script-demo.el` example opens with exactly this:

```
(defvar script-demo--dir "target/script-demo")
```

The non-local exit forms: `catch`, `unwind-protect`, `condition-case`

The last family is the most powerful: the forms that let control jump out of the normal flow. `catch` and `throw` are a labeled escape — `(catch 'tag body...)` establishes a catch point, and `(throw 'tag value)` anywhere in the dynamic extent of the body unwinds straight back to the matching catch, which returns `value`. `unwind-protect` is elisp's `try/finally` — `(unwind-protect body cleanup...)` runs the body and guarantees the cleanup forms run afterward, whether the body returns normally or throws or errors. `condition-case` is elisp's `try/catch` — it runs a body and, if the body signals an error, dispatches on the error's condition to a handler.

These three are special forms, and their lowering is the most interesting in the compiler because of *how* it is done: the compiler rewrites each of them into an ordinary call to an internal intrinsic, wrapping the protected and handler code in zero-argument lambdas. `(catch tag body...)` becomes a call to `--catch--` with the tag and a thunk; `(unwind-protect body cleanup...)` becomes a call to `--unwind--` with two thunks; `(condition-case var body handlers...)` becomes a call to `--condition-case--` with the quoted variable, a body thunk, and a list of condition-and-handler thunks. These `--catch--`/`--unwind--`/`--condition-case--` names are not subrs in the library; they are intercepted inside the runtime's central `call_function` dispatch, which implements the unwinding against the host's pending-throw and pending-error state and the specstack-unwinding machinery from Chapter 6. We will return to the mechanism in the errors chapter; the point here is that the compiler's trick — rewrite the special form into a call with thunked subexpressions — is what lets these powerful forms be implemented in the host without teaching the VM anything about elisp's error model. The `errors.el` example exercises all three paths and their happy and sad cases, and the data-flow it asserts (`(catch 'tag (throw 'tag 42))` is 42, cleanup runs even when the body errors, a signalled error is recovered) is the behavioral contract the lowering has to honor.

What is not a special form

It is worth being precise about the boundary, because it is a common source of confusion when reading elisp. `let` and `let*` are special forms (they appear in the compiler's dispatch and got their own chapter),

as are the twenty just surveyed. But `when-let`, `if-let`, `dolist`, `dotimes`, `cl-loop`, `pcase`, `cl-defun`, `ignore-errors`, and the rest of the familiar elisp vocabulary are *not* special forms — they are macros, defined in elisp in the prelude, that expand into the special forms. This is the same stratification Emacs uses, and it is load-bearing: keeping the special-form set small (twenty-two forms the compiler must understand) and building everything else as macros means the compiler stays small and the language stays extensible. The next chapter is about that expansion machinery.

Chapter 8 – Macros and Backquote

A macro is a function that runs at compile time and returns code. Where a function takes values and returns a value, a macro takes *unevaluated forms* and returns a new form, which is then compiled in place of the original. This is the feature that lets a language extend its own syntax — `dolist`, `cl-loop`, `pcase`, and the entire prelude are macros — and it is the feature most responsible for elisp’s expressive reach. elisprs implements macros fully, and this chapter is about how expansion fits into the compile pipeline, how it interacts with the eager backquote from Chapter 3, and how the prelude uses macros to build the standard library out of the twenty-two special forms.

Macros are closures with a flag

We have already seen the representational fact that makes macros cheap to implement: a macro is an `Obj::Closure` with `is_macro: true`, sharing the exact shape and machinery of a function. The only operational differences are two. First, a macro’s arguments are *not* evaluated before the closure runs — the unevaluated forms are passed straight in as the macro’s parameters. Second, the macro’s result is not the final value; it is a new form that gets compiled. Everything else — the parameter parsing, the body chunk, the way the closure is invoked — is identical to a function. The runtime even reuses one function, `run_closure`, for both normal application and macro expansion; the difference is only whether the arguments were evaluated on the way in.

`defmacro` installs such a closure in a symbol’s function cell with the macro flag set, and it deliberately does *not* capture a lexical environment, because a macro runs at expansion time and has no meaningful runtime scope to close over. The `functions.el` example’s `my-incr` is a minimal macro:

```
(defmacro my-incr (place) (list 'setq place (list '1+ place)))
```

Given `(my-incr counter)`, the macro receives the *form* `counter` (not its value) as `place`, and returns the list `(setq counter (1+ counter))`, which is then compiled. The example’s comment says exactly this: “my-incr expands to `(setq counter (1+ counter))` before lowering.”

Expansion happens before lowering

The pipeline is explicit about ordering. The library’s `eval_forms` reads all top-level forms, then for each form it macro-expands *and then* lowers and runs. Expansion strictly precedes compilation. This matters because of a sequencing subtlety: a macro must be defined before the forms that use it are compiled. elisprs handles this by compiling and running form by form, so that an earlier form’s `defmacro` has installed the macro into the obarray before a later form that uses the macro is expanded. The `cl-defun.el` example’s header comment names this exact constraint: “Defining a macro at top level (not inside one form) is what makes the macro available to the forms that follow, so this lives in an example script rather than the single-form eval tests.”

There is one more wrinkle the library handles for faithfulness to Emacs’s top-level behavior: a literal top-level `(progn ...)` is *spliced* into its subforms before this form-by-form processing, recursively. This is what `splice_top_forms` does, and the reason is that in Emacs a `defmacro` inside a top-level `progn` is in effect for later subforms of that same `progn`. By flattening top-level prologs into the form sequence, elisprs makes a macro defined in a progn visible to its progn-siblings, matching Emacs.

Three grades of expansion

The runtime exposes three elisp-visible expansion operations, and they differ in how far they go. `macroexpand-1` expands the outermost form exactly one step: if the head is a macro, it runs the macro on the unevaluated argument forms and returns the result; otherwise it returns the form unchanged. `macroexpand` expands the head repeatedly until it is no longer a macro call — a fixpoint on the outermost form — but does not descend into subforms. `macroexpand-all` expands fully and recursively, into every subform, which is what the compiler driver uses before lowering. The `meta-eval.el` example checks the one-step form directly:

```
(should (equal (macroexpand '(my-when c x)) '(if c (progn x))))
```

There is a deliberate stopping rule in the recursive expander: it does not descend into `quote`. Quoted data is data, not code, and expanding inside it would corrupt literals — `'(when x)` is a literal three-element list, not a `when` form to expand. The expander recognizes `quote` and leaves its contents alone, which is the correct and necessary behavior and exactly what Emacs does.

Backquote, revisited as a macro tool

Chapter 3 explained that the reader expands backquote eagerly, turning ``(a ,b ,@c)` into a tree of `cons`, `append`, and `quote` calls at read time. The payoff of that design lands here, in macro writing, because backquote is the tool that makes macros readable. Without it, `my-incr` had to be written with explicit `list` calls: `(list 'setq place (list '1+ place))`. With backquote, the same macro reads as a template of the code it produces, with commas marking the holes:

```
;; the cl-defmacro form from cl-defun.el:  
(cl-defmacro unless2 (cond &rest body)  
  `(if ,cond nil (progn ,@body)))
```

The backquoted body is a near-picture of the output code: an `if` on the unquoted condition, `nil` in the then-branch, and a `progn` splicing in the body forms. Because the reader already decomposed this into `cons` and `append` calls when it read the macro definition, the macro body, when run, simply executes those calls and builds the output structure. The `meta-eval.el` example's `swap!` shows backquote combined with `gensym` for hygiene — the temporary variable is an uninterned symbol spliced into the template — and `accumulate` shows `,@` splicing a body:

```
(defmacro accumulate (var init &rest body)  
  "Bind VAR to an empty list; BODY pushes to it; return it reversed."  
  `(let ((,var nil)) ,@body (nreverse ,var)))
```

The CHANGELOG records that backquote support extends to vector templates and even to backquote *patterns* in `pcase` — ``[,a ,b]` evaluates to a vector, and the same shape works as a pattern that destructures one — which is only possible because the reader's eager expansion leaves a recognizable `cons/append/vconcat` shape behind.

The prelude is macros all the way down

The clearest demonstration of the macro system's power is the prelude itself: a large body of elisp, loaded on top of the primitives, that defines the familiar standard library almost entirely as macros and functions over the twenty-two special forms. The control constructs everyone uses are prelude macros. `dolist`, which iterates a list, expands to a `while` over the list's tail:

```
(defmacro dolist (spec &rest body)
  (let ((var (car spec)) (lst (cadr spec)) (result (car (cddr spec))))
    `(let ((,var nil) (--dolist-tail-- ,lst))
      (while --dolist-tail--
        (setq ,var (car --dolist-tail--))
        ,@body
        (setq --dolist-tail-- (cdr --dolist-tail--)))
      (setq ,var nil)
      ,result)))
```

That is the whole of `dolist`: a `let` introducing the loop variable and a hidden tail pointer, a `while` walking the tail, the body spliced in, and the result form returned. Every loop you write with `dolist` is, after expansion, a `while` — the one primitive loop from the last chapter. `push` is even smaller, and shows a macro making a compile-time decision about its expansion based on the shape of its argument:

```
(defmacro push (x place)
  (if (symbolp place)
      (list (quote setq) place (list (quote cons) x place))
      (list (quote setf) place (list (quote cons) x place))))
```

If the place is a plain symbol, `push` expands to a `setq` of `(cons x place)`; otherwise it expands to a `setf` so that generalized places (a `car`, an `nth`, a `gethash`) work too. This is a macro inspecting its arguments' structure and choosing its output accordingly — the essence of what macros are for. The remarkable thing about the prelude is its depth: `cl-loop` is a several-hundred-line macro that compiles loop keywords into a `let*/catch/while` driver; `cl-defstruct` generates constructors, predicates, and self-able accessors; `pcase` compiles structural patterns into a `cond`. All of it is elisp, expanded by the macro machinery this chapter described, into the special forms the last chapter described. Chapter 13 gives the prelude the full treatment it deserves. For now, the lesson is the architectural one: elisp's language surface is large, but its *core* is small — twenty-two special forms and a couple hundred primitive subrs — and the distance between the two is covered entirely by macros.

An expansion, watched all the way down

It helps to watch a single form travel through all three grades of expansion, because the grades are easy to confuse in the abstract. Take `my-when`, a one-line macro from the prelude pattern, and the assertion `meta-eval.el` makes about it:

```
(should (equal (macroexpand '(my-when c x)) '(if c (progn x))))
```

`macroexpand` runs the outermost macro to a fixpoint, so `(my-when c x)` becomes `(if c (progn x))` — the `my-when` macro received the *forms* `c` and `x` unevaluated, and returned an `if` whose then-branch is a `progn` of the body. Notice three things. First, the arguments were not evaluated: `c` is the literal symbol `c`, not the value of a variable `c`, because a macro takes forms, not values. Second, the result is itself a form, `(if c (progn x))`, which still has to be compiled — `macroexpand` produced code, not an answer. Third, `macroexpand` stopped after `my-when` resolved, because `if` is a special form, not a macro, so there is nothing left to expand at the head.

Now contrast the grades. `macroexpand-1` would have done exactly one step (here, the same result, because one step already reaches a non-macro head). `macroexpand` iterates the head to a fixpoint but does *not* descend into the subforms — it would leave a macro call nested inside the `progn` body unexpanded. `macroexpand-all`, which the compiler driver uses before lowering, descends recursively into every subform, expanding macros wherever they appear, with one deliberate stop: it does not descend into `quote`, because quoted data is data, not code, and expanding inside `'(when x)` would corrupt a literal three-element list into an `if`. That single stopping rule — expand everywhere except inside `quote` — is the whole difference between a recursive expander that works and one that silently rewrites your literals, and it is exactly what Emacs does. By the time the compiler of the next chapter sees a form, `macroexpand-all` has already run, so every macro is gone and only special forms and calls remain.

With the reader, the values, the symbols, the binding, the special forms, and the macros all in hand, we have a complete picture of the language as data and semantics. Now we descend into the machine: how a form becomes bytecode.

Chapter 9 – The Compiler: Lowering elisp to fusevm::Chunk

This is the chapter where the project’s central claim becomes mechanical. `elisprs` “compiles, it does not interpret,” as the engineering report puts it, and the compiler is `src/compiler.rs`: a recursive walk over a form that emits a `fusevm::Chunk`. The file’s own header is the most concise statement of the design in the codebase: “`elisp` does not run on a bespoke interpreter; it compiles to `fusevm::Chunk` and executes on `fusevm`. Heap objects ride as `Value::Obj` handles; the `ElispHost` (via `fusevm`’s extension handler) supplies the semantics and the (dynamic) binding environment.” Reading the compiler is reading the exact translation from `elisp` meaning to `fusevm` bytecode.

Chunks and the builder

A `fusevm::Chunk` is the unit of compiled code: a vector of opcodes, a constant pool, and a names table. The compiler builds one with a `ChunkBuilder`, emitting opcodes one at a time and adding constants as needed. The top-level entry, `compile_top`, makes a fresh builder, compiles one form into it, and returns the built chunk. The whole compiler is a set of functions that take the host, the builder, and a form, and push opcodes onto the builder. There is no separate optimization pass and no intermediate representation; the walk emits final bytecode directly.

The opcodes fall into two groups. There are *native* `fusevm` opcodes the VM executes itself — `LoadInt`, `LoadFloat`, `LoadConst`, `LoadUndef`, `LoadTrue`, `Pop`, `Dup`, `Add`, `Sub`, `Mul`, `Negate`, `NumLt`, `NumGt`, `NumLe`, `NumGe`, `NumEq`, `Jump`, `JumpIfFalse`, `JumpIfTrue` — and there are *extension* opcodes, `Op::Extended(id, arg)` and `Op::ExtendedWide(id, n)`, that the VM does not understand and hands back to `elisprs`. The extension opcodes are how every `elisp`-specific operation — a function call, a variable reference, a binding — gets expressed in a chunk the VM can carry without understanding. They are the subject of the next chapter; here we will see the compiler emit them.

The constant pool and baked-in handles

A chunk is three things — a vector of opcodes, a constant pool, and a names table — and the constant pool is where the object model and the compiler meet. When the compiler needs to emit a quoted literal, a symbol to look up, or a closure template, it adds the value to the constant pool and emits a `LoadConst` of its index. For an immediate like a string that is unremarkable, but for a compound value — a quoted list, an interned symbol — the constant is a `Value::Obj(u32)` handle into the host arena, and *that* is the subtle part. The handle is only meaningful against the arena that issued it, so a chunk’s constant pool is, in effect, a set of references into a specific heap. This is exactly why the bytecode cache of Chapter 11 and the AOT path of Chapter 18 have to serialize a *heap image* alongside the chunks: the baked-in handles in the constant pool must resolve to the right objects when the chunk runs against a rebuilt arena. The compiler does not see that complexity — it just adds a handle to the pool and moves on — but the handle-in-the-constant-pool is the seam the whole serialization apparatus exists to manage.

Two things the compiler deliberately does *not* do are worth naming, because their absence is a design choice. There is no separate optimization pass — the walk emits final bytecode directly, with no intermediate representation between the form and the chunk — and there is no constant folding beyond the native arithmetic

fast path. The reason the compiler can be this direct and still produce fast code is that the speed lives downstream, in fusevm's JIT: the compiler's job is a faithful, quick lowering, and the tiering JIT is what turns the hot stretches of that lowering into native code. A compiler that tried to optimize would be duplicating work the rented engine already does, which is precisely the kind of redundancy the frontend posture exists to avoid.

Compiling an atom

The walk dispatches on the form's type. An integer compiles to `LoadInt`, a float to `LoadFloat`, a string to a `LoadConst` of the string, `nil` (and `Bool(false)`) to `LoadUndef`, and `t` to `LoadTrue`. These are the immediates, and they compile to the VM's own load opcodes with no elisp involvement — a chunk full of arithmetic on integer literals is, after compilation, a chunk the VM and its JIT can run with zero callbacks into elisprs.

A symbol is the first interesting case, because a symbol means different things in different positions, and the compiler resolves the ambiguity here. A symbol whose name starts with a colon — a keyword — compiles to a `LoadConst` of itself: keywords self-evaluate. Any other symbol compiles to a `LoadConst` of the symbol followed by an `Extended(GETVAR)` op, which means “load this symbol, then replace it with its variable value.” That two-op sequence is a variable reference, and it routes at run time through the binding machinery — lexical scope first, then the value cell — that Chapter 6 described.

Compiling a call

A list form is a call, and `compile_call` is the heart of the compiler. It pulls the head symbol's name and dispatches. If the name is one of the twenty-two special forms, it routes to that form's dedicated lowering function — `compile_if`, `compile_let`, `compile_defun`, and so on. If the name is one of the three unsupported buffer-scoping forms, `save-excursion`, `save-current-buffer`, or `save-restriction`, it errors at compile time with a clear message: “special form `save-excursion` not yet lowered (buffer milestone).” This is the project's “loud failure” discipline made concrete — the engineering report singles out exactly these three forms as “the only unlowered forms,” and rather than miscompile them, the compiler refuses. Everything else is a general call.

The general call lowering is short and worth seeing in its essence. The operator is loaded — as a constant if it is a symbol (so the call machinery can resolve its function cell), or compiled to a closure if it is a literal lambda in head position. Then each argument is compiled in order, leaving its value on the stack. Then a single extension op, `Extended(CALL, argc)`, is emitted, carrying the argument count. At run time that op pops the arguments and the operator and performs the call. So a call `(f a b)` compiles to: load `f`, compile `a`, compile `b`, `CALL 2`. That is the universal calling convention, and it is the same whether `f` is a primitive subr or a user closure — the resolution of which happens at run time, in the host, when the `CALL` op fires.

There is an arity ceiling: the argument count rides in a single byte of the `CALL` op, so a call with more than 255 arguments is rejected at compile time. In practice nothing real approaches that, and `apply` exists for genuinely variadic spreading.

The native arithmetic fast path

Here is the cleverest part of the compiler, and the place the frontend architecture pays off most directly. A naive lowering would compile `(+ a b)` as a `CALL` of the `+` subr — load `+`, compile `a`, compile `b`, `CALL 2` — which would, on every single addition, pop back into the host to run the Rust `add` function. For a hot numeric loop that is a callback per iteration, and it defeats the JIT, because the JIT can only accelerate native VM opcodes, not host callbacks. So the compiler has a fast path: when the operator is a core arithmetic or comparison primitive *that has not been redefined*, it lowers the operation to native fusevm opcodes instead of a `CALL`. The compiler's comment states the motive exactly: “lower core arithmetic/comparison on un-redefined primitives to native fusevm ops, so hot loops are JIT/AOT-able instead of dispatching to the host on every operation.”

The fast path handles `+`, `-`, `*` as folds over the VM's `Add`, `Sub`, `Mul` opcodes, with the right identities for the zero-argument cases (`(+)` is 0, `(*)` is 1) and unary negation for `(- x)`. It handles `1+` and `1-` as an `Add/Sub` of a constant 1 — and there is a beautiful detail here: it uses `Add` with 1 rather than a dedicated increment opcode, because `Add` is float-contagious the way elisp `+` is, so `(1+ 1.0)` correctly yields `2.0` rather than truncating, whereas an integer increment opcode would mishandle the float. It handles the two-argument comparisons `<`, `>`, `<=`, `>=`, `=` as the VM's `NumLt`, `NumGt`, `NumLe`, `NumGe`, `NumEq` opcodes.

Two restrictions keep the fast path correct. First, it only fires when the primitive has not been redefined — the host's `is_primitive_fn` check confirms the symbol's function cell still holds the original subr — because if a program has redefined `+`, the native op would silently ignore the redefinition. Second, `/`, `%`, and `mod` stay on the host even though they look arithmetic, because elisp's integer division and remainder semantics, and its wrong-type signaling, differ from the VM's native division in ways the fast path would get wrong. The compiler's comment is explicit that native arithmetic “skips the host's wrong-type signaling — an accepted fast-path trade-off for numbers.” This is a real and deliberate semantic concession: native `+` on a non-number will not produce elisp's `wrong-type-argument` error the way the subr would. The trade is judged worth it because the alternative — a host callback per arithmetic op — would make numeric elisp unbearably slow and would wall off the JIT entirely. The `arithmetic.el` example is the regression gate that keeps the fast path honest: it asserts the full suite of integer and float results, and it runs on every commit.

Conditionals and the TRUTHY op

We have referred several times to the `TRUTHY` op; here is where it is emitted. The `compile_if` function compiles the condition, emits `Extended(TRUTHY)` to convert the elisp value to a VM boolean under the “only nil is false” rule, then emits a `JumpIfFalse` whose target it patches once it knows where the else-branch begins. It compiles the then-branch, emits a `Jump` past the else, patches the false-jump to land on the else, compiles the else as an implicit `progn`, and patches the final jump. The shape is the standard compile-time conditional: forward jumps with back-patched targets. `when`, `unless`, `cond`, `and`, `or`, and `while` all share the same `TRUTHY`-then-jump idiom, with variations — `unless` flips to `JumpIfTrue`, and/or `Dup` the value before testing so the deciding value survives, `while` jumps backward to form the loop. The compiler's header note that “elisp truthiness ≠ fusevm truthiness, so conditionals emit `Extended(TRUTHY)` before `JumpIfFalse` (the strykelang pattern)” tells you this is a shared technique across the fusevm frontends — every language with its own truthiness rule inserts a conversion op before the VM's branch.

Lowering let and let*

The two binding forms lower to the extension binding ops, and the difference between them is visible in the opcodes. `let` evaluates every initializer first — compiling each onto the stack, paired with its symbol — and then emits a single `ExtendedWide(LETBIND, n)` that opens a scope and installs all `n` bindings at once. The “wide” op carries the binding count `n` as a full integer rather than a byte. `let*` instead emits a `SCOPE_OPEN` to open one scope, then for each binding compiles the initializer and emits a `SPECBIND` to bind it immediately, so each initializer sees the previous bindings. Both then compile the body as a `progn` and emit `ExtendedWide(UNBIND, n)` to close the scope on exit. The lowering is a direct transcription of the semantics from Chapter 6: bind-all-at-once versus bind-one-at-a-time, with the dynamic-versus-lexical routing deferred to the host ops.

Lowering lambda and defun

A `lambda` compiles its body to a *sub-chunk* — a separate `fusevm:Chunk` — and stores that chunk in a closure template object along with the parsed parameters. The compiler loads the template as a constant and emits an `Extended(MAKE_CLOSURE)` op, which at run time captures the current lexical environment into a fresh closure. So a closure is template-plus-captured-environment: the template (params and body chunk) is

baked into the constant pool at compile time, and the environment is snapshotted at run time when `MAKE_CLOSURE` fires. This is what makes a closure close over the variables in scope where it was created.

`defun` is the same closure construction followed by an `Extended(FSET)` op that installs the closure in the named symbol's function cell. The compiler emits `MAKE_CLOSURE` for a `defun` (so a function defined inside a `let` captures that `let`), but skips it for a `defmacro` (a macro captures no environment). The sub-chunk-per-lambda structure is the recursive heart of the compiler: compiling a function compiles its body into a nested chunk, and calling that function later runs the nested chunk on a nested VM, as the next chapter describes.

Lowering the non-local exits

The catch, unwind-protect, and condition-case lowerings are the ones that rewrite a special form into an ordinary call, as Chapter 7 previewed. `compile_catch` builds a zero-argument lambda wrapping the body, then constructs the form `(--catch-- tag thunk)` and compiles *that* as a normal call. `compile_unwind` wraps the body and the cleanup in separate thunks and compiles `(--unwind-- body-thunk cleanup-thunk)`. `compile_condition_case` wraps the body and each handler in thunks, quotes the condition specifications, and compiles `(--condition-case-- 'var body-thunk (list ...))`. The thunks are what defer evaluation — a thunk is a closure that runs its body only when called — so the intrinsic in the host can decide *when* and *whether* to run the protected body, the cleanup, and the handlers. It is a clean way to express non-local control without any new opcodes: the control logic lives in the host intrinsic, and the compiler's only job is to package the subexpressions as callable thunks and emit a call.

With the compiler understood, the chunk it produces needs somewhere to run. That is `fusevm`, and the protocol between them.

Chapter 10 – The Shared Machine: fusevm and the Extension Protocol

Everything so far has pointed at a single object — the `fusevm::Chunk` — and a single claim — that `elisprs` runs it on `fusevm` rather than on a VM of its own. This chapter is about the other side of that handoff: what `fusevm` is, how a chunk runs on it, and exactly how an alien object heap of cons cells and symbols rides through a VM that knows nothing about Lisp. The connecting tissue is `fusevm`'s *extension protocol*, and it is the single most important interface in the system, because it is the hinge on which the entire frontend architecture turns.

What fusevm provides

`fusevm` is a language-agnostic bytecode virtual machine, written in Rust, with a three-tier Cranelift JIT and an ahead-of-time native compiler. It is shared across five languages — `zshrs`, `stryke`, `awkr`, `vimlr`, and `elisprs` — and its design goal is to be exactly the part of a language implementation that is *not* language-specific. It executes a stack machine: opcodes push and pop a value stack, control flows by jumps, and a constant pool and a names table travel with each chunk. It understands integers, floats, strings, booleans, and an opaque object handle, and it can do arithmetic, comparison, and branching on the values it understands. What it does not understand is anything language-specific — it has no notion of a cons cell, a symbol, a variable binding, or an error condition. That is the whole point. By staying ignorant of those things, `fusevm` can be the same VM for a shell, a Perl dialect, AWK, Vimscript, and Emacs Lisp, none of which agree on what a value or a binding is.

`elisprs` depends on `fusevm` with two features turned on, and the Cargo manifest is precise about what they buy. `jit-disk-cache` turns on “`fusevm`'s three-tier Cranelift JIT *and* its persistent on-disk cache of compiled native code (keyed by chunk hash), so hot elisp loops trace-compile and warm runs skip recompilation.” `aot` turns on the native-object emitter used by `--aot` and `--aot-exe`. The version pinned is 0.14.5. So `elisprs` gets, for free, a tiering JIT and a code cache and an AOT backend that it did not write — the dividend of being a frontend.

Running a chunk

The function that runs a chunk is `run_chunk`. It takes a `fusevm::Chunk`, clears any recorded host error, builds a `fusevm::VM` around the chunk, installs the two extension handlers (more on those in a moment), enables the tracing JIT, runs the VM, and maps the VM's result back to an elisp value or error. The “enables the tracing JIT” step is where the three-tier acceleration comes in: hot loops in the chunk trace-compile through Cranelift to native code, and with the disk cache on, that native code persists across runs so a warm invocation skips the recompile. A tight numeric loop in elisp — the kind the compiler's native arithmetic fast path produces — therefore becomes, after a few iterations, native machine code, with no per-operation callback into `elisprs` at all. That is the payoff chain in full: the compiler lowers `(+ a b)` to a native `Add`, `fusevm`'s JIT compiles the loop containing it to machine code, and the loop runs at native speed. The frontend wrote none of the JIT and gets all of the speed.

When `run_chunk` finishes, it maps the VM's outcome: a normal completion returns the value, a halted VM returns the top of its stack, and a VM error becomes an elisp error. A host-recorded error — set by a `subr` that signalled — takes precedence, so that an error raised inside a callback surfaces as the chunk's result even though the VM itself did not fault.

The extension protocol

Now the hinge. The compiler emits two kinds of opcode the VM does not understand: `Op::Extended(id, arg)`, carrying a 16-bit operation id and an 8-bit argument, and `Op::ExtendedWide(id, n)`, carrying an id and a full-width integer. When the VM executes one of these, it does not try to interpret it. It calls a registered *extension handler*, passing the id and the argument and a mutable reference to itself. `elisprs` registers two such handlers on every VM in `run_chunk` — `set_extension_handler` for the narrow ops and `set_extension_wide_handler` for the wide ones — and those handlers are where every elisp-specific operation actually happens.

The handlers are plain functions, and the operation ids are a small enumerated set defined in a `ops` module. They are the complete vocabulary of elisp-specific instructions, and the list is short enough to give in full, each with the one-line contract from the source:

- `TRUTHY` (0): pop a value, push its elisp truthiness as a boolean.
- `CALL` (1): the argument is the argument count; the stack holds the operator and the arguments; perform the call and push the result.
- `GETVAR` (2): pop a symbol, push its variable value.
- `SETVAR` (3): pop a value and a symbol, set the value cell, push the value.
- `FSET` (4): pop a definition and a symbol, set the function cell, push the symbol.
- `SPECBIND` (5): pop a symbol and a value, bind it in the current scope (the `let*` per-binding op).
- `LETBIND` (6): wide, with count `n`: open a scope, pop `n` value/symbol pairs, bind them all in parallel.
- `UNBIND` (7): wide: close the innermost scope, keeping the stack value.
- `SCOPE_OPEN` (8): open an empty lexical scope (for `let*`).
- `MAKE_CLOSURE` (9): pop a closure template, push one capturing the current environment.

Ten operations. That is the entire elisp-specific instruction set — everything beyond the generic stack machine that `elisprs` needs the VM to do, expressed as ten callbacks. Every variable reference is a `GETVAR`. Every function call is a `CALL`. Every binding is a `LETBIND`, `SPECBIND`, or `UNBIND`. Every conditional is preceded by a `TRUTHY`. Every closure is a `MAKE_CLOSURE`. The VM carries these opcodes in the instruction stream like any other, and when it reaches one it hands control to `elisprs`, which does the work against the object heap and pushes the result back. The VM never learns what a cons cell is; it only learns to call back when it sees an opcode it does not recognize.

Narrow and wide, and two handlers

The extension ops come in two widths, and the reason is the size of the argument they carry. The narrow form, `Op::Extended(id, arg)`, carries a 16-bit operation id and an 8-bit argument, which is enough for `CALL`'s argument count (capped at 255, the arity ceiling of Chapter 9) and for the ops that take no argument at all, like `TRUTHY` and `GETVAR`. The wide form, `Op::ExtendedWide(id, n)`, carries the id and a full-width integer, and it exists for the binding ops whose count can exceed a byte: `LETBIND` opens a scope and binds `n` variables in parallel, and `UNBIND` closes a scope, where `n` is the number of bindings — a `let` with three hundred bindings is absurd but legal, and the wide op accommodates it where the byte-sized narrow op could not. So `elisprs` registers *two* handlers on every VM in `run_chunk` — `set_extension_handler` for the narrow ops and `set_extension_wide_handler` for the wide ones — and the compiler picks the width per op: `LETBIND` and `UNBIND` are wide, everything else is narrow. It is a small detail, but it is the kind of thing that has to be exactly right, because the VM dispatches the two widths to two different callbacks, and a binding op emitted at the wrong width would be handed to the wrong handler and misread.

CALL, the re-entrant heart

The most important handler arm is `CALL`, because it is where elisp function application happens and where the borrow discipline from Chapter 4 becomes critical. When the handler sees `CALL` with an argument count, it

pops that many arguments off the VM stack, pops the operator, and dispatches to `call_function` — the single re-entrant entry point for invoking any elisp callable. `call_function` is careful in a way that the whole architecture depends on. Before it resolves the operator, it intercepts the higher-order and re-entrant intrinsics *by name* — `funcall`, `apply`, `mapcar`, `mapc`, `sort`, `maphash`, `eval`, the `macroexpand` family, `replace-regexp-in-string` with a function replacement, and the non-local-exit intrinsics `--catch--`, `--unwind--`, `--condition-case--` — and runs them *outside* any host borrow. The reason is the thread-local host: a subr runs via `with_host`, which borrows the host mutably, and if a higher-order subr like `mapcar` held that borrow while calling back into elisp to apply its function argument, the callee's own `with_host` would panic on the already-borrowed cell. By intercepting the re-entrant intrinsics before the borrow and letting them manage their own calls through `call_function`, the system keeps the borrow short and non-nested. An ordinary subr, which does not call back into elisp, runs inside a single short `with_host` borrow.

This is the subtle cost of separating the object heap from the VM: re-entrancy has to be threaded carefully through a single choke point. But it is a contained cost — one function, `call_function`, holds the discipline — and in exchange the VM stays utterly ignorant of elisp, which is what lets it be shared.

Closures and nested VMs

When `call_function` resolves the operator to a user closure rather than a subr, it runs the closure's body, and the body is a `fusevm::Chunk` of its own. Running it means running a *nested* VM: the closure's chunk gets its own VM, with its own value stack, its own extension handlers, executing to completion and returning a value into the caller. The closure machinery opens a lexical scope as a child of the closure's captured environment, binds the parameters into it, runs the body chunk, and on the way out unwinds any scopes the body leaked through a non-local exit. So a call into a user function is a recursive descent into another chunk on another VM — which is exactly why the `functions.el` example's comment can say “calling it runs that chunk on a nested fusevm VM.” Recursion in elisp is recursion in nested VMs: `(fact 5)` runs `fact`'s chunk, which calls `(fact 4)`, which runs the chunk again on yet another nested VM, all the way down. The fusevm engine is built to nest this way, and elisprs leans on that nesting for the entire user-function call graph.

The Obj handle, in both directions

The extension ops are one half of the bridge; the other half is the value that crosses it. fusevm understands integers, floats, strings, and booleans natively, and it reserves exactly one opaque variant — `Obj`, an object handle — for everything a hosted language needs to carry that the VM must not interpret. elisprs uses that one reservation for the entire heap: a cons cell is an `Obj`, a symbol is an `Obj`, a vector, a hash table, a closure, a record, a marker — all of them ride the VM stack as the same opaque handle, and the VM never looks inside. When a chunk pushes a fixnum literal, it pushes a native fusevm integer; when it pushes the result of `(cons 1 2)`, it pushes an `Obj` that indexes into the `ElispHost` heap. The two kinds of value sit side by side on the same stack, and the compiler knows which is which because it knows the shape of the code it emitted.

The consequence is a marshalling boundary that has to be crossed twice per subr call, and crossed cheaply. When a value flows *into* a subr, elisprs reads it off the VM stack and decides whether it is a native scalar to be wrapped as an elisp integer or float, or an `Obj` handle to be resolved against the heap into a cons or a symbol or a string. When a value flows *out*, the reverse: an elisp integer that fits fusevm's native integer becomes one, a heap object becomes an `Obj` handle. This is why the numeric fast path of Chapter 9 matters so much. If `(+ a b)` had to box its operands as heap objects, every arithmetic step would cross the marshalling boundary and allocate; because small integers ride the VM as *native* integers, a tight numeric loop never touches the heap at all, and the JIT can compile it to machine registers. The `Obj` handle is the pressure-relief valve that lets the generic VM stay generic — but the art of the frontend is in keeping as much traffic as possible in the native lane, so the valve is used only for the values that genuinely need the heap.

There is one asymmetry worth naming. Booleans cross the boundary specially, because elisp's notion of truth is not fusevm's. fusevm has a real boolean type and branches on it; elisp has no boolean type at all — `nil` is the empty list is false, and everything else, including the symbol `t`, is true. So the `TRUTHY` op exists precisely

to convert an elisp `Obj` (or scalar) into the native boolean the VM's `JumpIfFalse` wants, and it is emitted before every conditional. It is the smallest extension op and the most frequently executed, and it is the seam where two different theories of truth are stitched together. Chapter 63 traces it in full.

Why this protocol is the whole design

Step back and look at what the extension protocol accomplishes. It lets a language-agnostic VM run a language it knows nothing about, by reserving a single opcode family — `Extended` — as a callback into the language's own runtime, and a single value variant — `Obj` — as an opaque handle into the language's own heap. With those two reservations, fusevm can carry elisp's cons cells and symbols on its stack, run elisp's compiled bytecode in its dispatch loop, JIT elisp's hot loops to native code, and never once need to understand a single elisp concept. The ten extension ops are the complete contract: implement these ten callbacks and a value handle, and you have an elisp runtime on a borrowed engine. The engineering report's summary line — “By lowering to `fusevm::Chunk`, elisprs reuses the same VM, JIT, AOT, and on-disk cache as the sibling languages instead of maintaining a one-off interpreter” — is, at the level of mechanism, this protocol. It is the answer to the question the whole book opened with: how do you run a language without building it a machine? You define ten callbacks and rent the machine.

Chapter 11 — The Bytecode Cache

There is one more piece of the machine, and it is the one that makes the second run of a script dramatically cheaper than the first. `elisprs` caches compiled bytecode on disk in an `rkyv` shard at `~/.elisprs/scripts.rkyv`, and on a cache hit it skips the entire front of the pipeline — the reader, the macro expander, the compiler, and even the prelude rebuild — and runs the cached chunks directly. The cache is `src/cache.rs`, and it mirrors a pattern the sibling languages use: the file's header says it “mirrors `zshrs script_cache`.” Understanding it closes the loop on the frontend story, because the cache is only possible *because* `elisprs` compiles to a serializable chunk rather than interpreting a tree.

What gets cached, and why it is two things

A cache entry holds two things: the per-form compiled chunks, and a clean heap image. The chunks are the obvious half — the `fusevm::Chunk` for each top-level form, ready to run. The heap image is the subtle half, and it exists because of the handle model from Chapter 4. A compiled chunk's constant pool contains `Value::Obj` handles — baked-in references to symbols, quoted lists, and closure templates that live in the host arena. Those handles are only meaningful against the arena that issued them. To run a cached chunk, `elisprs` must first rebuild an arena in which those exact handles resolve to the right objects. The heap image is that arena, serialized: a snapshot of every user-and-prelude object, in order, so that on reload the objects re-allocate to the same handles the chunks expect. Run the chunks against a freshly rebuilt heap image and every baked-in handle lands on its intended object.

The cache file's structure reflects a serialization subtlety the header calls out. The outer container is a zero-copy `rkyv` archive, validated on read with `rkyv's check_archived_root`. But the inner per-form chunks and the heap image are *bincode*, not `rkyv`, “because `fusevm::Chunk/Value` are *serde*-owned, not `rkyv::Archive`.” So the cache is `rkyv` on the outside for cheap validated loading and *bincode* on the inside for the `fusevm` types that only speak *serde*. The header notes this is “the same split `zshrs` uses” — another place the frontend family shares a technique.

The key: path, mtime, and a schema fingerprint

A cache entry is keyed by three things: the absolute file path, the file's modification time in nanoseconds, and a *schema key*. The first two are obvious — a different file, or the same file edited, should miss. The schema key is the clever part, and it solves a problem that would otherwise make on-disk bytecode caching unsafe. A compiled chunk bakes in builtin arena handles and macro expansions. If the set of builtins changes — a sub added, removed, or reordered — or if the prelude changes, then a chunk compiled under the old layout would resolve its baked-in handles to the *wrong* builtins. So the cache must invalidate whenever the builtin layout or the prelude changes, even within a single released version number.

The schema key folds exactly those things into the key. It combines the `elisprs` version with a hash of the builtin object layout — a `builtin_fingerprint` that hashes the count and ordered names of every builtin symbol — and a hash of the prelude source. The cache's comment is precise about why: “any change to the registered subrs or the prelude must invalidate cached bytecode even within a single released version — otherwise stale chunks resolve handles to the wrong builtins. Folding the fingerprint into the key makes that automatic.” So editing `builtins::install` or a single prelude definition changes the fingerprint, changes the schema key, and silently invalidates every stale entry. A developer never has to remember to bump a cache version; the cache notices the layout changed and rebuilds. This is the kind of correctness-by-construction

that a handle-based object model *forces* you to get right, and the schema fingerprint is the mechanism that gets it right.

The fingerprint, and a debug seam

The fingerprint itself is cheap to compute and exact in what it covers. `builtin_fingerprint` folds the count and the ordered names of every builtin symbol into a 64-bit hash, so any change to the registered subrs — one added, one removed, one renamed, the order shuffled — moves the hash. The schema key combines that hash with a hash of the prelude source and the `elisprs` version, and the three together are what a cache entry is keyed on alongside the file path and its modification time. The shape of the check is therefore: same file, same mtime, same builtins, same prelude, same version → hit; anything else → miss and rebuild. A shard found under a *different* schema key is discarded wholesale, because its chunks bake in handles against a builtin layout that no longer exists, and resolving them would land on the wrong objects.

There is a small operational seam for watching the cache work: setting the environment variable `ELISPRS_CACHE_DEBUG` makes `eval_file` print, for each run, whether it was a hit or a miss and how many chunks it loaded.

```
ELISPRS_CACHE_DEBUG=1 elisp examples/stdlib.el
```

The first run reports a miss and writes the shard; the second reports a hit and the chunk count, skipping the read, the macro expansion, the compile, and the whole prelude rebuild. That a cache meant to be invisible ships with a switch to make it visible is the small tell that it is built to be understood and debugged rather than merely trusted — the same honesty the rest of the project shows about its own machinery.

The hit path and the clean heap image

On a file run, `eval_file` computes the file's mtime and the schema key, then asks the cache. On a hit, it resets the host, imports the cached heap image so the handles line up, and runs the cached chunks in order — no reading, no expanding, no compiling, no prelude load. The header states the saving plainly: “On a fresh hit, the per-form chunks + a clean heap image are loaded and run directly — skipping read / macro-expand / lower AND the prelude rebuild.” For a script that pulls in `cl-lib`, `seq`, and `subr-x` — which means loading a very large prelude — skipping the prelude rebuild is a substantial saving on its own, before the compile saving.

The phrase “clean heap image” carries weight. The image is exported by `export_heap_image_clean`, and “clean” means the symbol value cells are reset to a baseline captured right after the prelude loaded but before the user file ran. The reason is determinism: re-running cached chunks must reproduce the original execution exactly. If the cached image carried the value cells as they were at the *end* of the first run — with all the user program's global mutations already applied — then re-running the chunks would apply those mutations a second time, on top of an already-mutated state, and the second run would diverge from the first. By resetting the value cells to the clean post-prelude baseline, the image gives the cached chunks the same starting state the original run had, so they execute identically. Symbols the user program defined are reset to unbound; prelude symbols are reset to their post-prelude values. It is a careful piece of bookkeeping, and it is the difference between a cache that speeds things up and a cache that quietly corrupts the second run.

The miss path and writing back

On a miss, `eval_file` does the full work: reset the host, load the prelude, snapshot the clean baseline, read the file, splice top-level prologs, and then compile and run each form one at a time — capturing each chunk as it goes. After the run it exports the clean heap image and writes a new cache entry with the chunks and the image, keyed by path, mtime, and schema key. The write is best-effort: any failure to serialize or write simply skips caching, because the cache is an optimization, never a correctness requirement — a run with no writable `~/.elisprs` directory just recompiles every time, correctly, only slower. The write is also atomic in

the way good on-disk state always is: it writes to a temporary file and renames it into place, so a reader never sees a half-written shard. A shard found under a different schema key is discarded wholesale, since its chunks reference a builtin layout that no longer exists.

There is an environment-variable hook for visibility: setting `ELISPRS_CACHE_DEBUG` makes `eval_file` print whether each run was a cache hit or miss and how many chunks it loaded. It is a small thing, but it is the kind of operational seam that tells you the cache is meant to be understood and debugged, not just trusted.

Why the cache validates the architecture

The bytecode cache is the cleanest demonstration of why the frontend design is worth its costs. You cannot cache a tree-walker's work, because a tree-walker does no work ahead of time — it re-interprets the source on every run, and there is nothing durable to save. A compiler produces an artifact, the chunk, and an artifact can be written to disk and reloaded. `elisprs` compiles to a `fusevm::Chunk`, the chunk is serializable, and so the expensive front of the pipeline — read, expand, compile, and the large prelude load — can be done once and skipped forever after, as long as the file and the schema are unchanged. The handle-based object model, which seemed in Chapter 4 like an implementation detail of riding the VM, turns out to be exactly what makes the cache both possible and tricky: possible because objects can be serialized as a heap image, tricky because handles must line up across the save and restore, which is why the schema fingerprint and the clean-baseline machinery exist. The whole apparatus is a consequence of compiling rather than interpreting, and it is a consequence the sibling languages share, which is why the cache code can mirror `zshrs`'s. Rent the engine, compile to its chunk, and the cache comes nearly for free.

With the machine fully described — compiler, VM, extension protocol, and cache — the rest of the book turns to the surface a programmer actually uses: the library.

Chapter 12 – The Subr Library

Below the special forms and below the macros there is a layer of *primitives* — the functions implemented in Rust, called subrs, that everything else is built on. `car` is a subr. `+` is a subr. `format`, `string-match`, `make-hash-table`, `sha256` — all subrs. They are the floor of the language, the operations that cannot be written in elisp because they touch the host directly: arena manipulation, arithmetic, string bytes, the filesystem, subprocesses, the clock. This chapter is about that layer — how a subr is registered, how it is called, and what the library actually covers. The subrs live in `src/builtins.rs`, the largest source file in the project at over four thousand lines, and the breadth of it is most of why elisprs can run real `.el` scripts rather than toy ones.

How a subr is defined and registered

A subr is, mechanically, a plain Rust function with a fixed signature: it takes a mutable reference to the host and a slice of argument values, and it returns either a value or an error string. There is no macro magic, no attribute, no code generation — every primitive is an ordinary function of that shape, and the library aliases the return type to keep the signatures short. `car` reads its argument and returns the car of a cons (or nil for nil, or a `wrong-type-argument` error otherwise); `cons` allocates a fresh cell in the arena and returns its handle; `add` folds its arguments into an integer-or-float sum. Here is the shape, transcribed:

```
;; car, in Rust, returns the first field of a Cons handle:
;; match h.obj(&a[0]) {
;;   Some(Obj::Cons(x, _)) => Ok(x.clone()),
;;   _ if is_nil(&a[0])    => Ok(Value::Undef),
;;   -                    => Err("wrong-type-argument: listp ..."),
;; }
;; }
```

Registration happens in one place, a function called `install`, run once when a host is created. It builds a little helper and calls it once per subr with the name, the minimum arity, the maximum arity (or “no maximum” for a variadic), and the function pointer. So `install` is a long, flat list of registrations — `+` with minimum arity zero and no maximum, `car` with exactly one argument, `format` with at least one and no maximum, and so on for roughly two hundred names. Each registration interns the name and points its symbol’s function cell at a fresh `Obj::Subr` carrying the arity bounds and the function pointer. After `install` runs, the host records the arena length as the “builtin count” — the boundary between the primitives and everything a user or the prelude adds later, which is the boundary the heap image and the schema fingerprint key off.

Arity is enforced centrally, not by each subr. When a `CALL` resolves to a subr, the dispatcher checks the argument count against the recorded minimum and maximum and signals `wrong-number-of-arguments` if it is out of range, before the subr’s body ever runs. So a subr body can index its first argument directly, trusting that the arity check already guaranteed it exists, and reach for optional arguments with a safe accessor. The bounds are also exposed to elisp through `func-arity` and `subr-arity`, which read them back as a `(min . max)` pair.

Three hundred-odd primitives, and the aliases

The earlier sections called the primitive layer “roughly two hundred names.” The engineering report’s stat card records the figure at the time it was generated — “234 subrs · 22 special forms” — but the primitive layer has kept growing as more of the standard library was ported directly into Rust, and `install` now registers over three hundred subrs (323 at the time of writing). It is that *live* count, not the card’s snapshot, that the bytecode

cache's `builtin_fingerprint` hashes, so the count is load-bearing rather than decorative: the cache keys off the ordered names of every registered subr, and adding or reordering even one invalidates stale chunks.

A handful of those names point at the *same* Rust function — they are aliases, registered by handing `install`'s little `s(...)` helper the same function pointer under two names. `lsh` is an alias for `ash` (both register `ash_fn`); `not` is an alias for `null`; `cl-struct-p` is an alias for `recordp`; `subr-arity` is an alias for `func-arity`; `sxhash` is an alias for `sxhash-equal`. These are not aliases in the elisp `defalias` sense — they are not function-cell indirections that resolve through the obarray — but two distinct symbols whose function cells point at the same `Obj::Subr`. The distinction matters for the alias-chain resolver of Chapter 5: a `defalias` chain costs a hop per link (up to the 64-hop limit), but these built-in aliases cost nothing, because each name's function cell already holds the subr directly. The effect is the same to a caller — `(lsh 1 4)` and `(ash 1 4)` both run `ash_fn` — but the mechanism is a shared pointer, not an indirection.

Why arity is checked centrally

The arity bounds recorded at registration are enforced in one place — the dispatcher that fires on a `CALL` to a subr — rather than by each subr body. Before a subr's Rust function runs, the dispatcher compares the argument count against the recorded minimum and maximum and signals `wrong-number-of-arguments` if it is out of range. This is why a subr body can index its first argument directly without a length check: the central check has already guaranteed the argument exists. It is also why the arity bounds are part of the registration tuple rather than baked into each function — centralizing the check keeps every subr body shorter and keeps the `wrong-number-of-arguments` signaling uniform. A `min` of 0 with no maximum is the variadic case (Emacs's `many`), a `min` and `max` that are equal is a fixed-arity function, and the gap between them is the optional-argument range. `car` is `(1, 1)`, `+` is `(0, None)`, `format` is `(1, None)`, `%` is `(2, 2)`, `substring` is `(2, 3)` — one required string and up to two optional bounds.

A map of the primitive layer

The breadth is best appreciated by category, because the categories are what tell you whether a given `.el` file will run. The reference's own chapter structure — arithmetic, predicates, lists and sequences, symbols and binding, strings and characters, I/O and format, control and functional, alists and hash tables — is a faithful index, and the primitive layer populates all of it.

The **arithmetic and numeric** primitives are the ones the compiler's fast path shadows: `+`, `-`, `*`, `/`, `%`, `mod`, `1+`, `1-`, the comparisons `=`, `/=`, `<`, `>`, `<=`, `>=`. Beyond those are the float and rounding operations — `floor`, `ceiling`, `round`, `truncate`, `float`, `abs` — the transcendentals — `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `expt` — and the bit operations — `logand`, `logior`, `logxor`, `lognot`, `ash`, `lsh`. The `stats.el` and `stdlib.el` examples lean on these for real numeric work: variance, gcd, lcm, isqrt, the lot.

The **list and cons** primitives are the spine of Lisp: `cons`, `car`, `cdr`, `setcar`, `setcdr`, `list`, `append`, `reverse`, `length`, `nth`. Note that the mutators `setcar` and `setcdr` are here, in the primitive layer, because they touch a cons cell's arena slot directly — and they are the reason the object model needed real identity-bearing cons cells in the first place. The `lists.el` example's mutation test is a direct exercise of these primitives.

The **vector and array** primitives — `vector`, `make-vector`, `aref`, `aset`, `fillarray`, `vconcat` — operate on the `Obj::Vector` backing, and the **hash-table** primitives — `make-hash-table`, `gethash`, `puthash`, `remhash`, `clrhash`, `hash-table-count`, `hash-table-keys`, `hash-table-values` — operate on the association- vector `Obj::HashTable`. The `data-model.el` example tallies word lengths into a hash table and reads them back, exercising the whole table surface.

The **predicate and type** primitives are the `eq/eq/=equal` equality trio plus the type tests — `null`, `not`, `consp`, `listp`, `atom`, `symbolp`, `stringp`, `numberp`, `integerp`, `floatp`, `vectorp`, `functionp` — and `type-of`, which names a value's type. These matter for both correctness and dispatch; `pcase`'s `pred` patterns and `cl-defmethod`'s type specializers both bottom out in these tests.

The **symbol and binding** primitives expose the Lisp-2 machinery from Chapter 5 to elisp: `symbol-name`, `intern`, `make-symbol`, `set`, `symbol-value`, `boundp`, `makunbound`, `fset`, `fboundp`, `symbol-function`, `special-variable-p`. There is also a family of `sxhash` functions for the structural hashing the hash tables use.

The **string** primitives include construction and inspection — `concat`, `substring`, `split-string`, `length`, `make-string`, `string`, `char-to-string`, `string-to-char`, `string-to-list`, `string-to-vector` — case folding — `upcase`, `downcase` — conversion — `number-to-string`, `string-to-number` — and comparison — `string-prefix-p`, `string-suffix-p`, `string-search`, `string-distance`, `compare-strings`. The richer string surface that `subr-x` programmers expect — `string-trim`, `string-join`, `string-pad`, `string-replace` — is built in elisp in the prelude on top of these primitives, which is the next chapter's subject.

format, a primitive worth its own section

`format` deserves attention because it is one of the most-used functions in any elisp program and because its implementation is a small, complete formatting engine. It parses each `%` directive into a specification — flags, width, precision, conversion — matching the C-printf-derived grammar `%[-][0][+][ ][#][width][.prec]CONV`, and then formats the corresponding argument. The conversions are the full elisp set: `s` prints a value in `princ` (raw) form, `S` in `prin1` (readable) form, `d` formats a signed integer, `o/x/X` format octal and hexadecimal as sign-and-magnitude with optional `0x` prefix under the `#` flag, `c` makes a character, and `e/f/g` format floats in exponential, fixed, and general notation. It honors width and precision and the justification and zero-fill flags, keeping a sign or radix prefix ahead of zero padding, so that `%#010x` of 255 produces `0x000000ff`. There is even an explicit argument-index syntax, `%2$s`, that selects the second argument regardless of position. The `text-utils.el` and `stats.el` examples assert a battery of these directives — `(format "%05.2f|%-d|%-x|%-c" 3.14159 7 255 ?A)` is `"03.14|+7|ff|A"` — and the engine passes them all. `message`, `error`, and `user-error` all route through the same engine for their format-string handling, differing only in where the result goes.

A separate and independent `%`-engine handles `format-time-string`, with the strftime-style directives — `%Y`, `%m`, `%d`, `%H`, `%M`, `%S`, and the rest — because time formatting is a different grammar from value formatting and sharing the parser would have been a false economy.

The directive, parsed

Underneath the conversions is a small parser that reads each `%`-directive into a specification before formatting the argument. The grammar is the C-printf one, `%[flags][width][.prec]CONV`, and the parsed spec carries each piece as a field:

```
;; FmtSpec (builtins.rs), transcribed: a parsed %-directive.
;; left: bool ; '-' left-justify
;; zero: bool ; '0' zero-fill
;; plus: bool ; '+' force a sign on signed conversions
;; space: bool ; ' ' leading space on a non-negative
;; alt: bool ; '#' alternate form (0x prefix, keep %g zeros)
;; width: usize ; minimum field width
;; prec: Option<usize> ; precision after '.'
;; conv: char ; the conversion letter
```

Two details of the engine are worth pinning. First, the conversions are exactly `s S d o x X c e f g` — there is no `%i`, `%u`, or `%a` — and an *unknown* directive is emitted verbatim and consumes no argument, so a stray `%q` prints as `%q` rather than erroring or eating an argument. Second, the positional-argument syntax `%N$` is recognized with a deliberate constraint: `N` runs from 1 to 9, which is what lets the parser tell it apart from a `0` flag — a leading `0` is the zero-fill flag, a leading `1–9` begins an argument index. So `%2$s` selects the second argument regardless of position. The flag interactions are the fiddly part the engine gets right: zero-fill applies

only to right-justified numeric conversions and keeps a sign or `0x` prefix ahead of the padding, so `%#010x` of 255 is `0x000000ff` — the `0x` first, then the zeros, then the digits — which is precisely the C-printf behavior the `text-utils.el` and `stats.el` format tests assert.

The surprising reach: crypto, time, filesystem, processes

The primitive layer goes well past the language core into the operating-system surface a standalone script needs, and this is what separates elisprs from a toy. There are cryptographic and encoding primitives — `sha1`, `md5`, `secure-hash`, `base64-encode-string` and its decode and URL-safe variants, `url-hexify-string` — and notably the SHA and MD5 implementations are self-contained Rust, no external crate. There are time primitives — `current-time`, `float-time`, `format-time-string`, `decode-time`, `encode-time` — reaching the system clock through libc. There is a read-side and write-side filesystem surface — `file-exists-p`, `file-directory-p`, `file-readable-p`, and a directory lister on the read side; `write-region`, `delete-file`, `make-directory`, `rename-file`, `copy-file` on the write side. And there are subprocess primitives — `shell-command-to-string`, `call-process`, `process-lines` — that shell out and read results back. The `script-demo.el` example uses the whole stack at once: it writes a file, reads it back, parses it, serializes the summary to JSON, reads the JSON back, and shells out to `echo` and `printf`, all in one ERT-tested program. That a `.el` file can do end-to-end scripting — file I/O, subprocesses, JSON — outside Emacs is the entire promise of the project, and it rests on this primitive layer reaching past the language into the OS.

The internal primitives

A handful of registered names are bracketed with double dashes — `--current-directory--`, `--directory-files--`, `--push-output-capture--`, `--buffer-push--`, and a few more. These are *internal* primitives, not meant to be called directly; they are the low-level hooks the prelude wraps in friendlier public functions and macros. `with-output-to-string`, for instance, is a prelude macro that brackets its body with `--push-output-capture--` and `--pop-output-capture--` to redirect `princ/prin1` output into a string. The double-dash convention marks the seam between the Rust floor and the elisp surface built on top of it, and it is the same convention Emacs uses for its own internal subrs. Knowing the convention is useful when reading the prelude, because it tells you which calls are reaching down to the host and which are staying in elisp.

Chapter 13 — The Derived Prelude

If the subr library is the floor, the prelude is the building. It is a large body of elisp — `src/prelude.rs` holds it as one enormous string constant, thousands of lines — that is loaded form by form on top of the primitives every time a host starts. It defines the familiar standard library: the `cl-lib` functions, the `seq` functions, the `subr-x` string helpers, `pcase`, `cl-loop`, `cl-defstruct`, the JSON encoder, `rx`, the `map` library, and the ERT testing surface. The prelude is where elispr stops being a language core and becomes a usable elisp, and its existence is a direct statement of the project's philosophy: build breadth in elisp, on top of a small core, exactly the way Emacs bootstraps `subr.el`. The prelude's own header says so — it is the “[`DERIVED`] surface,” breadth “written in elisp on top of the primitives.”

How the prelude loads

The prelude is loaded once per host, lazily, by `load_prelude`. The mechanism is the same read-expand-compile-run pipeline any elisp goes through, run over the prelude's forms in order, and it is deliberately *best-effort*: if a single prelude definition fails to compile or run, it is reported to `stderr` and skipped, not made fatal. The reason is robustness — a broken or not-yet-supported definition in the prelude should degrade gracefully, leaving the rest of the library available, rather than taking the whole runtime down. The form-by-form loading matters for the same reason it matters for user files: an earlier prelude `defmacro` must be in effect before a later prelude form that uses it is compiled, and loading in order guarantees that. The prelude is also what the bytecode cache's schema fingerprint hashes, so that editing any prelude definition invalidates stale cached bytecode.

That same read-expand-compile-run pipeline is now reachable from elisp itself through a faithful `load` primitive. (`load FILE`) searches `load-path` — falling back to the working directory when it is empty — trying the standard suffixes `.el` and `.el.gz` in order, decompressing a resolved `.el.gz` in memory the way `jka-compr` does, and binds `load-file-name` and `load-true-file-name` while the file's forms run so a loaded file sees the same variables it would under `emacs -l require` is layered on top, loading a feature's file only when it is not already in the `features` list. The practical payoff is reach: with `load` in place, and with the ported declaration machinery it pulls in — `defgroup/defcustom/defface`, `define-widget` and `define-button-type` registries, `cl-generic`'s full method dispatch, coding-system and language-environment registries, keymaps and syntax tables — elispr can load stock Emacs libraries off disk rather than only running self-contained scripts, which is what turns “best-effort” from a fallback for a broken prelude form into a working strategy for real-world `.el`.

The ordering of the prelude is itself a small lesson in bootstrapping. It opens with the tiny, foundational helpers — the `c[ad]+r` family (`cadr`, `caddr`, `cddr`, and the rest), numeric helpers like `max`, `min`, `gcd`, `lcm`, list helpers like `nthcdr`, `last`, `number-sequence` — and then builds upward through membership and alist functions, the higher-order and sequence functions, the `cl-lib` subset, the `subr-x` macros, and finally the large self-contained sub-libraries: JSON, `rx`, `pcase`, `map`, and ERT. Each layer uses only what the layers below it have already defined, which is exactly the discipline that lets a standard library be written in its own language.

Best-effort, by design

One property of the loader deserves emphasis because it shapes how the prelude can evolve: loading is *best-effort*. The prelude's own header says it is “loaded once per host, form by form, best-effort,” and the

loader honors that literally — if a single definition fails to compile or run, the failure is reported to `stderr` and skipped, not made fatal. A broken or not-yet-supported definition degrades to its own absence rather than taking the whole runtime down with it. This is the right robustness posture for a body of library code that is still growing toward full Emacs parity: a definition that depends on a primitive elisp does not yet have to leave the other few hundred definitions intact, and it does. It also pairs with the form-by-form ordering to make the prelude self-bootstrapping in the same way a user file is — an earlier `defmacro` is in effect before a later form that uses it, and an earlier helper is defined before a later function that calls it — which is the discipline that lets `cl-loop` be written in terms of `while`, `pcase` in terms of `cond`, and the whole `seq` library in terms of `mapcar` and `cons`. The prelude is not a monolith loaded atomically; it is a sequence of small definitions, each standing on the ones before it, each able to fail in isolation, which is exactly how a standard library written in its own language has to be built if it is going to grow without becoming brittle.

seq: the sequence library

The `seq` library is one of the broadest pieces of the prelude, and it is the modern, generic interface to sequences — lists, vectors, and strings treated uniformly. The `stdlib.el` and `text-utils.el` examples exercise dozens of its functions, and the breadth is real: `seq-map`, `seq-filter`, `seq-reduce`, `seq-find`, `seq-some`, `seq-every-p`, `seq-take`, `seq-drop`, `seq-subseq`, `seq-uniq`, `seq-min`, `seq-max`, `seq-position`, `seq-into`, `seq-difference`, `seq-intersection`, `seq-partition`, `seq-group-by`, `seq-map-indexed`, `seq-keep`, `seq-mapcat`. A representative pipeline from `text-utils.el` filters, maps, and reduces in one expression:

```
(should (= (seq-reduce #'(+
                        (seq-map (lambda (x) (* x x))
                                (seq-filter #'cl-evenp (number-sequence 1 10))))
          220))
```

Every one of those `seq-` functions is elisp in the prelude, defined over `mapcar`, `while`, `cons`, and the other primitives. That a fluent functional pipeline runs at all is the prelude doing its job; that it runs *fast* is the compiler and JIT doing theirs, because each `seq-` call compiles to a `CALL` and each lambda body to a sub-chunk the JIT can warm.

cl-lib: the Common Lisp compatibility layer

The `cl-lib` subset is the largest single body in the prelude, and it brings the Common Lisp vocabulary that a great deal of real elisp depends on. The `stdlib.el` example's `std-cl` and `std-more-cl` tests walk a representative slice: `cl-mapcar`, `cl-subseq`, `cl-position`, `cl-count`, `cl-find`, `cl-remove-duplicates`, `cl-getf`, `cl-list*`, `cl-first` through `cl-tenth`, `cl-member`, `cl-remove`, `cl-substitute`, `cl-acons`. Beyond the sequence functions are the place-modifying macros — `cl-incf`, `cl-decf`, `cl-pushnew`, `cl-rotatef` — the binding and control macros — `cl-flet`, `cl-labels`, `cl-destructuring-bind`, `cl-case`, `cl-typecase`, `cl-block`, `cl-return-from` — and the heavyweight definers, `cl-defun`, `cl-defmacro`, `cl-defstruct`, `cl-defgeneric`, and `cl-defmethod`.

The heavyweights are worth pausing on because they show how much a macro can do. `cl-defun` accepts a full Common-Lisp lambda list — `&optional` with per-argument defaults, `&key`, `&rest`, `&aux` — which the special-form `lambda` does not support. It gets there by desugaring to a plain `&rest` function whose body is a `cl-destructuring-bind` that pulls the real parameters out of the rest list:

```
(defmacro cl-defun (name arglist &rest body)
  `(defun ,name (&rest --cl-args--)
    (cl-destructuring-bind ,arglist --cl-args-- ,@body)))
```

So all the lambda-list richness lives in one place, `cl-destructuring-bind`, and `cl-defun` is a thin wrapper. The `cl-defun.el` example proves the keyword and optional and rest handling all work:

```
(cl-defun greet (name &key (greeting "Hello") loud)
  (let ((s (concat greeting " " name)))
    (if loud (upcase s) s)))
;; (greet "Ada" :greeting "Hi") => "Hi, Ada"
;; (greet "Ada" :loud t)         => "HELLO, ADA"
```

`cl-defstruct` is even more ambitious: from a single declaration it generates a constructor, a type predicate, and a self-able accessor for each slot, and it supports `:include` inheritance by walking a parent chain. The `data-model.el` example defines a `shape` struct, two subtypes `circle` and `rect` with `:include shape`, and then defines generic functions over them with `cl-defmethod`:

```
(cl-defstruct shape name)
(cl-defstruct (circle (:include shape)) radius)
(cl-defstruct (rect (:include shape)) width height)

(cl-defgeneric area (s) "Area of shape S.")
(cl-defmethod area ((c circle)) (* float-pi (circle-radius c) (circle-radius c)))
(cl-defmethod area ((r rect)) (* (rect-width r) (rect-height r)))
```

That `cl-defmethod` dispatches on the runtime type of its argument — a circle calls one method, a rect another — is CLOS-style multiple dispatch, and the CHANGELOG records that it grew the full method-combination apparatus: `:before`, `:after`, and `:around` qualifiers, `cl-call-next-method`, and specificity ordering so a more specific specializes wins. All of it is elisp in the prelude, dispatching through the symbol machinery and the type predicates of the primitive layer. A struct instance, recall, is a vector tagged with a `cl-struct-NAME` symbol in slot zero, which is exactly the `#s(NAME ...)` shape the reader and printer agree on.

cl-loop: a compiler inside a macro

The single most impressive macro in the prelude is `cl-loop`, the Common Lisp iteration facility, which is a small language of its own with a vocabulary of clauses. The `cl-loop.el` example shows the range: numeric iteration (`for i from 1 to 5`), list iteration (`for x in, for c on`), the accumulators (`collect`, `append`, `sum`, `count`, `maximize`, `minimize`), conditionals (`when`, `unless`, `if/else`), the quantifiers (`always`, `never`, `thereis`), with bindings, and finally `return`.

```
(cl-loop for x in (list 1 2 3 4 5) when (cl-evenp x) collect x) ; => (2 4)
(cl-loop for i from 1 to 10 by 2 collect i)                    ; => (1 3 5 7 9)
(cl-loop for x in (list 1 2 3 4) sum x)                        ; => 10
```

`cl-loop` is a macro that *parses* this clause vocabulary at expansion time and *compiles* it into a `let*-catch-while` driver — the example's header says it “expands to a `(let* ... (catch ... (while ...)))` driver in the prelude.” So `cl-loop` is a miniature compiler, written in elisp, targeting the special forms. Its existence is the strongest possible argument for the macro-and-prelude architecture: a feature as large as the Common Lisp loop facility can be added to the language without touching the core compiler at all, because it expands into forms the core already understands. The CHANGELOG shows it being extended clause by clause — `for V across SEQ`, `for V being the hash-keys of`, `for V = INIT then STEP` — each a prelude edit, none a compiler change.

subr-x, shown

The `subr-x` library is the modern convenience layer — the string helpers and the threading and binding macros that a contemporary `.el` file reaches for — and because it is small and pure it is a good place to see a prelude definition rather than just be told one exists. The threading macros are compile-time AST rewrites: `thread-first` threads a value into the first argument position of each successive form, `thread-last` into the last. Their real prelude bodies are short loops that fold the accumulator into each form:

```
(defmacro thread-first (x &rest forms)
  (let ((acc x))
    (while forms
      (let ((form (car forms)))
        (setq acc (if (consp form) (cons (car form) (cons acc (cdr form))) (list form
~ acc))))
      (setq forms (cdr forms)))
    acc))
```

So `(thread-first 5 (- 1) (* 2))` rewrites to `(* (- 5 1) 2)` — 5 threaded into the first argument of `(- 1)`, then that into the first argument of `(* 2)` — and evaluates to 8, exactly as `stdlib.el` asserts. `thread-last` is the same loop with `append` instead of `cons`, so the accumulator lands in the *last* position; `stdlib.el` checks that threading `(list 1 2 3 4)` through `(seq-filter #'cl-evenp)` and then `(mapcar #'1+)` evaluates to `(3 5)`. Neither macro has any runtime cost — the threading is gone by the time the compiler sees the rewritten form.

The string helpers are equally direct. `string-trim` is a pair of regexp strips, and its definition shows the regexp surface of Chapter 15 in use inside the prelude:

```
(defun string-trim-left (s &optional regexp)
  (let ((re (concat "\\`\\(?:" (or regexp "[ \\t\\n\\r]+" ) "\\)")))
    (if (string-match re s) (substring s (match-end 0)) s)))
```

The `\`` anchor is the buffer-start anchor that the Chapter 15 translator maps to the engine's `\A`, and `string-trim-left` strips a leading whitespace run by matching it and slicing from `match-end`. The binding macros `if-let` and `when-let` are built on one recursive helper, `if-let--chain`, that nests a `let`-and-`if` per binding and short-circuits to the else branch the first time a bound value is `nil` — the “bind these in sequence, but bail if any is false” idiom that `stdlib.el`'s `std-macros` test exercises. Every one of these is elisp over the primitives, loaded at startup, which is why a section that began by promising “the prelude is macros all the way down” can show you the macros and have them be real.

pcase, rx, json, map, and the rest

The prelude's other self-contained libraries each follow the same pattern of compiling a small sub-language into core forms. `pcase` compiles structural patterns into a `cond`: its core macro binds the scrutinee once and maps each clause through a pattern compiler that emits tests and bindings. The `pcase.el` and `data-model.el` examples use the pattern vocabulary — wildcards, literals, binders, `pred`, `guard`, `and`, `or`, and the backquote patterns the reader's eager expansion made possible. `rx` compiles an S-expression regular expression into a regexp string, so you can write `(rx (group (+ digit)))` instead of a backslash-laden string, and the CHANGELOG notes it is even wired as a `pcase` pattern. The JSON library encodes and parses JSON entirely in elisp, which is what lets `script-demo.el` round-trip data to disk as JSON. The `map` library gives a generic interface over alists, plists, and hash tables — `map-elt`, `map-keys`, `map-values`, `map-into`. And `subr-x` provides the modern string helpers — `string-trim`, `string-join`, `string-pad`, `string-replace`, `string-blank-p` — and the threading and binding macros — `thread-first`, `thread-last`, `if-let`, `when-let`. The `stdlib.el` example's `std-macros` test exercises the threading macros directly:

```
(should (= (thread-first 5 (- 1) (* 2)) 8))
(should (equal (thread-last (list 1 2 3 4) (seq-filter #'cl-evenp) (mapcar #'1+))
              (list 3 5)))
```

Every one of these libraries is elisp, loaded at startup, expanding into the twenty-two special forms and the two hundred primitive subrs. The prelude is, in a sense, the proof that the core is complete: if a standard library this large can be built on top of the core entirely in elisp, then the core has everything a Lisp needs. The next chapter takes the one prelude system that is most entangled with the core — the error and testing surface — and follows it down through the layers.

Chapter 14 – Control Flow, Errors, and Non-Local Exit

Errors in elisp are not exceptions bolted onto the side of the language; they are a first-class control-flow mechanism with their own vocabulary — `signal`, `error`, `condition-case`, `unwind-protect`, `catch`, `throw` — and their own data model, the condition system, in which errors are symbols arranged in an inheritance hierarchy. elisprs implements the whole of this, split across three layers exactly as the rest of the language is: the special-form lowering in the compiler, the intrinsic implementations in the host, and the condition hierarchy and convenience macros in the prelude. This chapter follows error handling down through all three, because it is the clearest case study in how the layers cooperate, and it ends with the ERT testing surface, which is built entirely out of the error system.

Signalling: how an error is raised

At the bottom, an error is a Rust `Result::Err`. A subr that fails returns an `Err` with a message string, and that `Err` propagates up through the VM as a halt. But a bare error string is not enough for elisp, because `condition-case` needs to dispatch on the error's *condition symbol* and bind the handler variable to the error's *data*. So when a subr signals, it does two things: it stashes a structured `(error-symbol . data)` cons on the host's pending-error field, and it returns the `Err`. The four signalling subrs — `error`, `user-error`, `signal`, and the wrong-type and arithmetic errors raised inline — all follow this protocol. `(error "boom")` builds an `(error "boom")` condition cons and returns an error; `(signal 'bad-token '("x"))` builds a `(bad-token "x")` cons. The CHANGELOG records the fix that made this faithful: `condition-case` now “binds the handler variable to the real `(ERROR-SYMBOL . DATA)` object, preserving `signal`'s data list,” so that catching `(signal 'wrong-type-argument '(integerp 5))` binds the handler variable to `(wrong-type-argument integerp 5)` and `(cadr e)` is `integerp`. The `data-model.el` example asserts exactly this data-preservation:

```
(should (equal (condition-case e (signal 'bad-token '("oops"))
                      (error (cadr e)))
              "oops"))
```

`condition-case`, `catch`, `unwind-protect`: the intrinsics

The three handler forms are, as Chapter 9 showed, compiled into calls to internal intrinsics — `--condition-case--`, `--catch--`, `--unwind--` — with their subexpressions wrapped in thunks. The intrinsics are implemented in the host's `call_function`, intercepted by name before any host borrow, and they implement the control logic against the pending-error and pending-throw state.

`--condition-case--` runs the body thunk. If it returns normally, that value is the result. If it signals — leaving a condition on the pending-error field — the intrinsic walks the handler list, and for each handler asks whether the signalled condition *matches* the handler's condition specification. Matching is where the condition hierarchy comes in: a handler for `error` catches everything that inherits from `error`, a handler for `arith-error` catches only arithmetic errors, and the umbrella `error` condition sits at the top so `(error ...)` handlers are the catch-all. The match test consults the condition symbols' `error-conditions` property, which the prelude seeds. On a match, the intrinsic binds the handler variable to the condition cons and runs

the handler thunk; on no match, it re-raises. The `errors.el` example proves both the happy path and the recovery path:

```
(should (eq (condition-case nil (error "boom") (error 'recovered)) 'recovered))
(should (= (condition-case nil (+ 1 2) (error 'unused)) 3))
```

`--catch--` and `--throw--` implement the labeled escape. `throw` checks the active catch-tag stack, and on a match sets the pending-throw field and returns a sentinel error; the matching `--catch--` intrinsic sees the pending throw, confirms the tag, and returns the thrown value. A `throw` with no matching catch signals a `no-catch` error instead, so a stray throw fails loudly. `--unwind--` runs its body thunk and then, no matter how the body exited — normally, by throw, or by error — runs its cleanup thunk, which is the `try/finally` guarantee. The `errors.el` example's unwind test confirms the cleanup runs even when the body errors:

```
(let ((cleaned nil))
  (ignore-errors
    (unwind-protect (error "boom")
      (setq cleaned t)))
  (should (eq cleaned t)))
```

This is also where the specstack unwinding from Chapter 6 earns its keep: when a throw or error jumps out of the middle of a `let`, the dynamic bindings that `let` made must be restored even though its normal `UNBIND` was skipped. The host's scope-recovery machinery, which tracks the specstack depth at each scope entry, unwinds the dynamic bindings back to the right depth as the non-local exit passes through. The `errors.el` example's own comment about a known over-unwinding edge in one nested-throw case is the honest acknowledgment that this cross-boundary unwinding is the hardest part of the error system to get exactly right.

The condition hierarchy in the prelude

The condition *symbols* and their inheritance live in the prelude, seeded by a block of `define-error` registrations. `define-error` takes a condition symbol, a message, and an optional parent, and records the condition's place in the hierarchy by setting its `error-conditions` property to the list of itself plus all its ancestors. The prelude seeds the standard conditions — `error` at the root, then `user-error`, `arith-error`, `wrong-type-argument`, `wrong-number-of-arguments`, `void-variable`, `void-function`, `args-out-of-range`, `end-of-file`, and the rest — so that `condition-case`'s matching test can ask whether `arith-error` inherits from `error` and get the right answer. The `data-model.el` example builds its *own* hierarchy on top of this, demonstrating that the mechanism is user-extensible:

```
(define-error 'parse-error "Parse error")
(define-error 'bad-token "Bad token" 'parse-error)
;; signalling bad-token is caught by a parse-error handler:
(should (eq (condition-case e (signal 'bad-token '("x"))
  (parse-error 'caught-parent))
  'caught-parent)))
```

A `bad-token` is caught by a `parse-error` handler because `define-error` made `bad-token` inherit from `parse-error`. This is the genuine Emacs condition system, implemented in elisp over the symbol property machinery, and it is what makes `error-message-string` able to render a condition the way Emacs does — the very rendering the command-line driver uses to turn an internal `void-variable: foo` into “Symbol's value as variable is void: foo.”

define-error, shown, and the seeded conditions

The condition hierarchy is, mechanically, two property writes per condition, and the real `define-error` makes that concrete. It computes the condition's `error-conditions` list as itself prepended to all its parents' conditions, and stores both that list and the human message on the symbol's property list:

```
(defun define-error (name message &optional parent)
  (let* ((parent (or parent 'error))
        (parents (if (listp parent) parent (list parent)))
        (conds (cons name (apply (function append)
                                   (mapcar (lambda (p) (get p 'error-conditions))
                                           parents)))))
    (put name 'error-conditions conds)
    (put name 'error-message message)
    name))
```

So `(define-error 'bad-token "Bad token" 'parse-error)` sets `bad-token`'s `error-conditions` to `(bad-token parse-error error)` — itself plus the inherited chain — which is exactly what makes a `parse-error` handler catch a signalled `bad-token`: the `condition-case` matcher tests whether the handler's condition appears in the signalled condition's `error-conditions` list, and it does. The prelude seeds the standard conditions the same way, a block of `define-error` calls that plants the umbrella `error` at the root and hangs the rest off it:

```
(define-error 'error "error" nil)
(define-error 'user-error "")
(define-error 'arith-error "Arithmetic error")
(define-error 'wrong-type-argument "Wrong type argument")
(define-error 'wrong-number-of-arguments "Wrong number of arguments")
(define-error 'void-variable "Symbol's value as variable is void")
(define-error 'void-function "Symbol's function definition is void")
(define-error 'args-out-of-range "Args out of range")
(define-error 'end-of-file "End of file during parsing")
```

The messages are not arbitrary — they are the exact strings Emacs uses, which is what lets `error-message-string` render `void-variable` as “Symbol's value as variable is void” rather than some elisp-specific phrasing. The two CLOS conditions `cl-no-applicable-method` and `cl-no-next-method` from Chapter 22 are registered the same way, later in the prelude, so the generic-function dispatch can signal them and a handler can catch them. The whole condition system, including the rendering the command-line driver leans on, is this — `put` of two properties per condition, with the hierarchy encoded as the `error-conditions` list — and nothing about it is in the host beyond the `get/put` primitives it builds on.

The convenience macros

On top of the intrinsics, the prelude defines the convenience macros everyone reaches for. `ignore-errors` wraps its body in a `condition-case` that catches `error` and returns `nil`, the idiom for “try this, but don't let it fail the program.” The `errors.el` example uses it directly: `(ignore-errors (error "x"))` is `nil`, `(ignore-errors 123)` is `123`. There is also `ignore-error` (singular) for catching a specific condition, `with-demoted-errors` for turning errors into messages, and `with-suppressed-warnings`. Each is a thin `condition-case` wrapper, and each exists because the raw `condition-case` is verbose for the common cases.

ERT: a test framework built from the error system

The clearest demonstration that the error system is a complete control-flow mechanism is that an entire testing framework is built out of it, in elisp, in the prelude. ERT — Emacs Lisp Regression Testing — is the framework every example file uses, and its assertions are nothing but signals and handlers. `should` evaluates a form and, if the result is `nil`, signals `ert-test-failed` with data describing the failure; otherwise it returns the value. `should-not` is the negation. `should-error` is the most interesting: it runs its body inside a `condition-case` that catches `error`, and it *fails* if no error was signalled, optionally checking that the signalled condition matches a `:type`. The `errors.el` and `arithmetic.el` examples lean on the type check:

```
(should-error (/ 1 0) :type 'arith-error)
(should-error (car 5) :type 'wrong-type-argument)
```

`should-error` only passes if the body raised, and with `:type` only if it raised the right condition — which means it is exercising the entire signal-and-match path on every assertion. `ert-deftest` registers a named test, stripping an optional docstring and the `:expected-result` keyword that marks a test as an expected failure (the `ert.el` example’s XFAIL for the not-yet-implemented `current-buffer`). `skip-unless` signals `ert-test-skipped` to short-circuit a test whose precondition is unmet. And `ert-run-tests-batch-and-exit` runs every registered test inside a `condition-case`, classifies each as pass, fail, skip, or expected-failure by which condition it caught, and exits non-zero if any test failed unexpectedly. That last function is why every example file is a CI gate: it turns a signalled `ert-test-failed` into a non-zero process exit, so a broken assertion breaks the build.

The whole of ERT — `should`, `should-not`, `should-error`, `skip-unless`, `ert-deftest`, the batch runner — is elisp in the prelude, built on `condition-case`, `signal`, and the condition hierarchy. There is no special test-runner support in the core. A testing framework, like the Common Lisp loop facility, like the struct system, like `pcase`, is just more elisp expanding into the special forms. That is the architecture’s final word: give a Lisp a small, correct core and a macro system, and the rest of the language — including the tools that test the language — can be written in itself.

How `should` explains a failure

A good assertion does not just fail; it explains. When `(should (= (+ 1 2) 4))` fails, ERT does not merely report that the form was false — it reports the form *and the values its subexpressions produced*, so the message reads like “`(should (= (+ 1 2) 4))`” with “`(+ 1 2)`” explained as having been `3`. This explain machinery is built into the `should` macro itself, and it is a small piece of compile-time cleverness worth seeing. When the asserted form’s head is one of a recognized set of comparison functions, `should` expands to code that evaluates each argument into a fresh temporary, tests the predicate on the temporaries, and — on failure — signals `ert-test-failed` with a `:values` plist pairing each argument form with the value it took:

```
;; should (builtins-aware arm), transcribed shape:
;; (let ((g1 (+ 1 2)) (g2 4))
;;   (if (= g1 g2) t
;;       (signal 'ert-test-failed
;;               (list 'should '(= (+ 1 2) 4) :values (list (cons '(+ 1 2) g1) ...)))))
```

The `tests/ert.rs` unit tests assert exactly this behavior — catching the signalled `ert-test-failed` from `(should (= (+ 1 2) 4))` and confirming the message contains both the form `should (= (+ 1 2) 4)` and the explained value `(+ 1 2) . 3`. That is the difference between a test framework that says “an assertion failed somewhere” and one that tells you the subexpression and its actual value, and it is implemented entirely in the `should` macro by expanding the comparison into let-bound temporaries it can report. The simpler arm, for a non-comparison form, just signals with the form itself; the comparison-aware arm is the one that does the explaining, and it is why a failing elisprs test reads the way a failing Emacs test does.

With control flow and errors in hand, the remaining surface is the data a script actually pushes around: text, patterns, and buffers.

Chapter 15 – Strings, Characters, and Regular Expressions

A great deal of what real elisp does is move text around: parse a line, extract a field, match a pattern, substitute a replacement, join a list of pieces. elisprs makes this work, and the most interesting part of it is regular expressions, because Emacs regular expressions have a syntax unlike any other engine's, and elisprs has to bridge that syntax to a Rust regex engine while preserving Emacs's exact match semantics. This chapter is about strings, characters, and — at length — the regexp bridge.

Strings and characters

A string in elisprs is a fusevm immediate, not a heap object, which means string data flows through the VM directly and the string subrs operate on it host-side. A character is an integer, as the reader chapter established, so character operations are integer operations — `?A` is 65, `(char-to-string ?A)` is "A", `(string-to-char "A")` is 65. The string subrs cover construction and inspection at the primitive layer — `concat`, `substring`, `split-string`, `length`, `upcase`, `downcase`, `number-to-string`, `string-to-number` — and the prelude adds the `subr-x` modern surface on top: `string-trim` and its directional variants, `string-join`, `string-pad`, `string-replace`, `string-blank-p`, `string-reverse`, `string-remove-prefix` and `-suffix`. The `strings.el` and `stdlib.el` examples walk the whole surface, and one detail worth noting is that everything counts in *characters*, not bytes — `(length "hello")` is 5, and a multibyte string's length is its character count. This matters enormously for the regexp engine, because Emacs reports match positions in character indices, and a faithful implementation must do the same even when the underlying engine works in bytes.

The `text-utils.el` example is the best single illustration of string work in anger: it parses `key=value` lines, trims whitespace, sorts an alist by key, pads keys to a common width, and joins the result into aligned lines:

```
(defun parse-kv (line)
  "Parse \"key=value\" into a (KEY . VALUE) cons, trimming whitespace."
  (let ((parts (split-string line "=")))
    (cons (string-trim (car parts)) (string-trim (cadr parts)))))
```

Every function in that — `split-string`, `string-trim`, `car`, `cdr`, `cons` — is in the library, primitive or prelude, and the example runs as a CI gate, so the string surface is exercised on every commit.

Emacs regexps invert the convention

Here is the syntactic fact that makes elisp regexps strange to anyone coming from PCRE or POSIX extended regular expressions. In a PCRE-style engine, the metacharacters for grouping, alternation, and bounded repetition are bare: `(abc)` is a group, `a|b` is alternation, `a{2,3}` is a bounded repeat, and to match a literal parenthesis you escape it, `\(`. Emacs inverts this exactly. In an Emacs regexp, the *bare* characters `(`, `)`, `|`, `{`, `}` are literals — they match themselves — and the *backslashed* forms `\(`, `\)`, `\|`, `\{`, `\}` are the metacharacters. So an Emacs regexp for “one or more lowercase letters, a dash, one or more digits” is `\([a-z]+\)-\([0-9]+\)`, with backslashed grouping parens, where a PCRE would write `([a-z]+)-([0-9]+)`. This is not a small cosmetic difference; it is a complete inversion of which characters are special, and any code copied from a real `.el` file is full of backslashed parens.

elisp's regexp engine cannot ask a Rust regex library to understand Emacs syntax, because the library speaks PCRE. So `src/regexp.rs` is a *translator*: it walks an Emacs regexp character by character and emits an equivalent pattern in the regex crate's dialect, inverting the convention back. Its header states the problem precisely: "Emacs regexps invert the convention of POSIX-ERE / PCRE engines: grouping, alternation and bounded repetition are spelled with a leading backslash, while the bare characters are literals. The `regex` crate is the opposite. This module walks an elisp pattern and emits an equivalent pattern in the crate's dialect so the engine can be reused wholesale instead of writing a matcher by hand."

What the translator handles

The translation is a careful mapping, and its scope is the common, portable subset of Emacs regexp syntax. The grouping inversion is the core: `\(` becomes `(`, `\)` becomes `)`, `\|` becomes `|`, `\{ /\}` become `{ /}`, and conversely the bare `(, ), |, {, }` get escaped so they match literally. Shy groups `\(?:...\)` translate to the crate's `(?:...)`. The anchors translate too: Emacs's buffer-start `\^` and buffer-end `\'` become the crate's `\A` and `\Z`, and the word boundaries `\<` and `\>` become `\b`. The word and syntax escapes — `\w`, `\W`, `\b`, `\B`, and the syntax-class forms `\s-` (whitespace) and `\sw` (word) — map to the crate's equivalents. Character alternatives in brackets, `[a-z]`, `[^...]`, and POSIX classes `[:alpha:]`, pass through because both dialects agree on them, with care taken for the corner cases — a `]` as the first character of a class is a literal, and a backslash inside an Emacs character class is an ordinary character, not an escape.

There is a subtlety in numbered groups that the translator handles thoughtfully. Emacs supports explicitly-numbered groups, `\(?:1:...\)`, which the Rust engine has no syntax for. The translator drops the explicit number and emits a plain capturing group, which gives the correct match-data index whenever the explicit numbers are sequential — the common case, as in font-lock patterns like `\(?:1:...\)\(?:2:...\)`. Non-sequential explicit numbering is not preserved, a limitation the source comment names openly. This is the recurring pattern of the project: handle the common case correctly, document the uncommon case as unsupported, never silently misbehave.

The translation, arm by arm

The translator is a character walk with a `match` at its center, and reading the arms in one place is the clearest way to see the inversion concretely. The top-level walk escapes the bare metacharacters so they match literally and dispatches a backslash to the escape handler:

```
;; translate (regexp.rs), the shape of it:
;; '\\ ' => translate_escape ; a backslashed metachar
;; '(' ')' '{' '}' '|' => emit "(" "\)" " ..." ; bare = literal, so escape it
;; '[' => copy_class ; a character alternative
;; else (+ * ? . ^ $ ...) => emit verbatim ; these agree in both dialects
```

Inside `translate_escape`, the backslashed forms drop their backslash to become the engine's metacharacters: `\(` becomes `(`, `\)` becomes `)`, `\|` becomes `|`, and `\{ /\}` become `{ /}`. The shy group `\(?:` copies through as `(?:`, and the explicitly-numbered group `\(?:N:` becomes a plain `(` with the number dropped. The anchors and classes map across the dialects' different spellings:

```
;; translate_escape's mappings, selected:
;; \^ => \A (buffer start) \w => \w \W => \W
;; \' => \Z (buffer end) \b => \b \B => \B
;; \< \> => \b (word boundaries) \s- => \s \sw => \w \SC => \S
;; \_< \_> => \b (approximated, symbol boundaries)
;; \= => {} (point - no analogue; matches empty)
;; \1 .. \9 => \1 .. \9 (backreferences, passed through unchanged)
```

Several of these are honest approximations the source names openly: the symbol boundaries `\_<` and `\_>` are approximated with a plain word boundary because the engine has no symbol-syntax concept, the point assertion

`\=` has no analogue and compiles to an empty match, and a syntax class the translator does not specifically recognize falls back to `\s` (or `\5`) “so the pattern still compiles.” The character class is copied through by `copy_class`, which preserves a leading `^` negation and a literal leading `]`, and which treats a backslash inside the class as an ordinary character — doubling it for the engine — because in an Emacs character class a backslash is not an escape. POSIX classes like `[:digit:]` pass through whole, which `tests/regexp.rs` confirms: `(string-match "[[:digit:]]+" "abc123")` is 3. This arm-by-arm correspondence is the whole of why the engine can be reused wholesale instead of hand-writing an Emacs-syntax matcher.

The matching engine and backreferences

The translated pattern is compiled and run by `fancy-regex`, the matching engine the Cargo manifest pins. The choice of `fancy-regex` over the plain `regex` crate is deliberate and is worth understanding, because it resolves a genuine tension. The plain `regex` crate is linear-time and very fast, but it cannot do backreferences, because backreferences make matching non-regular and require backtracking. `fancy-regex` wraps the linear engine and adds a backtracking fallback only for the patterns that need it, so ordinary patterns stay on the fast linear path and backreference patterns get a working backtracker. The Cargo manifest spells out the reasoning: `fancy-regex` “adds a backtracking fallback for backreferences (`\1.. \9`) while keeping the linear `regex` crate fast path for patterns that don’t need it.”

This is a place where the documentation in the repository disagrees with itself across time, and it is worth flagging for honesty. An older note in the README says pattern backreferences are *rejected* because the engine does not backtrack. That note predates the move to `fancy-regex`; the current source supports backreferences, the translator passes `\1` through `\9` straight to the backtracking engine (both dialects spell them the same way), and the `regexp.el` example asserts them working:

```
(ert-deftest regexp-backreferences ()
  "Backreferences \\1..\\9 match the same text an earlier group captured."
  (should (= (string-match "\\(\\.\\)\\1" "abccba") 2))
  (should (null (string-match "\\(\\.\\)\\1" "abcdef"))))
  (should (= (string-match "\\(ab\\.\\)\\1+" "xabababy") 1)))
```

`\\(\\.\\)\\1` matches a doubled character — the `\\1` must match the *same* character the group captured, not just any character — which is the defining test of a real backreference, and it passes. The example is the authority here; backreferences work in the version this book describes.

Match data, in character positions

When a match succeeds, elisp does not just return a boolean; it records *match data* — the start and end positions of the whole match and of each capturing group — which the program then reads with `match-beginning`, `match-end`, and `match-string`. elisprs stores this match data on the host, and crucially it stores it in *character* positions, bridging from the byte offsets `fancy-regex` reports. The `regexp.el` example asserts this explicitly with a multibyte string:

```
(ert-deftest regexp-unicode-positions ()
  "Match positions count characters, not bytes."
  (string-match "é" "aébé")
  (should (= (match-beginning 0) 1))
  (should (= (match-end 0) 2)))
```

The `é` is at character index 1 even though it is two bytes, and `match-beginning` and `match-end` report 1 and 2 — character positions — because elisprs converts the engine’s byte spans to character spans before storing them. This character-versus-byte bridging is exactly the kind of fidelity detail that separates a working elisp `regexp` from a plausible-looking one, and the example pins it down. `string-match` returns the match start (or nil), `match-string` slices the recorded span out of the subject, `match-data/set-match-data` serialize the spans to and from a flat list, and `save-match-data` shields the match state across a nested search — all

asserted in the example.

Replacement and case adaptation

`replace-regexp-in-string` is the substitution workhorse, and its replacement-template handling is the last piece of the regexp surface. A replacement string can refer to the whole match with `\&` and to a captured group with `\N`, and `elisp` expands these in the replacement. The `regexp.el` example shows both the group reference and the whole-match reference:

```
(should (equal (replace-regexp-in-string "\\([a-z]+\\)=\\([0-9]+\\)" "\\2:\\1" "x=1
~ yy=22")
               "1:x 22:yy"))
(should (equal (replace-regexp-in-string "[0-9]+" "<\\&>" "a1b22")
               "a<1>b<22>"))
```

The first swaps two captured groups via `\1` and `\2`; the second wraps each run of digits in angle brackets via `&`. There is also a literal flag that suppresses template expansion, and case adaptation that, by default, makes the replacement match the case of the text it replaces. The `rx` macro from the prelude sits above all of this, letting a program build a regexp from an S-expression rather than a backslash-heavy string, and the `buffers.el` and `script-demo.el` examples use the regexp surface against buffer text, which is the bridge to the next chapter.

Chapter 16 — Buffers and Text Processing

The buffer is the central object of Emacs — the editable text container that a window displays and that point moves through — and it is exactly the part of elisp you would expect to be missing from an elisp-without-the-editor. And the full editor object model *is* missing: there is no window system, no display, no interactive command loop, no notion of the user’s current buffer as an editor concept. But there is something narrower and, for scripting, more important: a minimal buffer model — text plus point — that backs `with-temp-buffer`, `insert`, point motion, search and replace, and file I/O. This is the read-process-write pipeline a standalone `.el` script actually uses, and `elisp` implements enough of it to run real text-processing programs. This chapter is about that minimal buffer model, what it includes, and where its edges are.

What a buffer is here

A buffer in `elisp` is, at bottom, a vector of characters plus a one-based point — a cursor position into the text. The host keeps a stack of these buffers, and the buffer primitives operate on the current top of stack. `insert` adds text at point; `point`, `point-min`, and `point-max` report positions; `goto-char` moves point; `buffer-string` returns the whole text; `buffer-substring` returns a span. The `buffers.el` example’s header describes the model exactly: “A minimal editing-buffer model (text + point) backs `with-temp-buffer`, `insert`, point motion, search/replace, and `insert-file-contents/write-region`. This is the read -> process -> write pipeline a standalone `.el` script actually uses.”

The most important construct is `with-temp-buffer`, which creates a scratch buffer, makes it current for the duration of its body, and discards it afterward. This is the idiom for “process some text” that does not need a persistent buffer, and it is how nearly all scripting text manipulation is done. The example’s `wc-lines` counts lines by inserting text into a temp buffer and calling `count-lines` over the whole extent:

```
(defun wc-lines (text)
  "Count lines in TEXT using a temp buffer."
  (with-temp-buffer
    (insert text)
    (count-lines (point-min) (point-max)))))
```

Point motion and searching

The buffer model supports point motion and searching, which is what makes it useful for parsing. `forward-line`, `beginning-of-line`, `end-of-line`, `forward-word`, `forward-char`, and their backward counterparts move point; `looking-at` tests whether text at point matches a regex; `char-after` and `char-before` read the characters adjacent to point. The `buffers.el` example’s motion test walks through a multi-line buffer checking that point lands where it should:

```
(with-temp-buffer
  (insert "alpha\nbeta\ngamma")
  (goto-char (point-min))
  (forward-line 1)
  (should (= (point) 7))
  (should (looking-at "beta"))
  (end-of-line)
  (should (= (char-before) ?a)))
```

The real power comes from `re-search-forward`, which combines point motion with the regexp engine of the last chapter: it searches forward from point for a pattern, moves point to the end of the match, and sets the match data so the captured groups can be read with `match-string`. This is the foundation of buffer parsing, and the `buffers.el` example's `collect-numbers` shows the loop idiom — search repeatedly until the search fails, collecting each match:

```
(defun collect-numbers (text)
  "Return all integers appearing in TEXT, in order."
  (with-temp-buffer
    (insert text)
    (goto-char (point-min))
    (let (nums)
      (while (re-search-forward "[0-9]+" nil t)
        (push (string-to-number (match-string 0)) nums))
      (reverse nums))))
```

The match data here is buffer-relative — the spans are positions in the buffer text, and `match-string 0` slices the whole match out of the current buffer — which is the same match-data machinery as the string case, flagged as coming from a buffer rather than a string. `replace-match` substitutes a replacement at the last match, so the search-and-replace loop — search forward, replace, repeat — works, and the example's `uppercase-words-via-buffer` uses exactly it to uppercase every run of letters:

```
(defun uppercase-words-via-buffer (text)
  "Uppercase every run of letters in TEXT via re-search-forward/replace-match."
  (with-temp-buffer
    (insert text)
    (goto-char (point-min))
    (while (re-search-forward "[a-z]+" nil t)
      (replace-match (uppercase (match-string 0)) t))
    (buffer-string)))
```

The buffer registry and the full vocabulary

The buffer model is, in the host, an `EditBuffer`, and its state has grown well past text and point: the source comment now describes “a char vector, a 1-based point, narrowing bounds, the mark, plus the buffer-local variable slots and the local keymap slot,” with per-character text-property plists and a list of live markers alongside. The host keeps a *registry* of these buffers, each with a name, and a notion of the current buffer that the primitives act on; a killed buffer keeps its slot so any `Obj::Buffer` handle still resolves and `buffer-live-p` can report it dead. `with-temp-buffer` is now written in terms of that registry — it expands to `generate-new-buffer` for a fresh “`*temp*`”, `with-current-buffer` to select it for the body, and `kill-buffer` in an `unwind-protect` cleanup so the temporary is discarded even on a non-local exit. That is the whole of the scratch-buffer mechanism: allocate a named buffer, select it, run the body, and guarantee the kill.

On top of that two-field model the primitive layer is surprisingly broad. The position queries are `point`, `point-min`, `point-max`, `bolp`, `eolp`, `bobp`, `eobp`, `line-beginning-position/pos-bol`, `line-end-position/pos-eol`, `current-column`, and `line-number-at-pos`. The motions are `goto-char`, `forward-char`, `backward-char`, `forward-line`, `beginning-of-line`, `end-of-line`, `forward-word`, `backward-word`, and the character-skipping `skip-chars-forward/skip-chars-backward`. The reads

are `char-after`, `char-before`, `buffer-string`, `buffer-substring`, and `buffer-size`. The edits are `insert`, `insert-char`, `delete-char`, `delete-region`, and `erase-buffer`. And the searches are `search-forward`, `search-backward`, `re-search-forward`, `re-search-backward`, `looking-at`, and `looking-at-p`. One simplification the source flags: `forward-word` treats a word as a run of alphanumerics and does not consult a syntax table for its notion of word constituents, even though the runtime now carries a syntax-table subsystem — the motion primitive simply keeps its own alphanumeric rule. The `buffers.el` motion test walks the vocabulary and pins where point lands — after `(forward-line 1)` over `"alpha\nbeta\ngamma"`, point is at 7, `(looking-at "beta")` holds, and after `(forward-word)` from the start of the second line point is at 11. This is enough vocabulary to parse and edit text, which is the bar a scripting buffer has to clear; it is not the full Emacs buffer object, which is the line Chapter 16 draws deliberately.

File I/O and the script pipeline

The buffer model connects to the filesystem through `insert-file-contents`, which reads a file's contents into the current buffer at point, and `write-region`, which writes a span of the buffer (or a string) to a file. Together with `with-temp-buffer` these form the complete read-process-write loop: read a file into a temp buffer, process it with searches and point motion, and write the result back out. The `buffers.el` example's file round-trip proves the loop closes:

```
(let ((f "target/elp-buffers-example.txt")
      (payload "line one\nline two\n"))
  (write-region payload nil f)
  (unwind-protect
    (should (equal (with-temp-buffer (insert-file-contents f) (buffer-string))
                  payload))
    (delete-file f)))
```

The `script-demo.el` example is the fullest demonstration, parsing a file of `name age` lines into an alist by scanning it in a temp buffer with `re-search-forward`:

```
(defun script-demo--parse-people (file)
  "Parse \"NAME AGE\" lines into an alist of (NAME . AGE).\"
  (with-temp-buffer
    (insert-file-contents file)
    (goto-char (point-min))
    (let (rows)
      (while (re-search-forward "^\\([a-z]+\\) \\([0-9]+\\)$" nil t)
        (push (cons (match-string 1) (string-to-number (match-string 2))) rows))
      (nreverse rows))))
```

This is real scripting: a file on disk, parsed with a buffer and a regexp, into a data structure, with no Emacs anywhere. There are also region-and-word case operations — `upcase-region`, `capitalize-region`, `downcase-word` — and in-place edits like `subst-char-in-region` and `transpose-chars`, all of which the `buffers.el` example exercises, so the buffer surface is broad enough for genuine text munging.

Where the edges are

It is important to be precise about what the buffer model does and does not do, because the project is precise about it and a reader should neither over-claim nor understate. The buffer is no longer just text plus point: it is a named object with a current-buffer notion the primitives act on, live markers that move with edits, per-character text properties, narrowing bounds, buffer-local variables, and a local keymap slot. The buffer-scoping special forms that were once refused now work — `save-excursion`, `save-current-buffer`, and `save-restriction` ship as prelude macros over marker-backed intrinsics (`--se-push--/--se-pop--`, `--save-restriction--/--restore-restriction--`), so they save and restore the current buffer, point, and narrowing across their bodies. And the `ert.el` example, which once carried an *expected-failure* test

asserting the buffer object model was not yet real, now carries a plain passing test that pins it:

```
(ert-deftest ert-buffer-object-model ()
  "The buffer/editor object model: `current-buffer' returns a real buffer object."
  (should (bufferp (current-buffer)))
  (should (stringp (buffer-name)))
  (should (buffer-live-p (current-buffer))))
```

What remains out of scope is the *display and interaction* surface, not the object model: there are no overlays, no windows or frames, no interactive command loop, and `forward-word` still uses a plain alphanumeric rule rather than consulting the syntax table. One reader-level nuance persists too — the `#(...)` *propertized-string literal* is parsed and its properties discarded, even though `propertize` and `put-text-property` build and preserve text properties at run time. So the honest summary is this: elisprs has a real, working buffer object model — named buffers, markers, narrowing, buffer-local variables, text properties, and the save-scoping forms — sufficient for the read-process-write scripting pipeline and for loading real Emacs libraries that touch buffers, and it stops deliberately at the live display and the interactive editor, which is the right line for a tool whose purpose is running `.el` as standalone programs.

With the data surface covered — values, strings, regexps, buffers — the last technical chapters turn to the tools that surround the language: the editor integration and the native compiler.

Chapter 17 – Tooling: The Language Server and the Debugger

A language that runs `.el` outside Emacs needs the things Emacs provided for free: editor intelligence and a debugger. `elisprs` ships both, as protocol servers that any modern editor can speak to. `elisp --lsp` is a Language Server Protocol server that gives completion, hover documentation, signature help, document symbols, and diagnostics; `elisp --dap` is a Debug Adapter Protocol server that gives breakpoints, stepping, and variable inspection. Both speak their protocol over stdio as JSON-RPC, and both are real, working servers — the engineering report marks them “Working” with the note “Full stdio servers.” This chapter is about what each provides and, more interestingly, how each is built, because the two make an instructive contrast: the language server works from a *static* picture of the language, while the debugger works against the *live* runtime.

The language server

The LSP server is `src/lsp.rs`, and it advertises a focused, deliberate capability set. When an editor connects, the server’s initialize response declares five features: full-document text synchronization, completion triggered on `(`, hover, document symbols, and signature help triggered on `(` and space. It does not advertise go-to-definition, find-references, rename, or formatting — the capability set is exactly the read-only intelligence that helps you write a form correctly, and nothing that rewrites your code. The transport is the `lsp-server` crate’s stdio connection, and the main loop owns a map of open documents, updating it on `didOpen` and `didChange` and dispatching each request to a handler.

The interesting design decision is where the server’s knowledge comes from. Completion, hover, and signature help all draw from a single *static corpus* — a hand-maintained table of entries, each carrying a name, a kind (special form or function), a signature, and a documentation string. There are two slices: one for the special forms and one for the subrs, the latter mirroring the `install` registration in the builtins. The server does *not* introspect the live obarray; it consults this static table. A representative entry, transcribed:

```
;; an LSP corpus entry, conceptually:
;;   name: "string-match"
;;   kind: Function
;;   sig: "(string-match REGEXP STRING &optional START)"
;;   doc: "Search STRING for REGEXP from START; set match data,
;;        return match index or nil."
```

This static-corpus choice has a clean payoff: the same table is the source for the rendered HTML reference. The Cargo manifest describes a `gen-docs` binary whose job is to render “the LSP hover/completion corpus as a static reference page,” producing `docs/reference.html`. So the documentation a programmer sees on hover in their editor and the documentation on the reference website are the same text, written once, consumed twice. There is one table of signatures and docstrings, and it feeds both the live editor and the published reference, which is exactly the kind of single-source-of-truth arrangement that keeps documentation from drifting out of sync with itself.

Completion is intentionally simple: it returns *every* entry in the corpus, with each item’s kind, signature as detail, and documentation attached, and leaves the prefix-filtering to the editor. Hover finds the symbol under the cursor, looks it up in the corpus, and renders its signature in an elisp code fence followed by its docstring. Signature help walks backward from the cursor to the innermost open paren, reads the head symbol, and shows

that function’s signature. Document symbols scans the text for `defun`, `defmacro`, `defvar`, and `defconst` forms and reports them as functions and variables, which is what populates an editor’s outline view.

Diagnostics are the one feature that does not come from the corpus, and they are deliberately shallow. The diagnostics scanner is a hand-rolled character pass that mirrors the reader’s lexical rules without invoking the compiler or the VM — it does not type-check or evaluate anything. It flags exactly three classes of problem: an unterminated string, unmatched parentheses (an extra `)` or an unclosed `(`), and backquote or unquote, which it reports as not-yet-supported in the diagnostics context. It skips comments and character literals as it scans. This is purely lexical well-formedness checking — “will the reader choke on this?” — pushed to the editor on every open and change. It is a modest feature, and the server is honest about its modesty: there is no semantic analysis, because semantic analysis would mean running the compiler on every keystroke, and the server is built to be cheap and static.

The capability set, precisely

It is worth being exact about what the server advertises, because the capability set is itself a design statement. On `initialize` the server declares five things and no more: full-document text synchronization, completion triggered on `(`, hover, document symbols, and signature help triggered on `(` and space. It does *not* advertise go-to-definition, find-references, rename, or formatting. The omissions are deliberate: those four are the features that *rewrite* or *cross-reference* code, and they would require the server to build and maintain a semantic model of the program, which is exactly the work the static-corpus design refuses. What the server offers is the read-only intelligence that helps you type a form correctly — what are the arguments to this function, what is its docstring, what functions exist — and nothing that touches the meaning of your code.

The four corpus-backed features each have a precise shape. Completion returns *every* entry in the corpus, with each item’s kind, signature, and docstring attached, and leaves the prefix-filtering to the editor — simpler and faster than computing a filtered set server-side. Hover finds the symbol under the cursor, looks it up, and renders its signature in an elisp code fence followed by its docstring. Signature help walks backward from the cursor to the innermost open paren, reads the head symbol, and shows that function’s signature, so the hint tracks which call you are inside. Document symbols is the one feature that reads the buffer rather than the corpus: it scans the text for `defun`, `defmacro`, `defvar`, and `defconst` forms and reports them as functions and variables, which is what populates an editor’s outline view. Four features off a static table, one off a lexical scan, and not a single one that runs the compiler — that is the server’s whole shape, and it is the same leverage-off-shared-machinery economy the rest of the project shows.

The debugger

The debug adapter is `src/dap.rs`, and it is the live counterpart to the static language server. Where the LSP server never runs your code, the debugger runs it for real — each top-level form compiled to a chunk and executed on `fusevm`, in-process, sharing the same host so that globals defined by one form persist to the next. The execution model is the key design fact, and it is coarser than a typical source debugger: breakpoints and stepping are at *top-level-form* granularity, not line or expression granularity. You break before a form runs, not in the middle of it. The debugger maps each top-level form to its starting line in the source, and a breakpoint on a line that begins a form stops execution just before that form is compiled and run.

The adapter handles the standard DAP request vocabulary. The initial loop handles `initialize` (advertising configuration-done, evaluate-for-hovers, and terminate support), `setBreakpoints` (recording the breakpoint line numbers), `launch` (recording the program), `configurationDone` (which kicks off the run), and `threads` (reporting a single main thread). When the run reaches a form that is on a breakpoint line, or when the debugger is stepping, it emits a `stopped` event and enters a paused loop that handles the inspection requests: `continue` resumes, `next/stepIn/stepOut` all advance to the next top-level form (they are collapsed to the same behavior, consistent with the form-level granularity), `stackTrace` reports a single synthetic frame named `toplevel`, `scopes` reports a single `Globals` scope, `variables` enumerates the global bindings, and `evaluate` runs an expression string in the live host and returns its printed value.

The `variables` and `evaluate` requests are where the debugger's *liveness* shows, because they read and run against the actual host arena. `variables` walks the arena and reports every symbol that has a bound value cell, printing each value — so the variable panel is a live snapshot of the dynamic global binding state, the real value cells from Chapter 5. `evaluate` reads, compiles, and runs the expression you type in the debug console against the live host, returning the printed result, so you can inspect and even mutate state at a breakpoint by evaluating arbitrary elisp. This is the sharp contrast with the language server: the LSP corpus is a frozen table written by hand, but the debugger's variable view and eval are reflections of the running program's actual heap.

Capturing the debuggee's output

There is one piece of plumbing in the debugger worth describing because it is a clean solution to a real problem. The debuggee runs in-process and writes its program output — `princ`, `prin1`, `print` — to standard output. But the DAP protocol *also* runs over standard output as JSON-RPC. If the debuggee printed directly to fd 1, its output would corrupt the protocol stream and break the debug session. The Cargo manifest names the fix in its dependency note: libc gives the DAP server “a pipe + `dup2` to capture the debuggee's stdout so program output streams as `output` events instead of corrupting the JSON-RPC channel.”

The mechanism, in order: the debugger saves the original stdout file descriptor, creates a pipe, and uses `dup2` to point fd 1 at the pipe's write end, so anything the debuggee prints goes into the pipe rather than to the real terminal. It keeps the *saved* original stdout as the channel for the actual DAP JSON. A reader thread drains the pipe and wraps each chunk of captured output in a DAP `output` event, which the editor displays in its debug console. When the session ends, the debugger restores the real stdout. The net effect is that the debuggee's `princ` output shows up as console output events in the editor, the DAP protocol stays clean on the saved descriptor, and `message` output — which goes to stderr, not stdout — passes through untouched. It is a small, careful piece of file-descriptor surgery, and it is the kind of detail that distinguishes a working debug adapter from a sketch of one.

The shape of the tooling

The two servers together tell you something about the project's priorities. Both speak standard protocols over stdio, so they work with any editor — VS Code, Neovim, Emacs itself — without bespoke integration. Both are scoped deliberately: the language server offers read-only intelligence from a static corpus that doubles as the published reference, and the debugger offers form-level breakpoints with live variable inspection against the real heap. Neither overreaches. The language server does not try to do semantic analysis it cannot do cheaply; the debugger does not pretend to expression-level stepping it does not have. And both reuse infrastructure — the LSP corpus feeds the docs, the debugger reuses the same compile-and-run pipeline as the rest of the runtime — which is the same economy that runs through the whole project. The tooling is built the way the language is built: small, honest, and leveraged off shared machinery.

Chapter 18 – Ahead-of-Time Compilation

The last capability is the one that most fully cashes in the frontend bet: ahead-of-time compilation to a standalone native executable. `elisp --aot-exe program.el -o program` produces a native binary that runs the program with no `elisp` interpreter present at all. This is only possible because `elisp` compiles to a `fusevm::Chunk` and `fusevm` has an AOT backend; a tree-walking `elisp` could never produce a native binary, because there would be no compiled artifact to emit. The AOT path is the frontend architecture's farthest reach, and it is also, honestly, the place with the most remaining work, so this chapter describes both what it does and where it stops.

Two AOT modes

There are two related commands. `elisp --aot file.el -o out.o` lowers the file to a `fusevm` chunk and emits a native object file. `elisp --aot-exe file.el -o program` goes the whole way: it emits the object and then links it into a standalone executable. The driver is `src/aot.rs`, and the lowering it does is real — it loads the prelude, reads the file, macro-expands every form exactly as the interpreted driver does (so macros like `dolist` and `when` are expanded before lowering rather than failing as bare macro calls), and compiles the forms to a chunk via the same `compile_program` the rest of the system uses. The macro-expansion-before-lowering step is explicitly mirrored from the interpreted path, because skipping it would leave macro calls uncompiled.

Embedding the heap image

The AOT path faces the same handle problem as the bytecode cache, and it solves it the same way: by embedding a heap image. A compiled chunk's constants are `Value::Obj` handles into the host arena, and a freshly-started AOT binary has a fresh, empty arena in which those handles mean nothing. So the AOT compiler exports the user-and-prelude heap image and embeds it in the chunk — specifically, it serializes the image to JSON and stashes it in the chunk's names table behind a sentinel tag. `fusevm`'s AOT model bincode-serializes the chunk into the object file along with a native entry point that deserializes and runs it; the embedded heap image rides along in the names table.

At load time, the standalone binary has to rebuild the heap before running. This is what `src/aot_runtime.rs` does. `fusevm`'s AOT entry, before running the chunk, calls back into the frontend through a C symbol, `fusevm_aot_register_builtins`, and `elisp`'s implementation of that hook rebuilds the heap from the embedded image and installs the two extension handlers from Chapter 10. The standalone binary is therefore exactly: the AOT object (chunk plus embedded heap image), linked against the `elisp` runtime staticlib (which carries `fusevm`'s AOT runtime and the register-builtins hook), plus a tiny C `main` that calls `fusevm`'s embedded-run entry. The Cargo manifest's `crate-type` includes `staticlib` precisely so that this linking works — so “an AOT object can be linked against the `elisp` runtime into a standalone native executable.”

The linking step is real systems plumbing: `compile_executable` writes a small C main, invokes the C compiler to link the object, the C main, and the `elisp` staticlib together, and passes the platform libraries the Rust staticlib needs — on macOS the CoreFoundation and Security frameworks and a couple of system libraries, on Linux pthread, dl, m, and rt. The result is a self-contained executable.

What the standalone binary contains

It is worth assembling the pieces, because a standalone elisprs binary is a small layered artifact and naming the layers makes the whole path legible. At the bottom is the AOT object: the bincode-serialized `fusevm::Chunk` for the program, with the embedded heap image riding in its names table behind a sentinel tag, plus a native entry point that deserializes and runs the chunk. Above that is the elisprs runtime staticlib, which carries two things the object needs at load time — fusevm’s AOT runtime (the engine that runs an embedded chunk) and the `fusevm_aot_register_builtins` hook, elisprs’s implementation of which rebuilds the heap from the embedded image and installs the two extension handlers of Chapter 10. At the top is a tiny C `main` that calls fusevm’s embedded-run entry. The C compiler links the three together with the platform libraries, and the output is one executable that needs no `elisp` binary, no prelude file, and no shared library on the target machine.

The sequence at startup mirrors the layers in reverse: the C `main` calls into fusevm’s AOT runtime, which before running the chunk calls back through the `fusevm_aot_register_builtins` C symbol into the elisprs runtime, which rebuilds the heap image and registers the handlers, after which the chunk runs against a heap in which its baked-in handles resolve. That this works at all is the deepest reach of the frontend bet: the same `fusevm::Chunk` that the interpreter runs and the cache stores is the thing the AOT backend emits as native code, and the only elisprs-specific work in the whole native path is wiring the heap image into a hook the shared backend already calls. The backend is rented; the heap is owned; the binary is the two linked together.

Where AOT stops, honestly

The AOT path works, and it is marked “Working” in the engineering report, but it carries a real, documented limitation that the source is candid about, and a book grounded in the source should be equally candid. The limitation is in *constant reification*. The embedded heap image reliably reconstructs the prelude’s objects, because the prelude is deterministic — reloading it interns the same symbols to the same handles. But a *user* program’s interned symbols, quoted data, and closure templates are not yet fully reconstructed at load time from the image alone. The `aot_runtime.rs` source states the catch plainly: a user program’s “interned symbols / quoted data / closure templates are not yet reconstructed at load — that requires constant reification (compiling symbols/quotes/closures as runtime construction instead of baked heap handles). Until then, AOT objects run correctly only for programs whose constants are all reconstructed by the prelude/builtins.”

In other words: the AOT compiler emits a real native object, links a real native binary, and runs programs whose constants come from the prelude and builtins; but a program that bakes in its own quoted literals or symbols beyond what the prelude reconstructs needs a reification pass that is identified as future work in the compiler. This is the frontier of the project — the engineering report and README both frame full native emission with JIT tiers as the next step, “essentially for free, the way they do for the sibling frontends,” once the constant reification lands. It is worth stating exactly because the project states it exactly: the lowering and linking work today, the constant model has a known gap, and the gap is named rather than hidden.

Why AOT is the architecture’s farthest reach

Even with the constant-reification caveat, AOT is the clearest proof of the frontend thesis. A language that interprets a tree has nothing to compile ahead of time; there is no artifact. elisprs, because it lowers to a `fusevm::Chunk`, has an artifact at every stage — a chunk it can run immediately, a chunk it can cache to disk, and a chunk it can emit as a native object and link into a binary. The same compiled form flows through the interpreter, the cache, and the AOT compiler, because it is the same form. And the AOT backend, like the JIT and the cache, is fusevm’s, shared with the sibling languages, so the work to make elisp AOT-compile is the work to wire elisp’s heap image into a backend that already exists — not the work to write a backend. That is the whole bet restated at its limit: own the language, rent the engine, and the engine’s native compiler becomes your native compiler. The reach is not yet complete, but the shape of it — and why it is even plausible — is entirely the consequence of being a frontend.

With the machine, the library, and the tooling all described, the final movement of the book does what the introduction promised: it reads real programs, line by line.

Chapter 19 – The Numeric Tower

Arithmetic is the part of a language people assume works and notice only when it doesn't. elisp's numeric model has a handful of commitments that are easy to get subtly wrong — integers and floats are different types, a float anywhere in an expression contaminates the result, integer division signals on zero, and rounding rounds half to even — and a faithful elisp has to honor all of them. elisprs does, and it does so while routing the hottest operations through the compiler's native fast path so that a numeric loop runs as machine code. This chapter is the numeric tower from the bottom: how a number is represented, how the two number types interact, what the rounding and transcendental and bit-twiddling families actually compute, and where the model's honest edges are.

Two number types, one tower

There are exactly two numeric types in elisprs, and they are fusevm immediates, not heap objects: `Value::Int(i64)` and `Value::Float(f64)`. An integer is a signed 64-bit machine word; a float is an IEEE 754 double. Every numeric subr begins by coercing its arguments through a single helper that the source calls `as_num`, which returns a triple — the value as an `i64`, the value as an `f64`, and a boolean saying whether the original was a float:

```
;; as_num (builtins.rs), transcribed: every numeric op funnels through this.
;; Value::Int(n)    => (n, n as f64, false)
;; Value::Float(f) => (f as i64, f, true)
;; _               => Err("wrong-type-argument: numberp ...")
```

That third boolean is the contagion flag, and it is the whole of elisp's numeric type-promotion rule made mechanical. Each arithmetic op carries an `isf` ("is float") flag that is OR-ed across all its operands, and a result helper, `num_result(i, f, isf)`, returns a `Value::Float` if any operand was a float and a `Value::Int` otherwise. So `add` keeps *both* a running `i64` sum and a running `f64` sum as it folds, and at the end picks which one to box based on whether it ever saw a float. The consequence is the rule every elisp programmer knows: a float anywhere in a chain makes the whole result a float. The reader test pins it at the smallest scale — `(+ 1.5 2)` is `3.5`, not `3` — and `stats.el` leans on it constantly, because `(* 2.0 3)` is `6.0` and its printed form keeps the `.0`.

i64, not bignum

Here is the first honest edge, and it is worth stating plainly because Emacs itself made the opposite choice. elisprs integers are `i64` and do not promote to bignums on overflow. Modern Emacs (28+) carries GMP and silently widens a fixnum to a bignum when a computation exceeds the fixnum range; elisprs does not. An integer that overflows 64 bits wraps or, in a debug build, panics, depending on the operation. The plain arithmetic ops `+`, `-`, `*` use ordinary Rust `+=` and `*=`, so they follow Rust's overflow discipline — checked (panicking) in debug, wrapping in release. A few operations are deliberately written to *wrap* rather than panic, because their domain makes wraparound the sensible behavior: the shifts `ash` and `lsh` use `wrapping_shl`, `expt` uses `wrapping_pow`, `abs` uses `wrapping_abs` (so that `(abs most-negative-fixnum)` does not panic), and the radix path of `string-to-number` uses `wrapping_mul`. The practical upshot is that elisprs is faithful to Emacs across the entire fixnum range — which is where real configuration and scripting code lives — and diverges only at the bignum frontier, a divergence the project would close by widening the value type rather than by bolting a bignum onto the side. For now the tower is a 64-bit tower, and the book marks that as the settled truth it is.

The arithmetic operators and their fast path

The core operators are subrs — `+` (min arity 0), `-` (0), `*` (0), `/` (1), `%` (exactly 2), `mod` (exactly 2), `1+` and `1-` (exactly 1). The zero-argument identities are the Common Lisp ones the reader expects: `(+)` is `0` and `(*)` is `1`. The variadic folds reduce left to right — `(- 10 3 2)` is `5`, `(/ 20 4)` is `5`, `(* 2 3 4)` is `24` — exactly as `arithmetic.el` asserts.

What makes elisp arithmetic fast rather than merely correct is the compiler's native fast path from Chapter 9. When `+`, `-`, `*`, `1+`, `1-`, and the comparisons have not been redefined, the compiler lowers them to the VM's own `Add`, `Sub`, `Mul`, `Negate`, and `Num*` opcodes instead of emitting a `CALL` to the subr. So the subr bodies in `builtins.rs` and the native opcodes in the chunk are two implementations of the same arithmetic, and they must agree. The fast path is why `1+` is lowered as `Add` of a constant `1` rather than a dedicated increment: `Add` is float-contagious the way elisp `+` is, so `(1+ 1.0)` is `2.0`, where an integer-increment opcode would have truncated it. The split is per operator, and it is worth tabulating because it is exactly the boundary between “runs as machine code” and “calls back into the host”:

```
;; native fast path (lowered to fusevm opcodes when un-redefined):  
;; + - * 1+ 1- < > <= >= =  
;; always on the host (subrs, never native):  
;; / % mod logand logior logxor ... floor ceiling round ...
```

The reason `/`, `%`, and `mod` stay on the host is semantic, not incidental. elisp's integer division truncates toward zero, its `mod` follows the sign of the divisor while `%` follows the dividend, and both *signal* on a zero divisor — and the VM's native division agrees with none of those exactly. So division is one place the fast-path trade (skip the host's wrong-type and arithmetic signaling for speed) would produce wrong answers, and the compiler declines it. `(/ 1 0)` and `(% 1 0)` raise `arith-error` from the host, which `arithmetic.el`'s `arith-errors` test asserts with `should-error ... :type 'arith-error`; float division does not signal, matching Emacs. `(mod -7 3)` is `2` — the divisor's sign — which `stats.el` checks.

Comparison chains

The comparison operators `=`, `/=`, `<`, `>`, `<=`, `>=` are variadic chains, not binary operators. `(< 1 2 3)` tests that the whole sequence is strictly increasing; `(= a b c)` tests that all three are numerically equal. `=` compares *numerically*, so `(= 1 1.0)` is `t` even though `1` and `1.0` are different types — this is the numeric equality that contrasts with `eq` and `eq1` in Chapter 23, where `(eq1 1 1.0)` is `nil`. The two-argument comparisons are the ones the compiler lowers to the VM's `NumLt`, `NumGt`, `NumLe`, `NumGe`, `NumEq`, so a tight loop's `(< i n)` test is a native compare with no callback, which is what lets the whole loop JIT.

Rounding: a family, and banker's rounding

Converting a float to an integer is not one operation but four, because there are four sensible ways to round, and elisp exposes all of them. `floor` rounds toward negative infinity, `ceiling` toward positive infinity, `truncate` toward zero, and `round` to the nearest integer. The directions matter on negatives, where they diverge, and `tests/math.rs` captures exactly that: `(floor 3.7)` is `3` but `(floor -3.1)` is `-4`; `(ceiling 3.2)` is `4`; `(truncate -3.7)` is `-3` — toward zero, not toward negative infinity. `float` goes the other way, coercing an integer to a float, so `(float 5)` is `5.0` and `(floatp (float 5))` is `t`.

`round` carries a subtlety that trips up everyone who assumes “round half up.” Like Emacs, elisprs rounds half to even — banker's rounding — so `(round 2.5)` is `2`, not `3`, and `(round 0.5)` is `0`. The source comment names it directly. Banker's rounding is the right default for accumulating sums because it does not bias upward the way round-half-up does, and matching Emacs here is the difference between a script that reproduces Emacs's numbers and one that drifts by a least-significant digit. There is also a parallel family that rounds but returns a *float* rather than an integer — `fround`, `ffloor`, `fceiling`, `ftruncate` — for when you want the rounded value to stay in the float type.

The two-argument forms of `floor`, `ceiling`, `round`, and `truncate` do a divide and round in one step and return *both* the quotient and the remainder, which the `cl-` wrappers surface as a two-element list: `stats.el` asserts `(cl-floor 17 5)` is `(3 2)` — three with a remainder of two.

Transcendentals and float internals

The float math layer is complete enough to do real numeric work, and the CHANGELOG records it arriving as a block where none existed before. The exponential and logarithmic functions are `exp`, `log` (with an optional second argument for the base), and `sqrt`; the trigonometric circle is `sin`, `cos`, `tan` and the inverses `asin`, `acos`, `atan`, where `atan` takes one or two arguments — the two-argument form is `atan2`, the quadrant-correct arctangent. `expt` is general exponentiation, integer or float, and `stats.el` checks `(expt 2 10)` is `1024`.

Below the elementary functions sits a layer of IEEE-754 plumbing that real numeric code occasionally needs: `ldexp` and `frexp` decompose and recompose a float into mantissa and exponent, `copysign` transfers a sign, `isnan` tests for NaN, and `logb` extracts the binary exponent. `logb` carries one of those exact Emacs-fidelity corner cases the project is careful about — `(logb 0)` returns the most-negative 62-bit fixnum, `-2305843009213693952`, because that is what Emacs returns for the undefined logarithm of zero. Printing floats is its own fidelity exercise, covered in Chapter 23, but the numeric payoff shows up here: `stats.el` asserts `(number-to-string (/ 1.0 4))` is `"0.25"` and `(format "%e" 12345.0)` is `"1.234500e+04"`, so the tower reads back the way Emacs writes it.

Bit operations

Integers double as bit vectors, and the bitwise family treats them as such: `logand`, `logior`, `logxor`, and the unary `lognot`, plus the shifts `ash` (the arithmetic, sign-preserving shift) and `lsh`, and the population count `logcount`. `tests/math.rs` works the truth table directly — with `12 = 1100` and `10 = 1010`, `(logand 12 10)` is `8`, `(logior 12 10)` is `14`, `(logxor 12 10)` is `6`, and `(lognot 0)` is `-1`. The variadic logical ops have the right identities: a single argument is itself, so `(logand 6)` is `6`. The shifts go both directions by the sign of the count — `(ash 1 4)` is `16` (left), `(ash 16 -2)` is `4` (right) — and `ash` preserves the sign bit on a right shift, so `(ash -8 -1)` is `-4`, an arithmetic shift rather than a logical one. These are the operations a `.el` file reaches for whenever it touches bytes, characters, or bitmask flags, and the radix reader from Chapter 3 (`#x`, `#o`, `#b`, `#NNi`) exists precisely to write their operands.

The derived numeric surface

The primitives are the floor; the prelude builds the rest in elisp on top of them. The numeric helpers everyone reaches for — `max`, `min`, `gcd`, `lcm`, `isqrt`, `abs`'s float-and-rounding companions, the predicates `zerop`, `cl-evenp`, `cl-oddp` — are prelude definitions over the primitives, exactly the bootstrapping pattern of Chapter 13. `stats.el` exercises the Common Lisp numeric vocabulary, `(cl-gcd 48 36)` is `12` and `(cl-lcm 4 6)` is `12`, and its `median` is a miniature lesson in the whole tower at once:

```
(defun median (xs)
  (let* ((s (sort (copy-sequence xs) #'<))
        (n (length s)))
    (if (cl-oddp n)
        (float (nth (/ n 2) s))
        (/ (+ (nth (1- (/ n 2)) s) (nth (/ n 2) s)) 2.0))))
```

Every numeric idea in this chapter is in those five lines: a `sort` with a numeric predicate, integer division `(/ n 2)` for indexing, the `cl-oddp` parity test, an explicit `float` coercion in the odd case so a single middle element comes back as a float, and a `/ ... 2.0` in the even case so the average is a float by contagion. That a function this small touches integer division, float coercion, contagion, and parity is the numeric tower in

working clothes — and because every operator in it is either a native fast-path op or a host subr that agrees with the fast path, it runs correctly whether the JIT has warmed the loop or not.

Chapter 20 – Lists, Sequences, and the Mapping Family

Lisp is a list processor; the name says so. The cons cell is the atom of structure, and almost everything a program builds — an association list, a tree, a queue, the argument list of a call, the body of a function — is cons cells chained together. Chapter 4 established what a cons cell *is*: an `Obj::Cons(Value, Value)` with identity, a real mutable arena slot. This chapter is about what you *do* with them: the primitive spine, the large derived list and sequence surface the prelude builds on top of it, and the higher-order mapping family that has to be threaded carefully through the host borrow because it re-enters elisp.

The cons spine

The irreducible list primitives are ten: `cons`, `car`, `cdr`, `setcar`, `setcdr`, `list`, `append`, `reverse`, `length`, and `nth`. `cons` allocates a fresh cell and returns its handle; `car` and `cdr` read the two fields; `setcar` and `setcdr` mutate them in place. The mutators are the reason the object model needed genuine identity-bearing cells, and `lists.el` makes the point its whole purpose:

```
(ert-deftest lists-mutation ()
  "In-place setcar/setcdr – impossible under an immutable value model."
  (let ((pair (cons 1 2)))
    (setcar pair 99)
    (setcdr pair 100)
    (should (equal pair (cons 99 100)))))
```

The docstring is the architectural claim restated: this test cannot pass under a value model that copies, only under one where `pair` names a slot that `setcar` reaches into. `car` and `cdr` are total on `nil` — `(car nil)` and `(cdr nil)` are both `nil`, the convention that lets list-walking code stop cleanly at the end — and signal `wrong-type-argument` on a non-list, which `arithmetic.el`'s sibling tests and `should-error (car 5)` :type `'wrong-type-argument` confirm.

`append` carries one detail worth flagging because it surprises people: it shares the tail. The final argument becomes the new list's tail *as-is*, not copied, and if that final argument is not a list the result is a dotted pair — `(append '(1 2) 3)` is `(1 2 . 3)`. This is not a bug; it is the documented Emacs behavior, and it falls directly out of building the result by consing the earlier elements onto the last argument unchanged.

The `clad+r` family and the derived list surface

Above the spine, the prelude defines the rest of the list vocabulary in elisp. The `c[ad]+r` family — `cadr`, `caddr`, `cddr`, `caar`, `cdar`, and the rest up to four levels — are compositions of `car` and `cdr`, each a one-line `defun`. The list-shape helpers are here too: `nthcdr` to drop a prefix, `last` and `butlast` for the ends, `number-sequence` to build a range, `take` to keep a prefix, `flatten-tree` to flatten, `delete-dups` to remove duplicates, `reverse` and `nreverse` to flip, and `mapconcat` to render a list as a delimited string. The `lists.el` example walks a representative slice:

```
(ert-deftest lists-derived ()
  "Prelude-defined helpers: delete-dups / number-sequence / mapconcat."
  (should (equal (delete-dups (list 1 1 2 3 3 1)) (list 1 2 3)))
  (should (equal (number-sequence 1 5) (list 1 2 3 4 5)))
  (should (equal (mapconcat 'number-to-string (list 1 2 3) "-") "1-2-3")))
```

`mapconcat` deserves a closer look because it is also, quietly, the join function the rest of the library leans on. There is no separate `string-join` written in elisp; `mapconcat` is the join, and its real prelude body coerces any sequence to a list and walks it, interposing the separator:

```
(defun mapconcat (f seq &optional sep)
  ;; SEQ may be any sequence (list/vector/string); coerce to a list of elements.
  (setq sep (or sep ""))
  (let ((l (append seq nil)) (r "") (first t))
    (while l
      (if first (setq first nil) (setq r (concat r sep)))
      (setq r (concat r (funcall f (car l))))
      (setq l (cdr l)))
    r))
```

That `(append seq nil)` idiom in the first line is the prelude's standard “coerce any sequence to a list” move — appending onto `nil` copies a list, and the same call flattens a vector or a string into a list of elements, which is how a function written for lists transparently accepts vectors and strings.

Membership and association lists

An association list — an alist — is a list of `(key . value)` cons pairs, and it is the most-used data structure in elisp configuration because it is cheap, ordered, and printable. The lookup family is `memq/member` for membership (by `eq` and by `equal` respectively), `assq/assoc` for keyed lookup returning the whole pair, `rassoc` for reverse lookup by value, and `alist-get` for the value alone. The distinction between the `q` variants and the plain ones is the equality test — `memq` and `assq` compare with `eq` (identity, fast, right for symbols), `member` and `assoc` with `equal` (structural, right for strings and lists) — and choosing the wrong one is a real bug, because `(assq "key" alist)` will silently fail to find a string key that `(assoc "key" alist)` would find. The dotted-pair reader from Chapter 3 exists precisely so that an alist literal `'((a . 1) (b . 2))` reads as pairs rather than two-element lists.

The mapping family and the borrow discipline

The higher-order functions — the ones that take a function and apply it across a structure — are `funcall`, `apply`, `mapcar`, `mapc`, `sort`, and `maphash`. These are special in the implementation in a way that ordinary subrs are not, and the reason is the thread-local host borrow from Chapter 4 and Chapter 10. An ordinary subr runs inside a single `with_host` borrow, does its work against the arena, and returns. But `mapcar` has to *call back into elisp* to apply its function argument to each element, and that callee will itself try to borrow the host — which would deadlock on the already-held borrow. So the mapping family is intercepted by name inside `call_function`, *before* any host borrow is taken, and each manages its own re-entrant calls through `call_function` rather than running as a borrowed subr. The source comment is explicit: these “are intercepted in `host::call_function` (they re-enter elisp, so they can't run inside a host borrow).”

The interception list, in the order the dispatch checks it, is `funcall`, `apply`, `mapcar`, `mapc`, `sort`, `maphash`, `eval`, the `macroexpand` family, and a function-valued `replace-regexp-in-string`, followed by the three non-local-exit intrinsics. `funcall` calls its first argument on the rest; `apply` does the same but splices its final argument as a list of trailing arguments, which is how a program spreads a computed argument list into a call. `mapcar` collects the results into a fresh list; `mapc` runs for effect and returns its list argument unchanged. `meta-eval.el`'s simplest assertion shows the pair in use — `(mapcar #'1+ '(1 2 3))` is `(2 3 4)` — and the `ert.el` example proves the higher-order machinery is just ordinary elisp:

```
(ert-deftest ert-uses-the-language ()
  "Tests are ordinary elisp - closures, recursion, higher-order all work."
  (let ((double (lambda (x) (* x 2))))
    (should (= (funcall double 21) 42)))
  (should (equal (mapcar #'1+ '(1 2 3)) '(2 3 4)))))
```

sort, in three calling conventions

`sort` is the most elaborate of the intercepted intrinsics, because it accepts three different calling conventions and an internal stable merge. The classic form is `(sort SEQ PRED)` with a two-argument predicate; the Emacs-30 keyword form is `(sort SEQ :lessp PRED :key KEY :reverse FLAG :in-place FLAG)`; and the bare form `(sort SEQ)` sorts by the default value ordering. Under the hood it is a stable merge sort — `merge_sort_by` — so equal elements keep their input order, which matters whenever the sort key is coarser than the elements. The `stats.el` median and the `text-utils.el` table renderer both sort with a custom predicate, the latter keying on a string field:

```
(defun render-table (alist)
  "Render ALIST as aligned \"key : value\" lines sorted by key."
  (let* ((sorted (sort (copy-sequence alist)
    (lambda (a b) (string< (car a) (car b)))))
    (width (seq-max (cons 0 (mapcar (lambda (kv) (length (car kv))) sorted)))))
    (mapconcat (lambda (kv)
      (format "%s : %s" (string-pad (car kv) width) (cdr kv)))
      sorted "\n"))))
```

The `(copy-sequence alist)` is defensive — `sort` may rearrange in place, so a program that wants to keep the original sorts a copy — and the rest is a pipeline: `mapcar` to extract key lengths, `seq-max` to find the column width, `string-pad` to align, `mapconcat` to join. Five library functions, four of them prelude elisp, composed into the kind of report formatting a script does constantly.

seq: one generic interface over lists, vectors, and strings

The `seq` library is the modern, type-generic sequence interface, and its design principle is that a list, a vector, and a string are all sequences and should be operable by one vocabulary. Every `seq-` function is elisp in the prelude, and they are short — `seq-reduce`, `seq-map`, `seq-filter`, and `seq-remove` are each a line or two over `mapcar`, `mapc`, and a `while` walk:

```
(defun seq-reduce (f l init)
  (setq l (append l nil))
  (while l (setq init (funcall f init (car l))) (setq l (cdr l)))
  init)
(defun seq-filter (pred l)
  (setq l (append l nil))
  (let ((r nil))
    (while l (if (funcall pred (car l)) (setq r (cons (car l) r))) (setq l (cdr l)))
    (reverse r)))
```

The `(append l nil)` coercion appears again — it is what makes `seq-filter` work on a vector or string as readily as a list. On top of these the prelude builds the broad surface: `seq-uniq` (which delegates to `delete-dups` when no custom test is given), `seq-partition` (chunking a sequence into runs of N, preserving the sequence's own type), `seq-map-indexed`, `seq-keep` (map-and-drop-nils), `seq-mapcat` (map-and-concatenate), `seq-group-by`, `seq-into`, `seq-max`, `seq-find`, `seq-take`, `seq-drop`, `seq-subseq`. `stdlib.el` walks the less obvious ones:

```
(ert-deftest std-more-seq ()
  (should (equal (seq-keep (lambda (x) (if (cl-evenp x) x nil)) (list 1 2 3 4)) (list 2
    4)))
  (should (equal (seq-mapcat (lambda (x) (list x x)) (list 1 2)) (list 1 1 2 2)))
  (should (equal (seq-group-by #'cl-evenp (list 1 2 3 4))
    (list (cons nil (list 1 3)) (cons t (list 2 4))))))
```

`seq-group-by` returns an alist keyed by the grouping function's result, and the groups appear in first-encounter order of their key — a deliberate match to Emacs's own quirk, which the example documents in a comment so that a reader does not mistake the order for an accident. The canonical `seq` pipeline, filter-map-reduce in one expression, is `text-utils.el`'s, and it is the showcase for the whole chapter: each `seq-` call compiles to a `CALL`, each lambda body to a sub-chunk the JIT can warm, and the fluent functional surface runs at compiled speed even though every function in it is elisp.

```
(should (= (seq-reduce #'+
  (seq-map (lambda (x) (* x x))
    (seq-filter #'cl-evenp (number-sequence 1 10)))
  0)
  220))
```

Chapter 21 — Tables and Records: Hash Tables, Property Lists, and the map Library

Not every lookup wants a linear scan down an alist. When a program accumulates counts, memoizes results, or indexes a large table, it reaches for a hash table; when it hangs metadata off a symbol, it uses a property list; and when it wants one interface that spans all of these plus alists, it uses the `map` library. This chapter covers the three keyed structures and the generic interface that unifies them — the data structures that, together with the cons spine, make up the working storage of real elisp.

The hash table, which is an association vector

elisp's hash table is, perhaps surprisingly, not a hash table in the data-structure sense. The `Obj::HashTable` variant is an association vector — a `Vec<Value, Value>` of key-value pairs scanned linearly — with a `test` discriminant selecting the comparison: `0` for `eq`, `1` for `eq1`, `2` for `equal`. The source is candid that this is a size-appropriate choice rather than an asymptotic one: the comment calls the linear scan “fine for the table sizes elisp config uses.” And it is — the tables a `.el` file builds are typically tens of entries, where a linear scan over a contiguous vector beats the constant factors of real hashing. The interface, though, is the full Emacs hash-table surface, so code written against a real hash table runs unchanged: `make-hash-table` (with a `:test` keyword), `gethash` (with an optional default), `puthash`, `remhash`, `clrhash`, `hash-table-count`, `hash-table-keys`, `hash-table-values`, `hash-table-test`, `hash-table-p`, and `copy-hash-table`. `maphash` — which calls a function on every pair — is, like the list mappers, intercepted in `call_function` because it re-enters elisp.

The `data-model.el` example uses a hash table the way real code does, as a counter, and reads it back with `cl-loop`:

```
(ert-deftest data-model-hash-aggregation ()
  "Tally word lengths into a hash table, then read it back with cl-loop."
  (let ((counts (make-hash-table :test 'eq1))
        (words '("a" "bb" "cc" "ddd" "ee"))))
    (dolist (w words)
      (cl-incf (gethash (length w) counts 0)))
    (should (= (gethash 2 counts) 3))
    (let ((pairs (cl-loop for k being the hash-keys of counts using (hash-values v)
                          collect (cons k v))))
      (should (= (seq-reduce #' + (mapcar #'cdr pairs) 0) 5)))))
```

Three things are happening at once. `(gethash (length w) counts 0)` reads with a default of `0`, so an absent key starts at zero; `cl-incf` of a `gethash` place increments it back into the table, which works because `cl-incf` expands to a `setf` and the struct-and-place machinery knows how to set a `gethash` place; and the `cl-loop for k being the hash-keys of counts using (hash-values v)` clause iterates keys and values in parallel — one of the `being the hash-keys` iteration forms the CHANGELOG records `cl-loop` learning. A hash-table literal can also be *read* and *printed*: `#s(hash-table test eq data (k1 v1 k2 v2))` round-trips through the reader of Chapter 3 and the printer of Chapter 4.

Property lists, hung off symbols

A property list is a flat list of alternating keys and values — `(k1 v1 k2 v2 ...)` — and its most important use is the property list every symbol carries, reached with `get` and `put`. This is the symbol's third storage cell, alongside the value and function cells of Chapter 5, and it is how elisp attaches arbitrary metadata to a name. `meta-eval.el` shows the raw operation:

```
(put 'my-sym 'color 'blue)
(should (eq (get 'my-sym 'color) 'blue))
```

The property-list mechanism is not a curiosity; it is load-bearing infrastructure the rest of the system rides on. The condition system of Chapter 14 stores each error symbol's place in the hierarchy as its `error-conditions` property and its human message as `error-message` — `define-error` is, mechanically, two `put` calls. The struct system of the next section records slot indices and parentage in properties and global alists the same way. So when this book says the obarray *is* the variable table and the function table, the property list is the third thing it is: the metadata table. `get` and `put` are the public door to it, and a great deal of elisp's reflective power — advice, font-lock keywords, custom variable declarations — is property-list manipulation underneath.

The map library: one interface, four backings

The `map` library is the generic keyed interface, and its conceit is that an alist, a plist, a hash table, and an array are all maps and should answer one vocabulary. `map-elt` is the generic getter, and its body is a `cond` over the backing type that shows the unification concretely:

```
(defun map-elt (map key &optional default testfn)
  (cond
    ((hash-table-p map) (gethash key map default))
    ((listp map)
     (let ((entry (assoc key map (or testfn #'equal))))
       (if entry (cdr entry) default)))
    ((arrayp map)
     (if (and (integerp key) (>= key 0) (< key (length map)))
         (aref map key)
         default))
    (t default)))
```

A hash-table lookup becomes `gethash`, a list lookup becomes `assoc`, an array lookup becomes a bounds-checked `aref` — one function, three implementations chosen at run time by type. On top of `map-elt` the library builds `map-keys`, `map-values`, `map-pairs`, and `map-apply` (all of which funnel through a single `map-do` traversal), and the conversion verb `map-into`, which materializes any map as a chosen concrete type. `map-into` even understands the keyword-spec form for a hash table, so you can ask for a specific test:

```
;; map--into's hash-table arm (prelude):
;; (hash-table :test TEST ...) builds a table with that test;
;; bare 'hash-table defaults to :test 'equal.
```

The `text-utils.el` example uses the library end to end, building a hash table from an alist and reading it back generically:

```
(let ((m (map-into '((a . 1) (b . 2) (c . 3)) 'hash-table)))
  (should (= (map-elt m 'b) 2))
  (should (= (seq-reduce #'+ (map-values m) 0) 6)))
```

The same three lines would work if `m` were an alist or a plist, because every call goes through the generic interface. That is the library's whole point: a function that takes “a map” does not have to know or care which of the four shapes it got, which is exactly the polymorphism a configuration system wants when a user might supply data in any of them.

Choosing among the three

The practical guidance falls out of the implementations. An alist is ordered, cheap to build by consing, trivially printable, and right up to a few dozen entries — and it is what most elisp uses by default. A plist is the same idea flattened, and its canonical home is the symbol property list and keyword-argument parsing. A hash table is what you reach for when the table is large enough that the linear alist scan would hurt, or when you want destructive in-place update semantics — though in elisprs, where the hash table is itself a linear-scan vector, the asymptotic argument is softer than in Emacs and the choice is more about interface than speed. The `map` library lets a program defer the choice, accepting whichever the caller brought. Records and structs, the fourth keyed shape, are a vector with a tag, and they are the subject of the next chapter.

Chapter 22 – Structures and Generic Functions

The two most ambitious systems the prelude builds are an object system: structures with inheritance, defined by `cl-defstruct`, and generic functions with CLOS-style multiple dispatch and method combination, defined by `cl-defgeneric` and `cl-defmethod`. Both are elisp, expanded by the macro machinery of Chapter 8 into the special forms of Chapter 7 and the primitives of Chapter 12. That an object system of this richness is *derived* — that no part of the compiler or the VM knows what a struct or a method is — is the strongest single demonstration of the macro-and-prelude architecture, and this chapter follows both systems down through their expansions.

cl-defstruct: a tagged vector and its generated functions

A struct instance is a vector whose slot zero holds a tag symbol `cl-struct-NAME` and whose remaining slots hold the field values. From a single `cl-defstruct` declaration the macro generates a family of functions: a constructor `make-NAME` that accepts keyword arguments with per-slot defaults, a predicate `NAME-p`, an accessor `NAME-SLOT` for each slot, and a copier `copy-NAME`. The accessors are setf-able — a `(setf (NAME-slot obj) v)` works — because the macro registers each accessor's slot index in a global alist `cl-struct--slot-index` that the `setf` machinery consults. The accessor generation is a straightforward loop emitting one `defun` per slot:

```
;; cl-defstruct accessor generation (prelude), per slot at index j:
;; (defun NAME-SLOT (--s--) (aref --s-- j))
;; ;; and record j in cl-struct--slot-index so (setf (NAME-SLOT x) v) works
```

Inheritance is the interesting part. A `(:include PARENT)` option prepends the parent's slots — in order, so the parent's fields occupy the same indices in the child as they did in the parent — and records the tag parentage in a global alist `cl-struct--parent`. The predicate then accepts subtypes by walking that parentage chain with a helper:

```
(defun cl-struct--is-a (tag target)
  (let ((res nil))
    (while (and tag (not res))
      (if (eq tag target) (setq res t) (setq tag (cdr (assq tag cl-struct--parent)))))
    res))
```

So `shape-p` returns true for a `circle` because `circle`'s tag chains up to `shape`'s. The `data-model.el` example builds exactly this hierarchy:

```
(cl-defstruct shape name)
(cl-defstruct (circle (:include shape)) radius)
(cl-defstruct (rect (:include shape)) width height)
```

Because `circle` includes `shape`, a circle has both a `name` slot (inherited, accessed by `shape-name`) and a `radius` slot, and it satisfies both `circle-p` and `shape-p`. The instance prints as Emacs record syntax — `#s(circle shape-name-value radius-value)` — because the printer of Chapter 4 recognizes the `cl-struct-` tagged vector and renders it that way; `type-of` returns the struct name, and `recordp` and

`cl-struct-p` recognize it. The reader's `#s(NAME ...)` macro from Chapter 3 reads that syntax back, so a struct instance round-trips through print and read, which is what lets `script-demo.el` serialize structured data to disk.

cl-defgeneric and cl-defmethod: dispatch on type

A generic function is one name with several implementations, each specialized on the types of its arguments, and the implementation selected at call time is the one whose specializers best match the actual arguments. `cl-defgeneric` declares the name; `cl-defmethod` adds a method. The `data-model.el` example defines `area` over the shape hierarchy:

```
(cl-defgeneric area (s) "Area of shape S.")
(cl-defmethod area ((c circle)) (* float-pi (circle-radius c) (circle-radius c)))
(cl-defmethod area ((r rect)) (* (rect-width r) (rect-height r)))
```

`(area c)` on a circle runs the circle method; on a rect, the rect method. The mechanism is entirely in the prelude. Methods are stored per generic name in a table `cl--generic-table` as `(QUALIFIER SPECS . FN)` entries, and a call selects the applicable methods — those whose specializers all match the argument types — and orders them by specificity. The specificity rule is the CLOS one the CHANGELOG records: a more specific specialized wins, so `integer` beats `number` and an `(eq VALUE)` specialized beats a type. The specialized vocabulary includes plain type names, `(eq V)` for matching a specific value, and `(head V)` for matching a cons by its car. A call with no applicable method signals `cl-no-applicable-method`, which is the generic-function analogue of `void-function`.

The macro expansion shows how a method becomes two things — a registration and a dispatcher:

```
;; cl-defmethod NAME ((arg TYPE)...) BODY expands (roughly) to:
;; (progn
;;   (cl--add-method 'NAME QUALIFIER (list SPECS...) (lambda (arg...) BODY))
;;   (defun NAME (first --args--) (cl--generic-dispatch 'NAME --args--))
;;   'NAME)
```

Each `cl-defmethod` adds its lambda to the table and (re)installs `NAME` as a thin dispatcher that calls `cl--generic-dispatch`, which does the selection and ordering at call time. So the generic function `area` is, after expansion, an ordinary `defun` whose body looks up the applicable method — no compiler support, no VM support, just the symbol machinery and the type predicates of Chapter 12.

Method combination

The CHANGELOG records `cl-defmethod` growing the full CLOS method-combination apparatus, and it is the part that turns “dispatch on type” into a genuine object system. A method can carry a qualifier — `:before`, `:after`, or `:around` — that changes when it runs relative to the primary method. All applicable `:before` methods run first (most specific first), then the most specific primary method, then all `:after` methods (most specific last); `:around` methods wrap the whole thing and can choose whether and when to call the rest. The engine that assembles this is `cl--chain`, which wires each method's `cl-call-next-method` to invoke the next method in the chain:

```
(defun cl-call-next-method (&rest newargs)
  (let ((args (if newargs newargs cl--cnm-args)))
    (if cl--cnm-next (funcall cl--cnm-next args) (signal 'cl-no-next-method nil)))
  (defun cl-next-method-p () (and cl--cnm-next t))
```

Inside a primary method, `cl-call-next-method` runs the next-most-specific primary method — the mechanism by which a subtype's method can extend rather than replace its supertype's — and `cl-next-method-p` tests whether there is one. Calling `cl-call-next-method` past the end of the chain signals `cl-no-next-method`. The combination logic sorts the applicable methods into the four buckets and assembles the effective

method:

```
;; cl--generic-dispatch's effective-method assembly (prelude):  
;;   before's run in order (most specific first)  
;;   primary chain (cl--chain) – first primary, cl-call-next-method to the rest  
;;   after's run in reverse (most specific last)  
;;   around's wrap the whole core, innermost-out
```

This is the same method-combination model Common Lisp's CLOS defines, implemented in a few hundred lines of prelude elisp over closures, alists, and the dispatch table. It is worth stepping back to see what has happened: starting from twenty-two special forms and a couple hundred primitives, the prelude has built structures with inheritance, generic functions with multiple dispatch, and a method-combination protocol — an object system fully the equal of the one a Lisp ships built in — without adding a single instruction to the compiler or the VM. The distance from the small core to the large surface is, as Chapter 8 promised, covered entirely by macros.

Chapter 23 – Reading, Printing, and Equality

Three operations form a closed loop at the center of any Lisp: reading turns text into a value, printing turns a value back into text, and equality asks whether two values are the same. The loop has to be coherent — what the printer writes, the reader must read back to an equal value — and equality has to come in the right grades, because “the same” means different things for a symbol, a number, and a list. Chapters 3 and 4 introduced the reader and the printer as stages of the pipeline; this chapter returns to them as a *contract*, the `prin1/read` round-trip, and adds the third corner of the loop, the `eq/eql/equal` family that the whole object model rests on.

Three grades of equality

elisp has three equality predicates, and using the wrong one is one of the most common elisp bugs. `eq` is identity: two values are `eq` if they are the same object. `eql` is `eq` plus by-value comparison of numbers of the same type. `equal` is structural: two values are `equal` if they have the same shape and `equal` components all the way down. The CHANGELOG records the fix that split `eq` and `eql` into separate subrs and made `eq` honestly identity-based on floats:

```
;; the three grades, by example:
;; (eq 'foo 'foo)      => t      ; interned to the same symbol
;; (eq 1 1)           => t      ; small integers are immediates
;; (eq 1.0 1.0)       => nil    ; distinct float objects, not the same object
;; (eql 1.0 1.0)      => t      ; same type, same value
;; (equal '(1 2) '(1 2)) => t   ; same structure
;; (eq '(1 2) '(1 2)) => nil    ; two different cons chains
```

`(eq 1.0 1.0)` is `nil` because the two floats are distinct objects, and `eql` exists precisely to paper over that — it compares floats by their bit pattern, so two floats with the same value are `eql` even though they are not `eq`. The host carries an identity-style comparison, `values_eq`, used internally for things like matching a `throw` tag against a `catch` tag; it compares integers numerically, floats by `to_bits`, and objects by arena handle, which is exactly the `eq` semantics. The user-facing `eq`, `eql`, and `equal` subrs build on the same notion of handle identity, adding by-value and structural layers on top.

Which to use is a decision with teeth. `eq` is right and fast for symbols and small integers and is what `memq/assq` use; `eql` is the right default for numbers when you might have floats; `equal` is what you need for strings and lists, and is what `member/assoc` and the ERT `should (equal ...)` assertions use. The examples are careful about it throughout — `lists.el` uses `(equal pair (cons 99 100))` to compare structure, while `demo.el` uses `(eq ... 'recovered)` to compare a symbol result.

Structural hashing

Where there is equality there must be hashing, because a hash table’s `equal` test needs a hash function consistent with `equal`. elisprs provides the `sxhash` family: `sxhash-equal` (aliased `sxhash`), `sxhash-eq`, and `sxhash-eql`, one per equality grade. The source is honest about a compatibility boundary: the hash values are *not* bit-compatible with GNU Emacs’s — they are an implementation detail — but they are internally self-consistent, which is the property that actually matters. Two objects that are `equal` hash to the same

`sxhash-equal` value, within a bounded structural depth, so the hash table's `equal` test works correctly even though a hash computed by `elisp`s would not match one computed by Emacs. This is the right contract to promise: consistency within a run, not cross-implementation byte-identity, because nothing portable should depend on a specific hash value.

Reading a value back

The reader is not only the front of the compile pipeline; it is also a runtime function a program can call. `read` and `read-from-string` parse text into a value at run time, and they are the inverse of the printer. Internally `read-from-string` is `read_one`, which returns both the parsed form and the index just past it, so a program can read successive forms from a string. `meta-eval.el` exercises the round-trip and the eval-of-read composition directly:

```
(ert-deftest meta-eval-read ()
  (should (= (eval '(+ 1 2 3) t) 6))
  (should (equal (read "(a (b c) . d)") '(a (b c) . d)))
  (should (= (eval (read "(* 6 7)") t) 42)))
```

`(read "(a (b c) . d)")` is the reader's full power in one call — nested lists and a dotted tail — producing structure `equal` to the quoted literal, and `(eval (read "(* 6 7)") t)` reads *and* runs, the classic “code is data” demonstration. The `t` second argument to `eval` requests lexical evaluation. Reading and evaluating arbitrary text at run time is what makes the debugger's `evaluate` request and the REPL possible: both read a string, compile the result, and run it.

The printer's round-trip contract

The printer's job is to be the reader's inverse: print a value so that reading the output reproduces an `equal` value. This is the `prin1` (readable) mode, distinct from `princ` (raw, human-facing) mode, and it is what `%S` selects in `format` and what `-e` uses to show a result. The round-trip contract is a set of small fidelities, each of which the examples pin down:

- A string prints with its quotes and escapes under `prin1`, raw under `princ`: `(format "%S" "x")` is `"x"`, `(format "%s" "x")` is `x`.
- A dotted pair prints with the `" . "` separator: `(1 . 2)` prints as exactly that, and reads back to the same cons.
- A float keeps its `.0` so it does not collapse to an integer: `(* 2.0 3)` prints as `6.0`, not `6`, preserving its floatness. Non-finite floats print in Emacs's read syntax, `0.0e+NaN` and `1.0e+INF`, which the reader of Chapter 3 reads back.
- A struct instance prints as `#s(NAME ...)` record syntax, a hash table as `#s(hash-table ...)`, both of which the reader's `#s(...)` macro accepts.
- A symbol whose name would not read back cleanly is escaped; the empty-named symbol prints as `##`.
- Quote forms abbreviate under `print-quoted`: `(quote x)` prints as `'x`, `(function x)` as `#'x`.

`text-utils.el` asserts the compound case in one line — `(format "%S" '(1 "two" 3.0))` is `"(1 \"two\" 3.0)"` — a list whose elements are an integer, a quoted string, and a float-with-`.0`, each printed in the form that reads back. And the full-precision float case, `(number-to-string (/ 22.0 7))` is `"3.142857142857143"`, shows the printer choosing the shortest decimal that reads back to the same double, which is the modern Emacs float-printing behavior.

Bounding the print: level and length

A self-referential or enormous structure would print forever, so the printer honors two dynamic variables that cap it. `print-level` limits how deeply nested the printer descends before it truncates a sub-structure with

..., and `print-length` limits how many elements of a list or vector it prints before truncating. Both default to “no limit” (read as nil), and both are dynamic variables a program can bind around a print to tame a large value — the same mechanism Emacs uses to keep `*Messages*` from filling with a circular list. There is also the model’s one sharp edge from Chapter 4: a handle that does not resolve to a live object prints as `#<dangling>`, the printer’s honest signal that an arena handle has outlived or mismatched its arena.

Why the loop has to close

It is worth saying why this matters beyond tidiness. The read/print round-trip is not a convenience; it is load-bearing infrastructure. The bytecode cache of Chapter 11 and the AOT heap image of Chapter 18 both depend on serializing the object heap and reconstructing it so that handles line up — a form of round-trip at the object level. The `script-demo.el` example serializes data to JSON, writes it to disk, reads it back, and asserts equality, which only works because the encode and decode agree. And `pcase`’s backquote patterns, from Chapter 8, work because the reader’s eager backquote expansion leaves a structure the pattern matcher can recognize — reader and matcher agreeing on a shape. Every one of these is the same property: the reader and the printer (or the encoder and decoder, or the serializer and the deserializer) are inverses, and a value that goes out comes back equal. A Lisp whose reader and printer disagreed would be a Lisp where you could not trust your own data, and the care this chapter describes — the dotted separator, the preserved `.0`, the escaped symbol, the record syntax, the consistent hash — is what earns that trust.

Chapter 24 – The Operating-System Surface

There is a line between a language that can compute and a language that can *do work*, and it runs through the operating system. A `.el` file that can add numbers and walk lists but cannot read a file, run a subprocess, ask the clock the time, or hash a string is a toy; a `.el` file that can do all of those is a scripting language. `elisprs` is on the working side of that line, and the reason is a layer of primitive subrs that reach past the language core into the OS. This chapter is the map of that surface — files, processes, time, and cryptography — and the recurring design note that everything in it is self-contained, reaching the system through `libc` and hand-written Rust rather than a tower of external crates.

The filesystem, read side and write side

The file primitives split cleanly into inspection, reading, and writing. The predicates answer the questions a script asks before it touches a path: `file-exists-p`, `file-directory-p`, `file-regular-p`, `file-readable-p`, `file-writable-p`, and `file-symlink-p`. The access checks route through `libc::access`, and a leading `~/` is expanded against `$HOME` by a helper the source calls `fs_expand`, so a path written the way a user writes it resolves the way a user expects. On the read side, `insert-file-contents` pulls a file's bytes into the current buffer at point, and an internal `--directory-files--` primitive lists a directory, wrapped by the prelude into the friendlier `directory-files`. On the write side, `write-region` writes a span of the buffer or a string to a file, and the mutation verbs `delete-file`, `make-directory`, `rename-file`, and `copy-file` round out the surface.

Together these close the read-process-write loop of Chapter 16. The `buffers.el` example's file round-trip is the loop at its smallest — write a payload, read it back, assert equality, clean up — and it uses `unwind-protect` to guarantee the temporary file is deleted even if the assertion fails:

```
(ert-deftest buffers-file-roundtrip ()
  "write-region then insert-file-contents reproduces the text."
  (let ((f "target/elp-buffers-example.txt")
        (payload "line one\nline two\n"))
    (write-region payload nil f)
    (unwind-protect
      (should (equal (with-temp-buffer (insert-file-contents f) (buffer-string))
                     payload))
      (delete-file f))))
```

Subprocesses

A script that cannot shell out is sealed off from the rest of the system, so `elisprs` ships the three subprocess primitives a `.el` file actually uses. `shell-command-to-string` runs a command through `sh -c` and returns its standard output as a string; `call-process` runs a program directly with an argument vector; and `process-lines` runs a program and returns its output split into a list of lines. All three are built on Rust's `std::process::Command`. The `script-demo.el` example proves the round-trip works with no Emacs in sight:

```
(ert-deftest script-demo-shell ()
  "Shell out and read the result back."
  (should (equal (string-trim (shell-command-to-string "echo elisprs")) "elisprs"))
  (should (equal (process-lines "printf" "x\ny\n") ('("x" "y")))))
```

`(shell-command-to-string "echo elisprs")` returns `"elisprs\n"`, which `string-trim` cleans to `"elisprs"`, and `(process-lines "printf" "x\ny\n")` returns the two lines as a list. That a `.el` file can launch `echo` and `printf` and read their output back is, as the project frames it, the entire promise realized: the editor's language, doing a shell script's job.

Time

Time is a small subsystem with two grammars. The clock primitives — `current-time`, `float-time`, `current-time-string`, `decode-time`, and `encode-time` — reach the system clock and the calendar through libc, calling `localtime_r` and `gmtime_r` to break a timestamp into fields and `mktime` and `timegm` to put it back together. `float-time` returns the current time as a float of seconds, the form a script uses to measure an elapsed interval; `decode-time` explodes a timestamp into a list of calendar fields; `encode-time` is its inverse.

The formatting side, `format-time-string`, deserves its own note because it is a *separate* %-engine from the `format` of Chapter 12. The two grammars are different — `format`'s `%d` is a decimal integer, `format-time-string`'s `%d` is the day of the month — so sharing a parser would have been a false economy, and the source keeps them apart. `format-time-string` implements the strftime vocabulary by hand: the date fields `%Y` `%y` `%m` `%d` `%e` `%j`, the time fields `%H` `%k` `%I` `%l` `%M` `%S`, the named fields `%a` `%A` `%b` `%B` `%p` `%P`, the zone fields `%Z` `%z`, and the composite forms `%F` `%T` `%R` `%D` `%c`, plus the literals `%n` `%t` `%%`. Each is computed from the libc-broken time fields directly, which is why `format-time-string` works in a standalone binary with no Emacs time machinery underneath. The environment primitives `getenv` and `setenv` sit alongside, reaching the process environment.

Cryptography and encoding, with no crates

The most striking thing about the crypto layer is what it does *not* depend on. The hash functions are self-contained Rust — the source comment is explicit, “secure-hash / sha1 / md5 (self-contained, no crates)” — with hand-written `sha1_bytes`, `sha256_bytes`, `sha512_bytes` (and the `sha224/sha384` truncations), and `md5_bytes` implementations and a local `to_hex`. The base64 codec is the same, with its own `B64_STD` and `B64_URL` alphabet tables. There is no `sha2` crate, no `base64` crate, no `openssl`. This is a deliberate posture that serves the single-binary goal: a tool that compiles to a standalone executable and links into an AOT binary should not drag a dependency tree behind it, and a hash function is small enough to own rather than rent.

The public surface is the Emacs one. `sha1` and `md5` hash a string or buffer region; `secure-hash` is the dispatcher that takes an algorithm name and routes to `md5`, `sha1`, `sha224`, `sha256`, `sha384`, or `sha512`, signaling an error on any other name — the accepted set is itself readable through `secure-hash-algorithms`. The encoders are `base64-encode-string` and `base64-decode-string` with their URL-safe variants `base64url-encode-string` and `base64url-decode-string`, plus the percent-encoding pair `url-hexify-string` and `url-unhex-string`. The structural-hash family `sxhash-equal/sxhash-eq/sxhash-eql` from Chapter 23 lives here too, since it is hashing of a different kind. And `random` rounds out the primitives a script reaches for, returning a pseudo-random integer.

The shape of the OS surface

Step back and the layer has a consistent character. It is *broad enough* — files, processes, time, hashing, encoding — to run a real end-to-end script, which is the bar the `script-demo.el` example sets when it writes

a file, parses it in a buffer, summarizes it, round-trips the summary through JSON on disk, and shells out to two programs, all in one ERT-tested file. And it is *self-contained* — libc for the system calls, hand-written Rust for the algorithms, no external crates for crypto or encoding — which is what lets the whole thing compile into a standalone binary that needs nothing on the target machine. The OS surface is the part of elisprs that most directly cashes the project's thesis: the editor's language, taken out of the editor, has to reach the operating system the editor used to mediate, and this layer is that reach.

Chapter 25 — Self-Contained Sub-Languages: JSON, rx, and pcase

A recurring move in the prelude is to take a small problem domain — matching structured values, writing a regular expression, serializing data — and build a *sub-language* for it: a notation with its own grammar, compiled (or interpreted) down into core forms or strings, entirely in elisp. Chapter 8 promised that the distance from the small core to the large surface is covered by macros; this chapter makes the promise concrete with three of the most self-contained examples. Each takes a different compilation target — `pcase` compiles to a `cond`, `rx` compiles to a regexp string, JSON compiles to and from text — and all three are prelude elisp with no compiler or VM support.

pcase: patterns compiled to a cond

`pcase` is structural dispatch: a scrutinee is tested against a series of patterns, and the first matching pattern's body runs, with any variables the pattern bound in scope. The surface looks primitive — like a `case` that can destructure — but it is entirely derived, and the `pcase.el` example shows the pattern vocabulary:

```
(defun classify (x)
  (pcase x
    ((pred stringp) 'string)
    ((and n (guard (and (integerp n) (< n 0)))) 'negative)
    (0 'zero)
    ((pred integerp) 'positive-int)
    ((or 'yes 'no) 'bool-symbol)
    (_ 'unknown)))
```

The macro itself is almost nothing — it binds the scrutinee once and maps each clause through a clause compiler, building a `cond`:

```
(defmacro pcase (expr &rest clauses)
  `(let ((--pcase-v-- ,expr))
     (cond ,@(mapcar (function pcase--clause) clauses))))
```

The work is in `pcase--compile`, which turns a pattern into a pair (`TESTS . BINDS`) — a list of boolean tests over the value and a list of `let*` bindings the pattern introduces. The dispatch is a `cond` over the pattern's shape: the wildcard `_` is no test and no binding; a literal is an `equal` test; a bare symbol is a binder that captures the whole value; and a list pattern dispatches on its head — `quote`, `pred`, `guard`, `and`, `or`, `cons`, and the others. The `and` and `or` arms are where the recursion shows, each compiling its subpatterns and combining their tests:

```

((eq head 'and)
 (let ((tests nil) (binds nil))
  (dolist (p (cdr pat))
   (let ((r (pcase--compile p val)))
    (setq tests (append tests (car r)))
    (setq binds (append binds (cdr r)))))
  (cons tests binds)))
((eq head 'cons)
 (let ((cr (pcase--compile (nth 1 pat) (list 'car-safe val)))
       (cd (pcase--compile (nth 2 pat) (list 'cdr-safe val))))
  (cons (cons (list 'consp val) (append (car cr) (car cd)))
        (append (cdr cr) (cdr cd)))))

```

The `cons` arm is the heart of destructuring: it tests that the value is a cons, then recursively compiles the car-pattern against (`car-safe val`) and the cdr-pattern against (`cdr-safe val`), threading the sub-tests and sub-bindings up. This is how a nested pattern like ``(,a . ,rest)` decomposes a structure — and because the reader's eager backquote from Chapter 3 already turned that template into `cons` calls, the pattern compiler can recognize it.

One implementation detail in `pcase--clause` is worth quoting because it is a scar from a real bug, the kind the project documents rather than hides. The clause builder wraps both the test and the body in a `let*` so that a `guard` in the test sees the binders, and the comment explains why the whole thing compiles to a `cond` rather than a `catch/throw`:

```

;; This `cond' shape (like `cl-case') expands cleanly when nested in a
;; macro-produced `defun' (e.g. an ERT `should'); a `catch'/'`throw' shape does
;; not — it miscompiles to a "void variable" error there.

```

That is the lexical-scope-leak-on-non-local-exit edge from Chapter 6 and Chapter 14, seen from the macro author's side: a `catch/throw`-based expansion tripped the specstack over-unwind, so `pcase` is deliberately built on `cond` to route around it. The CHANGELOG records the matching host-side fix and the pattern vocabulary growing — (`app FN PAT`) to match a pattern against a function's result, (`pred FN`) and (`app FN ...`) accepting a `lambda`, backquoted vector patterns with exact-length matching, and (`rx ...`) itself wired in as a pattern.

rx: a regexp written as an S-expression

A regular expression written as a backslash-laden string is hard to read and harder to write — the Emacs convention from Chapter 15, where grouping is `\(...\)`, makes it worse. `rx` is the antidote: a notation that writes the regexp as a nested S-expression and compiles it to a regexp string at macro-expansion time. The macro entry is a one-liner over a sequence compiler, and the clause translator dispatches on each form's head:

```

(defun rx--list (form)
  (let ((head (car form)) (args (cdr form)))
    (cond
     ((memq head '(seq sequence : and)) (rx--seq args))
     ((memq head '(or |))
      ;; Emacs folds all-single-character alternatives into a char class.
      (if (and args (rx--all-1char-p args))
          (concat "[" (mapconcat 'rx--1char args "") "]")
          (concat "\\(?:" (mapconcat 'rx--form args "\\|") "\\)")))
     ((memq head '(group submatch)) (concat "\\(" (rx--seq args) "\\)"))
     ((memq head '(zero-or-more * 0+)) (concat (rx--quant-body args) "*"))
     ((memq head '(one-or-more + 1+)) (concat (rx--quant-body args) "+"))
     ((memq head '(zero-or-one opt optional)) (concat (rx--quant-body args) "?")))))

```

So (`group (one-or-more digit)`) compiles to `\([0-9]+\)`, (`or "cat" "dog"`) to a shy-grouped alternation, and a `seq` to its concatenated parts. The `or` arm carries a genuine fidelity detail: when every alternative

is a single character, Emacs folds the alternation into a character class — `(or "a" "b" "c")` becomes `[abc]` rather than `\(?:a|b|c\)` — and `rx--list` reproduces that fold exactly. The output is an ordinary Emacs regexp string, which then flows into the Chapter 15 translator and the `fancy-regex` engine like any other pattern. The CHANGELOG notes `rx` was also wired as a `pcase` pattern, so `(rx ...)` can match a string inside a `pcase` clause — two sub-languages composed.

JSON: serialization in elisp

The JSON library is the third sub-language, and its target is text in both directions: `json-encode` turns an elisp value into a JSON string, and `json-read-from-string` parses one back. Both are written in elisp in the prelude, which is what lets a standalone `.el` script use JSON as a serialization format with no external library. The encoder maps elisp shapes to JSON shapes — an alist or plist to an object, a list or vector to an array, a number to a number, a string to a string, `t` to `true`, `nil` to `null` or `false` depending on context — and the parser reverses it, with dynamic variables like `json-object-type` and `json-key-type` controlling how a JSON object is materialized. The `script-demo.el` example round-trips data through JSON on disk:

```
(write-region (json-encode people) nil jf)
(let ((back (let ((json-object-type 'alist) (json-key-type 'string))
              (json-read-from-string
               (with-temp-buffer (insert-file-contents jf) (buffer-string))))))
    (should (= (cdr (assoc "alice" back)) 30))))
```

The `people` alist is encoded to a JSON string, written to a file, read back into a buffer, parsed with `json-object-type` bound to `alist` so the object comes back as an alist with string keys, and asserted. This is the round-trip contract of Chapter 23 applied to a foreign format: the encoder and the decoder are inverses, so data that goes out as JSON comes back as an `equal` structure. That the whole encoder and parser are elisp — not a binding to a C JSON library — is the same self-containment the crypto layer showed, and it is what makes JSON a portable capability of a standalone elisp binary.

Three notations, one architecture

The three sub-languages compile to three different targets — a `cond` of tests and bindings, a regexp string, JSON text — and yet they share one architecture: a small grammar, a translator written in elisp over the primitives, and an output the rest of the system already understands. None of them required a new special form, a new opcode, or a host change. They are the macro-and-prelude thesis at its most expressive: when the core is small and correct and the macro system is real, a new notation is a library, not a language change. The next chapter takes the single most elaborate of these notations — the Common Lisp loop facility — and gives it the full treatment, because it is the proof of the thesis at its limit.

Chapter 26 — The cl-loop Facility

`cl-loop` is the largest macro in the prelude and the most complete demonstration of what a macro can be. It is the Common Lisp iteration facility — a small, idiosyncratic sub-language of *clauses* that reads almost like English (`for i from 1 to 10 collect i`) and compiles into an efficient loop. It has a numeric iterator, several list and sequence iterators, a family of accumulators, conditional clauses, quantifiers, and a finalizer, and it parses all of that at macro-expansion time and emits a driver. Chapter 13 introduced it; this chapter gives it the full treatment it earns, because it is the single strongest argument for the architecture: a feature the size of a language subsystem, added entirely in the prelude, touching neither the compiler nor the VM.

The driver shape

Every `cl-loop` compiles to the same skeleton, which the `cl-loop.el` example's header names exactly: a `(let* ... (catch ... (while ...)))` driver. The `let*` binds the loop variables and the accumulators; the `catch` establishes the target for an early `return` or a satisfied quantifier; and the `while` is the loop itself — the one primitive loop from Chapter 7, the only loop the compiler understands. Everything `cl-loop` offers is a way of generating the right `let*` bindings, the right `while` condition and body, and the right `finally` epilogue. So the facility is, precisely, a compiler: it parses a clause stream into an intermediate picture of the loop and generates the three special forms that realize it.

The iterators

The `for` clause introduces an iteration variable, and it comes in several flavors selected by the keyword that follows. The numeric iterator is `for V from A to B`, with `below` for an exclusive upper bound, `downto/above` for descending, and `by STEP` for a stride. `cl-loop.el` walks the numeric range thoroughly:

```
(ert-deftest cl-loop-numeric ()
  (should (equal (cl-loop for i from 1 to 5 collect i) (list 1 2 3 4 5)))
  (should (equal (cl-loop for i from 0 below 5 collect i) (list 0 1 2 3 4)))
  (should (equal (cl-loop for i from 10 downto 7 collect i) (list 10 9 8 7)))
  (should (equal (cl-loop for i from 1 to 10 by 2 collect i) (list 1 3 5 7 9))))
```

The list iterators are `for V in LIST` (each element) and `for V on LIST` (each successive `cdr`). Beyond those, the CHANGELOG records the facility learning more iterators clause by clause: `for V across SEQ` for vectors and strings, `for V being [the|each] {elements|hash-keys|hash-values} of SOURCE` for hash tables and sequences, and the general `for V = INIT [then STEP]` for an arbitrary recurrence. The hash-key form is what `data-model.el` uses to read a tally back out:

```
(cl-loop for k being the hash-keys of counts using (hash-values v)
  collect (cons k v))
```

The `using (hash-values v)` sub-clause binds a second variable to the value alongside each key — parallel iteration over a hash table, expressed as one clause.

The accumulators

An accumulator clause collects a result across iterations and becomes the loop's value. `collect` builds a list, `append` and `nconc` splice lists, `sum` and `count` total, and `maximize` and `minimize` track extremes. `stats.el` runs the numeric accumulators over a range:

```
(ert-deftest stats-cl-loop-accumulators ()
  (let ((xs (number-sequence 1 10)))
    (should (= (cl-loop for x in xs sum x) 55))
    (should (= (cl-loop for x in xs maximize x) 10))
    (should (= (cl-loop for x in xs count (cl-evenp x)) 5))
    (should (equal (cl-loop for x in xs when (> x 7) collect x) '(8 9 10)))))
```

An accumulator can name its destination with `into`, which binds the accumulating variable so the loop body or the `finally` clause can see it rather than returning it directly. `cl-loop.el` shows both the plain accumulator and the `into` form, the latter handed to a `finally return`:

```
(ert-deftest cl-loop-with-into ()
  (should (= (cl-loop with total = 0 for x in (list 1 2 3) do (setq total (+ total x))
                    finally return total)
            6))
  (should (equal (cl-loop for x in (list 1 2 3) collect x into ys finally return (cons
    ~ 'done ys))
            (list 'done 1 2 3)))))
```

The `with` clause binds a loop-scoped variable before iteration begins — `with total = 0` — and `finally return` supplies the loop's value explicitly, overriding the accumulator. The `do` clause runs arbitrary body forms for effect each iteration.

Conditionals and quantifiers

A clause can be guarded. `when COND` and `unless COND` gate the clause that follows — most often an accumulator — and `if COND ... else ...` chooses between two. The `cl-loop.el` conditionals test shows the gated accumulators and the `if/else` pair:

```
(ert-deftest cl-loop-conditionals ()
  (should (equal (cl-loop for x in (list 1 2 3 4 5) when (cl-evenp x) collect x) (list 2
    ~ 4)))
  (should (equal (cl-loop for x in (list 1 2 3) if (cl-oddp x) collect x else collect (-
    ~ x))
            (list 1 -2 3)))
  (should (cl-loop for x in (list 2 4 6) always (cl-evenp x)))
  (should (cl-loop for x in (list 1 3 5) never (cl-evenp x)))
  (should (= (cl-loop for x in (list 1 2 3) thereis (and (cl-evenp x) x)) 2)))
```

The last three lines are the quantifiers, and they are why the driver has a `catch`. `always` returns `t` only if its condition holds for every iteration, short-circuiting to `nil` the moment it fails; `never` is its negation; and `thereis` returns the first non-nil value its expression produces, short-circuiting on success. Each of these has to be able to *exit the loop early* with a value, and the early exit is a `throw` to the `catch` the driver established — which is exactly why the skeleton is `catch` wrapped around `while` rather than a bare `while`. (`cl-loop ... thereis (and (cl-evenp x) x)`) returns `2`, the first even element, by throwing it the instant it is found.

A compiler in the prelude

Read `cl-loop`'s clause vocabulary as a whole — numeric and list and sequence and hash iterators, half a dozen accumulators, `into` destinations, `with` bindings, `when/unless/if/else` guards, three quantifiers,

`do` bodies, `finally return` — and what it is becomes unmistakable: a small programming language, with a parser that reads the clause stream and a code generator that emits a `let*/catch/ while` driver. And it is written in elisp, in the prelude, expanding into the same three special forms the compiler already understood before `cl-loop` existed. The CHANGELOG's history of the facility is a list of *prelude edits* — `for V across`, `for V being the hash-keys of`, `for V = INIT then STEP`, `when COND return X`, an accepted `named NAME`, destructuring `for PATTERN in LIST` — not one of which touched `compiler.rs`. That is the architecture's thesis at its limit: the Common Lisp loop facility, one of the larger pieces of machinery in any Lisp, added to elisprs without the core knowing it happened. If a sub-language this large can be a library, the core is complete — and the next chapter, the tour, watches the whole stack run in real programs.

Chapter 27 – A Tour of Real Programs

Everything to this point has been explanation. This chapter is reading. We take complete programs from the `examples` directory — every one of them an ERT-tested file that the continuous-integration system runs on every commit through the release binary on both Linux and macOS — and walk them in detail, watching the machinery from the earlier chapters appear in real code. These are not toy snippets written for the book; they are the project's own examples, self-testing and load-bearing, and reading them is the best way to see how the pieces compose.

demo.el: the whole pipeline in thirty lines

We met `demo.el` in Chapter 2, but now we can read it knowing what every line touches. It opens with four `message` calls that exercise the print and format surface:

```
(message "== elisprs demo ==")
(message "sum = %d" (+ 1 2 3 4 5))
(message "list = %S" (list 1 2 3))
(message "vec[1] = %d" (aref (vector 10 20 30) 1))
```

The first is a plain string to stderr. The second computes `(+ 1 2 3 4 5)` — and this is the native arithmetic fast path from Chapter 9, lowered to a fold of VM `Add` opcodes, no host callback — and formats it with `%d`. The third builds a real list of cons cells in the arena and prints it readably with `%S`. The fourth builds a vector, indexes it with `aref`, and formats the integer result. Four lines, and already the immediate-value arithmetic path, the cons heap, the vector backing, and the format engine are all in motion.

Then the tests. `demo-arithmetic` asserts the sum and `(1+ 41)` — another fast-path op, the `Add-with-one` lowering. `demo-cons-and-mutation` is the one that proves the object model:

```
(ert-deftest demo-cons-and-mutation ()
  "Real cons cells with identity - in-place setcar works."
  (let ((p (cons 1 2)))
    (should (equal p (cons 1 2)))
    (setcar p 99)
    (should (= (car p) 99))))
```

`(cons 1 2)` allocates a cell; `p` is bound (lexically, since this is an ordinary `let` of a non-special variable) to its handle. The first `should` checks structural equality against a fresh `(1 . 2)`. Then `setcar` mutates the cell `p` points at, in place — this is the primitive that touches the arena slot directly — and `(car p)` now reads 99. The docstring's claim, "real cons cells with identity," is the whole point: this test would be impossible under an immutable value model, and it passes because `Obj::Cons` is a genuine mutable arena cell. The file ends with `(ert-run-tests-batch-and-exit)`, which turns any failed `should` into a non-zero exit, making the file a CI gate.

arithmetic.el: the fast path under test

`arithmetic.el` is the regression gate for the compiler's native arithmetic lowering, and reading it is reading the contract the fast path must honor:

```
(ert-deftest arith-basic ()
  "Sum, product, difference, quotient, remainder."
  (should (= (+ 1 2 3 4 5) 15))
  (should (= (* 2 3 4) 24))
  (should (= (- 10 3 2) 5))
  (should (= (/ 20 4) 5))
  (should (= (% 17 5) 2))
  (should (= (+ 1 (* 2 (- 5 2))) 7)))
```

The first three operations — `+`, `*`, `-` — are the ones the compiler lowers to native VM folds. The last expression nests them, `(+ 1 (* 2 (- 5 2)))`, and the whole thing compiles to a sequence of `LoadInt` and `Add/Mul/Sub` opcodes with no callbacks — a chunk the JIT could swallow whole. But notice `/` and `%`: these are *not* on the fast path, by the deliberate design from Chapter 9, because elisp's integer division and remainder semantics differ from the VM's native division. They stay on the host as subrs. The test asserts both kinds in one place, which is exactly right: it pins the fast-path operations and the host-path operations to the same expected results, so a regression in either shows up. The `arith-errors` test then confirms the host-path operations signal correctly — `(/ 1 0)` raises `arith-error` — which the native path, skipping wrong-type signaling, could not do, another reason division stays on the host.

control-flow.el: truthiness and the loop macros

`control-flow.el` exercises the conditionals and loops, and its `classify` function is the canonical `cond`:

```
(defun classify (n)
  (cond ((< n 0) 'neg)
        ((= n 0) 'zero)
        (t 'pos)))
```

Each clause's test — `(< n 0)`, `(= n 0)`, `t` — compiles to a value, a `TRUTHY` op, and a conditional jump, with the bodies jumping past the rest on a match. The `<` and `=` are fast-path comparisons, lowered to `NumLt` and `NumEq`. The `cf-logic` test then proves the value-returning short-circuit of `and` and `or` — `(and 1 2 3)` is 3, not `t` — and `cf-loops` walks all three looping constructs:

```
(let ((i 0) (total 0))
  (while (< i 5) (setq total (+ total i)) (setq i (1+ i)))
  (should (= total 10)))
(let ((sum 0))
  (dotimes (k 4) (setq sum (+ sum k)))
  (should (= sum 6)))
```

The `while` is the primitive loop, compiled to a backward jump. The `dotimes` is a prelude macro that *expands into* a `while`. So both loops, after expansion, are the same primitive — the only loop the compiler knows — and the difference is entirely in the macro layer. The `setqs` find their variables in the enclosing lexical scope and mutate them in place, which is what makes the accumulation visible across iterations. This one example touches the whole control stack: special-form conditionals, the `TRUTHY` op, short-circuit connectives, the primitive `while`, the macro-expanded `dotimes`, and lexical `setq`.

functions.el: closures, recursion, macros

`functions.el` is the function chapter made executable. Its definitions cover the cases:

```
(defun square (x) (* x x))
(defun fact (n) (if (< n 2) 1 (* n (fact (1- n)))))
(defun fib (n) (if (< n 2) n (+ (fib (- n 1)) (fib (- n 2)))))
(defmacro my-incr (place) (list 'setq place (list '1+ place)))
```

`square` compiles its body to a sub-chunk and installs a closure in the function cell. `fact` and `fib` are recursive — and recall from Chapter 10 that each recursive call runs the function's chunk on a nested VM, so `(fact 5)` is five nested VM invocations deep at its base. The `fn-recursion` test asserts `(fact 5)` is 120 and `(fib 10)` is 55, real recursion through real nested chunks. The `fn-lambda` test proves anonymous functions are first-class values, funcalled directly and bound to a variable. And `fn-let-and-macro` proves the macro expands at compile time:

```
(let ((counter 41))
  (my-incr counter)
  (should (= counter 42)))
```

`my-incr` receives the form `counter` unevaluated, returns `(setq counter (1+ counter))`, and that expansion is what compiles and runs. The example's comment says it exactly: “my-incr expands to `(setq counter (1+ counter))` before lowering.” This single file exercises closures, sub-chunks, nested-VM recursion, first-class functions, and compile-time macro expansion — the whole function and macro story from Chapters 7 and 8.

pcase.el: structural dispatch

`pcase.el` shows the pattern matcher, a prelude macro that compiles patterns to a `cond`:

```
(defun classify (x)
  (pcase x
    ((pred stringp) 'string)
    ((and n (guard (and (integerp n) (< n 0)))) 'negative)
    (0 'zero)
    ((pred integerp) 'positive-int)
    ((or 'yes 'no) 'bool-symbol)
    (_ 'unknown)))
```

Read the patterns in order, because order is the dispatch. `(pred stringp)` matches if `stringp` returns non-nil — a predicate pattern bottoming out in the type primitive. `(and n (guard ...))` binds the value to `n` and then requires a guard expression to hold — a conjunction of a binder and a guard. `0` is a literal pattern, matching only the integer zero. `(or 'yes 'no)` matches either quoted symbol. And `_` is the wildcard, the catch-all. The `pcase` macro compiles each of these to the appropriate test-and-bind and chains them in a `cond`, so the whole `classify` becomes a single `let` binding the scrutinee and a `cond` walking the patterns. The test confirms each branch: `(classify "hi")` is `string`, `(classify -4)` is `negative`, `(classify 3.5)` is `unknown`. This is the reader's eager backquote, the macro system, the type predicates, and the special-form `cond` all cooperating to deliver a feature that looks primitive but is entirely derived.

cl-loop.el: a sub-language compiled in the prelude

`cl-loop.el` is the showcase for the most elaborate prelude macro. Its tests read like a tour of the Common Lisp loop facility:

```
(should (equal (cl-loop for i from 1 to 5 collect i) (list 1 2 3 4 5)))
(should (equal (cl-loop for x in (list 1 2 3) collect (* x x)) (list 1 4 9)))
(should (= (cl-loop for x in (list 1 2 3 4) sum x) 10))
(should (equal (cl-loop for x in (list 1 2 3 4 5) when (cl-evenp x) collect x) (list 2 4)))
(should (= (cl-loop with total = 0 for x in (list 1 2 3) do (setq total (+ total x))
                    finally return total) 6))
```

Each of these is parsed by the `cl-loop` macro at expansion time — the `for/in/collect/sum/when/with/finally` keywords are a grammar the macro reads — and compiled into a `let*-catch-while` driver, as the file's header notes. So a `cl-loop` with a numeric range and a collect

accumulator becomes a `let*` binding the loop variable and an accumulator list, wrapped in a `catch` for early exit, driving a `while`. The remarkable thing, worth repeating because it is the architecture's thesis in one example, is that this entire facility — a hundreds-of-lines clause compiler — lives in the prelude as `elisp`, expanding into the same three special forms (`let*`, `catch`, `while`) the compiler already understood. No core change was needed to add the Common Lisp loop to the language.

data-model.el: structs, generics, and a custom error hierarchy

`data-model.el` is the most feature-dense example, exercising the OO and error surfaces together. It defines a struct hierarchy with inheritance and dispatches generic functions over it:

```
(cl-defstruct shape name)
(cl-defstruct (circle (:include shape)) radius)
(cl-defmethod area ((c circle)) (* float-pi (circle-radius c) (circle-radius c)))
(cl-defmethod area ((r rect)) (* (rect-width r) (rect-height r)))
```

`cl-defstruct` generates the constructor `make-circle`, the predicate `circle-p`, and the accessor `circle-radius`, and `:include shape` makes `circle` inherit the `name` slot and satisfy `shape-p`. `cl-defmethod` installs type-dispatched methods, so `(area c)` on a circle runs the circle method and on a rect runs the rect method. The test confirms the inherited accessor (`shape-name c`) works, the subtype predicate (`shape-p c`) holds, and the dispatch picks the right method. Then it uses `pcase` for bucketing and, in a separate test, builds a *custom error hierarchy* and proves inheritance-based catching:

```
(define-error 'parse-error "Parse error")
(define-error 'bad-token "Bad token" 'parse-error)
(should (eq (condition-case e (signal 'bad-token '("x"))
      (parse-error 'caught-parent))
    'caught-parent))
```

A signalled `bad-token` is caught by a `parse-error` handler because `define-error` made it inherit. This one file touches the struct system, multiple dispatch, hash-table aggregation, `cl-loop`, `pcase`, the condition hierarchy, and `condition-case` — almost the entire derived surface, all in `elisp`, all on top of the small core.

text-utils.el and script-demo.el: real scripting

The last two examples are the ones that look like work a person would actually do. `text-utils.el` parses configuration lines, sorts and aligns them, and runs `seq` pipelines; its `render-table` sorts an alist by key, computes a column width, and joins padded lines — the kind of report-formatting a script does constantly. `script-demo.el` goes end-to-end: it writes a file, parses it in a temp buffer with `re-search-forward`, summarizes with `seq` and `cl`, round-trips the summary through JSON on disk, and shells out to subprocesses:

```
(ert-deftest script-demo-shell ()
  "Shell out and read the result back."
  (should (equal (string-trim (shell-command-to-string "echo elisprs")) "elisprs"))
  (should (equal (process-lines "printf" "x\\ny\\n") '("x" "y"))))
```

That a `.el` file shells out to `echo` and `printf` and reads their output back, with no Emacs anywhere, is the entire promise of the project realized in one test. The file's own header says it "passes under GNU Emacs too" — which is the deepest validation possible: the same program, byte for byte, runs under real Emacs and under `elisprs` and produces the same results. That cross-checking against the reference implementation is what lets the project claim fidelity rather than mere plausibility, and it is why the examples in this book can be trusted as more than illustrations. They are executable specifications, run on every commit, against two implementations.

stats.el: the numeric tower in working clothes

`stats.el` is the numeric chapter made executable, and it is a good example of how much of the tower a single ordinary function touches. Its `median` sorts a copy, indexes the middle, and is careful about types — (`float ...`) in the odd case, `/ ... 2.0` in the even case — so a median comes back as a float either way. The file's accumulator test runs the `cl-loop` math clauses over a range, and its float-printing test is the one that pins the printer's fidelity:

```
(ert-deftest stats-float-printing ()
  (should (equal (number-to-string (/ 1.0 4)) "0.25"))
  (should (equal (number-to-string (* 2.0 3)) "6.0"))
  (should (equal (format "%.3f" float-pi) "3.142"))
  (should (equal (format "%g" 0.0001) "0.0001"))
  (should (equal (format "%e" 12345.0) "1.234500e+04")))
```

Every line here is a contract from Chapter 19 and Chapter 23 at once: `(/ 1.0 4)` is a float by contagion and prints `"0.25"`; `(* 2.0 3)` keeps its `.0` and prints `"6.0"` rather than collapsing to `6`; `%.3f`, `%g`, and `%e` exercise three of the format engine's float conversions with their C-printf semantics. The `stats-integer-math` test does the same for the integer side — `(expt 2 10)` is `1024`, `(cl-gcd 48 36)` is `12`, `(cl-floor 17 5)` is `(3 2)`, `(mod -7 3)` is `2` — so the two number types are pinned together, and a regression in either the fast-path lowering or the host subr shows up as a failed assertion on the next commit.

lists.el and strings.el: identity and text

These two small files anchor the two most fundamental data types. `lists.el` exists to prove cons-cell identity, and its mutation test is the whole argument in four lines: a `(cons 1 2)`, a `setcar` and a `setcdr`, and an `equal` check against `(cons 99 100)` — a test that cannot pass under a value model that copies. Its derived test exercises the prelude list surface, `delete-dups` and `number-sequence` and `mapconcat`, each elisp over the primitives. `strings.el` does the same for text, and its join test shows that `mapconcat` is the join function, used two ways:

```
(ert-deftest strings-join ()
  "mapconcat as string join."
  (should (equal (mapconcat 'identity (list "a" "b" "c") ", ") "a, b, c"))
  (should (equal (mapconcat 'number-to-string (number-sequence 1 4) "") "1234")))
```

`mapconcat` with `identity` joins strings with a separator; with `number-to-string` and an empty separator it renders a number sequence into a single string. The file's header is a one-line statement of where strings live in the architecture: "Strings flow through fusevm as values; format/concat/mapconcat run host-side via the subr table." A string is an immediate, not a heap object, so it rides the VM directly and its operations run host-side without an arena round-trip — the small efficiency from Chapter 4 in everyday use.

errors.el: the three handler forms under test

`errors.el` is the control-flow chapter's regression gate, and it proves the happy and sad paths of all three non-local-exit forms. Its `condition-case` test asserts both that an error is recovered and that the no-error path returns the body value; its `should-error` test asserts that signals are caught and that the `:type` check discriminates the condition; and its `unwind-protect` test proves the cleanup runs even when the body errors:

```
(ert-deftest err-unwind-protect ()
  "Cleanup runs even when the body errors."
  (let ((cleaned nil))
    (ignore-errors
      (unwind-protect (error "boom")
        (setq cleaned t))))
    (should (eq cleaned t))))
```

The `(error "boom")` in the protected position throws, the `unwind-protect` runs its cleanup `(setq cleaned t)` anyway, and the surrounding `ignore-errors` swallows the re-raised error so the test can check that `cleaned` was set. This single test exercises the whole stack from Chapter 14: the compiler's rewrite of `unwind-protect` into a `--unwind--` call with `thunks`, the host intrinsic that guarantees the cleanup, the `ignore-errors` macro's `condition-case`, and the specstack unwinding that keeps the binding state consistent as control jumps out. The `should-error ... :type 'arith-error` line is the one that exercises the condition hierarchy, because it only passes if `(/ 1 0)` raised *and* the raised condition matched `arith-error`.

meta-eval.el: the metaprogramming surface

`meta-eval.el` is where the language looks at itself: macros, `eval`, `read`, `gensym` hygiene, and recursion through named-let and `cl-labels`. Its `swap!` macro is the canonical hygiene example, using `gensym` for a temporary that no source text can name:

```
(defmacro swap! (a b)
  (let ((tmp (gensym)))
    `(let ((,tmp ,a)) (setq ,a ,b) (setq ,b ,tmp))))
```

The `(gensym)` produces an uninterned symbol — distinct from any symbol the reader could produce, as Chapter 5 established — so swapping `x` and `y` can never be broken by the user happening to have a variable named `tmp`. The file's recursion test proves two forms of named recursion work, `cl-labels` for mutually-visible local functions and `named-let` for a tail-recursive loop:

```
(ert-deftest meta-recursion ()
  (cl-labels ((fib (n) (if (< n 2) n (+ (fib (- n 1)) (fib (- n 2))))))
    (should (= (fib 10) 55)))
  (should (= (named-let loop ((n 5) (acc 1))
    (if (= n 0) acc (loop (1- n) (* acc n))))
    120)))
```

`cl-labels` binds `fib` as a local recursive function — each recursive call runs the closure's chunk on a nested VM, as Chapter 10 described — and `named-let` writes a factorial as a loop that calls itself by name. Both are prelude macros expanding into the core, and both exercise the closure-and-nested-VM machinery that makes recursion work. The `meta-eval-read` test closes the metaprogramming loop, reading a dotted list back from text and evaluating read code, the demonstrations from Chapter 23 in the example suite.

regexp.el: the Emacs-regexp bridge

`regexp.el` is the proof that the Chapter 15 regexp bridge works end to end, and its two most pointed tests are the Unicode-position test and the backreference test. The position test asserts that match data counts characters, not bytes — `é` is at character index 1 even though it is two bytes — and the backreference test asserts that `\1` matches the *same* text an earlier group captured, the defining property of a real backreference:

```
(ert-deftest regexp-backreferences ()
  "Backreferences \\1..\\9 match the same text an earlier group captured."
  (should (= (string-match "\\(\\.\\)\\1" "abccba") 2))
  (should (null (string-match "\\(\\.\\)\\1" "abcdef"))))
  (should (equal (replace-regexp-in-string "\\b\\(\\w+\\) \\1\\b" "\\1"
    "the the cat cat sat")
    "the cat sat"))
  (should (= (string-match "\\(ab\\)\\1+" "xabababy") 1)))
```

`\\(\\.\\)\\1` matches a doubled character, finding the `cc` in `"abccba"` at index 2 and failing entirely on `"abcdef"` where no character doubles; the replacement form collapses doubled words by matching `\\(\\w+\\) \\1` and replacing with the single capture, turning `"the the cat cat sat"` into `"the cat sat"`. This only works because the move to `fancy-regex` gave the engine a backtracking fallback, and the example is the authority that settles the documentation's own historical disagreement about whether backreferences are supported: they are, in the version this book describes, and the CI runs this test on every commit.

ert.el: the framework testing itself

The last file in the tour is the one that closes the loop on the whole self-testing discipline. `ert.el` is a demonstration of ERT that is itself ERT-tested, and its two most interesting tests are the one proving tests are ordinary elisp and the one documenting a known limitation as an expected failure:

```
(ert-deftest ert-known-limitation ()
  "An expected failure (XFAIL) documents a not-yet-implemented feature without
  reddening the suite."
  :expected-result :failed
  (should (bufferp (current-buffer))))
```

The `:expected-result :failed` marks the test as an XFAIL: it asserts something that does *not* yet work — `(bufferp (current-buffer))`, the named-buffer object model from Chapter 16's edges — and the batch runner counts it as expected rather than failing the suite. This is the project documenting its own boundary inside the test suite, the most honest place to put a limitation: not in a roadmap that can drift, but in a test that will start passing the day the feature lands and can then have its `:expected-result` removed. The unit tests in `tests/ert.rs` confirm the runner's bookkeeping precisely — an XFAIL that fails counts zero, an XFAIL that *passes* counts one (an unexpected success is itself a failure), and a passing test alongside an XFAIL keeps the count at zero — which is the meta-level guarantee that the framework every other example relies on is itself correct.

buffers.el: the temp-buffer pipeline

`buffers.el` is the buffer chapter made executable, and its three tests trace the read-process-write loop end to end. The motion test walks point through a multi-line buffer; the file round-trip writes and reads a payload; and `upcase-words-via-buffer` is the search-and-replace loop in miniature, searching for runs of letters and replacing each with its upcased form in place. The `collect-numbers` helper is the parsing idiom every buffer-scanning script uses — search repeatedly until the search fails, collecting each match:

```
(defun collect-numbers (text)
  "Return all integers appearing in TEXT, in order."
  (with-temp-buffer
    (insert text)
    (goto-char (point-min))
    (let (nums)
      (while (re-search-forward "[0-9]+" nil t)
        (push (string-to-number (match-string 0)) nums))
      (nreverse nums))))
```

The `(re-search-forward "[0-9]+" nil t)` advances point past each match and sets the buffer-relative match data, `(match-string 0)` slices the matched digits out of the buffer, `string-to-number` parses them, and the `while` loops until the search returns `nil`. Every primitive in it — `with-temp-buffer`, `insert`, `goto-char`, `re-search-forward`, `match-string`, `push`, `nreverse` — is in the library, and the whole function is the parsing half of a script that reads a file and pulls structure out of it. This is the model Chapter 16 described, doing real work in fifteen lines.

cl-defun.el: full lambda lists

`cl-defun.el` is the showcase for the Common Lisp lambda list, the keyword-and-default argument machinery the special-form `lambda` does not provide. Its `greet` takes a required positional, a `&key` argument with a default, and a boolean flag:

```
(cl-defun greet (name &key (greeting "Hello") loud)
  (let ((s (concat greeting ", " name)))
    (if loud (upcase s) s)))
```

`(greet "Ada")` is "Hello, Ada" — the default greeting; `(greet "Ada" :greeting "Hi")` is "Hi, Ada" — the keyword override; `(greet "Ada" :loud t)` is "HELLO, ADA" — the flag taking effect. The mechanism, from Chapter 13, is that `cl-defun` desugars to a plain `&rest` function whose body is a `cl-structuring-bind` that pulls the real parameters — keywords, defaults, and all — out of the rest list. The example's `cl-structuring-and-values` test exercises the binder directly, proving `&optional` and `&key` defaults work and that the multiple-values forms return lists. The file's header names the constraint that makes it an example script rather than a unit test: a macro defined at top level is what makes it available to the forms that follow, so a file that defines `greet` with `cl-defun` and then calls it has to be run as a whole, form by form — exactly the top-level processing of Chapter 8.

stdlib.el: the derived surface, exercised

`stdlib.el` is the broadest single example, walking the prelude's derived surface — `subr-x`, `seq`, `cl-lib` — the way Emacs's own `subr.el` would be exercised. Its `std-macros` test runs the threading and binding macros, its `std-more-seq` test runs the less-obvious `seq` functions, and its `std-cl-case` test runs `cl-case` and the `while-let` loop:

```
(ert-deftest std-cl-case ()
  (should (eq (cl-case 2 (1 'one) (2 'two) (t 'other)) 'two))
  (should (eq (cl-case 'b ((a b) 'ab) (t 'no)) 'ab))
  (let ((acc 0) (n 3))
    (while-let ((x (if (> n 0) n nil)))
      (setq acc (+ acc x)) (setq n (1- n)))
    (should (= acc 6))))
```

`cl-case` dispatches on a value against literal keys — `2` matches the `(2 'two)` clause, and `'b` matches the `((a b) 'ab)` clause because the key is a *list* of values to match — and `while-let` loops as long as its binding stays non-nil, accumulating `3 + 2 + 1` to `6`. Every construct in the file is a prelude macro or function over the small core, and the file's header states the architecture in one line: these "are written in elisp in the prelude, on top of the host primitives — the way Emacs bootstraps `subr.el`." That a single example file can walk threading macros, sequence functions, `cl-case`, and `while-let` and have them all pass is the prelude doing its job, and the CI running this file on every commit is the proof that it keeps doing it.

What the tour shows

Reading these programs in sequence reveals the layering directly. The arithmetic lowers to native ops; the conditionals lower to TRUTHY-and-jump; the loops are `while` with macros on top; the functions are closures with sub-chunk bodies running on nested VMs; the structs and generics and loop facility and pattern matcher are all elisp in the prelude expanding into a small set of special forms; the errors are a condition hierarchy over a Rust signal channel; and the whole thing reaches the filesystem and subprocesses through the primitive sublayer. Every example is a different cross-section of the same machine, and the machine is the one the book has described: a reader and a compiler that own the elisp semantics, lowering everything to a `fusevm::Chunk` that runs on a borrowed engine. The examples are the proof that the architecture is not just elegant but *sufficient* — that a small core, a macro system, and a borrowed VM are enough to run real Emacs Lisp outside of Emacs.

Chapter 28 – The Catalogue: Numbers and the Math Library

The first twenty-seven chapters were the *why*: the architecture, the machine, the systems, walked as an argument. The chapters that follow are the *what*, walked as a systematic reference — a catalogue of the primitive layer, family by family, every entry carrying the signature the language server reports on hover, its precise behavior, its edges, and an example you can run. This is the same corpus that feeds the editor’s completion and generates `docs/reference.html`, turned into prose you can read front to back, and it is grounded in `builtins.rs` and the ERT tests that gate every commit. Where a behavior is asserted in a test, the catalogue says so; where it is faithful Emacs behavior the project deliberately reproduces, the catalogue says that too. We begin where every language begins, with numbers.

The four operations and their identities

The arithmetic operators are variadic folds. `(+ &rest NUMBERS)` returns the sum, `(* &rest NUMBERS)` the product, and each has the Common Lisp identity for the empty case — `(+)` is `0` and `(*)` is `1` — so they compose cleanly in a `reduce`. `(- &rest NUMBERS)` is overloaded by arity: with one argument it negates, `(- 5)` is `-5`; with more it subtracts the rest from the first, `(- 10 3 2)` is `5`. `(/ DIVIDEND &rest DIVISORS)` divides left to right, and it is the first operator with a sharp edge: integer division *truncates toward zero*, so `(/ 7 2)` is `3`, not `3.5`, and `(/ -7 2)` is `-3`. To get a float quotient, make an operand a float — `(/ 7 2.0)` is `3.5` — because a float anywhere makes the whole fold a float by the contagion rule of Chapter 19. Dividing an integer by zero signals `arith-error`; dividing a float by zero does not, following IEEE-754 and Emacs.

```
(+ 1 2 3 4 5)      ; => 15
(* 2 3 4)          ; => 24
(- 10 3 2)         ; => 5
(/ 20 4)           ; => 5
(/ 7 2)            ; => 3      (integer truncation)
(/ 7 2.0)          ; => 3.5    (float by contagion)
```

The three folding operators `+`, `-`, `*` are the ones the compiler lowers to native VM opcodes when they have not been redefined, so a tight loop full of them runs as machine code with no callback into the host. `/` stays on the host, by the deliberate design of Chapter 9, precisely because its truncation and zero-signaling differ from the VM’s native division.

Remainder and modulo

Two operators compute remainders, and the difference between them is the sign rule. `(% X Y)` is the integer remainder whose sign follows the *dividend* `X`, so `(% 17 5)` is `2` and `(% -17 5)` is `-2`. `(mod X Y)` is the modulo whose sign follows the *divisor* `Y`, so `(mod -7 3)` is `2`, a positive result, which `stats.el` asserts. This is exactly the C-versus-mathematical-modulo distinction, and elisp gives you both so you can pick the one your algorithm needs — `%` for “the remainder of a division,” `mod` for “the representative in `[0, Y)`,” the latter being what you want for wrapping an index around a ring. Both signal `arith-error` on a zero divisor in the integer case; `mod` of floats checks for a zero divisor too.

Increment, decrement, and the comparison chain

`(1+ NUMBER)` and `(1- NUMBER)` add and subtract one, and they are not mere conveniences — they are the canonical loop-counter steps, and the compiler lowers each as a native `Add/Sub` of a constant `1` rather than a dedicated increment opcode. The reason is contagion: `Add` is float-preserving, so `(1+ 1.0)` is `2.0`, where an integer increment would have truncated the float. The comparison operators `=`, `<`, `>`, `<=`, `>=` are *chains*, not binary operators: each takes any number of arguments and tests the whole sequence. `(< 1 2 3)` is true because the sequence strictly increases; `(<= 1 1 2)` is true because it is non-decreasing; `(= a b c)` is true only if all three are numerically equal. The numeric `=` compares *across types*, so `(= 1 1.0)` is `t` even though `1` and `1.0` are not `eql` — a distinction that matters constantly and is the reason Chapter 23 separates numeric equality from object equality. The two-argument comparisons are the ones the compiler lowers to the VM's `NumLt`, `NumGt`, `NumLe`, `NumGe`, `NumEq`, so a loop's `(< i n)` test is a native compare.

Float conversion and the rounding family

`(float ARG)` coerces a number to a float, so `(float 5)` is `5.0` and `(floatp (float 5))` is `t`. Converting the other way — float to integer — is four operations, because there are four sensible roundings. `(floor ARG &optional DIVISOR)` rounds toward negative infinity, `(ceiling ...)` toward positive infinity, `(truncate ...)` toward zero, and `(round ...)` to the nearest integer. On negatives they diverge, and `tests/math.rs` pins the divergence: `(floor 3.7)` is `3` but `(floor -3.1)` is `-4`; `(truncate -3.7)` is `-3`, toward zero, not toward negative infinity. `round` rounds half to even — banker's rounding — so `(round 2.5)` is `2` and `(round 0.5)` is `0`, matching Emacs and avoiding the upward bias of round-half-up. Each of the four takes an optional `DIVISOR`, dividing and rounding in one step and returning the quotient; the `cl-` wrappers expose the remainder too, so `(cl-floor 17 5)` is `(3 2)`. A parallel family — `fround`, `ffloor`, `fceiling`, `ftruncate` — does the same rounding but returns a *float*, for when the rounded value should stay in the float type.

```
(floor 3.7)      ; => 3
(floor -3.1)     ; => -4
(ceiling 3.2)    ; => 4
(truncate -3.7)  ; => -3
(round 2.5)       ; => 2      (banker's rounding)
(fround 2.5)     ; => 2.0    (rounded, but a float)
```

The bitwise and shift operators

Integers double as bit vectors. `(logand &rest INTS)`, `(logior &rest INTS)`, and `(logxor &rest INTS)` are the variadic bitwise AND, inclusive OR, and exclusive OR; `(lognot NUMBER)` is the unary one's complement. `tests/math.rs` works the truth table — with `12 = 1100` and `10 = 1010`, `(logand 12 10)` is `8`, `(logior 12 10)` is `14`, `(logxor 12 10)` is `6`, and `(lognot 0)` is `-1`. The variadic forms have the right identities: `(logand 6)` with a single argument is `6`. `(ash VALUE COUNT)` is the arithmetic shift — left for positive `COUNT`, right for negative — and it preserves the sign bit on a right shift, so `(ash -8 -1)` is `-4`. `(lsh VALUE COUNT)` is the logical shift, registered as an alias of `ash`. `(logcount VALUE)` is the population count: the number of one bits in a non-negative value, or of zero bits in a negative one. These are the operators a `.el` file reaches for whenever it manipulates bytes, character codes, or bitmask flags, and the radix reader `(#x, #o, #b)` exists to write their operands legibly.

The transcendental library

The float math layer is complete enough for real numeric work. `(expt BASE EXPONENT)` is general exponentiation, integer or float — `(expt 2 10)` is `1024`. `(sqrt ARG)` is the square root, `(exp ARG)` is *e* raised to ARG, and `(log ARG &optional BASE)` is the natural logarithm or, with a second argument, the logarithm

to that base. The trigonometric circle is `(sin ARG)`, `(cos ARG)`, `(tan ARG)` in radians, with the inverses `(asin ARG)`, `(acos ARG)`, and `(atan Y &optional X)` — where the two-argument form is `atan2`, using the signs of both arguments to pick the correct quadrant. None of these is behind an import; they are primitive subrs, present the moment the host starts, and `stats.el` uses them for real statistics.

IEEE-754 plumbing

Below the elementary functions sits a layer of float-representation primitives that numeric code occasionally needs. `(ldexp SGNFCAND EXPONENT)` builds a float as `SGNFCAND * 2**EXPONENT`, and `(frexp X)` is its inverse, returning `(SIGNIFICAND . EXPONENT)` with the significand normalized to `[0.5, 1)`. `(copysign X1 X2)` returns the magnitude of `X1` with the sign of `X2`. `(isnan X)` tests whether a float is a NaN. And `(logb ARG)` returns the binary exponent — the floor of the base-2 logarithm of the magnitude — with the Emacs-faithful corner case that `(logb 0)` returns the most-negative 62-bit fixnum, `-2305843009213693952`, the conventional value for the undefined logarithm of zero. These are the workhorses underneath a serious numeric routine, and their presence is what lets `stats.el`'s distribution and variance code read like the textbook rather than like a workaround.

Randomness and conversion

`(random &optional LIMIT)` returns a pseudo-random integer; with a positive integer `LIMIT` it returns a value in `[0, LIMIT)`, the form you use to pick a random index. `(abs ARG)` is the absolute value, written with a wrapping negation so that taking the absolute value of the most-negative fixnum does not panic. The two conversion primitives bridge numbers and their text: `(number-to-string NUMBER)` renders a number as a string — `(number-to-string (/ 1.0 4))` is `"0.25"`, and an integer-valued float keeps its `.0`, so `(number-to-string (* 2.0 3))` is `"6.0"` — and `(string-to-number STRING &optional BASE)` parses the number at the front of a string in the given radix, defaulting to base 10. Together they are the round-trip a script does whenever it reads a number from text or writes one back, and the printer's shortest-decimal float rendering, from Chapter 23, is what makes that round-trip exact.

Chapter 29 — The Catalogue: Predicates, Equality, and Types

Before a program transforms a value it usually has to ask what the value *is*, and elisp answers with a dense layer of predicates — functions that return non-`nil` for “yes” and `nil` for “no.” This chapter catalogues the type tests, the three grades of equality, and the reflective predicates that ask about functions, macros, and variables. They are the floor of every dispatch in the language: `pcase`’s `pred` patterns, `cl-defmethod`’s type specializers, and `cond`’s tests all bottom out here.

The three grades of equality

elisp has three equality predicates, and choosing the wrong one is one of the most common bugs in the language. `(eq A B)` is identity: non-`nil` when A and B are the same object. For interned symbols and small integers — which are immediates — `eq` is reliable, so `(eq 'foo 'foo)` and `(eq 1 1)` are both `t`. But two distinct floats are not the same object, so `(eq 1.0 1.0)` is `nil`. `(eql A B)` closes that gap: it is `eq` plus by-value comparison of numbers of the same type, so `(eql 1.0 1.0)` is `t` while `(eql 1 1.0)` is still `nil` because the types differ. `(equal A B)` is structural: non-`nil` when A and B have the same shape and `equal` components all the way down, so `(equal '(1 2) '(1 2))` is `t` even though the two lists are different cons chains. The rule of thumb the examples follow: `eq` for symbols and small integers and the `memq/assq` family; `eql` for numbers when floats are possible; `equal` for strings, lists, and vectors, and for every ERT `should (equal ...)` assertion.

Negation and the nil tests

`(null OBJECT)` and `(not OBJECT)` are the same function — `not` is registered as an alias of `null` — both returning `t` exactly when their argument is `nil`. They read differently by intent: `null` asks “is this the empty list / false value,” `not` negates a boolean, but mechanically they are one subr. `(zerop NUMBER)` tests for numeric zero, true for both `0` and `0.0`, and it is the idiomatic “is this zero” rather than `(= n 0)`.

The type tests

The type predicates form a complete partition of the value space. `(consp OBJECT)` is true for a cons cell, `(atom OBJECT)` is its exact negation — true for everything that is *not* a cons, including `nil`, symbols, numbers, and strings. `(listp OBJECT)` is true for a cons or `nil`, because `nil` is the empty list, which is why a list-walking loop tests `listp` or `consp` at each step. The leaf-type tests are `(symbolp OBJECT)`, `(stringp OBJECT)`, `(numberp OBJECT)`, `(integerp OBJECT)`, `(floatp OBJECT)`, and `(vectorp OBJECT)`, each true for exactly its type; `numberp` is the union of `integerp` and `floatp`. `(char-or-string-p OBJECT)` is true for a character (which is an integer) or a string, the test a function uses when it accepts either. And `(type-of OBJECT)` names the type as a symbol — `integer`, `float`, `string`, `symbol`, `cons`, `vector`, `hash-table`, or a struct’s name — which is the reflective primitive `cl-defmethod` dispatch and `pcase` type checks consult.

```

(cons ' (1 . 2)) ; => t
(atom 5)         ; => t
(listp nil)      ; => t      (nil is the empty list)
(numberp 3.5)    ; => t
(integerp 3.5)   ; => nil
(type-of "x")    ; => string
(type-of [1 2])  ; => vector

```

Records and callables

Two predicates recognize struct instances: `(recordp OBJECT)` and its alias `(cl-struct-p OBJECT)` are both true for a record — a `cl-defstruct` instance, which is a tagged vector — so a generic function can ask whether it received a struct before dispatching on its exact type. The callable predicates are a small reflective family. `(functionp OBJECT)` is true for anything callable — a subr, a closure, or a symbol with a function binding. `(subrp OBJECT)` is true only for a primitive (a built-in subr), `(macrosp OBJECT)` only for a macro, and `(special-form-p OBJECT)` only for one of the twenty-two special forms. These three carve the callable space into its three kinds, and they are how introspective code — a documentation generator, an advice system — tells a primitive from a user function from a macro.

The reflective variable and character predicates

`(special-variable-p SYMBOL)` asks whether a symbol has been declared special by `defvar` or `defconst` — whether, by Chapter 6's routing, a `let` of it would bind dynamically rather than lexically. `(boundp SYMBOL)` and `(fboundp SYMBOL)`, catalogued fully in the next chapter, test whether the value and function cells are non-void. On the character side, `(char-uppercase-p CHAR)` tests whether a character is uppercase, and `(char-equal C1 C2)` compares two characters for equality, case-insensitively when `case-fold-search` is non-nil — the one predicate here whose answer depends on a dynamic variable, which is itself a small lesson in how much of elisp's behavior is parameterized by the dynamic binding the language is built on.

Chapter 30 — The Catalogue: Cons Cells, Lists, and Sequences

The cons cell is the atom of Lisp structure, and the list built from cons cells is the language's default container. This chapter catalogues the primitive list operations — the ten subrs that build, read, mutate, and measure cons structure — and then the large derived list-and-sequence surface the prelude layers on top, because in practice a program reaches for both without noticing the seam. The primitives are the spine; the prelude is the muscle.

Building and reading cons structure

`(cons CAR CDR)` allocates a fresh cell in the arena and returns its handle, with the two fields holding any values at all — which is what makes a dotted pair `(cons 1 2)`, whose cdr is not a list, as real as a proper list. `(car LIST)` and `(cdr LIST)` read the two fields, and both are *total on nil*: `(car nil)` and `(cdr nil)` are both `nil`, the convention that lets a list walk stop cleanly at the end without a guard. On a non-list they signal `wrong-type-argument`, which is why `(car 5)` raises and `should-error (car 5) :type 'wrong-type-argument` passes. `(list &rest OBJECTS)` builds a proper list of its arguments, the everyday constructor, and `(nth N LIST)` returns the Nth element, zero-indexed, or `nil` past the end.

Mutating cons structure

`(setcar CELL VALUE)` and `(setcdr CELL VALUE)` are the mutators, and they are the reason the object model needed genuine identity-bearing cells. `setcar` overwrites the car field of the very cell its argument points at, in place, so every other reference to that cell sees the change — the property `lists.el` exists to prove:

```
(let ((pair (cons 1 2)))
  (setcar pair 99)
  (setcdr pair 100)
  (equal pair (cons 99 100))) ; => t
```

This is impossible under a value model that copies and trivial under one where a cons is a mutable arena slot. The mutators are the foundation of in-place data structures — a queue that appends by `setcdr` on its tail pointer, a destructive sort that relinks cells — and they are the concrete payoff of Chapter 4's insistence on real cons cells.

Length, reversal, and concatenation

`(length SEQUENCE)` returns the number of elements, and it is generic over the three sequence types: a list's length is its element count, a vector's is its size, a string's is its *character* count (not its byte count, which matters for multibyte text). `(reverse SEQUENCE)` returns a reversed *copy*, leaving the original intact, while the prelude's `nreverse` reverses destructively by relinking — the choice between them is the choice between safety and allocation. `(append &rest SEQUENCES)` concatenates lists into one, and it carries the tail-sharing subtlety from Chapter 20: the final argument becomes the result's tail as-is, not copied, so `(append '(1 2)`

3) is the dotted (1 2 . 3) and (append seq nil) is the idiom for copying a list or coercing any sequence to one.

```
(length "héllö")      ; => 5      (characters, not bytes)
(reverse '(1 2 3))    ; => (3 2 1)
(append '(1 2) '(3 4)) ; => (1 2 3 4)
(append '(1 2) 3)     ; => (1 2 . 3) (tail shared, dotted)
```

The derived list surface

On top of the ten primitives the prelude defines the rest of the list vocabulary in elisp. The `c[ad]+r` family — `cadr`, `caddr`, `cddr`, `caar`, `cdar`, and the rest up to four levels — are compositions of `car` and `cdr`, each a one-line definition; they read a path into nested structure, so `cadr` is the second element and `cddr` drops two. The shape helpers are `nthcdr` to drop a prefix of N, `last` and `butlast` for the ends, `take` to keep a prefix, `flatten-tree` to flatten arbitrary nesting, `delete-dups` to remove duplicates by `equal`, and `number-sequence` to build a range. `mapconcat` renders a list as a delimited string — and, lacking a separate elisp join, it *is* the join function, coercing any sequence to a list and interposing a separator. `lists.el` walks a representative slice — (`delete-dups (list 1 1 2 3 3 1)`) is (1 2 3), (`number-sequence 1 5`) is (1 2 3 4 5), (`mapconcat 'number-to-string (list 1 2 3) "-"`) is "1-2-3".

Association lists and membership

An association list is a list of (key . value) pairs, and its lookup family is keyed by equality grade. `memq` and `member` test membership — by `eq` and by `equal` respectively — returning the tail beginning at the found element or nil. `assq` and `assoc` look up a key and return the whole matching pair — again by `eq` and by `equal` — and `rassoc` looks up by *value* instead of key. `alist-get` returns just the value for a key, with an optional default, and is the modern accessor. The *q*-versus-plain distinction is the same equality choice as everywhere: `assq` for symbol keys (fast, identity-based), `assoc` for string or list keys (structural), and getting it wrong means a string key silently not being found. The dotted-pair reader exists so an alist literal '((a . 1) (b . 2)) reads as pairs.

The seq library: one interface over three types

The `seq` library is the modern generic sequence interface, treating lists, vectors, and strings uniformly, and every `seq-` function is elisp in the prelude over the primitives. The transformers are `seq-map`, `seq-filter`, `seq-remove`, and `seq-reduce`; the searches are `seq-find`, `seq-some`, `seq-every-p`, `seq-position`, `seq-contains-p`; the slicers are `seq-take`, `seq-drop`, `seq-subseq`; and the resapers are `seq-uniq`, `seq-partition` (chunking into runs of N, preserving the sequence's type), `seq-group-by` (an alist keyed by a function, in first-encounter order), `seq-map-indexed`, `seq-keep` (map-and-drop-nils), and `seq-mapcat` (map-and-concatenate). `seq-into` converts between sequence types and `seq-max/seq-min` find extremes. The canonical pipeline is the filter-map-reduce that `text-utils.el` runs in one expression — squaring the even numbers from 1 to 10 and summing them to 220 — and because each `seq-` call compiles to a `CALL` and each lambda to a sub-chunk the JIT can warm, the fluent functional surface runs at compiled speed even though every function in it is elisp.

Chapter 31 – The Catalogue: Symbols, Binding, and Evaluation

A symbol is the most semantically loaded object in elisp — the carrier of the Lisp-2, the key of the obarray, the holder of value, function, and property cells — and the operations on symbols are how a program reads and writes those cells, interns names, and reaches into the binding machinery from elisp. This chapter catalogues the symbol and binding primitives and the four evaluation primitives that turn data into action.

Naming and interning

`(symbol-name SYMBOL)` returns a symbol's print name as a string, and `(intern NAME)` is its inverse — returning the canonical interned symbol for a name, creating one if none exists, so that two `intern`s of the same name yield the same object and `(eq (intern "foo") 'foo)` is `t`. `(intern-soft NAME)` is the non-creating lookup: it returns the interned symbol if one already exists and `nil` otherwise, the right tool for *testing* whether a name is known without the side effect of creating it. `(make-symbol NAME)` deliberately does *not* intern — it builds a fresh, uninterned symbol every call, so `(eq (make-symbol "g") (make-symbol "g"))` is `nil`, which `tests/builtins.rs` asserts. That distinction is the foundation of `gensym` and macro hygiene: an uninterned symbol cannot be named by any source text, so a macro's temporary can never collide with a user's variable.

The value cell

`(set SYMBOL VALUE)` writes a symbol's value cell, the dynamic counterpart to the lexical `setq`, and `(symbol-value SYMBOL)` reads it, signaling `void-variable` if the cell is void. `(boundp SYMBOL)` tests whether the value cell is non-void — true after a `set` or `defvar`, false for a name never assigned — and `(makunbound SYMBOL)` voids it again, returning the symbol. `(special-variable-p SYMBOL)` reports whether the symbol has been declared special by `defvar/defconst`, which is what routes a `let` of it to the dynamic specstack rather than the lexical scope. These five are the elisp-visible window onto the value-cell half of the Lisp-2, and the dynamic-binding example from the test suite shows them cooperating with `let`: `(defvar *dyn* 1)` then `(let ((*dyn* 42)) (rd))` returns `42`, because the dynamic binding is visible to `rd` through the value cell during the `let`'s extent.

The function cell

`(fset SYMBOL DEFINITION)` writes a symbol's function cell and `(symbol-function SYMBOL)` reads it, returning the closure, subr, or alias symbol it holds, or `nil` if void. `(fboundp SYMBOL)` tests whether the function cell is non-void. `(indirect-function OBJECT &optional NOERROR)` follows a function-cell alias chain — a symbol whose function cell holds another symbol — to the underlying callable, the operation that resolves `defalias` indirection, bounded at 64 hops. The arity of any callable is readable with `(func-arity FUNCTION)`, which returns `(MIN . MAX)` where `MAX` may be the symbol `many` for a variadic function; `(subr-arity SUBR)` is its alias for primitives. These are the reflective operations a program uses to inspect and rewrite the function namespace — advice systems, mocking, dynamic dispatch tables built by hand — all by reading and writing function cells directly, because in a Lisp-2 the obarray *is* the function table.

The four evaluation primitives

Four subrs turn data into action. `(eval FORM &optional LEXICAL)` compiles and runs a form at run time, the “code is data” primitive — `(eval (read "(+ 1 2)"))` is 3, and the optional `LEXICAL` argument (passed as `t`) requests lexical evaluation. `(funcall FUNCTION &rest ARGS)` calls a function on explicit arguments — `(funcall (make-adder 10) 5)` is 15, calling a closure returned from another function. `(apply FUNCTION &rest ARGS LIST)` is `funcall` with its final argument *spliced* as a list of trailing arguments, the primitive for spreading a computed argument list into a call. And `(identity ARG)` returns its argument unchanged — trivial, but the canonical “do nothing” function you hand to a higher-order operation when you want it to pass elements through, as `mapconcat`’s `identity` join shows. All four are intercepted in `call_function` before the host borrow, because they re-enter elisp and must not run inside a borrow held by their caller.

Chapter 32 – The Catalogue: Strings, Characters, and Format

Text is what most real elisp moves around, and the string layer is correspondingly broad. This chapter catalogues the string and character primitives — construction, inspection, slicing, case folding, comparison, conversion — and then the `format` engine, which is its own small formatting language and one of the most-used functions in any program. Throughout, the unifying fact from Chapter 4 holds: a string is a fusevm immediate, so string data flows through the VM directly and these subrs operate on it host-side, and everything counts in *characters*, not bytes.

Construction and decomposition

`(concat &rest STRINGS)` is the workhorse constructor, concatenating its arguments into one string. `(make-string COUNT CHARACTER &optional MULTIBYTE)` builds a string of COUNT copies of a character — `(make-string 3 65)` is "AAA", since 65 is `?A` — and `(string &rest CHARACTERS)` assembles a string from individual character codes. Going the other way, `(string-to-list STRING)` and `(string-to-vector STRING)` explode a string into a list or vector of its character codes, and `(char-to-string CHAR)` and `(string-to-char STRING)` bridge a single character and a one-character string — `(string-to-char "A")` is 65, the integer, because a character *is* an integer.

Slicing and searching

`(substring STRING &optional FROM TO)` extracts a substring by character index, and it accepts negative indices that count from the end — `(substring "hello world" 0 5)` is "hello" and `(substring "hello" -3)` is "llo", both asserted in `eval.rs`. Reaching past the end signals `args-out-of-range`, which `condition-case` can catch to yield the condition `(args-out-of-range "abc" 0 10)`. `(split-string STRING &optional SEPARATORS OMIT-NULLS TRIM)` divides a string on a separator regexp, defaulting to whitespace and omitting empty fields — `(split-string "a b c")` is ("a" "b" "c") — and it is the front of nearly every parsing pipeline. `(string-search NEEDLE HAYSTACK &optional START)` returns the character index of a literal substring, or `nil` — `(string-search "lo" "hello")` is 3 — distinct from the regexp `string-match` of Chapter 35 in that it is a plain, fast literal search.

Case, comparison, and distance

`(upcase OBJ)` and `(downcase OBJ)` fold case, and they are polymorphic over a string or a single character — `(upcase "abc")` is "ABC" and `(upcase ?a)` is `?A` (65 → 97... rather, 97 → 65). `(string-prefix-p PREFIX STRING &optional IGNORE-CASE)` and `(string-suffix-p SUFFIX STRING ...)` test the ends — `(string-prefix-p "foo" "foobar")` is `t` — and `(string-empty-p STRING)` tests for the empty string. `(compare-strings STR1 START1 END1 STR2 START2 END2 &optional IGNORE-CASE)` does a fine-grained comparison of substrings, returning `t` for equal or a signed mismatch position. And `(string-distance STRING1 STRING2 &optional BYTECOMPARE)` is the Levenshtein edit distance, the fuzzy-matching primitive behind spell-check and record linkage. The prelude adds the lexicographic comparators `string</string-lessp`, `string>`, and `string=/string-equal` on top of these. `(string-join`

`STRING$ &optional SEPARATOR`) joins a list of strings with a separator — `(string-join (split-string "a, b ,c" " " *, *)) "|")` is `"a|b|c"` — the named join alongside `mapconcat`'s general one.

The format engine

`(format STRING &rest ARGS)` is a complete formatting language compressed into one subr. Its directives follow the C-printf grammar `%[flags][width][.prec]CONV`, and the conversions are `s` (a value in `princ`/raw form), `S` (in `prin1`/readable form), `d` (signed decimal), `o/x/X` (octal and hex), `c` (a character), and `e/f/g` (float in exponential, fixed, and general notation). The flags are `-` (left-justify), `0` (zero-fill), `+` (force sign), space (leading space on non-negatives), and `#` (alternate form). `text-utils.el` exercises a battery in one line — `(format "%05.2f|%+d|%x|%c" 3.14159 7 255 ?A)` is `"03.14|+7|ff|A"` — and `tests/builtins.rs` pins the sign-before-zeros rule: `(format "%05d" -42)` is `"-0042"`, the sign kept ahead of the zero padding. The `%S` conversion renders re-readably, so `(format "%S" '(1 "two" 3.0))` is `"(1 \"two\" 3.0)"`, the round-trip contract of Chapter 23 inside a format string. `message` and `error` and `user-error` all route through this same engine, differing only in where the result goes — `message` to `stderr`, `error` into a signalled condition.

Chapter 33 – The Catalogue: Vectors, Hash Tables, and Records

Beyond cons cells, elisp has three more compound structures: the vector (a flat, fixed-length, mutable array), the hash table (a keyed store), and the record (a tagged vector that backs structs). This chapter catalogues all three, because together with lists they are the working storage of every real program, and each has a small, precise primitive surface.

Vectors

A vector is a fixed-length array with constant-time indexing, read from `[1 2 3]` literals or built by subr. `(vector &rest OBJECTS)` builds one from its arguments, and `(make-vector LENGTH INIT)` builds one of a given length with every slot the same initial value. `(aref ARRAY IDX)` reads the `IDX`'th element and `(aset ARRAY IDX VALUE)` writes it in place — `(progn (setq v (make-vector 3 0)) (aset v 1 9) v)` is `[0 9 0]`, the in-place mutation that distinguishes a vector from a list. `(fillarray ARRAY ITEM)` sets every slot to the same value, and `(vconcat &rest SEQUENCES)` concatenates sequences into a fresh vector — note it accepts any sequences, so it can build a vector out of lists and strings. `aref` and `aset` are also the operations on strings-as-arrays and on records, since all three are vector-backed, which is why `(setf (aref v 1) 7)` works through the generalized-place machinery. The reader's `[a b c]` is self-evaluating with *unevaluated* elements, so a bare vector is a literal, and `vectorp` and `length` recognize and measure it.

Hash tables

`(make-hash-table &rest KEYWORD-ARGS)` creates an empty hash table, accepting `:test` (one of `eq`, `eq1`, `equal`) and `:size`; the implementation is the association vector of Chapter 21, a linear-scan store sized for the tens-of-entries tables elisp config actually builds. `(gethash KEY TABLE &optional DEFAULT)` reads, returning `DEFAULT` (or `nil`) when the key is absent — `(gethash 'missing (make-hash-table) 'default)` is `default` — and `(puthash KEY VALUE TABLE)` writes. `(remhash KEY TABLE)` removes a key and `(clrhash TABLE)` empties the table. The inspection family is `(hash-table-count TABLE)`, `(hash-table-keys TABLE)`, `(hash-table-values TABLE)`, `(hash-table-test TABLE)`, and `(hash-table-p OBJECT)`, and `(copy-hash-table TABLE)` makes a shallow copy. `maphash` — catalogued among the higher-order functions — iterates the table, and the counter idiom `(cl-incf (gethash key counts 0))` reads with a default of zero and writes the increment back, which is how `data-model.el` tallies word lengths. A hash table both reads and prints as `#s(hash-table ...)`, round-tripping through the reader and printer.

```
(let ((h (make-hash-table :test 'equal)))
  (puthash "a" 1 h)
  (gethash "a" h))                ; => 1

(let ((h (make-hash-table)) (s 0))
  (puthash 1 10 h) (puthash 2 20 h)
  (maphash (lambda (k v) (setq s (+ s v))) h)
  s)                               ; => 30
```

Records

A record is a tagged vector — slot zero holds a type tag, the rest hold fields — and it is the representation `cl-defstruct` instances use. `(recordp OBJECT)` and its alias `(cl-struct-p OBJECT)` recognize one, and `(type-of OBJECT)` on a record returns its type symbol rather than `vector`, because the tag identifies it. The struct machinery of Chapter 22 generates the constructor, predicate, and accessors over this representation, and the accessors are `aref` reads at fixed slot indices, made setf-able by the slot-index registry — `(setf (sf-x p) 99)` and `(cl-incf (sf-y p) 10)` both work, as `eval.rs` asserts, because the accessor is a generalized place. The record prints as `#s(NAME ...)` and reads back through the `#s(...)` reader macro, the same round-trip the hash table enjoys, which is what lets a struct instance be serialized to disk and restored — the property `script-demo.el` leans on when it persists data.

Chapter 34 — The Catalogue: Printing, Reading, and Output

A program's contact with the outside world, on the value side, is printing and reading: turning a value into text and text back into a value. This chapter catalogues the output primitives — which differ by where they write and whether they quote — and the reader subs that parse text into structure at run time, closing the read/print loop of Chapter 23 at the level of individual functions.

princ, prin1, print, and terpri

The output primitives split on two axes: readable versus raw, and where the newline goes. `(princ OBJECT)` writes a value in raw, human-facing form — a string without its quotes — to standard output. `(prin1 OBJECT)` writes it in *read* syntax, re-readable — a string *with* its quotes and escapes — so that `prin1` output fed back to `read` reproduces the value. Both return the printed value as their result, distinct from the stdout side effect, which is why `(prin1 'sym)` evaluates to `sym` while also writing `sym`. `(print OBJECT &optional STREAM)` is `prin1` surrounded by newlines, the quick-and-dirty debugging print, and `(terpri &optional STREAM)` outputs a single newline. `(prin1-to-string OBJECT)` is `prin1` captured into a string rather than written, the function you use to *get* the readable text of a value without emitting it. The crucial split, from Chapter 2, is that these program-output functions go to standard output, while `message` goes to standard error — so a script's real result and its progress chatter land on different file descriptors, which the debugger exploits to keep program output from corrupting the protocol stream.

read and read-from-string

`(read STRING)` parses and returns one Lisp object from a string — the run-time face of the reader, the inverse of `prin1`. `(read-from-string STRING &optional START END)` parses one object and returns a cons of the object and the index just past it, so a program can read successive forms by feeding the next index back in: `(read-from-string "(a b)")` is `((a b) . 5)` and `(read-from-string "42 foo")` is `(42 . 2)`, the `2` pointing at the space after `42`. That second return value is what makes a read loop possible. Reading composes with `eval` to run text as code — `(eval (read "(+ 1 2)"))` is `3` — the demonstration that in a Lisp the reader and the evaluator are the whole front end, reachable from within the language itself. The same reader handles dotted pairs, vectors, the `#s(...)` record and hash-table syntax, and the radix and character literals of Chapter 3, so anything `prin1` writes, `read` reads back to an `equal` value.

Chapter 35 – The Catalogue: Regular Expressions and Match Data

Regular expressions are how elisp dissects text, and the regexp surface is two layers: the matching subrs that run a pattern against a string, and the match-data subrs that read back where the match and its groups landed. This chapter catalogues both, against the backdrop of Chapter 15's central facts — Emacs regexps invert the metacharacter convention (grouping is `\(...\)`), the engine is `fancy-regexp` so backreferences work, and match positions are reported in *character* indices.

Matching against a string

`(string-match REGEXP STRING &optional START)` searches a string for a pattern from an optional start index, sets the match data as a side effect, and returns the character index where the match began, or `nil` — `(string-match "world" "hello world")` is 6. `(string-match-p REGEXP STRING &optional START)` is the same search but *preserves* the existing match data, the variant you use when you only want a yes/no and must not clobber an outer match in progress. `(regexp-quote STRING)` escapes a string's regexp metacharacters so it matches literally — `(regexp-quote "a.b*c")` is `"a\\.b\\*c"` — the function you call before splicing user text into a pattern. `(save-match-data &rest BODY)` runs its body while shielding the caller's match data from any search the body performs, the macro that makes a function safe to call from inside another function's match loop.

Reading the match data

When a match succeeds, the positions of the whole match (group 0) and each parenthesized subgroup (groups 1, 2, ...) are recorded, and four subrs read them. `(match-beginning N)` and `(match-end N)` return the character positions where the Nth subexpression began and ended, and crucially these are *character* positions even when the engine works in bytes — `(progn (string-match "é" "aébé") (list (match-beginning 0) (match-end 0)))` is `(1 2)`, the `é` at character index 1 though it is two bytes. `(match-string N &optional STRING)` returns the text the Nth subexpression matched, slicing it out of the subject. `(match-data)` serializes all the positions to a flat list `(beg0 end0 beg1 end1 ...)`, and `(set-match-data LIST)` restores them — the pair that lets a program save, manipulate, and restore match state, as `predicates-io.rs` shows with `(set-match-data '(2 5))` then `(match-end 0)` returning 5.

Backreferences and replacement

The defining test of a real backreference is that `\1` matches the same text an earlier group captured, and elisprs passes it: `(string-match "\\(\\.\\)\\1" "abccba")` is 2, finding the doubled `cc`, because the move to `fancy-regexp` gave the engine a backtracking fallback. `(replace-regexp-in-string REGEXP REP STRING &optional FIXEDCASE LITERAL)` is the substitution workhorse: it replaces every match of a pattern, with the replacement template understanding `&` for the whole match and `\N` for the Nth group unless `LITERAL` is set. The group-swap idiom is `(replace-regexp-in-string "\\([a-z]+\\)=\\([0-9]+\\)" "\\2:\\1" "x=1 yy=22")` → `"1:x 22:yy"`, swapping each `key=value` into `value:key`. Above all of this sits the `rx` macro from Chapter 25, which builds the backslash-heavy pattern strings from readable

S-expressions, and the buffer search subrs of the next chapter, which run the same engine against buffer text rather than a string argument.

Chapter 36 – The Catalogue: Buffers and Text Motion

The buffer is a vector of characters and a point, and the buffer subrs are how a script reads, navigates, searches, and edits that text. This chapter catalogues the full buffer surface — far broader than the two-field model suggests — because it is the read-process-write substrate every standalone text-processing `.el` program runs on, the working vocabulary behind `with-temp-buffer`.

Insertion, extent, and content

`(insert &rest ARGS)` inserts strings or characters at point, the primitive that fills a buffer; `(insert-char CHARACTER &optional COUNT INHERIT)` inserts repeated copies of one character. `(buffer-string)` returns the whole buffer as a string and `(buffer-substring START END)` (with its `-no-properties` variant) returns a span, the readers that pull text back out. `(buffer-size &optional BUFFER)` is the character count. The deletion verbs are `(delete-region START END)`, `(delete-char N &optional KILLFLAG)`, and `(erase-buffer)`, which empties it. The two internal primitives `--buffer-push--` and `--buffer-pop--` are the machinery `with-temp-buffer` brackets with `unwind-protect`, pushing a fresh scratch buffer and discarding it.

Point and position

`(point)` returns the current position as a one-based integer, and `(point-min)` and `(point-max)` the accessible bounds. The line-position queries `(line-beginning-position &optional N)` / `(pos-bol &optional N)` and `(line-end-position ...)` / `(pos-eol ...)` return positions without moving point, while the predicates `(bolp)`, `(eolp)`, `(bobp)`, and `(eobp)` test whether point is at a line or buffer boundary. `(current-column)` and `(line-number-at-pos &optional POS ABSOLUTE)` report the column and line number, and `(count-lines START END)` counts lines in a span — the function `buffers.el`'s `wc-lines` uses over `(point-min)` and `(point-max)`. `(char-after &optional POS)` and `(char-before &optional POS)` read the characters adjacent to a position, returning `nil` at the buffer ends.

Motion

`(goto-char POSITION)` sets point absolutely; the relative motions are `(forward-char &optional N)` and `(backward-char ...)` by characters, `(forward-line &optional N)` by lines (returning the count of lines it could not move, the idiom for detecting end-of-buffer), `(beginning-of-line ...)` and `(end-of-line ...)`, and `(forward-word &optional N)` and `(backward-word ...)` by words — where a word, in this model, is a run of alphanumerics, with no syntax-table refinement. `(skip-chars-forward STRING &optional LIM)` and `(skip-chars-backward ...)` move point over a set of characters and return the distance moved, the fine-grained scanner for hand-written parsers.

Searching and replacing in a buffer

(`search-forward` `STRING` &optional `BOUND NOERROR COUNT`) and (`search-backward` ...) do literal searches that leave point at the match; (`re-search-forward` `REGEXP` ...) and (`re-search-backward` ...) do regexp searches, setting the match data so the groups can be read with `match-string`. (`looking-at` `REGEXP` &optional `INHIBIT-MODIFY`) tests whether text at point matches a pattern (setting match data), and (`looking-at-p` `REGEXP`) does so without disturbing it. (`replace-match` `NEWTEXT` &optional `FIXEDCASE LITERAL STRING SUBEXP`) substitutes at the last match. The canonical parsing loop is `re-search-forward` driving a `while` until it returns `nil`, collecting each match — the shape `script-demo.el` uses to parse `NAME AGE` lines and `buffers.el` uses to collect every integer in a text. These buffer searches run the same translated `fancy-regex` engine as the string `string-match` of the previous chapter; the only difference is that the subject is the buffer and point advances past each match.

Chapter 37 — The Catalogue: Files, Processes, Time, and Crypto

This chapter catalogues the operating-system surface — the primitives that reach past the language into the filesystem, the process table, the clock, the environment, and the cryptographic algorithms. It is the layer that separates a toy from a scripting language, and its defining engineering trait is self-containment: `libc` for the system calls, hand-written Rust for the algorithms, no external crates.

File inspection and manipulation

The file predicates answer the questions a script asks before it acts: `(file-exists-p FILENAME)`, `(file-directory-p FILENAME)`, `(file-regular-p FILENAME)`, `(file-readable-p FILENAME)`, `(file-writable-p FILENAME)`, and `(file-symlink-p FILENAME)` — the last returning the link target rather than `t`. Access checks route through `libc::access`, and a leading `~/` expands against `$HOME`. The content operations are `(insert-file-contents FILENAME &optional VISIT BEG END REPLACE)`, which reads a file into the current buffer at point, and `(write-region START END FILENAME &optional APPEND ...)`, which writes a buffer span or a string out. The mutation verbs are `(delete-file FILENAME &optional TRASH)`, `(make-directory DIR &optional PARENTS)`, `(rename-file FILE NEWNAME ...)`, and `(copy-file FILE NEWNAME ...)`. Alongside these host primitives, the prelude provides a large *pure* path-manipulation library — `file-name-directory`, `file-name-nondirectory`, `file-name-extension`, `file-name-sans-extension`, `file-name-base`, `file-name-concat`, `expand-file-name`, `file-relative-name`, and `file-name-split` — that operates on path *strings* without touching the filesystem, which is why a script can compute a path before deciding whether to read it. The read-process-write loop these compose into is the whole point: `script-demo.el` writes a `people.txt`, reads it back in a temp buffer, and asserts the round-trip.

Subprocesses

`(shell-command-to-string COMMAND)` runs a command through `sh -c` and returns its standard output as a string; `(call-process PROGRAM &optional INFILE DESTINATION DISPLAY &rest ARGS)` runs a program directly with an argument vector and returns its exit status; and `(process-lines PROGRAM &rest ARGS)` runs a program and returns its output split into a list of lines. The three together let a `.el` file orchestrate other programs — `(string-trim (shell-command-to-string "echo elisprs"))` is `"elisprs"` and `(process-lines "printf" "x\ny\n")` is `("x" "y")` — which is the editor's language doing a shell script's job.

Time and environment

The clock primitives reach `libc` through `localtime_r`, `gmtime_r`, `mktime`, and `timegm`. `(current-time)` returns the time as a `(HIGH LOW USEC PSEC)` list, `(float-time &optional TIME)` returns seconds since the epoch as a float (the form for measuring an interval), and `(current-time-string ...)` returns a human string. `(decode-time &optional TIME ZONE)` explodes a timestamp into calendar components and `(encode-time ...)` reassembles them. `(format-time-string FORMAT &optional TIME ZONE)`

is a *separate* %-engine from `format`, implementing the strftime vocabulary — `%Y %m %d %H %M %S %a %A %b %B %p %z %F %T` and the rest — because date formatting is a different grammar from value formatting. The environment pair is `(getenv VARIABLE)` and `(setenv VARIABLE &optional VALUE ...)`, reaching the process environment.

Cryptography, encoding, and hashing

The hash functions are self-contained Rust — no `sha2`, no `openssl`. `(sha1 OBJECT ...)` and `(md5 OBJECT ...)` hash a string or buffer region to a hex string, and `(secure-hash ALGORITHM OBJECT ...)` dispatches on an algorithm name across the SHA family — `md5`, `sha1`, `sha224`, `sha256`, `sha384`, `sha512`, all hand-written — signaling on an unsupported one, and `(secure-hash-algorithms)` returns the accepted set. The base64 codec is `(base64-encode-string STRING &optional NO-LINE-BREAK)` and `(base64-decode-string STRING ...)` with the URL-safe variants `(base64url-encode-string ...)` and `(base64url-decode-string ...)`, each over its own alphabet table; the percent-encoding pair is `(url-hexify-string STRING ...)` and `(url-unhex-string STRING ...)`. The structural-hash family `(sxhash-equal OBJ)` (aliased `sxhash`), `(sxhash-eq OBJ)`, and `(sxhash-eql OBJ)` hashes values by equality grade — not bit-compatible with Emacs, but internally consistent so that `equal` objects hash alike, which is what the `equal` hash table needs. Together this layer is what lets a standalone elisp binary hash, encode, and checksum with nothing linked in but its own runtime.

Chapter 38 – The Catalogue: Control, Errors, and Higher-Order Functions

The last catalogue chapter covers the operations that direct control rather than move data: the error-signalling subrs, the non-local-exit `throw`, and the higher-order functions that take a function and apply it across a structure. These are the subrs most entangled with the runtime, because several of them re-enter elisp and so are intercepted in `call_function` before any host borrow is taken.

Signalling and non-local exit

`(error FORMAT &rest ARGS)` signals an error whose message is a formatted string — the everyday “something went wrong” — and `(user-error FORMAT &rest ARGS)` signals one marked as a user’s mistake rather than a program bug, which tools can present differently. `(signal ERROR-SYMBOL DATA)` is the low-level primitive both build on: it raises the condition named by a symbol with an associated data list, and the condition system of Chapter 14 dispatches on the symbol’s `error-conditions` hierarchy. The data list is preserved through the handler, so `(condition-case e (signal 'arith-error '(1 2)) (error e))` binds `e` to `(arith-error 1 2)`. `(throw TAG VALUE)` is the labeled escape, paired with the `catch` special form — it unwinds straight to the matching `catch`, which returns the thrown value, and `(catch 'x (+ 1 (throw 'x 99))) 999` is `99` because the throw aborts the enclosing expression mid-evaluation. A throw with no matching catch signals `no-catch`, so a stray throw fails loudly rather than silently.

The higher-order functions

Four higher-order subrs apply a function across a structure, and all four are intercepted by name in `call_function` because they call back into elisp and must not hold the host borrow while doing so. `mapcar` applies a function to every element of a sequence and collects the results into a fresh list; `mapc` does the same for effect, returning its sequence argument unchanged. `sort` orders a sequence by a predicate — a stable merge sort, accepting the classic `(sort SEQ PRED)`, the Emacs-30 keyword form `(sort SEQ :lessp PRED :key KEY ...)`, and the bare `(sort SEQ)` — and it sorts a vector to a vector and a list to a list, keeping equal elements in input order, so `(sort (vector 5 3 1 4 2) #'<)` is `[1 2 3 4 5]`. `maphash` iterates a hash table’s key-value pairs. Alongside these, `funcall` and `apply` from Chapter 31 are the direct callers, and `eval`, the `macroexpand` family, and a function-valued `replace-regexp-in-string` round out the intercepted set — the complete list of operations that re-enter the language. The interception is the visible cost of separating the object heap from the VM: re-entrant calls thread through one choke point so the thread-local host borrow stays short and non-nested, the discipline that buys the whole frontend separation.

```
(mapcar #'1+ '(1 2 3))           ; => (2 3 4)
(sort (vector 5 3 1 4 2) #'<)    ; => [1 2 3 4 5]
(catch 'x (+ 1 (throw 'x 99))) 99 ; => 99
(condition-case e (car 5) (error e)) ; => (wrong-type-argument listp 5)
```

Chapter 39 – The Prelude Library: `cl-lib` in Full

The catalogue so far has been the floor — the subrs written in Rust. This chapter and the next walk the *building*: the derived library written in elisp in `src/prelude.rs`, a single 3,711-line raw string loaded from form on top of the primitives. The prelude’s own header states the philosophy exactly — it is “the large `[DERIVED]` surface ... written in elisp on top of the Rust primitives. This is how the port gets breadth without hand-coding a thousand subrs — exactly how Emacs bootstraps `subr.el`.” Everything in it “uses only primitives or earlier prelude definitions, and every macro is defined before its first use,” which is the bootstrapping discipline that lets a standard library be written in its own language. The prelude declares, in its `features` variable, exactly which Emacs libraries it claims to provide: `emacs`, `cl-lib`, `cl-macs`, `cl-seq`, `cl-extra`, `seq`, `subr-x`, `map`, `rx`, `pcase`, `gv`, and `ert`. This chapter takes the largest of them, the Common Lisp compatibility layer.

The sequence functions

`cl-lib` brings the Common Lisp sequence vocabulary that a great deal of real elisp depends on, and it is implemented over `mapcar`, `while`, and the keyword-argument parser `cl--getkey`. The filtering pair `cl-remove-if` and `cl-remove-if-not` (with the destructive `cl-delete-if/-not`) drop or keep elements by a predicate, honoring a `:count` limit:

```
(defun cl-remove-if (pred seq &rest keys)
  (let ((count (cl--getkey keys :count nil)) (lst (append seq nil)) (r nil) (n 0))
    (while lst
      (if (and (funcall pred (car lst)) (or (null count) (< n count)))
          (setq n (1+ n))
          (setq r (cons (car lst) r)))
      (setq lst (cdr lst)))
    (cl--like (nreverse r) seq)))
```

The `(append seq nil)` coercion and the `cl--like` re-typing at the end are the pattern that makes these functions polymorphic over lists, vectors, and strings: coerce to a list, work, then convert back to the input’s type. `script-demo.el` uses exactly this — `(cl-remove-if (lambda (p) (< (cdr p) 30)) people)` keeps the people aged 30 or more. The search family is `cl-find`, `cl-find-if`, `cl-position`, `cl-position-if`, `cl-count`, `cl-count-if`, `cl-some`, `cl-every`, `cl-notany`, and `cl-notevery`, all keyword-driven with `:test`, `:key`, `:start`, `:end`, and `:from-end`:

```
(defun cl-position (item seq &rest keys)
  (let ((test (cl--getkey keys :test 'eql)) (key (cl--getkey keys :key 'identity))
        (start (cl--getkey keys :start 0)) (end (cl--getkey keys :end nil))
        (from-end (cl--getkey keys :from-end nil))
        (lst (append seq nil)) (i 0) (r nil))
    (while (and lst (or from-end (not r)))
      (when (and (cl--in-bounds i start end) (funcall test item (funcall key (car lst))))
        (setq r i))
      (setq i (1+ i) lst (cdr lst)))
    r))
```

The reducers and transformers are `cl-reduce`, `cl-mapcar`, `cl-map`, `cl-mapcan`, `cl-remove-duplicates` (and destructive `cl-delete-duplicates`), `cl-substitute`, `cl-subseq`, and the sort `cl-sort/cl-`

`stable-sort`. `cl-reduce` is the most feature-complete, supporting `:initial-value`, `:key`, and `:from-end`, and falling back to calling the function with no arguments for the identity element when the sequence is empty:

```
(defun cl-reduce (f seq &rest keys)
  (let* ((l (append seq nil))
        (key (plist-get keys :key))
        (has-init (plist-member keys :initial-value))
        (init (plist-get keys :initial-value)))
    (when key (setq l (mapcar (lambda (x) (funcall key x)) l)))
    (if (plist-get keys :from-end)
        (let ((rl (reverse l)) (acc nil))
          (cond (has-init (setq acc init))
                ((null rl) (setq acc (funcall f) rl nil))
                (t (setq acc (car rl) rl (cdr rl))))
          (while rl (setq acc (funcall f (car rl) acc) rl (cdr rl)))
          acc)
        (if has-init (seq-reduce f l init)
            (if (null l) (funcall f) (seq-reduce f (cdr l) (car l)))))))
```

The set operations `cl-union`, `cl-intersection`, `cl-set-difference`, `cl-adjoin`, `cl-subsetp`, and `cl-merge` round out the sequence layer, and the ordinal accessors `cl-first` through `cl-tenth`, `cl-rest`, and `cl-caddr` give the Common Lisp names for list positions — each a plain `defun` over `car/cdr`, not a `defalias`.

The place-modifying macros

A *place* is a generalized location — a variable, a `car`, an `aref`, a `gethash` — and the place macros read-modify-write one. They all expand through `setf`, the generalized-assignment macro whose full place-expander (`setf--expand`) understands the accessor forms. `cl-incf` and `cl-decf` add and subtract:

```
(defmacro cl-incf (place &rest amt) `(setf ,place (+ ,place ,(if amt (car amt) 1))))
(defmacro cl-decf (place &rest amt) `(setf ,place (- ,place ,(if amt (car amt) 1))))
```

Because the place can be a `gethash`, (`cl-incf (gethash key counts 0)`) increments a hash-table entry — the counter idiom from `data-model.el` — and because it can be an `aref` or a struct accessor, (`cl-incf (sf-y p) 10`) increments a struct slot. `cl-pushnew` conses an element onto a list place unless it is already present:

```
(defmacro cl-pushnew (x place &rest _keys)
  `(if (memql ,x ,place) ,place (setf ,place (cons ,x ,place))))
```

The rest of the family — `cl-rotatef` (rotate several places' values), `cl-shiftf` (shift values along), `cl-psetf` (parallel set), `cl-callf` and `cl-callf2` (apply a function to a place) — all expand to `setf` over temporaries, and the non-`cl-` aliases `incf`, `decf`, `push`, `pop`, and `setf` are defined in the earlier control-macros section so that plain elisp has them too.

Binding and control macros

`cl-lib` provides the local-function and destructuring binders the special-form `lambda` does not. `cl-flet` and `cl-flet*` bind local functions, `cl-labels` binds mutually-recursive ones (so a local `fib` can call itself), and `cl-macrolet` binds local macros — (`cl-macrolet ((sq (x) (list '* x x))) (sq 5)`) is 25. `cl-destructuring-bind` binds a whole lambda-list against a value, and it is the engine under `cl-defun`:

```
(defmacro cl-destructuring-bind (arglist expr &rest body)
  `(let ((--cl-db-v-- ,expr))
    (let* ,(cl-db--binds arglist '--cl-db-v-- ) ,@body)))
```

The dispatch macros `cl-case` and `cl-typecase` are multi-way conditionals — `cl-case` on a value against literal keys (a key that is a *list* matches any member), compiling to a `cond`:

```
(defmacro cl-case (expr &rest clauses)
  `(let ((--cl-case-v-- ,expr))
    (cond ,@(mapcar
      (lambda (clause)
        (let ((key (car clause)) (body (cdr clause)))
          (cond ((memq key '(t otherwise)) (cons t body))
                ((listp key) (cons (list 'memq '--cl-case-v-- (list 'quote key))
                                   body)))
          (t (cons (list 'eql '--cl-case-v-- (list 'quote key)) body))))
      clauses))))
```

`cl-block` and `cl-return-from` establish a named escape (over `catch/throw`), `cl-assert` and `cl-check-type` are the assertion macros, and `cl-do/cl-do*` are the general iteration macros alongside `cl-loop`. The numeric helpers `cl-floor`, `cl-ceiling`, `cl-truncate`, `cl-round`, `cl-mod`, `cl-rem`, `cl-gcd`, `cl-lcm`, `cl-isqrt`, and `cl-signum` give the Common Lisp arithmetic that `stats.el` exercises — `(cl-gcd 48 36)` is 12, `(cl-floor 17 5)` is (3 2). The whole of `cl-lib`, several hundred definitions, is elisp expanding into the special forms and primitives — the single largest demonstration that the core is complete.

Chapter 40 – The Prelude Library: seq, map, subr-x, and the Rest

The remaining prelude families are the modern conveniences — the generic sequence and map interfaces, the `subr-x` string and binding helpers, the pure path library, JSON, and a grab-bag of time, property-list, and printing functions. This chapter walks them, closing the catalogue of what a `.el` program can reach without leaving the language.

seq, the generic sequence interface

The `seq` library treats lists, vectors, and strings uniformly, and it is split across two prelude sections — a small list-oriented core and a fuller `seq.el` block. The transformers and searches — `seq-map`, `seq-filter`, `seq-remove`, `seq-reduce`, `seq-find`, `seq-some`, `seq-every-p`, `seq-count`, `seq-do` — are each a short loop over the `(append seq nil)` coercion:

```
(defun seq-find (pred l &optional default)
  (setq l (append l nil))
  (let ((res default))
    (while l (if (funcall pred (car l)) (progn (setq res (car l)) (setq l nil)) (setq l
      (cdr l))))
    res))
```

The reshapers are richer. `seq-uniq` removes duplicates, delegating to `delete-dups` when no custom test is given and otherwise scanning with the test; `seq-partition` chunks a sequence into runs of N, preserving the input's type so a string partitions into strings; and `seq-group-by` buckets elements into an alist keyed by a function, in first-encounter order:

```
(defun seq-group-by (fn seq)
  (let ((result nil))
    (dolist (x (append seq nil))
      (let* ((key (funcall fn x)) (cell (assoc key result)))
        (if cell (setcdr cell (cons x (cdr cell)))
            (setq result (cons (cons key (list x)) result))))
      (nreverse (mapcar (lambda (c) (cons (car c) (reverse (cdr c)))) result))))
```

`text-utils.el` documents the order quirk explicitly — it looks groups up by key rather than depending on their order — because the first-encounter ordering is an Emacs behavior the example reproduces rather than an accident. `seq-mapn` zips N sequences in parallel, stopping at the shortest — `(seq-mapn #'(1 2) '(3 4))` is `(4 6)` — and the rest of the family covers `seq-take/seq-drop/seq-subseq`, `seq-min/seq-max`, `seq-position`, `seq-into`, the set operations `seq-difference/seq-intersection/seq-union`, the sorts `seq-sort/seq-sort-by`, the indexed variants `seq-map-indexed` and `seq-do-indexed`, and `seq-keep` and `seq-mapcat`. The destructuring binder `seq-let` binds a pattern of variables against a sequence's elements.

map, the generic keyed interface

The `map` library is the generic interface over alists, plists, hash tables, and arrays, dispatching on the backing type at run time. `map-elt` is the getter, a `cond` over the four types, and `map-do` is the traversal every other function funnels through:

```
(defun map-do (function map)
  (cond
    ((hash-table-p map) (maphash function map) nil)
    ((listp map)
     (dolist (pair map) (funcall function (car pair) (cdr pair))) nil)
    ((arrayp map)
     (dotimes (i (length map)) (funcall function i (aref map i))) nil)))
```

On top of `map-do` and `map-elt` sit `map-keys`, `map-values`, `map-pairs`, `map-apply`, `map-filter`, `map-remove`, `map-some`, `map-every-p`, `map-length`, `map-empty-p`, and the converters `map-into` and `map-insert`, plus the mutators `map-put!` and `map-delete` and the combinators `map-merge` and `map-merge-with`. `map-nested-elt` walks a path of keys into nested maps. `text-utils.el` builds a hash table from an alist with `map-into` and reads it back generically — `(map-elt m 'b)` is 2, `(map-values m)` summed is 6 — and the same three lines would work over an alist or a plist, which is the library's whole point.

subr-x and the binding macros

`subr-x` is the modern convenience layer. The string helpers are the trim family — `string-trim`, `string-trim-left`, `string-trim-right`, each a regexp strip anchored with `\`` or `\'`:

```
(defun string-trim-left (s &optional regexp)
  (let ((re (concat "\\`\\(?:" (or regexp "[ \\t\\n\\r]+" ) "\\`"))))
    (if (string-match re s) (substring s (match-end 0)) s)))
```

— plus `string-blank-p`, `string-clean-whitespace`, `string-remove-prefix/suffix`, `string-pad`, `string-chop-newline`, `string-fill`, `string-limit`, and the `string-replace`, `string-reverse`, and `string-lines` that round out the surface. The binding macros are the threading and conditional-let family. `thread-first` and `thread-last` are compile-time rewrites (Chapter 13); `if-let`, `when-let`, their `*` variants, and `and-let*` all delegate to one helper, `if-let--chain`, that nests a `let-and-if` per binding and short-circuits on the first `nil`:

```
(defmacro if-let (bindings then &rest else)
  (if-let--chain (if-let--norm bindings) then (cons 'progn else)))
(defmacro when-let (bindings &rest body)
  (if-let--chain (if-let--norm bindings) (cons 'progn body) nil))
```

`named-let` writes a tail-recursive loop as a named local function, `letrec` binds mutually-recursive values, `cl-flet/cl-labels` (shared with `cl-lib`), and the `let-alist` macro binds dotted accessors over an alist.

The pure path library, JSON, and the grab-bag

The prelude carries a *pure* file-name library — `file-name-directory`, `file-name-nondirectory`, `file-name-extension`, `file-name-sans-extension`, `file-name-base`, `file-name-as-directory`, `directory-file-name`, `file-name-concat`, `file-name-absolute-p`, `file-name-split`, `expand-file-name`, `file-relative-name`, and more — that manipulates path strings without touching the filesystem, so a script can compute paths before deciding to act on them. The JSON library is elisp end to end: `json-encode` maps elisp values to JSON text (with `json--encode-string`, `json--encode-kv`, and the alist/plist detectors), and `json-read-from-string` parses the other way through a small recursive-descent reader (`json--read-string`, `json--read-number`, `json--read-array`, `json--read-object`), with the

dynamic variables `json-object-type`, `json-array-type`, `json-key-type`, `json-null`, and `json-false` controlling the materialization — which is how `script-demo.el` round-trips data to disk as JSON.

The grab-bag is genuinely broad. The time helpers `time-add`, `time-subtract`, `time-less-p`, `time-convert`, `format-seconds`, and the `decoded-time-*` accessors build on the clock primitives. The property-list surface `get`, `put`, `symbol-plist`, `setplist`, `function-get`, and `function-put` is here, along with `define-error` itself and the seventeen `(define-error ...)` registrations that seed the condition hierarchy — `error`, `user-error`, `quit`, `arith-error`, `wrong-type-argument`, `void-variable`, `void-function`, `end-of-file`, and the rest, plus the two CLOS conditions `cl-no-applicable-method` and `cl-no-next-method`. The printing helpers `pp`, `pp-to-string`, `prin1-to-string`'s `cl-` variants, and `format-message` are derived, as are the display-width functions `string-width`, `char-width`, and `truncate-string-to-width`, and the module system `provide/require/featurep` over the `features` list. Two redefinitions are worth a footnote: `setf` is defined twice — a simple version early and the full place-expander late — and `cl-first` likewise, the later definition winning in each case; this is deliberate bootstrapping, an early stub so dependent code can load, replaced by the full version once its dependencies exist. And there is exactly one `defalias` in the entire prelude, `(defalias 'string-split 'split-string)` — every other apparent alias is a plain `defun` wrapper. Taken whole, the prelude is the proof the catalogue has been building toward: a standard library of this breadth, written in elisp, expanding into twenty-two special forms and a couple hundred primitives, is what a complete Lisp core makes possible.

Chapter 41 — The Compiler, Form by Form

Chapter 9 explained the compiler’s shape and its cleverest move, the native arithmetic fast path. This chapter is the complete lowering reference: every special form, the exact opcodes it emits, in order, transcribed from `src/compiler.rs`. It is the kind of chapter you read once to understand the machine and then keep for when you want to know exactly what a given form becomes. A useful structural fact first: several special forms share a lowering function, because they differ only by a flag. `let` and `let*` share `compile_let` (a sequential flag); `when` and `unless` share `compile_when` (a polarity flag); `and` and `or` share `compile_andor`; `defun` and `defmacro` share `compile_defun` (an `is_macro` flag); `defvar` and `defconst` share `compile_defvar`; and `quote` and `function` are inlined directly in `compile_call`. There is no separate `compile_let_star` or `compile_unless`.

Atoms and symbols

The walk dispatches on the form’s type. An integer compiles to `LoadInt`, a float to `LoadFloat`, a string to a `LoadConst`, `nil` (and `Bool(false)`) to `LoadUndef`, and `t` to `LoadTrue` — the immediates, lowered to the VM’s own load opcodes with no elisp involvement. A symbol is the first interesting case: a keyword (a name starting with `:`) compiles to a `LoadConst` of itself, because keywords self-evaluate, while any other symbol compiles to a two-op sequence — `LoadConst` of the symbol, then `Extended(GETVAR)` — which loads the symbol and replaces it with its variable value at run time, routing through the lexical-then-global binding search.

```
;; compile_form on a symbol in value position:
;;   LoadConst(sym)           ; push the symbol object
;;   Extended(GETVAR, 0)      ; replace it with its variable value
;; a keyword :foo compiles to just LoadConst(:foo)
```

quote, function, progn, and prog1

`quote` is inlined in `compile_call` as a bare `load_const` of its argument — `(quote x)` emits one `LoadConst`. `function` is the same for a symbol argument, but when its argument is a `(lambda ...)` form it compiles that lambda as a closure instead, the fix that made `#'(lambda ...)` build a callable. `progn` compiles each body form with a `Pop` between them, keeping only the last value; `prog1` keeps the *first* form’s value and emits a `Pop` after each subsequent form.

The conditionals

`compile_if` is the canonical back-patched conditional: it compiles the condition, emits `Extended(TRUTHY)` to convert the value under the “only nil is false” rule, emits a `JumpIfFalse` whose target it patches once the else-branch position is known, compiles the then-branch, emits a `Jump` past the else, and compiles the else as an implicit `progn`.

```
;; compile_if:
;;   <cond>
;;   Extended(TRUTHY, 0)
;;   JumpIfFalse(-> else)
;;   <then>
;;   Jump(-> end)
;;   [else:] <progn else-body>
;;   [end]
```

`compile_when` handles both `when` and `unless` with a polarity flag — `when` emits `JumpIfFalse`, `unless` emits `JumpIfTrue` — and both push `LoadUndef` on the skipped path so the one-armed conditional returns `nil` when the body does not run. `compile_cond` walks the clauses, each emitting a test, a `TRUTHY`, and a `JumpIfFalse` to the next clause; a clause with a test but no body is special-cased to `Dup` the test value and keep it when truthy, the `(cond (x) ...)` idiom that returns `x` — which is why `(cond (nil 1) (42))` is `42`. A final `LoadUndef` is the no-clause-matched value.

The boolean connectives and the loop

`compile_andor` handles `and` and `or`. An empty `(and)` is `LoadTrue` and an empty `(or)` is `LoadUndef`. For each non-last form it emits the form, a `Dup`, a `TRUTHY`, and a short-circuit jump — `JumpIfFalse` for `and`, `JumpIfTrue` for `or` — followed by a `Pop`; the last form is left on the stack, and every short-circuit jump is patched to the end, so the deciding value survives. This is what makes `(and 1 2 3)` return `3` and `(or nil nil 5)` return `5` — the *value*, not a boolean.

```
;; compile_andor, per non-last form:
;;   <form>
;;   Dup
;;   Extended(TRUTHY, 0)
;;   JumpIfFalse(-> end) ; (JumpIfTrue for `or`)
;;   Pop
;; ... last form left on the stack; jumps patched to end
```

`compile_while` is the only loop: it records the start position, compiles the condition, emits `TRUTHY` and a `JumpIfFalse` to the exit, compiles the body, emits a `Pop` to discard each iteration's value, emits an unconditional `Jump` back to the start, and ends with `LoadUndef` because `while` returns `nil`.

let, let*, and setq

The two binding forms diverge exactly where their semantics do. `let` (parallel) compiles every initializer onto the stack, each paired with a `LoadConst` of its symbol, then emits a single `ExtendedWide(LETBIND, n)` that opens a scope and binds all `n` at once. `let*` (sequential) emits `Extended(SCOPE_OPEN)` to open one scope, then for each binding compiles the initializer, loads the symbol, and emits `Extended(SPECBIND)` to bind it immediately — so each initializer sees the previous bindings. Both compile the body and emit `ExtendedWide(UNBIND, n)` to close the scope.

```
;; compile_let, parallel `let':
;;   <init_1> LoadConst(sym_1) ... <init_n> LoadConst(sym_n)
;;   ExtendedWide(LETBIND, n)
;;   <progn body>
;;   ExtendedWide(UNBIND, n)
;; sequential `let*':
;;   Extended(SCOPE_OPEN, 0)
;;   <init_1> LoadConst(sym_1) Extended(SPECBIND, 0) ... (repeated)
;;   <progn body>
;;   ExtendedWide(UNBIND, n)
```

`compile_setq` walks its symbol/value pairs, emitting `LoadConst(sym)`, the value, and `Extended(SETVAR)` for each, with a `Pop` between pairs and the last value left on the stack; an empty `(setq)` is `LoadUndef`.

lambda, defun, and defvar

`compile_lambda` allocates an `Obj::Closure template` — parsed params, the compiled body sub-chunk, the `is_macro` flag, and `env: None` — loads it as a constant, and emits `Extended(MAKE_CLOSURE)`, which captures the current lexical environment at run time. `compile_defun` (for both `defun` and `defmacro`) loads the name and the template, emits `Extended(MAKE_CLOSURE)` only when it is *not* a macro (a `defun` captures its defining environment; a macro does not), and emits `Extended(FSET)` to install the closure in the function cell, leaving the symbol on the stack.

```
;; compile_defun:
;;   LoadConst(name)
;;   LoadConst(template)
;;   Extended(MAKE_CLOSURE, 0) ; omitted for defmacro
;;   Extended(FSET, 0)       ; sets the function cell, leaves the symbol
```

`compile_defvar` (for `defvar` and `defconst`) calls `h.set_special(fname)` at *compile time* — marking the symbol special so later `lets` of it bind dynamically — and, if an initializer is present, emits `LoadConst(name)`, the init, `Extended(SETVAR)`, and a `Pop`, always ending with `LoadConst(name)` so the form returns the symbol.

The general call and the fast path

Any list whose head is not a special form is a call. `compile_call` first tries the native arithmetic fast path — but only when `h.is_primitive_fn(name)` confirms the symbol's function cell still holds the original `Obj::Subr`, the redefinition guard. `try_native_op` handles `+/*/-` as folds over `Add/Mul/Sub` (with the empty-case identities and unary `Negate`), the two-argument comparisons as `NumLt/NumGt/NumLe/ NumGe/NumEq`, and `1+/1-` as an `Add/Sub` of a literal `1` — explicitly not a dedicated increment opcode, because `Add` is float-contagious so `(1+ 1.0)` is `2.0`. `/`, `%`, and `mod` fall through and stay on the host to preserve integer-division and zero-signaling semantics. When the fast path declines, the general lowering loads the operator (as a constant for a symbol, or as a closure for a head-position lambda), compiles each argument, and emits `Extended(CALL, argc)` with the argument count in a byte — the universal calling convention, the same for a subr and a closure.

What the compiler refuses

Three forms are recognized and deliberately *not* lowered: `save-excursion`, `save-current-buffer`, and `save-restriction`. `is_unsupported_special` flags them, and `compile_call` returns an error — “special form `save-excursion` not yet lowered (buffer milestone)” — rather than miscompiling them. This is the project's loud-failure discipline in the compiler itself: a form whose correct lowering does not yet exist is refused at compile time, never silently mistranslated. The complete special-form dispatch table is exactly twenty-two names plus these three rejected ones, and everything else — `dolist`, `pcase`, `cl-loop`, the whole familiar vocabulary — is a macro that expands into this set before the compiler ever sees it.

Chapter 42 – The Extension Handler, Op by Op

The compiler emits ten extension opcodes the VM does not understand; this chapter is the host side, the handlers that execute them, transcribed from `src/host.rs`. It is the mirror of the previous chapter: where Chapter 41 showed what each form *emits*, this shows what each emitted op *does* against the object heap. Together they are the complete account of how an elisp form becomes an effect.

Running a chunk

`run_chunk` is the entry. It clears any recorded host error, builds a `fusevm::VM` around the chunk, installs the two extension handlers, enables the tracing JIT, runs, and maps the outcome:

```
;; run_chunk (host.rs), transcribed:
;;   with_host(|h| h.error = None);
;;   let mut vm = VM::new(chunk);
;;   vm.set_extension_handler(Box::new(ext_dispatch));           // narrow ops
;;   vm.set_extension_wide_handler(Box::new(ext_dispatch_wide)); // wide ops
;;   vm.enable_tracing_jit();
;;   let outcome = vm.run();
;;   if let Some(e) = with_host(|h| h.take_error()) { return Err(e); }
;;   match outcome {
;;     VMResult::Ok(v)    => Ok(v),
;;     VMResult::Halted  => Ok(top-of-stack or Undef),
;;     VMResult::Error(e) => Err(e),
;;   }
;; }
```

Two subtleties. The host error — set by a subr that signalled — takes *precedence* over the VM's own outcome, so an error raised inside a callback surfaces as the chunk's result even though the VM did not fault. And a `Halted` VM (one that ran off the end after an `abort`) returns its top-of-stack value. The tracing JIT enabled here is what turns the hot native-arithmetic loops of the previous chapter into machine code.

The narrow handler, op by op

`ext_dispatch(vm, id, arg)` handles the eight narrow ops. Each arm pops from the VM stack, does its work against the host through `with_host`, and pushes a result:

- **TRUTHY** (0): pops `v`, pushes `Bool(el_truthy(&v))` — the elisp truthiness of the value, the conversion before every conditional jump.
- **CALL** (1): pops `arg` arguments (reversing them) and the operator, calls `call_function`, and pushes the result — or calls `abort(vm, e)` on an error.
- **GETVAR** (2): pops a symbol, pushes `h.get_value(&sym)`, the variable read.
- **SETVAR** (3): pops a value and a symbol, sets the value cell, pushes the value back.
- **FSET** (4): pops a definition and a symbol, sets the function cell, pushes the symbol.
- **SPECBIND** (5): pops a symbol and a value, binds it into the already-open scope — the `let*` per-binding op.
- **SCOPE_OPEN** (8): opens an empty lexical scope, for `let*`.

- **MAKE_CLOSURE** (9): pops a closure template, pushes a closure capturing the current environment via `instantiate_closure`.

```
;; CALL arm (ext_dispatch), transcribed:
;;   let symv = vm.pop();
;;   match call_function(&symv, &args) {
;;     Ok(v) => vm.push(v),
;;     Err(e) => abort(vm, e),
;;   }
```

`abort` records the error on the host and sets `vm.ip` past the end of the chunk, which halts the VM and is why `run_chunk` checks the host error first.

The wide handler

`ext_dispatch_wide(vm, id, n)` handles the two wide ops, whose count `n` can exceed a byte. **LETBIND** (6) pops `n` symbol/value pairs, opens a fresh scope, and binds them in parallel (reversed, so source order is preserved); **UNBIND** (7) ignores `n` and closes the innermost scope.

```
;; LETBIND arm (ext_dispatch_wide):
;;   with_host(|h| {
;;     h.open_scope();
;;     for (sym, val) in pairs.into_iter().rev() { h.bind_value(&sym, val); }
;;   });
```

The narrow **SPECBIND** and the wide **LETBIND** are the difference between `let*` and `let`: `let*` opens one scope and binds one at a time as it goes, `let` evaluates all the initializers first and binds them in a single parallel **LETBIND**.

call_function: the re-entrant heart

CALL routes through `call_function`, the single entry for invoking any callable, and it does two things in order. First it intercepts the re-entrant intrinsics *by name*, before any host borrow, because they call back into elisp and a borrow held across the callback would deadlock the callee's own `with_host`. The intercepted names are `funcall`, `apply`, `mapcar`, `mapc`, `sort`, `maphash`, `eval`, `macroexpand-1`, `macroexpand`, `macroexpand-all`, a function-valued `replace-regexp-in-string`, and the three non-local-exit intrinsics `--catch--`, `--unwind--`, `--condition-case--`. Second, for everything else it resolves the operator through `resolve_function` and dispatches:

```
;; call_function dispatch (host.rs), transcribed:
;;   match h.resolve_function(f)? {
;;     Resolved::Subr { f, min, max, name } => {
;;       if args.len() < min || max.is_some_and(|m| args.len() > m) {
;;         return Err("wrong-number-of-arguments: ...");
;;       }
;;       with_host(|h| f(h, args))    // arity-checked, runs in one short borrow
;;     }
;;     Resolved::Closure { params, body, is_macro, env } => {
;;       if is_macro { return Err("macro called as a function ..."); }
;;       run_closure(&params, &body, env, args)  // nested VM
;;     }
;;   }
```

A subr is arity-checked centrally, then run inside a single short `with_host` borrow. A closure routes to `run_closure`, which opens a scope as a child of the closure's captured environment, binds the parameters, runs the body chunk on a *nested* VM via `run_chunk`, and unwinds scopes to the entry depth on the way out —

which is why recursion is recursion in nested VMs, and why a macro accidentally called as a function is rejected here rather than misbehaving. This dispatch, plus the ten handler arms above, is the entire bridge between the alien object heap and the language-agnostic VM: ten callbacks and a value handle, exactly as Chapter 10 promised.

Chapter 43 – Worked Example: script-demo.el, End to End

The tour of Chapter 27 read programs in excerpts; these worked-example chapters read them *whole*, top to bottom, because seeing a complete file is the only way to watch the layers compose into a real script. We begin with `script-demo.el`, the file the project ships to prove the standalone-scripting surface, and we read every line. Its header states the ambition plainly: it “exercises the standalone-scripting surface composed together the way a real .el program would: write/read files, scan a buffer with regexps, summarize with seq/cl, round-trip through JSON, and shell out,” and the same file “passes under GNU Emacs.”

The preamble

```
(require 'cl-lib)
(require 'seq)
(require 'subr-x)
(require 'json)

(defvar script-demo--dir "target/script-demo")
```

The four `require` forms declare the prelude libraries the file uses; in `elisp`s they are satisfied by the `features` list the prelude provides, so `require` succeeds without loading anything from disk. The `defvar` declares a special variable for the working directory — special, by Chapter 6’s rule, so any later `let` of it would bind dynamically, though here it is just a module-level constant.

Setup and the buffer parser

```
(defun script-demo--setup ()
  (make-directory script-demo--dir t)
  (write-region "alice 30\nbob 25\ncarol 41\n" nil
    (file-name-concat script-demo--dir "people.txt")))

(defun script-demo--parse-people (file)
  "Parse \"NAME AGE\" lines into an alist of (NAME . AGE)."
  (with-temp-buffer
    (insert-file-contents file)
    (goto-char (point-min))
    (let (rows)
      (while (re-search-forward "^\\([a-z]+\\) \\([0-9]+\\)$" nil t)
        (push (cons (match-string 1) (string-to-number (match-string 2))) rows))
      (nreverse rows))))
```

`script-demo--setup` is the OS surface of Chapter 37: `make-directory` with a non-`nil` `PARENTS` creates the tree, and `write-region` writes a literal string to a file whose path `file-name-concat` (a pure path function from the prelude) builds. `parse-people` is the read-process-write loop of Chapter 16 and the buffer catalogue of Chapter 36, in miniature. `with-temp-buffer` pushes a scratch buffer; `insert-file-contents` reads the file into it; `goto-char` moves point to the start; and the `while` drives `re-search-forward` over an Emacs

regex — note the backslashed grouping `\(...\)` of Chapter 15 — anchored with `^...$`, matching each `NAME AGE` line. Each iteration sets the buffer-relative match data, `match-string 1` and `match-string 2` slice the two groups out, `string-to-number` parses the age, and `push` conses the `(NAME . AGE)` pair onto an accumulator that `nreverse` flips back to source order. The result is an alist built entirely from buffer scanning, with no Emacs anywhere.

Parse, summarize, and the seq/cl pipeline

```
(ert-deftest script-demo-parse-and-summarize ()
  "Read a file, parse with a buffer, summarize with seq/cl."
  (script-demo--setup)
  (let* ((people (script-demo--parse-people
                  (file-name-concat script-demo--dir "people.txt")))
         (ages (mapcar #'cdr people)))
    (should (equal (mapcar #'car people) '("alice" "bob" "carol")))
    (should (= (seq-reduce #'+ ages 0) 96))
    (should (= (seq-max ages) 41))
    (should (equal (cl-remove-if (lambda (p) (< (cdr p) 30)) people)
                  '(("alice" . 30) ("carol" . 41))))))
```

This is the summary half. `mapcar #'cdr` extracts the ages; `mapcar #'car` the names. `seq-reduce #'+` sums the ages to 96 and `seq-max` finds 41 — the `seq` library of Chapter 30, each call a prelude function over the primitives. `cl-remove-if` from Chapter 39 drops the people younger than 30, keeping alice and carol, demonstrating the keyword-less common case of the Common Lisp filter. The `#'car`, `#'cdr`, `#'+` function quotes reach the function cells of those symbols, the Lisp-2 in everyday use.

The JSON round-trip

```
(ert-deftest script-demo-json-roundtrip ()
  "Serialize the summary to JSON on disk and read it back."
  (script-demo--setup)
  (let* ((people (script-demo--parse-people
                  (file-name-concat script-demo--dir "people.txt")))
         (jf (file-name-concat script-demo--dir "people.json")))
    (write-region (json-encode people) nil jf)
    (let ((back (let ((json-object-type 'alist) (json-key-type 'string))
                  (json-read-from-string
                   (with-temp-buffer (insert-file-contents jf) (buffer-string))))))
      (should (= (cdr (assoc "alice" back)) 30))
      (should (= (cdr (assoc "carol" back)) 41)))))
```

This is the read/print round-trip of Chapter 23 applied to a foreign format. `json-encode` — elisp in the prelude, Chapter 40 — serializes the alist to a JSON string, `write-region` writes it, and the inner `let` binds the dynamic variables `json-object-type` and `json-key-type` around `json-read-from-string` so the parse materializes the object as an alist with string keys. The `with-temp-buffer/insert-file-contents/buffer-string` idiom reads the file back into a string. The two `assoc` lookups confirm the data survived the trip — `(assoc "alice" back)` finds the pair whose `cdr` is 30 — which only works because the encoder and decoder are inverses, and because `assoc` uses `equal` so a string key is found.

Shelling out and cleanup

```
(ert-deftest script-demo-shell ()
  "Shell out and read the result back."
  (should (equal (string-trim (shell-command-to-string "echo elisprs")) "elisprs"))
  (should (equal (process-lines "printf" "x\ny\n") ('("x" "y")))))

(ert-deftest script-demo-cleanup ()
  "Tidy the scratch files."
  (dolist (f '("people.txt" "people.json"))
    (let ((p (file-name-concat script-demo--dir f)))
      (when (file-exists-p p) (delete-file p))))
  (should t))

(ert-run-tests-batch-and-exit)
```

`shell-command-to-string` runs `echo` through `sh -c` and `string-trim` cleans the trailing newline; `process-lines` runs `printf` and splits its output into a list — the subprocess primitives of Chapter 37. The cleanup test uses `dolist` (a prelude macro over `while`) to delete each scratch file, guarding with `file-exists-p` so a missing file is not an error. And `ert-run-tests-batch-and-exit` is the line that makes the file a CI gate: it runs every `ert-deftest` and exits non-zero if any failed, so this end-to-end script — file I/O, buffer parsing, seq/cl summarizing, JSON round-tripping, and subprocesses — is asserted to behave exactly as shown, on every commit, against both `elisprs` and real Emacs. It is the whole promise of the project in one file.

Chapter 44 – Worked Example: data-model.el, End to End

`data-model.el` is the densest example in the corpus, exercising the object system, the pattern matcher, hash aggregation, and the condition hierarchy together. Reading it whole shows how the derived surface of Chapters 22, 25, and 14 composes into the kind of code a real package is built from. Its header: it “exercises the OO/data surface the way real packages do: `cl-defstruct` with `:include` inheritance, `cl-defmethod` dispatch, `pcase` destructuring, hash-table aggregation, and `cl-loop`.”

The struct hierarchy and generic functions

```
(cl-defstruct shape name)
(cl-defstruct (circle (:include shape)) radius)
(cl-defstruct (rect (:include shape)) width height)

(cl-defgeneric area (s) "Area of shape S.")
(cl-defmethod area ((c circle)) (* float-pi (circle-radius c) (circle-radius c)))
(cl-defmethod area ((r rect)) (* (rect-width r) (rect-height r)))
```

The three `cl-defstruct` forms build the hierarchy of Chapter 22. `shape` has one slot, `name`; `circle` and `rect` each (`:include shape`), so they inherit the `name` slot at the same index and satisfy `shape-p`. Each declaration generates a constructor (`make-circle`), a predicate (`circle-p`), and accessors (`circle-radius`), all elisp expanded from the macro. `cl-defgeneric` declares the generic `area`, and the two `cl-defmethod` forms add type-specialized methods — `((c circle))` dispatches on a circle, `((r rect))` on a rect — installed in the dispatch table `cl--generic-table` and selected at call time by the argument’s type. `float-pi` is the prelude’s constant.

pcase bucketing

```
(defun classify-size (s)
  "Bucket S by area using pcase guards."
  (let ((a (area s)))
    (pcase a
      ((pred (lambda (x) (< x 10)))) 'small)
      ((and n (guard (< n 100))) 'medium)
      (_ 'large))))
```

`classify-size` computes the area by generic dispatch, then runs it through `pcase` from Chapter 25. The first pattern `(pred ...)` matches if the lambda returns non-nil — area under 10 is `small`. The second `(and n (guard ...))` binds the value to `n` and then requires the guard `(< n 100)` — under 100 is `medium`. The wildcard `_` catches the rest as `large`. The whole `pcase` compiles to a `let` binding the scrutinee and a `cond` walking the patterns, all derived.

Generics, inheritance, and dispatch under test

```
(ert-deftest data-model-generics-and-inheritance ()
  "cl-defmethod dispatches on struct type; :include shares the `name' slot."
  (let ((c (make-circle :name "c" :radius 2))
        (r (make-rect :name "r" :width 3 :height 4)))
    (should (equal (shape-name c) "c")) ; inherited accessor
    (should (shape-p c)) ; subtype satisfies parent pred
    (should (= (area r) 12))
    (should (< (abs (- (area c) 12.566)) 0.01))
    (should (eq (classify-size (make-rect :width 2 :height 2)) 'small))
    (should (eq (classify-size r) 'medium))
    (should (eq (classify-size (make-rect :width 20 :height 20)) 'large))))
```

The constructors take keyword arguments with the slot names. `(shape-name c)` reads the *inherited* slot through the parent's accessor — proof that `:include` shares the slot — and `(shape-p c)` is true for a circle because the predicate walks the tag-parentage chain. `(area r)` dispatches to the rect method and returns 12; `(area c)` dispatches to the circle method and returns $\pi r^2 \approx 12.566$, checked within a tolerance because it is a float. The three `classify-size` calls confirm the three *pcase* buckets, area driving the dispatch. This single test exercises struct construction, inheritance, keyword constructors, type-dispatched generics, and guarded pattern matching — the whole object surface.

Hash aggregation with cl-loop

```
(ert-deftest data-model-hash-aggregation ()
  "Tally word lengths into a hash table, then read it back with cl-loop."
  (let ((counts (make-hash-table :test 'eql))
        (words '("a" "bb" "cc" "ddd" "ee")))
    (dolist (w words)
      (cl-incf (gethash (length w) counts 0)))
    (should (= (gethash 2 counts) 3))
    (should (= (gethash 1 counts) 1))
    (let ((pairs (cl-loop for k being the hash-keys of counts using (hash-values v)
                          collect (cons k v))))
      (should (= (seq-reduce #' + (mapcar #'cdr pairs) 0) 5)))))
```

The hash table of Chapter 33 is used as a counter. `dolist` walks the words; for each, `(gethash (length w) counts 0)` reads the current count for that length with a default of zero, and `cl-incf` — a place macro from Chapter 39, expanding through `setf` over the `gethash` place — writes the increment back. After the loop, length 2 has count 3 (`bb`, `cc`, `ee`) and length 1 has count 1 (`a`). The `cl-loop` reads the table back with the `for k being the hash-keys of ... using (hash-values v)` clause from Chapter 26 — parallel iteration over keys and values — collecting `(k . v)` pairs, whose values sum to 5, the word count. Three subsystems — hash tables, place macros, and the loop facility — in one short test.

The custom error hierarchy

```
(ert-deftest data-model-error-handling ()
  "Custom error hierarchy with condition-case inheritance."
  (define-error 'parse-error "Parse error")
  (define-error 'bad-token "Bad token" 'parse-error)
  (should (eq (condition-case e (signal 'bad-token '("x"))
    (parse-error 'caught-parent))
    'caught-parent))
  (should (equal (condition-case e (signal 'bad-token '("oops"))
    (error (cadr e)))
    "oops"))))

(ert-run-tests-batch-and-exit)
```

This builds a *user* condition hierarchy on the mechanism of Chapter 14. `define-error` registers `parse-error` under the root `error`, then `bad-token` under `parse-error`, by setting each symbol's `error-conditions` property to itself plus its ancestors. The first assertion signals `bad-token` and catches it with a `parse-error` handler — which works because `bad-token`'s condition list contains `parse-error`. The second catches the same signal with an `error` handler and reads `(cadr e)` to get the signal's data, `"oops"`, proving the data list is preserved through the handler. The whole condition system is elisp over `put/get`, and a user can extend it exactly as the standard conditions were seeded. `data-model.el` thus touches structs, inheritance, generics, `pcase`, hash tables, `cl-loop`, and the condition system — almost the entire derived surface, all on the small core.

Chapter 45 – Worked Example: text- utils.el and stats.el

The last two worked examples are the ones that look most like everyday work: `text-utils.el` formats and reshapes text, and `stats.el` does numeric analysis. Reading them whole shows the string, format, map, seq, and numeric surfaces in the kind of code a person actually writes.

text-utils.el: parsing and rendering a table

```
(defun parse-kv (line)
  "Parse \"key=value\" into a (KEY . VALUE) cons, trimming whitespace."
  (let ((parts (split-string line "=")))
    (cons (string-trim (car parts)) (string-trim (cadr parts)))))

(defun render-table (alist)
  "Render ALIST as aligned \"key : value\" lines sorted by key."
  (let* ((sorted (sort (copy-sequence alist)
                       (lambda (a b) (string< (car a) (car b))))))
    (width (seq-max (cons 0 (mapcar (lambda (kv) (length (car kv))) sorted)))))
    (mapconcat (lambda (kv)
                  (format "%s : %s" (string-pad (car kv) width) (cdr kv)))
                sorted "\n")))
```

`parse-kv` splits a `key=value` line on `=` and `string-trim`s both halves into a cons — `split-string` and `string-trim` from Chapters 32 and 40. `render-table` is a report formatter: `sort` orders a copy of the alist by key with a `string<` predicate (the stable merge sort of Chapter 38, copying because `sort` may rearrange in place); `seq-max` over the key lengths, seeded with 0 so an empty alist is safe, finds the column width; and `mapconcat` joins each entry as a `format`-aligned line, `string-pad` padding the key to the width. Five library functions composed into the report-formatting a script does constantly.

```
(ert-deftest text-utils-format-directives ()
  (should (equal (format "%05.2f|%+d|%x|%c" 3.14159 7 255 ?A) "03.14|+7|ff|A"))
  (should (equal (format "%-8s|%8s|" "L" "R") "L          |          R|"))
  (should (equal (format "%5" '(1 "two" 3.0)) "(1 \"two\" 3.0)"))
  (should (equal (number-to-string (/ 22.0 7)) "3.142857142857143")))
```

The `format-directives` test pins the engine of Chapter 32: `%05.2f` is width-5, precision-2, zero-filled; `%+d` forces a sign; `%x` is hex; `%c` is a character. The `%-8s` and `%8s` show left- and right-justification. `%5` renders re-readably — a list whose float element keeps its `.0`. And the full-precision `(/ 22.0 7)` prints the shortest decimal that reads back, the printer's float fidelity from Chapter 23.

```
(ert-deftest text-utils-seq-and-map ()
  (let ((m (map-into '((a . 1) (b . 2) (c . 3)) 'hash-table)))
    (should (= (map-elt m 'b) 2))
    (should (= (seq-reduce #'(+) (map-values m) 0) 6)))
  (should (= (seq-reduce #'(+)
    (seq-map (lambda (x) (* x x))
      (seq-filter #'cl-evenp (number-sequence 1 10)))
    0)
    220))
  (let ((g (seq-group-by (lambda (n) (if (cl-evenp n) 'even 'odd)) '(1 2 3 4 5))))
    (should (equal (assq 'odd g) '(odd 1 3 5)))
    (should (equal (assq 'even g) '(even 2 4)))))
```

The `map` library of Chapter 40 builds a hash table from an alist with `map-into` and reads it back generically — `map-elt` and `map-values`. The `seq` pipeline squares the even numbers from 1 to 10 and sums them to 220, the showcase filter-map-reduce. And `seq-group-by` buckets by parity, looked up by key with `assq` because the group order is an Emacs quirk the example deliberately does not depend on. The `string-ops` test rounds it out — `string-join` over a `split-string`, `string-pad`, `string-trim`, `mapconcat` with `upcase`, and `string-replace` — the whole string surface in five assertions.

stats.el: the numeric surface

```
(defun mean (xs) (/ (float (seq-reduce #'(+) xs 0)) (length xs)))

(defun median (xs)
  (let* ((s (sort (copy-sequence xs) #'<))
        (n (length s)))
    (if (cl-oddp n)
        (float (nth (/ n 2) s))
        (/ (+ (nth (1- (/ n 2)) s) (nth (/ n 2) s)) 2.0))))

(defun variance (xs)
  (let ((m (mean xs)))
    (/ (seq-reduce (lambda (acc x) (+ acc (* (- x m) (- x m)))) xs 0.0)
       (length xs))))
```

The three statistics functions are the numeric tower of Chapter 19 in working clothes. `mean` sums with `seq-reduce`, coerces the integer sum to a `float` so the division is a float division, and divides by the count. `median` sorts a copy with the numeric `<`, indexes the middle with integer division (`/ n 2`), and is careful about types — `float` in the odd case, `/ ... 2.0` in the even — so a median is a float either way. `variance` accumulates squared deviations from the mean, seeding the reduce with `0.0` so the accumulator is a float from the start. Every numeric idea — contagion, integer division, explicit float coercion, the stable sort — is in those nine lines.

```
(ert-deftest stats-basic ()
  (let ((data '(2 4 4 4 5 5 7 9)))
    (should (= (mean data) 5.0))
    (should (= (median data) 4.5))
    (should (= (variance data) 4.0))
    (should (= (sqrt (variance data)) 2.0))
    (should (= (seq-max data) 9))
    (should (= (seq-min data) 2))))
```

The basic test checks the three functions against a known dataset — mean 5.0, median 4.5, variance 4.0, standard deviation (`sqrt (variance ...)`) exactly 2.0 — plus `seq-max` and `seq-min`. The `cl-loop-accumulators`, `float-printing`, and `integer-math` tests (walked in Chapter 27) round out the numeric coverage, and `(ert-run-tests-batch-and-exit)` makes the whole file a gate. Together `text-utils.el`

and `stats.el` show the data-munging and numeric surfaces doing real work — and like every example in the book, they pass byte for byte under both `elisp` and GNU Emacs, which is the cross-check that turns these programs from illustrations into executable specifications.

Chapter 46 – The Test Suite as Specification: Values and Control

elisprs is gated by two kinds of test, and both are executable specifications. The ERT files in `examples/` are elisp the project runs under both elisprs and GNU Emacs; the Rust integration tests in `tests/` drive the interpreter directly through a tiny harness — `reset_host()`, `eval_str(src)`, then `print(&v, true)` — so each test is an elisp form and its exact printed result. This chapter and the next walk the most revealing of those results, organized by topic, because reading them is the most precise way to learn what the language actually does: every line below is a behavior the CI asserts on every commit.

Arithmetic and numbers

```
(+ 1 2 3)           ; => 6
(* 2 (+ 3 4))       ; => 14
(/ 7.0 2)           ; => 3.5
(- 5)               ; => -5
(1+ 41)             ; => 42
```

These pin the basics from Chapter 19: the variadic folds, the contagion that makes `(/ 7.0 2)` a float, the unary negation `(- 5)`, and the native-fast-path increment `(1+ 41)`. The float result `3.5` is the contagion rule made visible — one float operand and the whole quotient is a float — and it is asserted rather than assumed, so a regression in either the subr or the native lowering shows up here.

Cons cells and lists

```
(cons 1 2)           ; => (1 . 2)
(quote (a b . c))    ; => (a b . c)
(progn (setq p (cons 1 2)) (setcar p 9) p) ; => (9 . 2)
(nth 2 (list 10 20 30 40)) ; => 30
(append (list 1 2) (list 3 4)) ; => (1 2 3 4)
```

The dotted pair `(1 . 2)` prints with the `.` separator, the printer/reader agreement of Chapter 23; the quoted improper list `(a b . c)` round-trips; and the `setcar` sequence proves in-place mutation — `p` is `(9 . 2)` after `setcar` reaches into the very cell it points at. `nth` and `append` are the list primitives of Chapter 30 doing exactly what their signatures say.

Equality, by grade

```
(eq (quote a) (quote a)) ; => t
(equal (list 1 2) (list 1 2)) ; => t
(eq (list 1 2) (list 1 2)) ; => nil
```

These three are the equality lesson of Chapter 23 in three lines: two reads of `'a` intern to the same symbol so `eq` holds; two freshly-consed `(1 2)` lists are structurally `equal` but are different objects, so `equal` is `t` and `eq`

is `nil`. This is the bug that the choice between `memq` and `member`, `assq` and `assoc`, turns on.

Strings

```
(substring "hello world" 0 5) ; => "hello"
(substring "hello" -3)       ; => "llo"
(split-string "a b c")      ; => ("a" "b" "c")
(string-prefix-p "foo" "foobar") ; => t
(make-string 3 65)          ; => "AAA"
(string-search "lo" "hello") ; => 3
```

`substring` slices by character index and accepts negative indices counting from the end; `split-string` defaults to whitespace and omits empty fields; `make-string` builds from a character code (65 is `?A`); and `string-search` does a literal search returning a character index. The strings print with their quotes because the harness prints in readable form — `"hello"`, not `hello`.

Vectors and sequences

```
(progn (setq v (make-vector 3 0)) (aset v 1 9) v) ; => [0 9 0]
(seq-filter (lambda (x) (> x 2)) (list 1 2 3 4 5)) ; => (3 4 5)
(seq-reduce (lambda (a b) (+ a b)) (list 1 2 3 4) 0) ; => 10
(remove 3 [1 2 3]) ; => [1 2]
(seq-contains-p "abc" ?b) ; => t
```

The `make-vector/aset` sequence prints `[0 9 0]`, the in-place vector mutation of Chapter 33. `seq-filter` and `seq-reduce` are the generic sequence functions of Chapter 30, and `(remove 3 [1 2 3])` returning `[1 2]` shows `remove` operating on a vector and returning a vector — the type-preserving polymorphism the prelude's `(append seq nil)` coercion buys. `(seq-contains-p "abc" ?b)` is `t` because a string is a sequence of character codes and `?b` is one of them.

Control flow

```
(when t 1 2 3) ; => 3
(cond (nil 1) (42)) ; => 42
(let ((acc 0) (i 1))
  (while (<= i 5) (setq acc (+ acc i)) (setq i (1+ i)))
  acc) ; => 15
(catch 'done (throw 'done 42)) ; => 42
(catch 'x (+ 1 (throw 'x 99)) 999) ; => 99
(let ((log nil))
  (catch 'x (unwind-protect (throw 'x 1) (setq log 'cleaned))))
  log) ; => cleaned
```

`(when t 1 2 3)` returns the last body value, 3, its implicit `progn`. `(cond (nil 1) (42))` is the body-less clause idiom of Chapter 41 — the clause `(42)` has a test but no body, so it returns the test value when truthy. The `while` accumulates 1 through 5 to 15, the lexical `setq` mutating the `let` bindings in place. The two `catch/throw` lines show the labeled escape, the second proving a throw aborts the enclosing expression mid-evaluation — 999 is never reached. And the `unwind-protect` line proves the cleanup runs even when the body throws: `log` is `cleaned` even though the `throw` jumped past the normal flow, the non-local-exit guarantee of Chapter 14.

Chapter 47 – The Test Suite as Specification: Closures, Errors, and Objects

The previous chapter walked the value and control core; this one continues through the parts of the language that are most entangled with the runtime — closures and binding, the condition system, hash tables and structs, the regex bridge, and the macro-and-eval machinery — each pinned by a CI-gated assertion.

Closures, recursion, and binding

```
(progn (defun fact (n) (if (<= n 1) 1 (* n (fact (1- n))))) (fact 6)) ; => 720
(defun make-adder (n) (lambda (x) (+ x n)))
(funcall (make-adder 10) 5) ; => 15
(let ((c 0)) (defun bump () (setq c (1+ c))) (bump) (bump) (bump) c) ; => 3
(defvar *dyn* 1) (defun rd () *dyn*) (let ((*dyn* 42)) (rd)) ; => 42
```

`fact` is recursion through nested VMs — `(fact 6)` is 720, each call running the function's chunk on a fresh VM. `make-adder` returns a closure that captured `n`, and `funcall`ing it with 5 yields 15 — the lexical capture of Chapter 6. `bump` closes over the `let`'s `c` and mutates it across three calls to 3, the shared mutable cell. And the `*dyn*` example is dynamic binding: `rd` reads the special variable through its value cell, and the `let` of `*dyn*` is visible to `rd` during its extent, so the result is 42, not 1 — the two binding disciplines, each demonstrated in one line.

Errors and the condition system

```
(condition-case e (car 5) (error e)) ; => (wrong-type-argument listp 5)
(condition-case e (substring "abc" 0 10) (error e)) ; => (args-out-of-range "abc" 0 10)
(condition-case e (signal 'arith-error '(1 2)) (error e)) ; => (arith-error 1 2)
(condition-case e (signal 'arith-error '(1)) ((arith-error error) 'caught)) ; => caught
(progn (define-error 'my-err "My" 'arith-error)
      (condition-case e (signal 'my-err nil) (arith-error 'parent))) ; => parent
```

These pin the error data model of Chapter 14. Catching `(car 5)` binds `e` to the full condition `(wrong-type-argument listp 5)` — the symbol plus its data — and an out-of-range `substring` yields `(args-out-of-range "abc" 0 10)`, the offending arguments preserved. The `signal` lines show the data list surviving the handler, and the fourth shows a handler matching a *list* of conditions `(arith-error error)`. The last builds a custom condition `my-err` under `arith-error` and confirms an `arith-error` handler catches it — inheritance through the `error-conditions` property, user-extensible.

Hash tables and structs

```
(let ((h (make-hash-table :test 'equal))) (puthash "a" 1 h) (gethash "a" h)) ; => 1
(let ((h (make-hash-table)) (s 0))
  (puthash 1 10 h) (puthash 2 20 h)
  (maphash (lambda (k v) (setq s (+ s v))) h) s) ; => 30
(gethash 'missing (make-hash-table) 'default) ; => default
(cl-defstruct point x y)
(let ((p (make-point :x 3 :y 4))) (list (point-x p) (point-y p))) ; => (3 4)
(cl-defstruct dd (x 0) (y 10))
(let ((p (make-dd :x 3))) (list (dd-x p) (dd-y p))) ; => (3 10)
(cl-defstruct sf x y)
(let ((p (make-sf :x 1 :y 2))) (setf (sf-x p) 99) (cl-incf (sf-y p) 10) (list (sf-x p)
  ~ (sf-y p))) ; => (99 12)
```

The hash table reads, writes, iterates, and defaults exactly as Chapter 33 catalogues — `maphash` summing values to 30, `gethash` returning the default `default` for a missing key. The struct examples show the generated constructor and accessors, the `(x 0)/(y 10)` default slot values (`make-dd :x 3` leaves `y` at its default 10), and the settable accessors — `(setf (sf-x p) 99)` and `(cl-incf (sf-y p) 10)` mutating slots through the generalized-place machinery of Chapters 22 and 39.

Regexp

```
(string-match "world" "hello world") ; => 6
(progn (string-match "é" "aébé") (list (match-beginning 0) (match-end 0))) ; => (1 2)
(string-match "\\(\\.\\)\\1" "abccba") ; => 2
(replace-regexp-in-string "\\([a-z]+\\)=\\([0-9]+\\)" "\\2:\\1" "x=1 yy=22") ; => "1:x
  ~ 22:yy"
(regexp-quote "a.b*c") ; => "a\\.b\\*c"
```

`string-match` returns the character index of a match; the Unicode test reports `(1 2)` — character positions, not bytes, the fidelity of Chapter 15. The backreference `\\(\\.\\)\\1` finds the doubled `cc` at index 2, working because of `fancy-regex`. The replacement swaps two groups with `\\2:\\1`, and `regexp-quote` escapes metacharacters so they match literally. Every backslash here is doubled at the Rust level but single in the elisp the form represents.

Backquote, pcase, and macros

```
(let ((x 5)) `(a ,x c)) ; => (a 5 c)
(let ((xs (list 2 3))) `(1 ,@xs 4)) ; => (1 2 3 4)
(pcase 2 (1 'one) (2 'two) (_ 'other)) ; => two
(pcase 42 (x (list 'got x))) ; => (got 42)
(defmacro my-when (c &rest body) `(if ,c (progn ,@body) nil))
(my-when t 1 2 3) ; => 3
(macroexpand '(push 1 lst)) ; => (setq lst (cons 1 lst))
(cl-macrolet ((sq (x) (list '* x x))) (sq 5)) ; => 25
(eval (read "(+ 1 2)")) ; => 3
(read-from-string "(a b)") ; => ((a b) . 5)
(cl-loop for i below 5 collect i) ; => (0 1 2 3 4)
(cl-loop for i below 10 by 3 collect i) ; => (0 3 6 9)
```

The backquote lines show the eager expansion of Chapter 3: `,x` splices a value, `,@xs` splices a list. `pcase` dispatches on a literal and binds a wildcard variable. The macro `my-when` expands and runs to 3, and `(macroexpand '(push 1 lst))` shows the expansion itself — `(setq lst (cons 1 lst))`, the compile-

time decision of Chapter 8. `cl-macrolet` binds a local macro; `(eval (read ...))` reads and runs text; `read-from-string` returns the object and the next index; and the two `cl-loop` forms show the numeric iterator. The smaller test files add their own spot-checks — `(plist-get '(:a 1 :b 2) :b)` is 2, `(cl-reduce (lambda (a b) (+ a b)) '(1 2 3 4))` is 10, `(pcase-let ((a 2) (b 3)) (+ a b))` is 5, `(mod -7 3)` is 2 (the divisor-signed modulo), `(eq (make-symbol "g") (make-symbol "g"))` is `nil` (fresh uninterned symbols), `(sort (vector 5 3 1 4 2) #'<)` is `[1 2 3 4 5]` (stable, type-preserving), `(format "%05d" -42)` is `"-0042"` (sign before zeros), and `(append "ab" nil)` is `(97 98)` (a string flattens to character codes). Each is a single behavior, asserted on every commit, and together they are the most precise specification the language has.

Chapter 48 – The Reader Syntax Reference

Chapter 3 explained how the reader works; this chapter is the complete catalogue of *what* it reads — every literal and syntactic form, with examples. It is the reference you reach for when a `.el` file uses a notation you do not recognize, and it is grounded in `reader.rs` and the reader's own tests. The reader produces heap objects directly, so every form here yields a value the rest of the system manipulates.

Integers and the trailing-dot rule

A bare run of digits with an optional sign is an integer: `42`, `-7`, `+19`. The corner case everyone misses is the trailing decimal point: `1.` is the *integer* `1`, not a float, and `-3.` is `-3`, because a decimal point with no fractional digits does not make a float. The reader implements this by trying an `i64` parse, then stripping a trailing `.` and trying again, before falling back to a float. Integers are `i64`; there is no bignum promotion, so the range is the 64-bit fixnum range.

Floats, including the infinities

A token with a fractional part or an exponent is a float: `1.5`, `3.0`, `1.e3`, `6.02e23`, `-0.5`. The non-finite floats have Emacs read syntax that the reader accepts and the printer emits, closing the round-trip: `1.0e+INF` and `-1.0e+INF` for the infinities and `0.0e+NaN` for not-a-number, with the sign preserved. An integer-valued float keeps its `.0` on printing — `(* 2.0 3)` prints `6.0` — so floatness survives a round-trip.

Numbers in other bases

The radix reader macros read integers in other bases. `#x1f` is hex (31), `#o17` is octal (15), `#b101` is binary (5), and the general `#NNrDIGITS` reads in base `NN` for any base 2 through 36 — `#16rFF` is 255, `#36rZ` is 35. Each accepts an optional sign after the prefix, so `#x-10` is -16. These exist because real code dealing with bytes, characters, and bitmasks uses them freely.

Character literals

A question mark introduces a character, and a character *is* an integer — `?A` is 65, `?0` is 48, `?` (question mark space) is 32. The escape vocabulary is the shared one: `?\n`, `?\t`, `?\r`, `?\e`, `?\a`, `?\b`, `?\v`, `?\f`, `?\d` (delete, 127), and `?\0`; the numeric forms `?\x41` (hex), `?\101` (octal), `?é` (four-hex Unicode), `?\U0001F600` (eight-hex), and `?\N{U+1F600}` (codepoint by name). On top of these are the modifiers — `?\C-` control, `?\M-` meta, `?\S-` shift, `?\H-` hyper, `?\s-` super, `?\A-` alt — each setting a high bit (meta is bit 27, control bit 26, and so on), and they compose, so `?\C-\M-a` carries both. Control folds the base character: `?\C-a` is 1, `?\C-A` is also 1, `?\C-?` is 127. The reader's tests pin the simple end — `(list ?\n ?\t ?\e ?\0)` is `(10 9 27 0)`.

Strings

A double quote begins a string, and the escape vocabulary is the character one — `\n`, `\t`, the hex/Unicode/octal forms — plus two string-specific wrinkles. A backslash before an actual newline is a *line continuation*: the newline is elided, so a long literal can wrap across source lines. And `\s` without a following dash is the space character. A string prints with its quotes and escapes under `prin1`, raw under `princ`.

Symbols and keywords

A token that is not a number and not a `#`-form is interned as a symbol. The delimiters that end a token are whitespace and the eight structural characters `( ) [ ] " ' ` , ;`; anything else extends the symbol, which is why `1+`, `<=`, and `string-match-p` are single symbols. A backslash escapes a delimiter into a name — `foo\ bar` is one symbol with a space — and any escape also forces the token to be a symbol, never a number, which is how you would name a symbol `1` (as `\1`). The empty-named symbol prints as `##`. A token beginning with a colon is a keyword — `:greeting` — and keywords self-evaluate, so the compiler loads one as a constant rather than looking it up. `nil` reads as the VM's `Undef` and `t` as boolean true.

Quote, function, and backquote

The reader macros for evaluation control are `'x` for `(quote x)`, `#'x` for `(function x)`, ``x` for backquote, `,x` for unquote, and `,@x` for unquote-splicing. Backquote is expanded *eagerly at read time* into a tree of `cons`, `append`, `vconcat`, and `quote` calls, which is what makes `pcase` backquote patterns recognizable — ``(a ,b ,@c)` is decomposed before the compiler ever sees it.

```
'foo           ; => (quote foo)
#'car          ; => (function car)
`(a ,b ,@c)    ; => a cons/append tree built at read time
```

Vectors, dotted pairs, and records

Square brackets read a vector with *unevaluated* elements, so `[a b c]` is a literal vector of three symbols, self-evaluating, distinct from the call `(a b c)`. A list may be improper: `(a b . c)` reads with the final `cdr` being `c` rather than `nil`, the dotted pair that backs association lists. The `#s(...)` reader macro reads two things: `#s(hash- table test eq data (k1 v1 ...))` is a literal hash table, and `#s(NAME slot...)` is a record — the print syntax of a `cl-defstruct` instance — read into a tagged vector. This is the syntax the printer emits for records and hash tables, so they round-trip. The `#(...)` form reads a propertized string, but the text properties are *dropped*, since this model does not carry them.

What the reader does not read

Two notations are deliberately out of scope. The circular-and-shared-structure syntax `#N=` and `#N#` is not implemented — there is no label table — so a `#` followed by a digit is routed to the radix reader and errors if no `r` follows. And text properties on `#(...)` strings are parsed but discarded. Comments — `;` to end of line — and whitespace are skipped by `skip_ws` before every form, which is why a paren inside a comment or a string never counts as structure. These are the edges; everything else above is the complete surface the reader accepts.

Chapter 49 – The Condition System and Error Reference

Errors in elisp are a control-flow mechanism with a data model: conditions are symbols arranged in an inheritance hierarchy, signalled with data, and dispatched by handlers. This chapter is the reference for that system — the standard conditions, how the hierarchy is encoded, how matching works, and the convenience macros over it — grounded in the prelude's `define-error` block and Chapter 14.

How a condition is encoded

`define-error` records a condition's place in the hierarchy as two symbol properties: an `error-conditions` list (the symbol plus all its ancestors) and an `error-message` string. `(define-error 'bad-token "Bad token" 'parse-error)` sets `bad-token`'s `error-conditions` to `(bad-token parse-error error)`, so a handler for any of those three catches a signalled `bad-token`. The whole hierarchy is this list-of-ancestors encoding, and `condition-case` matching is a membership test: a handler matches if its condition appears in the signalled condition's `error-conditions`.

The standard conditions

The prelude seeds the standard hierarchy with a block of registrations, all descending from the umbrella `error`:

```
(define-error 'error "error" nil)
(define-error 'user-error "")
(define-error 'quit "Quit")
(define-error 'args-out-of-range "Args out of range")
(define-error 'arith-error "Arithmetic error")
(define-error 'wrong-type-argument "Wrong type argument")
(define-error 'wrong-number-of-arguments "Wrong number of arguments")
(define-error 'void-variable "Symbol's value as variable is void")
(define-error 'void-function "Symbol's function definition is void")
(define-error 'invalid-function "Invalid function")
(define-error 'wrong-length-argument "Wrong length argument")
(define-error 'invalid-regexp "Invalid regexp")
(define-error 'end-of-file "End of file during parsing")
(define-error 'cl-assertion-failed "Assertion failed")
```

The messages are the exact Emacs strings, which is what lets `error-message-string` render a condition the way Emacs does — the rendering the command-line driver uses to turn an internal `void-variable: foo` into “Symbol's value as variable is void: foo.” Two more conditions, `cl-no-applicable-method` and `cl-no-next-method`, are registered later by the generic-function engine, so a `cl-defmethod` dispatch failure signals a catchable condition. These are the conditions a runtime raises: `car` of a non-list signals `wrong-type-argument`, an out-of-range `substring` signals `args-out-of-range`, integer division by zero signals `arith-error`, a void variable read signals `void-variable`.

Signalling and catching

(`signal SYMBOL DATA`) is the primitive that raises a condition with a data list; (`error FORMAT ...`) and (`user-error FORMAT ...`) are the formatted-message wrappers. `condition-case` runs a body and dispatches on the signalled condition:

```
(condition-case e
  (signal 'bad-token '("oops"))
  (parse-error (list 'caught-by-parent (cadr e)))
  (error 'caught-by-root))
```

The handler whose condition matches first wins, and the handler variable — `e` — is bound to the full (`SYMBOL . DATA`) condition, so (`cadr e`) reads the first data element, `"oops"`. A handler may name a *list* of conditions, (`(arith-error error) ...`), matching any of them. The data list is preserved exactly through the handler, which is what makes (`condition-case e (signal 'arith-error '(1 2)) (error e)`) yield (`arith-error 1 2`).

catch, unwind-protect, and the convenience macros

`catch` and `throw` are the non-condition escape — a labeled jump that returns a value, unrelated to the error hierarchy — and `unwind-protect` is the `try/finally` that guarantees its cleanup runs on any exit. Over `condition-case` the prelude builds the convenience macros: `ignore-errors` wraps its body to catch `error` and return `nil`, `ignore-error` catches a specific condition, `with-demoted-errors` turns an error into a message and continues, and `with-suppressed-warnings` and `with-no-warnings` quiet the compiler. ERT's `should-error` is the testing face: it runs a body inside a `condition-case` and *fails* if no error was signalled, optionally checking a `:type` — so (`should-error (/ 1 0) :type 'arith-error`) passes only if the body raised exactly an `arith-error`. Every one of these is elisp over the intrinsics and the hierarchy, and a user extends the hierarchy with `define-error` exactly as the standard conditions were seeded — which `data-model.el` proves by building its own `parse-error/bad-token` hierarchy and catching across it.

Chapter 50 — Minor Libraries: kbd, Time, Completion, and Display

The prelude carries several smaller libraries that round out the standard surface — key notation, time arithmetic, string completion, and display-width measurement. None is large enough for its own movement, but each is real, used by actual `.el` code, and worth cataloguing so the reader knows it is there. All are elisp over the primitives.

Key notation: kbd and key-description

`kbd` parses Emacs key notation into the internal key-sequence representation — the function you use to write (`kbd "C-c C-x"`) for a key binding. It is built from a small tokenizer (`kbd--token`, `kbd--ctrl`, `kbd--char`, `kbd--sym`) that reads each space-separated chunk, applies the `C-`, `M-`, `S-`, `H-`, `s-`, `A-` modifiers as the character-modifier bits of Chapter 3, and recognizes the named keys `RET`, `TAB`, `SPC`, `ESC`, `DEL`. The inverse pair `key-description` and `single-key-description` render a key sequence back to its notation. Outside the editor there are no keymaps to bind into, but the parsing is faithful, so code that constructs key sequences as data — a help generator, a which-key table — works.

Time arithmetic

On top of the clock primitives of Chapter 37, the prelude builds a time-arithmetic library. `time-add` and `time-subtract` add and subtract time values, `time-less-p` and `time-equal-p` compare them, and `time-convert` and `time-to-seconds` change representation. `format-seconds` renders a duration in seconds as a human string, and `current-time-zone` reports the offset. The decoded-time accessors — `decoded-time-second`, `decoded-time-minute`, `decoded-time-hour`, `decoded-time-day`, `decoded-time-month`, `decoded-time-year`, `decoded-time-weekday`, `decoded-time-dst`, `decoded-time-zone` — read the fields of a `decode-time` list by name rather than by position, which is how modern elisp avoids the magic indices the raw list would require. `ngettext` picks a singular or plural message form by a count, the small `i18n` helper that belongs with the formatting functions.

Completion

The completion library does prefix completion over a collection of strings — the engine behind an editor’s minibuffer completion, here usable as a pure string operation. `all-completions` returns every string in a collection that has a given prefix, `try-completion` returns the longest common extension of the prefix (or `t` if it is an exact unique match), and `test-completion` tests whether a string is a complete member. They are built on the helpers `completion--common-prefix` and `completion--elements`, and they accept a list, a vector, or a hash table as the collection. Outside the minibuffer they are still useful: a command dispatcher that completes a partial command name, a configuration system that resolves an abbreviation, both reach for `try-completion`.

Display width

Text has two lengths — its character count and its *display* width, which differ when a string contains wide characters (CJK) or combining marks. `string-width` returns the display width, `char-width` the width of one character, and `truncate-string-to-width` cuts a string to fit a column budget without splitting a wide character. `string-glyph-split` splits into grapheme clusters rather than codepoints. These are the functions a report formatter needs to align columns correctly when the data is not pure ASCII — `render-table`'s `string-pad` aligns by character count, but a CJK-aware version would pad by `string-width`. The prelude also carries the pretty-printers `pp` and `pp-to-string`, which print a value with line breaks and indentation for human reading, and their `cl-` variants `cl-prin1` and `cl-prin1-to-string`.

Generalized variables: `gv` and `setf`

Underneath the place macros of Chapter 39 is the generalized-variable machinery — Emacs's `gv` — which the prelude implements as the full `setf` place-expander `setf--expand`. A *place* is any form that names a settable location, and `setf` knows how to invert the common accessors: `(setf (car x) v)` becomes `(setcar x v)`, `(setf (aref v i) x)` becomes `(aset v i x)`, `(setf (gethash k h) x)` becomes `(puthash k x h)`, and a struct accessor `(setf (point-x p) v)` becomes an `aset` at the registered slot index. Every place macro — `cl-incf`, `cl-push`, `cl-rotatef`, `push`, `pop` — expands through this one expander, which is why they all work uniformly on any place. The expander is the quiet foundation that makes “modify a location” a composable operation rather than a special case per accessor, and it is, like everything in these minor libraries, elisp over the primitive floor.

Chapter 51 — Region, Word, and the `subr.el` Grab-Bag

The prelude's largest section is a grab-bag of `subr.el`-style helpers — the region and word operations on buffers, the small string and list utilities, and the type predicates that did not fit the earlier catalogues. This chapter sweeps them, because together they are most of what makes `elisp`s feel like Emacs Lisp rather than a subset, and every one is a prelude definition over the primitive floor.

Region and word operations

The buffer model supports region-wide and word-wide edits built on the point-and-text primitives of Chapter 36. The case operations on a region are `upcase-region`, `downcase-region`, and `capitalize-region`, and on a word `upcase-word`, `downcase-word`, and `capitalize-word`, each moving point and rewriting the span. `subst-char-in-region` replaces one character with another throughout a span, the in-place edit a sanitizer uses. The structural edits are `transpose-chars` (swap the two characters around point), `kill-line` (delete to end of line), `newline` and `open-line` (insert line breaks), and `insert-before-markers`. The counting functions are `count-words`, `how-many` (count regexp matches in a region), `buffer-hash` (a content hash), and `following-char`/`preceding-char` (the characters at point, as integers). These are the operations a text-munging script reaches for once it is past simple search-and-replace, and `buffers.el` exercises the case and transpose operations directly.

String utilities

Beyond the `subr-x` trim-and-pad family, the prelude carries a wider string surface. `capitalize` and `upcase-initials` case the start of words; `propertize` attaches text properties (minimally, since this model does not preserve them); `string-reverse` flips character order; `rot13-string` applies the classic cipher; and `string-version-lessp` compares version strings numerically so `"foo10"` sorts after `"foo9"`. The quoting pair `combine-and-quote-strings` and `split-string-and-unquote` round-trip a list of strings through a shell-quoted line, and `make-separator-line` builds a divider. The comparison predicates `string=`/`string-equal`, `string<`/`string-lessp`, `string>`, and the case-insensitive `string-equal-ignore-case` give the lexicographic vocabulary, and `string-blank-p`, `string-empty-p`, and `string-prefix-p`/`string-suffix-p` the tests. `format-spec` is a named-field formatter — `%n`-style substitution from an alist — distinct from the positional `format`.

List and sequence utilities

The list grab-bag fills in the operations the core ten primitives do not cover. `flatten-tree` and `flatten-list` flatten arbitrary nesting; `copy-tree` deep-copies cons structure and `copy-sequence` shallow-copies any sequence; `ensure-list` wraps a non-list in a one-element list; `mapcan` maps and concatenates destructively; and `proper-list-p` returns the length of a proper list or `nil` for an improper or circular one. The alist mutators `assq-delete-all`, `assoc-delete-all`, and `rassq-delete-all` remove entries by key or value, and `add-to-list` conses an element onto a list variable unless already present — `(progn (defvar al '(a`

b)) (add-to-list 'a1 'c) a1) is (c a b), as `library.rs` asserts. The set operations `seq-union`, `seq-difference`, and `seq-intersection`, and the boolean `xor`, complete the list algebra.

The remaining predicates

The type-predicate surface is broader than Chapter 29's core. `booleanp` tests for `t` or `nil`; `characterp` for a valid character code; `sequencep` for any sequence (list, vector, or string); `arrayp` for a vector or string; `keywordp` for a `:`-led symbol; `natnump` and `wholenump` for non-negative integers; `proper-list-p` for a proper list; `string-or-null-p` and `number-or-marker-p` for the union types; and `nlistp` for the negation of `listp`. These are the fine-grained tests that `cl-typecase` dispatch and defensive argument-checking lean on, each a one-line `defun` over the primitive type tests. Taken with everything in this chapter, they are the long tail of the standard library — individually small, collectively the difference between a language that runs a toy `.el` file and one that runs a real package, and all of it elisp on the small core.

Chapter 52 – Limitations, Edges, and the Moving Frontier

A book grounded in the source has to be as precise about what does *not* work as about what does, and elisprs makes that easy because the project is candid about its own edges — it would rather fail loudly than misbehave quietly. This chapter collects the limitations scattered through the earlier chapters into one honest reference, each with its cause and its status, so a reader knows exactly where the moving frontier is.

The buffer-scoping special forms, now lowered

The three buffer-scoping forms — `save-excursion`, `save-current-buffer`, and `save-restriction` — were once the project's one named “buffer milestone,” recognized by the compiler and deliberately refused. They now work. The prelude defines them as macros over marker-backed intrinsics: `save-excursion` saves the current buffer and a point marker that tracks edits, `save-restriction` saves the narrowing bounds the same way, and `save-current-buffer` restores the selected buffer if it is still live, each wrapped in an `unwind-protect` so the restore runs on a non-local exit. Because macroexpansion rewrites them before the compiler sees a call, the old `is_unsupported_special` guard in `compile_call` is now a dormant fallback rather than a wall — it survives as belt-and- braces, but no path through a loaded prelude reaches it. There are no special forms the compiler refuses today.

The numeric model: i64, no bignum

elisprs integers are `i64` and do not promote to bignums on overflow, where modern Emacs carries GMP and widens. A computation that exceeds the 64-bit fixnum range wraps or, in a debug build, panics. A few operations are written to wrap deliberately — `ash/lsh` (`wrapping_shl`), `expt` (`wrapping_pow`), `abs` (`wrapping_abs`), the radix `string-to-number` (`wrapping_mul`) — so they stay defined at the boundary. The practical effect is full fidelity across the fixnum range, where real code lives, and divergence only at the bignum frontier, which would close by widening the value type rather than bolting on a bignum.

AOT constant reification

The ahead-of-time path lowers and links a real native binary, but it carries a named gap: a *user* program's interned symbols, quoted data, and closure templates are not yet fully reconstructed at load from the embedded heap image alone. The `aot_runtime.rs` source states it plainly — that reconstruction “requires constant reification (compiling symbols/quotes/closures as runtime construction instead of baked heap handles),” and until it lands, “AOT objects run correctly only for programs whose constants are all reconstructed by the prelude/builtins.” The lowering and linking work today; the user constant model is the frontier.

The buffer model's edges

The buffer object model is now real: named buffers with a current-buffer notion, markers that move with edits, narrowing, per-character text properties, buffer-local variables, and a local keymap. `current-buffer`,

`bufferp`, `buffer-name`, and `buffer-live-p` are live — what was an expected-failure test in `ert.el` is now a passing `ert-buffer-object-model` test. The edge that remains is the *display and interaction* surface, not the objects: there are no overlays, no windows or frames, and no interactive command loop. Two smaller seams persist — `forward-word` uses a plain alphanumeric rule and does not consult the syntax table, and the `#( . . . )` property-string *literal* is read with its properties discarded even though `propertyize` and `put-text-property` build and preserve properties at run time. Treat the live editor display as the out-of-scope line, and the buffer object surface as settled.

The hash table is a linear scan

The `Obj::HashTable` is an association vector scanned linearly, not a hashed store — the source calls it “fine for the table sizes elisp config uses,” which is true for the tens of entries a `.el` file builds but is not the asymptotic guarantee a large table would want. The interface is the full Emacs hash-table surface, so code runs unchanged; the difference is in the constant factors, not the behavior.

Hashing, equality, and the regexp corners

`sxhash` and its family return values that are *not* bit-compatible with GNU Emacs’s — they are an internal detail — but they are self-consistent, so `equal` objects hash alike within a bounded depth, which is what a hash table needs. The regexp translator handles the common, portable subset and names its approximations: non-sequential explicitly- numbered groups `\(?N: . . . \)` are not preserved (sequential ones are), the symbol boundaries `\_</\_>` are approximated with a plain word boundary, and the point assertion `\=` has no analogue and compiles to an empty match. The native arithmetic fast path skips the host’s wrong-type signaling — an accepted trade for numbers — so a native `+` on a non-number will not raise `wrong-type-argument` the way the subr would.

No garbage collector, and the over-unwind edge

The arena only grows; there is no garbage collector, which is adequate for run-to- completion scripts and REPL forms and is the deterministic property the bytecode cache exploits, but a long-lived process that allocated unboundedly would eventually want collection. And the hardest edge to get exactly right is the specstack unwinding across a non-local exit: the `errors.el` example documents a known case where a loop-break via `throw` inside a nested funcall can over-unwind the specstack, the honest acknowledgment that unwinding across the lexical/dynamic boundary is genuinely hard — the same boundary whose fix made `pcase` and `cl-loop` work inside an ERT assertion. None of these is hidden; each is named in the source or the tests, which is the posture that lets the rest of the book be trusted: where the frontier is, the project says so.

Chapter 53 – The Command Line and Tooling Reference

This chapter is the operational reference: how you invoke the `elisp` binary in each of its modes, how the REPL behaves, what the language server and debug adapter expose, how ahead-of-time compilation is driven, and the environment seams that control caching. It consolidates the operational facts scattered through Chapters 2, 11, 17, and 18 into one quick reference.

The modes of the elisp binary

`main` is a dispatcher that checks, in order, for `--version/-V`, `--help/-h`, `--lsp`, `--dap`, `--aot-exe`, `--aot`, `-e/--eval`, and finally the first non-flag argument as a file path; with no arguments it drops into the REPL. The modes are mutually exclusive and the precedence is fixed.

```
elisp program.el           # evaluate a file (for effect)
elisp -e "(+ 1 2)"         # evaluate one expression, print the result
elisp                     # start the REPL
elisp --lsp                # language server over stdio
elisp --dap                # debug adapter over stdio
elisp --aot program.el -o program.o # emit a native object
elisp --aot-exe program.el -o program # emit and link a standalone binary
elisp --version            # print the version
```

A file run completes with exit status zero, or prints `error: ...` to stderr and exits non-zero on a signalled error — which is what makes a file ending in `ert-run-tests-batch-and-exit` a CI gate. The `-e` form prints its result in readable (`prin1`) form, so a one-liner is a calculator. A second binary, `gen-docs`, renders the language-server corpus to `docs/reference.html` and is run before publishing the docs site; it shares the corpus with the live server, the single-source-of-truth arrangement.

The REPL

Run with no arguments, `elisp` reads a line, decides whether the parens balance, and evaluates a complete form. The prompt is `elisp>` for a fresh form and `...>` while accumulating an unbalanced one; the balance check counts open and close parens while ignoring those inside strings (with backslash escaping) and comments (`;` to end of line). Ctrl-D exits. Every form typed at the prompt goes through the same read, expand, lower, run pipeline as `-e`, so there is no corner-cutting interactive evaluator — the REPL is fast because the machine underneath is a compiler and a JIT.

The language server and debug adapter

`elisp --lsp` advertises five capabilities: full-document sync, completion on `(`, hover, document symbols, and signature help on `(` and space — read-only intelligence from the static corpus, with diagnostics (unterminated string, unmatched parens, unsupported backquote-in-context) from a lexical pass. It does not offer go-to-definition, rename, or formatting, by design. `elisp --dap` runs your code for real, with breakpoints and stepping at top-level-form granularity, a single synthetic `toplevel` frame, a `Globals` scope that enumerates

the live value cells, and an `evaluate` request that reads, compiles, and runs an expression against the live host. The debugger captures the debuggee's stdout through a `pipe/dup2` so program output streams as `output` events instead of corrupting the JSON-RPC channel, while `message` output (on stderr) passes through untouched.

Ahead-of-time compilation and the cache

`--aot` emits a native object and `--aot-exe` links it into a standalone executable that needs no `elisp` binary present. The standalone binary is the AOT object (chunk plus embedded heap image), linked against the `elisprs` runtime staticlib (carrying `fusevm`'s AOT runtime and the `fusevm_aot_register_builtins` hook) and a small C `main`, with the platform libraries the Rust staticlib needs — `CoreFoundation` and `Security` on macOS, `pthread/dl/m/rt` on Linux. The bytecode cache lives at `~/.elisprs/scripts.rkyv`, keyed by file path, modification time, and a schema fingerprint of the builtin layout and prelude; setting `ELISPRS_CACHE_DEBUG` makes each run report whether it hit or missed the cache and how many chunks it loaded. The cache is best-effort — a run with no writable `~/.elisprs` recompiles, correctly, only slower — and the AOT and cache paths both reuse `fusevm`'s backend, the dividend of being a frontend that this reference, like the whole book, keeps returning to.

Chapter 54 — The ERT Framework, From the Inside

Every example file in this book ends with `ert-run-tests-batch-and-exit`, and every assertion is a `should`. ERT — Emacs Lisp Regression Testing — is the framework the whole project is gated on, and it is itself elisp in the prelude, built entirely from the error system of Chapter 14. This chapter reads its implementation, because nothing demonstrates the completeness of the core better than a test framework written in the language it tests, with no special runtime support.

should: an assertion that explains

`should` evaluates a form and signals `ert-test-failed` if the result is `nil`, otherwise returning the value. The interesting part is that a *good* assertion explains its failure — when `(should (= (+ 1 2) 4))` fails, the message names the form and the values its subexpressions took. `should` achieves this at expansion time: when the asserted form's head is a recognized comparison, it expands into code that evaluates each argument into a fresh temporary, tests the predicate, and on failure signals with a `:values` plist pairing each argument form with its value.

```
;; should on a comparison expands (in shape) to:
;; (let ((g1 (+ 1 2)) (g2 4))
;;   (if (= g1 g2) t
;;       (signal 'ert-test-failed
;;               (list 'should '(= (+ 1 2) 4) :values (list (cons '(+ 1 2) g1) ...)))))
```

The helpers `ert--value-pairs` and `ert--argsyms` build the temporaries and the form/value pairs, and `ert--explain-fns` records which heads get this treatment. The `tests/ert.rs` integration tests confirm the result precisely — catching the `ert-test-failed` from `(should (= (+ 1 2) 4))` and asserting the message carries both the form `should (= (+ 1 2) 4)` and the explained value `(+ 1 2) . 3`. That is the difference between “an assertion failed somewhere” and “this subexpression was 3, not 4,” and it is implemented purely in the macro. `should-not` is the negation — it fails if the form is *non-nil*.

should-error and the type check

`should-error` is the most error-system-entangled assertion: it runs its body inside a `condition-case` that catches `error`, and it *fails* if no error was signalled, optionally checking that the signalled condition matches a `:type`.

```
(should-error (/ 1 0) :type 'arith-error)
(should-error (car 5) :type 'wrong-type-argument)
```

`should-error` only passes if the body raised, and with `:type` only if it raised the right condition — so each assertion exercises the entire signal-and-match path of Chapter 49, including the `error-conditions` hierarchy walk. It is the assertion that makes the error catalogue testable, and it is why `arithmetic.el` can pin `(/ 1 0)` to `arith-error` and `(car 5)` to `wrong-type-argument`.

ert-deftest, skip-unless, and the batch runner

`ert-deftest` registers a named test in the `ert--tests` table, stripping an optional docstring and recognizing the `:expected-result :failed` keyword that marks an expected failure — an XFAIL. `skip-unless` signals `ert-test-skipped` to short-circuit a test whose precondition is unmet, so a test that depends on an unimplemented feature can declare its own irrelevance rather than fail. `ert-run-tests-batch-and-exit` runs every registered test inside a `condition-case`, classifying each by which condition it caught: a clean return is a pass, an `ert-test-failed` is a fail, an `ert-test-skipped` is a skip, and an XFAIL that fails is an *expected* failure (counted as passing) while an XFAIL that *passes* is an unexpected success (counted as failing). It then exits non-zero if any test failed unexpectedly.

That classification logic is itself tested — `tests/ert.rs` confirms an XFAIL that fails counts zero failures, an XFAIL that passes counts one (the unexpected success), and a passing test alongside an XFAIL keeps the count at zero. This is the meta-level guarantee: the framework every other file relies on is itself asserted correct, so when `ert.el` marks `(bufferp (current-buffer))` as `:expected-result :failed`, the suite stays green and the test will flip to a real pass — and need its marker removed — the day the feature lands. The whole of ERT — `should`, `should-not`, `should-error`, `skip-unless`, `ert-deftest`, the batch runner, the explain machinery — is a few hundred lines of prelude elisp over `condition-case`, `signal`, and the condition hierarchy. A testing framework, like the loop facility and the object system, is just more elisp expanding into the special forms. That is the architecture's final word, made concrete in the tool that guards the architecture.

Chapter 55 – Inside cl-loop: A Compiler in the Prelude

`cl-loop` is the largest and most intricate macro in the prelude, and Chapter 26 walked its surface — the clause vocabulary and the driver shape. This chapter reads its *implementation*, because it is the clearest single proof that a macro can be a compiler: `cl-loop` parses a clause stream at expansion time and generates a `let*/catch/while` driver, all in elisp, touching neither the core compiler nor the VM.

The parser

The macro is backed by a small parser. `cl-loop--kw` recognizes loop keywords, `cl-loop--clause-p` classifies the next clause, and `cl-loop--accum` handles the accumulator clauses. The macro walks the clause list, and for each clause it appends to four growing pieces of the eventual driver: the `with-and-for` bindings (which become the `let*`), the loop-condition test (which becomes the `while` condition), the body forms (the `while` body), and the `finally` epilogue. The accumulators add a hidden accumulator variable to the bindings and a `setq` onto it in the body. So the parser is, precisely, a code generator: it reads the clause grammar and emits the four parts of a loop.

Generating the iterators

A `for V from A to B` clause binds `V` to `A` in the `let*`, adds `(<= V B)` to the condition, and adds `(setq V (1+ V))` (or the `by` step) to the end of the body. `for V in LIST` binds a hidden tail pointer to the list, tests it non-nil, binds `V` to its `car`, and advances the tail — exactly the `dolist` expansion of Chapter 8, generated inline. `for V across SEQ` indexes a sequence; `for V = INIT then STEP` binds and re-steps an arbitrary recurrence; and `for V being the hash-keys of TABLE using (hash-values U)` materializes the table's keys and values and walks them in parallel. The `stats.el` assertion `(cl-loop for x from 1 to 5 for y = (* x x) sum y)` is 55 because the parser generates two `for` clauses — a numeric counter and a `= ... then`-style recurrence — and a `sum` accumulator, all into one driver.

Generating the accumulators and exits

`collect` appends to a hidden list (kept in reverse and `nreversed` in the `finally`), `sum` and `count` add to a hidden number, and `maximize/minimize` track an extreme. `into V` names the accumulator so the body or `finally` can see it. The quantifiers are why the driver wraps the `while` in a `catch`: `always` returns `t` unless its condition fails, short-circuiting to `nil` with a `throw`; `never` is its negation; and `thereis` throws the first non-nil value its expression produces. So `(cl-loop for x in '(1 2 3) thereis (and (cl-evenp x) x))` is 2 because the parser generates a `throw` of the first even element to the `catch` the driver established. The conditional clauses `when/unless/if/else` wrap the clause that follows in an `if` in the generated body.

The whole compiler

Assemble the pieces and `(cl-loop for i from 1 to 5 collect i)` becomes, after expansion, roughly a `(let* ((i 1) (--acc-- nil)) (catch '--loop-- (while (<= i 5) (setq --acc-- (cons i --acc--)) (setq i (1+ i)))) (nreverse --acc--))` — a `let*` for the counter and accumulator, a `catch` for early exit, a `while` for the iteration, and a `finally`-style `nreverse` for the result. That is a real compiler: a parser that reads the clause grammar, a code generator that emits the four driver parts, and an output of three special forms the core already understood. The CHANGELOG's history of `cl-loop` is a list of parser extensions — a new `for` form, a new accumulator, a `named` clause — none of which touched `compiler.rs`, because each just taught the prelude parser one more clause. If a sub-language the size of the Common Lisp loop facility can be a prelude macro, the core is complete; `cl-loop` is that proof at its largest scale, and reading its implementation is reading the macro-and-prelude thesis carried all the way to a compiler written in the language it compiles.

Chapter 56 – Inside `pcase` and `rx`: Two More Prelude Compilers

`cl-loop` is not the only compiler hiding in the prelude. `pcase` compiles structural patterns into a `cond`, and `rx` compiles S-expression regular expressions into pattern strings. Both are elisp, both parse a small grammar and generate core forms, and reading their implementations completes the picture of the prelude as a stack of little compilers.

`pcase`: patterns to a `cond`

The `pcase` macro binds the scrutinee once and maps each clause through a clause compiler, assembling a `cond`:

```
(defmacro pcase (expr &rest clauses)
  `(let ((--pcase-v-- ,expr))
     (cond ,@(mapcar (function pcase--clause) clauses))))
```

The work is in `pcase--compile`, which turns one pattern into a pair (`TESTS . BINDS`) — a list of boolean tests over the value and a list of `let*` bindings the pattern introduces. The dispatch is a `cond` over the pattern's shape. The wildcard `_` is no test and no binding; a literal is an `equal` test; a bare symbol is a binder capturing the whole value; (`pred FN`) is a (`funcall FN val`) test; (`guard E`) is the expression as a test; and the compound patterns recurse. The `and` arm concatenates its subpatterns' tests and bindings, the `or` arm combines them disjunctively, and the `cons` arm is the heart of destructuring:

```
((eq head 'cons)
 (let ((cr (pcase--compile (nth 1 pat) (list 'car-safe val)))
       (cd (pcase--compile (nth 2 pat) (list 'cdr-safe val))))
  (cons (cons (list 'consp val) (append (car cr) (car cd)))
        (append (cdr cr) (cdr cd)))))
```

It tests that the value is a `cons`, then recursively compiles the `car`-pattern against (`car-safe val`) and the `cdr`-pattern against (`cdr-safe val`), threading the sub-tests and sub-bindings up — which is how a nested `(,a . ,rest)` decomposes a structure, working because the reader's eager backquote already turned the template into `cons` calls. A deliberate detail, quoted in the source, is why the whole thing is a `cond` rather than a `catch/throw`: a `cond` shape “expands cleanly when nested in a macro-produced `defun` (e.g. an ERT `should`); a `catch/throw` shape does not — it miscompiles to a ‘void variable’ error there,” the specstack-leak edge of Chapter 6 seen from the macro author's side. The pattern vocabulary extends to (`app FN PAT`) (match against a function's result), backquoted patterns (via `pcase--list->cons`), and (`rx ...`) as a pattern. The family macros `pcase-let`, `pcase-let*`, `pcase-exhaustive`, `pcase-dolist`, and `pcase-setq` build on the same compiler.

`rx`: S-expressions to a regexp

`rx` compiles a regexp written as a nested S-expression into an Emacs pattern string at expansion time, the antidote to the backslash-heavy syntax of Chapter 15. The entry `rx` delegates to `rx-to-string`, which dispatches each form through `rx--form`, `rx--seq`, and the list translator `rx--list`:

```

(defun rx--list (form)
  (let ((head (car form)) (args (cdr form)))
    (cond
      ((memq head '(seq sequence : and)) (rx--seq args))
      ((memq head '(or |))
       (if (and args (rx--all-1char-p args))
           (concat "[" (mapconcat 'rx--1char args "") "]")
           (concat "\\(?:" (mapconcat 'rx--form args "\\|" ) "\\)")))
      ((memq head '(group submatch)) (concat "\\(" (rx--seq args) "\\)"))
      ((memq head '(zero-or-more * 0+)) (concat (rx--quant-body args) "*"))
      ((memq head '(one-or-more + 1+)) (concat (rx--quant-body args) "+"))
      ((memq head '(zero-or-one opt optional)) (concat (rx--quant-body args) "?")))))

```

So `(group (one-or-more digit))` compiles to `\([0-9]+\)`, `(seq ...)` concatenates its parts, and `(or "a" "b" "c")` folds to the character class `[abc]` when every alternative is a single character — the Emacs fidelity detail that `rx--all-1char-p` checks. The character-set helpers `rx--charset`, `rx--class-in`, and `rx--syntax-code` handle `(any ...)`, `(in ...)`, and the syntax classes. The output is an ordinary Emacs regexp string, which then flows into the Chapter 15 translator and `fancy-regex` like any hand-written pattern. Both `pcase` and `rx` are, like `cl-loop`, parsers-and-generators written in elisp, compiling a small notation into forms the core already understands — three little compilers riding on a complete core.

Chapter 57 – The fusevm Engine and the Frontend Family

The whole book has turned on one claim: `elisprs` owns the language and rents the engine. This chapter is the reference for the engine it rents — what `fusevm` provides, how the rental is structured in the manifest, and why the bargain holds — consolidating the architecture’s economic case into one place.

What fusevm is

`fusevm` is a language-agnostic bytecode virtual machine written in Rust, with a three-tier Cranelift JIT and an ahead-of-time native compiler. It executes a stack machine: opcodes push and pop a value stack, control flows by jumps, and each chunk carries a constant pool and a names table. It understands a small set of value kinds — integers, floats, strings, booleans, and an opaque object handle — and it can do arithmetic, comparison, and branching on the ones it understands. What it does *not* understand is anything language-specific: it has no notion of a cons cell, a symbol, a binding, or an error condition. That ignorance is the design: by knowing nothing about any particular language, `fusevm` can be the same VM for a shell, a Perl dialect, AWK, Vimscript, and Emacs Lisp.

The frontend family

`fusevm` is shared across five languages — `zshrs` (a Z shell rewrite), `stryke` (a Perl-compatible scripting language), `awkr` (an AWK implementation), `vimlrs` (Vimscript), and `elisprs`, the fifth consumer. None of the five agrees with another on a value model, a binding discipline, or a syntax; what they share is the generic stack machine, which is the part that is genuinely the same for every compiled language. The economic consequence is direct: the VM, the three-tier JIT, the AOT emitter, and the on-disk native-code cache are written once and maintained once, and every improvement lands in all five at once. A JIT speedup for `stryke`’s hot loops speeds `elisprs`’s; a fix in the shared dispatch loop fixes every frontend. `elisprs` did not build a VM, a JIT, an AOT compiler, or a code cache, because four other languages had already built them and the build was reusable.

The rental, in the manifest

The rental is exact and itemized in `Cargo.toml`. `elisprs` depends on `fusevm = "0.14.5"` with two features. `jit-disk-cache` buys “`fusevm`’s three-tier Cranelift JIT *and* its persistent on-disk cache of compiled native code (keyed by chunk hash), so hot elisp loops trace-compile and warm runs skip recompilation.” `aot` buys the native-object emitter used by `--aot` and `--aot-exe`. The version pin is the line where the engine’s capabilities are fixed: when `fusevm`’s calling convention or its `Value` type changes, `elisprs` changes at the next bump; when the JIT gets faster, `elisprs` gets faster at the next bump. That coupling is the cost side of the bargain, and the feature flags are the bill — a tiering JIT, a code cache, and an AOT backend, for a version pin and a narrow protocol.

The one interface

The whole rental flows through a single interface: the value variant `Value::Obj(u32)`, an opaque handle into elisprs's own object arena, and the extension-opcode family `Op::Extended/Op::ExtendedWide`, a callback into elisprs's own runtime. With those two reservations, fusevm carries elisp's cons cells and symbols on its stack, runs elisp's compiled bytecode in its dispatch loop, JITs elisp's hot loops to native code, and never once needs to understand a single elisp concept. The ten extension ops of Chapter 42 are the complete contract: implement those ten callbacks and a value handle, and you have an elisp runtime on a borrowed engine. The cost is contained and honest — elisprs can only express what lowers to fusevm ops plus its callbacks, which is why the three buffer-scoping special forms that do not yet lower fail loudly rather than miscompiling. The benefit is everything else. That is the bargain the book opened with, stated one last time as a reference: own the personality of the language, rent the generic machine, and connect them with ten callbacks and a handle.

Chapter 58 – From Source to Result, One Form Traced

The book has described every stage of the pipeline separately. This chapter runs one tiny program through all of them at once, tracing a function definition and a call from text to result, naming the exact reader output, macro expansion, opcodes, and handler arms at each step. It is the synthesis chapter — the place where the reader, the compiler, and the extension handler stop being separate accounts and become one path.

The program

```
(defun sq (x) (* x x))
(sq 6)
```

Two top-level forms: a function definition and a call. The file evaluator reads all the forms, then compiles and runs each in turn, so the `defun` has installed `sq`'s function cell before the call to `sq` is compiled. We follow each form.

Reading

The reader turns the text of the first form into cons structure on the heap: a list whose head is the symbol `defun`, then the symbol `sq`, then the parameter list `(x)` (one symbol), then the body `(* x x)` (the symbol `*` and two `xs`). Every symbol is interned, so the two `xs` are the *same* object, and the whole form is a `Value::Obj` handle to the first cons. The second form reads to a two-element list, `sq` and the integer `6`. No AST type is involved — the reader produced the same heap objects the program will manipulate.

Macro-expanding

`macroexpand-all` runs over each form before lowering. Here there are no macros — `defun` and `*` are a special form and a primitive — so both forms pass through unchanged. Had the body been `(when c x)`, this is where `when` would have expanded; the stage runs regardless, and its no-op here is as much a part of the pipeline as its work elsewhere.

Compiling the defun

`compile_defun` lowers the first form. It compiles the body `(* x x)` into a *sub-chunk*: `x` is a parameter, so a lexical variable, and each reference compiles to `LoadConst(x)` then `Extended(GETVAR)`; `*` is an un-redefined primitive, so the fast path fires and `(* x x)` becomes two loads and a native `Mul`. That sub-chunk, the parsed parameter list, and `is_macro: false` go into an `Obj::Closure` template with `env: None`. The outer chunk then emits:

```
;; compile_defun on (defun sq (x) (* x x)):
;;   LoadConst(sq)           ; the name symbol
;;   LoadConst(template)     ; params + body sub-chunk
;;   Extended(MAKE_CLOSURE)    ; capture the (empty top-level) env into a closure
;;   Extended(FSET)           ; set sq's function cell, leave the symbol
```

Running the defun

`run_chunk` builds a VM around the outer chunk, installs the two extension handlers, and runs. `LoadConst` pushes the name and the template. `MAKE_CLOSURE` (handler op 9) pops the template and pushes a closure capturing the current lexical environment — empty, at top level. `FSET` (op 4) pops the closure and the symbol, sets `sq`'s function cell to the closure, and pushes the symbol. The form's value is the symbol `sq`, discarded because the file runs for effect; the lasting effect is that `sq`'s function cell now holds a closure whose body is the compiled `(* x x)`.

Compiling and running the call

`compile_call` lowers `(sq 6)`. `sq` is not a special form and not a primitive, so the fast path declines; the general lowering loads the operator as a constant, compiles the argument, and emits the call:

```
;; compile_call on (sq 6):
;;   LoadConst(sq)
;;   LoadInt(6)
;;   Extended(CALL, 1)
```

Running it: `LoadConst` and `LoadInt` push `sq` and `6`. `CALL` (op 1) pops the one argument and the operator and dispatches to `call_function`. `sq` resolves to a closure, so `run_closure` opens a lexical scope as a child of the closure's captured environment, binds the parameter `x` to `6`, and runs the body sub-chunk on a nested VM. In that nested VM: `LoadConst(x)` then `GETVAR` (op 2) reads `x` from the scope — `6`; the second pair reads `6` again; `Mul` multiplies them to `36`. The nested VM returns `36`, `run_closure` unwinds the scope to its entry depth, and `36` is pushed onto the outer VM's stack as the call's result.

The result

The outer chunk's value is `36`. Under `-e` the printer would render it as `36`; in a file it is discarded. That is the whole pipeline for two lines of elisp: the reader built heap objects, `macroexpand-all` passed them through, `compile_defun` built a closure template and a sub-chunk and emitted `MAKE_CLOSURE/FSET`, the handler installed the function, `compile_call` emitted a `CALL`, and the handler dispatched through `call_function` and `run_closure` into a nested VM that ran the native-arithmetic body to `36`. Every chapter of the machine appears in those steps, in order, and the trace is the same for any program — larger programs are only more of it. The reader and the compiler owned the elisp meaning; the borrowed VM ran the chunks and the nested chunks; ten extension callbacks carried everything elisp-specific across the boundary. That is elisprs, end to end.

Chapter 59 — Inside cl-defstruct and setf

The object system of Chapter 22 and the place macros of Chapter 39 meet in one piece of machinery: the generated accessors of `cl-defstruct` and the `setf` expander that makes them settable. This chapter reads that machinery, because it is the clearest example of a macro generating code that another macro then has to invert — the kind of two-sided metaprogramming that a complete macro system makes possible.

What cl-defstruct generates

A `cl-defstruct` declaration expands into a `progn` of definitions. For `(cl-defstruct point x y)` it generates a keyword constructor `make-point`, a predicate `point-p`, a copier `copy-point`, and two accessors `point-x` and `point-y`. The instance is a record — a tagged vector with `cl-struct-point` in slot 0 and the field values in slots 1 and 2 — so the accessors are `aref` reads at fixed indices and the constructor is a `record`/vector build:

```
;; cl-defstruct generates, per slot SLOT at index j (conceptually):  
;; (defun point-SLOT (s) (aref s j))  
;; ;; and registers j in cl-struct--slot-index so setf can invert it
```

The constructor accepts `:slot value` keyword arguments with per-slot defaults — `(cl-defstruct dd (x 0) (y 10))` makes `(make-dd :x 3)` leave `y` at its default `10`, as `eval.rs` asserts. The slot indices are recorded in the global `cl-struct--slot-index` table at definition time, which is the registry `setf` consults to invert an accessor.

Inheritance

A `(:include PARENT)` option prepends the parent's slots in order — so the parent's fields occupy the same indices in the child — and records the tag parentage in `cl-struct--parent`. The predicate then accepts subtypes by walking that chain:

```
(defun cl-struct--is-a (tag target)  
  (let ((res nil))  
    (while (and tag (not res))  
      (if (eq tag target) (setq res t) (setq tag (cdr (assq tag cl-struct--parent)))))  
    res))
```

So `shape-p` is true for a `circle` because `circle`'s tag chains up to `shape`'s, and `shape-name` reads slot 1 of a circle because `:include` put `name` at the same index the parent uses. `type-of` on a record returns its tag's struct name, and `recordp`/`cl-struct-p` recognize it; the printer renders it `#s(NAME ...)` and the reader reads that back, so a struct round-trips.

How `setf` inverts an accessor

A *place* is any form naming a settable location, and `setf` — through its expander `setf--expand` — knows how to invert the common accessors into their mutators. For the built-in places the inversions are direct: `(setf (car x) v)` becomes `(setcar x v)`, `(setf (cdr x) v)` becomes `(setcdr x v)`, `(setf (aref a i) v)` becomes `(aset a i v)`, `(setf (gethash k h) v)` becomes `(puthash k v h)`, and `(setf (symbol-value s) v)` becomes `(set s v)`. For a struct accessor, `setf` looks the slot index up in `cl-struct--slot-index` and emits the corresponding `aset` — so `(setf (point-x p) 99)` becomes an `aset` at the `x` slot. This is why the struct test `(setf (sf-x p) 99)` and `(cl-incf (sf-y p) 10)` both work: the accessor is a generalized place, and `setf` knows its inverse.

The two-sided metaprogramming

Step back and see the symmetry. `cl-defstruct` is a macro that *generates* accessor functions; `setf` is a macro that *inverts* them. The bridge between the two is the slot-index registry, written by the first and read by the second, so that a location named by a generated reader can be assigned through a derived inverter. Every place macro of Chapter 39 — `cl-incf`, `cl-decf`, `cl-push`, `cl-pop`, `cl-rotatef` — expands through `setf`, so all of them inherit the struct-accessor inversion for free, which is why `(cl-incf (sf-y p))` increments a struct slot with no special case. This is the macro system carrying a real design load: an object system whose locations are first-class places, assembled from two cooperating macros and a registry, all elisp on the primitive floor. It is the last and most intricate demonstration the book offers that the distance from twenty-two special forms to a full Common Lisp object system is covered, end to end, by macros.

Chapter 60 – The Heap Image and Determinism

Three of the system’s most important capabilities — the bytecode cache, ahead-of-time compilation, and the very ability to serialize compiled code — rest on one property that looked, in Chapter 4, like an implementation detail: the object heap is deterministic. This chapter draws that thread through the whole machine, because the handle model and the heap image it requires are the hinge on which durability turns.

Why a handle model forces a heap image

A compiled chunk’s constant pool holds `Value : Obj(u32)` handles — baked-in references to symbols, quoted lists, and closure templates that live in the host arena. A handle is only meaningful against the arena that issued it. So a chunk cannot be run, cached, or emitted in isolation: to run it, elisprs must first rebuild an arena in which those exact handles resolve to the right objects. That rebuilt arena, serialized, is the *heap image* — a snapshot of every user-and-prelude object, in order, so that on reload the objects re-allocate to the same handles the chunk expects. The heap image is the price of the handle model, and it is the same price the cache and the AOT path both pay.

Determinism makes the image possible

The heap image only works because allocation is deterministic. The arena is append-only — `alloc` pushes and never reuses, moves, or compacts — and the prelude loads the same way every run, so the same program allocates the same objects in the same order and gets the same handles every time. A chunk whose constant pool references handle 412 can trust that re-running against a rebuilt image puts the intended object back at slot 412. A garbage collector that moved objects, or an allocator that reused freed slots, would break that promise — the handle might resolve to a different object on the second run. The absence of a collector, which Chapter 4 presented as a simplification the project got away with, is revealed here as a positive enabling property: a deterministic, append-only arena is what makes a serializable heap image *correct*.

The cache: chunks plus a clean image

The bytecode cache stores, per file, the per-form compiled chunks and a *clean* heap image, in a shard at `~/.elisprs/scripts.rkyv`. The container is rkyv on the outside, for cheap validated loading, and bincode on the inside, because `fusevm : Chunk` and `Value` are serde-owned, not rkyv-archived — the same split zshrs uses. The key folds three things together: the file path, its modification time, and a schema fingerprint hashing the builtin layout and the prelude source, so any change to the registered subrs or the prelude invalidates stale chunks automatically. The image is “clean” in a precise sense: the symbol value cells are reset to the baseline captured right after the prelude loaded but before the user file ran. That reset is the subtle correctness condition — re-running the cached chunks must reproduce the original execution exactly, so they must start from the same state the first run did, not from the end-of-run state with the user program’s global mutations already applied. On a hit, `eval_file` resets the host, imports the clean image so the handles line up, and runs the cached chunks directly, skipping the reader, the macro expander, the compiler, and the entire prelude

rebuild. On a miss it does the full work and writes the shard back atomically (temp file plus rename), best-effort, because the cache is an optimization and never a correctness requirement.

AOT: the same image, embedded

Ahead-of-time compilation faces the identical handle problem and solves it the identical way. The AOT compiler exports the user-and-prelude heap image and embeds it in the chunk — serialized to JSON and stashed in the chunk's names table behind a sentinel tag. fusevm's AOT model bincode-serializes the chunk into the object file along with a native entry point, and the embedded image rides along. At load time the standalone binary rebuilds the heap before running: fusevm's AOT entry calls back through the C symbol `fusevm_aot_register_builtins`, and elisprs's implementation rebuilds the heap from the embedded image and installs the two extension handlers. The cache and the AOT path are the same idea applied twice — serialize the heap, reconstruct it so baked-in handles resolve — and they share the same constraint, which is also the AOT path's named limitation: a user program's interned symbols, quoted data, and closure templates are not yet fully reconstructed at load from the image alone, which awaits constant reification.

One property, three payoffs

The whole apparatus — builtin count, heap image, schema fingerprint, clean baseline — is a consequence of one decision: lower elisp to a serializable `fusevm:Chunk` whose constants are handles into a deterministic arena. That decision buys three things a tree-walking interpreter cannot have: a chunk you can run immediately, a chunk you can cache to disk and reload, and a chunk you can emit as native code. You cannot cache or compile a tree-walker's work, because it does none ahead of time; there is no artifact. elisprs has an artifact at every stage, the same artifact, which is why the same compiled form flows through the interpreter, the cache, and the AOT compiler. The handle model is the cost; the heap image is how the cost is paid; and determinism is what makes the payment correct. Rent the engine, compile to its chunk, keep the arena deterministic, and durability comes nearly for free.

Chapter 61 – What Runs Fast: The JIT, the Fast Path, and the Cache

A bytecode frontend exists, in the end, to be fast, and `elisp`'s speed comes from four mechanisms it largely did not have to build. This chapter is the reference for what makes `elisp` run quickly here, what stays on the slow path, and why the frontend posture is what delivers the speed.

The native arithmetic fast path

The first and most direct accelerator is the compiler's native arithmetic fast path of Chapter 9. A naive lowering would compile `(+ a b)` as a `CALL` of the `+` subr — a callback into the host on every single addition, which for a hot loop is a callback per iteration and which walls off the JIT, because the JIT can only accelerate native VM opcodes, not host callbacks. So the compiler lowers un-redefined `+`, `-`, `*`, `1+`, `1-`, and the two-argument comparisons to the VM's own `Add`, `Sub`, `Mul`, `Negate`, and `Num*` opcodes. A tight numeric loop becomes a stretch of native opcodes with no host involvement at all. Two guards keep it correct: the redefinition check `is_primitive_fn` confirms the symbol still holds the original subr, and `/`, `%`, and `mod` stay on the host to preserve integer-division and zero-signaling semantics. The accepted trade is that native arithmetic skips the host's wrong-type signaling — a native `+` on a non-number will not raise — which is judged worth it because the alternative is a callback per arithmetic op.

The tiering JIT and the native-code cache

The second accelerator is the one `elisp` rents whole: `fusevm`'s three-tier Cranelift JIT, enabled in `run_chunk` by `enable_tracing_jit`. Hot loops in a chunk trace-compile through Cranelift to native machine code, so the stretch of native arithmetic opcodes the fast path produced becomes, after a few iterations, actual machine instructions — the same kind the CPU would run for hand-written Rust. The `jit-disk-cache` feature adds a persistent on-disk cache of that compiled native code, keyed by chunk hash, so a warm run skips the recompilation entirely. This is the payoff chain in full: the compiler lowers `(+ a b)` to a native `Add`, the JIT compiles the loop containing it to machine code, the disk cache persists that machine code, and the loop runs at native speed on every subsequent run. The frontend wrote none of the JIT or its cache and gets all of the speed — the dividend of being a tenant.

The bytecode cache

The third accelerator is the per-file bytecode cache of Chapter 60, which attacks a different cost: the front of the pipeline. A second run of a script skips the reader, the macro expander, the compiler, and the large prelude rebuild, loading the cached chunks and a clean heap image and running them directly. For a script that pulls in `cl-lib`, `seq`, and `subr-x` — a very large prelude — skipping the prelude rebuild is a substantial saving on its own, before the compile saving. This is the cache that the JIT's native-code cache complements: one caches the *compiled chunk* so you skip compiling, the other caches the *compiled machine code* so you skip JITing.

AOT, and what stays slow

The fourth, the farthest reach, is ahead-of-time compilation to a standalone native binary that skips even the VM startup for the lowered subset. And it is worth being honest about what stays on the slow path. Every non-arithmetic subr call is a `CALL` that pops back into the host — `car`, `format`, `gethash`, every primitive that is not on the native fast path runs as a host callback. The hash table is a linear scan, fine for small tables but not asymptotically fast. The re-entrant intrinsics — `mapcar`, `sort`, `apply` — thread through `call_function` outside the host borrow, which is correct but not free. The speed story is therefore specific: numeric loops and the control flow around them run native, because the fast path and the JIT make them so; the rest runs as a competent bytecode interpreter with host callbacks. That is the right shape for the workloads elisprs targets — scripts whose inner loops are arithmetic and whose outer work is library calls — and it is the shape the frontend posture produces, because the part that is generic and worth optimizing (the VM and the JIT) is exactly the part that was already built, shared, and fast.

Chapter 62 – The Backquote Expander, Traced

Backquote is the tool that makes macros readable, and `elisp` expands it *eagerly, at read time*, in a function the reader calls `bq_expand`. Chapter 3 introduced the idea and Chapter 8 showed why it matters for `pcase`; this chapter traces the expansion itself, because watching a template decompose into `cons` and `append` calls is the clearest way to understand both how backquote works and why doing it at read time has the consequences it does.

The decomposition rules

A backquoted template is shorthand for a tree of calls that builds the templated structure at run time. The rules are the textbook ones, applied by `bq_expand` the moment it reads the backtick. A top-level ``x` is just `x`. A backquoted *list* is walked element by element, folding right: a plain element `e` becomes `(cons 'e rest)`, an unquoted element `,x` becomes `(cons x rest)`, and a splice `,@x` becomes `(append x rest)`. A dotted tail is handled by making the final `cdr` the tail's expansion. And a backquoted *vector* folds like a list and then wraps the result in `vconcat` to rebuild a vector.

A trace

Take the template ``(a ,b ,@c)`. The reader folds its three elements right to left, starting from the empty tail `'nil`:

```
;; `(a ,b ,@c) decomposes, right to left, to:
;;   start:      'nil
;;   ,@c (splice): (append c 'nil)
;;   ,b  (unquote): (cons b (append c 'nil))
;;   a   (plain):  (cons 'a (cons b (append c 'nil)))
```

So ``(a ,b ,@c)` is `(cons 'a (cons b (append c nil)))` by the time anyone looks at it — ordinary `elisp` the compiler lowers like any other expression. At run time, with `b` bound to `2` and `c` to `(3 4)`, it builds `(a 2 3 4)`: `a` consed onto `2` consed onto the appended `(3 4)`. The test suite pins the simple cases — `(let ((x 5))(a x c))` is `(a 5 c)`, and `(let ((xs (list 2 3))) (1 ,@xs 4))` is `(1 2 3 4)`, the splice flattening `xs` into place.

Why read-time expansion is load-bearing

Doing the decomposition at read time, rather than carrying an unexpanded backquote form into the compiler, has a consequence that ripples into `pcase`. Because a backquoted template is already a tree of `cons`, `append`, and `vconcat` calls by the time the macro expander or the pattern compiler sees it, `pcase` can recognize backquote *patterns* — ``(,a ,b)`, ``(,a . ,rest)`, nested forms, and the `vconcat`-headed vector patterns — by matching the shape the reader left behind. The pattern compiler's `cons` arm of Chapter 56 destructures exactly that shape. The reader's eager expansion is therefore not merely a convenience for macro authors; it is what makes structural patterns over backquoted templates *implementable*, because the matcher and the constructor agree on one representation. A macro body like `swap!`'s ``(let ((,tmp ,a)) (setq ,a ,b)`

`(setq ,b ,tmp)` is, after reading, a tree of conses with the unquoted holes spliced in — a near-picture of the code it produces, decomposed before the macro was ever defined. That is the whole value of backquote, and it is delivered entirely in the reader.

Chapter 63 — Truthiness and the TRUTHY Op

“Only nil is false” is the smallest rule in elisp and one of the most consequential, because it differs from the borrowed VM’s own idea of truth and so cannot be delegated to the VM’s native branch. This chapter traces how that one rule becomes a custom instruction in every compiled conditional — the **TRUTHY** extension op — and why that op exists.

The rule

In elisp the only false value is **nil**. The number zero is true. The empty string is true. The empty list is **nil** and so is false, but the empty vector `[]` is true. Every conditional in the language tests for non-nil, and this single rule propagates through **if**, **when**, **unless**, **cond**, **and**, **or**, and **while**. `demo.el` states it in a docstring — “elisp truthiness: only nil is false” — and asserts it: `(if (eq 1 1) "yes" "no")` is “yes”, `(when (cons (cons 1 2)) 'ok)` is **ok**, and `(when nil 'ok)` is **nil**.

The impedance with fusevm

`fusevm` has its own notion of truthiness — its **JumpIfFalse** and **JumpIfTrue** branch on the VM’s idea of a true value, which is not elisp’s. If the compiler let the VM’s native branch decide an elisp conditional directly, a value the VM considers false but elisp considers true (or the reverse) would branch the wrong way. So elisprs cannot simply emit a **JumpIfFalse** on an elisp condition. The fix is a custom opcode: the compiler emits **Extended(TRUTHY)** — handler op 0 — *before* every conditional jump, and the handler pops the elisp value and pushes the VM boolean that the elisp rule dictates. The conversion function, `el_truthy`, returns false for exactly two values, **nil** (the VM’s **Undef**) and boolean false, and true for everything else. After **TRUTHY** runs, the value on the stack is a VM boolean the native **JumpIfFalse** can branch on correctly.

```
;; every elisp conditional compiles its test then converts:
;; <cond>
;;   Extended(TRUTHY, 0)   ; pop v; push Bool(only-nil-is-false(v))
;;   JumpIfFalse(-> else)
```

Where it is emitted

The **TRUTHY** op appears in every conditional lowering of Chapter 41. **if** emits it before the **JumpIfFalse**; **when** and **unless** before their polarity jump; **cond** before each clause’s jump; **and** and **or** after **Dup**-ing the value so the deciding value survives the test; and **while** before the loop-exit jump. So a program full of conditionals is a program full of **TRUTHY** ops, one before every branch — which is why the book has insisted that “only nil is false” is not a documentation nicety but a custom instruction in the compiled code. The compiler’s own note records that this is a shared technique across the `fusevm` frontends: “elisp truthiness ≠ fusevm truthiness, so conditionals emit **Extended(TRUTHY)** before **JumpIfFalse** (the strykelang pattern).” Every language hosted on `fusevm` that has its own truthiness rule inserts its own conversion op before the VM’s branch, and **TRUTHY** is elisp’s. It is the cleanest possible example of the extension protocol earning its keep: a one-bit semantic

difference between the language and the engine, bridged by a single callback, emitted everywhere it is needed and nowhere it is not.

Chapter 64 – Binding, Traced: Lexical and Dynamic Side by Side

Binding is the most intricate part of the language half of elisprs, because the language runs two completely different binding disciplines at once and routes each variable to the right one. Chapter 6 explained the mechanisms; this chapter traces them, following a lexical binding and a dynamic binding through their runtime structures step by step, so the routing decision becomes concrete.

The two structures

Lexical binding uses a `Scope`: a mutable list of `(symbol, value)` pairs plus a link to a parent, forming a chain that a closure captures. Dynamic binding uses a `specstack`: a stack of `(symbol, previous-value)` entries, where the variable's current value always lives in the symbol's value cell and the specstack just remembers what to restore. The routing between them is one function, `bind_here`: if the symbol is special, bind on the specstack; else if a lexical scope is open, push into it; else write the global value cell.

Tracing a lexical binding

Consider `(let ((x 10)) (helper))` where `x` was never declared special and `helper` was defined elsewhere. The compiler lowered the `let` to evaluate `10`, load the symbol `x`, and emit `ExtendedWide(LETBIND, 1)`. At run time `LETBIND` opens a fresh scope and binds `x` to `10` in it. When the body calls `(helper)`, `helper` runs on a nested VM with its *own* captured environment — which does not include this scope, because `helper` was not defined inside the `let` — so `helper` cannot see `x`. That is lexical scope: `x` is visible only in the textual region of the `let`, and a function called from the body sees it only if it closed over it. On exit, `UNBIND` discards the scope.

Tracing a dynamic binding

Now `(defvar *d* 1)` then `(let ((*d* 42)) (reader-fn))` where `reader-fn` is `(defun reader-fn () *d*)`. The `defvar` set `*d*`'s special flag at compile time, so when the compiler lowered this `let`, `bind_here` will route `*d*` to the specstack. At run time, binding `*d*` pushes `(*d* . 1)` — the current value cell — onto the specstack and overwrites the value cell with `42`. When the body calls `(reader-fn)`, `reader-fn` reads `*d*` through its value cell — which now holds `42` — even though `reader-fn` was defined in a different place and knows nothing about this `let`. That is dynamic scope: the binding is visible to everything that runs during the `let`'s extent. The test asserts exactly this: `(defvar *dyn* 1) (defun rd () *dyn*) (let ((*dyn* 42)) (rd))` is `42`. On exit, the specstack entry is popped and `1` is restored to the value cell.

```
;; the routing, in elisp terms:
(defvar *log-level* 'info)      ; marks *log-level* special -> dynamic
(let ((*log-level* 'debug)      ; special -> pushed on the specstack
      (tmp (compute)))          ; ordinary -> pushed into the lexical scope
  (run-handlers))               ; handlers see *log-level* = debug dynamically
;; on exit: specstack restores *log-level*; the lexical scope is discarded
```

setq and unwinding

`setq` respects the same search: it tries the lexical scope first, mutating the slot in place if the variable is bound there, and otherwise writes the global value cell. This is why a `setq` inside a `let` updates the `let`'s binding and a `setq` at top level creates a global, and why the loop accumulators in the examples — `(setq total (+ total i))` inside a `let` — mutate the lexical binding visibly across iterations. The hardest case is the abnormal exit: if a `throw` or error unwinds out of the middle of a `let`, the `let`'s `UNBIND` never runs in the normal flow, yet the dynamic bindings must still be restored. The frame stack records, for each open scope, the specstack depth at which it was entered, and `unwind_scopes_to` restores the specstack and closes scopes back to a target depth as the non-local exit passes through — which is what makes `condition-case` and `catch` safe around code that binds dynamic variables, and what the over-unwind edge of Chapter 52 warns is the genuinely hard part. Two structures, one routing function, and a recovery mechanism at the boundaries: that is the whole of how `elisp` runs a Lisp-2 with both binding disciplines at once, traced from the routing decision to the restored value cell.

Chapter 65 – Interning, Traced

The symbol is the carrier of the Lisp-2, and interning is what gives symbols the identity everything else depends on. This chapter traces interning from a reader token to a stable arena handle, and follows the three things a symbol then carries — its two cells and its property list — because the whole personality of the language rests on a name resolving to the same object every time it is read.

From token to handle

When the reader reads a token that is not a number, it interns it. The obarray is a plain `HashMap<String, u32>` from name to arena handle inside the host. `intern("foo")` looks the name up: if a symbol with that name is already recorded, it returns that exact handle; if not, it allocates a fresh `Obj::Symbol`, records the name-to-handle mapping, and returns the new handle. The consequence is that two reads of `foo` anywhere in a program produce the *same* handle, which is what makes `(eq 'foo 'foo)` true — both `'foos` interned to one object. Symbol identity is the foundation of everything downstream: symbols are the natural keys for property lists, alists, and `pcase`'s literal patterns precisely because `eq` on them is a single integer comparison.

make-symbol and hygiene

`make-symbol` deliberately breaks the interning contract: it allocates a symbol with a name but never records it in the obarray, so each call yields a distinct object. `(eq (make-symbol "g") (make-symbol "g"))` is `nil`, which `tests/builtins.rs` asserts. This is the mechanism behind `gensym` and macro hygiene — an uninterned symbol cannot be named by any source text, because no read of any name will ever intern to it, so a macro's temporary variable built from `(gensym)` can never collide with a user's variable. `intern-soft` is the third member of the family: it performs the obarray lookup *without* allocating, so it tests whether a name is known and returns the symbol or `nil`, the right tool when you must not create a symbol merely by asking about one.

The three cells

Once interned, a symbol carries three stores. Its value cell, reached by `symbol-value` and written by `set`, holds the variable binding — void until assigned, signaling `void-variable` when read empty. Its function cell, reached by `symbol-function` and written by `fset`, holds the function binding — a subr, a closure, or another symbol. And its property list, reached by `get` and `put`, holds arbitrary metadata. These three are independent: `set` touches only the value cell, `fset` only the function cell, `put` only the plist, and a symbol can have any combination. The obarray is therefore three tables at once — the variable table, the function table, and the metadata table — all keyed by the same interned symbol, which is the Lisp-2 (two cells) plus the metadata store (a third) in one object.

Aliases and enumeration

Because the function cell can hold *another symbol*, a symbol can be an alias, and resolving a call follows the chain: `resolve_function` walks function-cell-to-symbol links up to 64 hops before signaling “function indirection too deep,” and `indirect-function` exposes that walk to elisp. This is how `defalias` works

and how `cl-first` could be an alias for `car` — though in the prelude `cl-first` is actually a plain `defun` wrapper, and the one real `defalias` is `string-split` for `split-string`. Finally, the obarray is enumerable: `mapatoms` applies a function to every interned symbol, the reflective operation that lets a program audit the whole namespace — find every symbol with a docstring, every function with a prefix. Interning gives identity; identity gives `eq`; `eq` gives the keys; and the three cells give the Lisp-2. That chain, traced from a reader token to a stable handle, is the foundation the rest of the language is built on.

Chapter 66 — Closures and Nested VMs, Traced

A user function is a closure, and calling it runs its compiled body on a nested VM. This chapter traces a closure from its construction through a call and into recursion, because the closure-and-nested-VM mechanism is the entire user-function call graph, and seeing it step by step is the clearest way to understand how recursion, lexical capture, and the parameter list all work at once.

Constructing a closure

A closure is an `Obj::Closure` with four fields: the parsed parameter list, the compiled body as a `fusevm::Chunk`, an `is_macro` flag, and a captured lexical environment. The compiler builds it in two stages. `compile_lambda` allocates a *template* — params, body chunk, `is_macro`, and `env: None` — and bakes it into the constant pool, then emits `Extended(MAKE_CLOSURE)`. At run time, `MAKE_CLOSURE` (handler op 9) pops the template and calls `instantiate_closure`, which clones the template's params, body, and flag and sets `env` to the *current* lexical environment. So a closure is template-plus-environment: the code half is fixed at compile time, the environment half is snapshotted at the moment of construction. That snapshot is what makes a function defined inside a `let` close over the `let`'s variables — `make-adder` returns a closure whose `env` holds the `n` it was built with, so `(funcall (make-adder 10) 5)` is 15.

Calling a closure

When `call_function` resolves an operator to a closure, it runs `run_closure`. The steps: open a lexical scope as a *child* of the closure's captured `env`; parse the arguments against the parameter list and bind them into the new scope; run the body chunk on a nested VM via `run_chunk`; and on the way out, unwind any scopes the body leaked through a non-local exit, back to the entry depth. The body runs on its own VM — its own value stack, its own extension handlers — executing to completion and returning a value into the caller. A macro accidentally invoked as a function is rejected at this point, because `is_macro` is checked before the body runs.

The parameter list

The argument binding follows a small state machine over the parameter list. Symbols are required parameters until `&optional` is seen, then optional parameters (defaulting to `nil` when the call omits them) until `&rest` is seen, then the single rest parameter, which collects all remaining arguments into a freshly-consed list. Too few required arguments, or too many when there is no rest parameter, signals `wrong-number-of-arguments`. The richer Common Lisp lambda-list features — `&key`, per-argument defaults, `&aux` — are not part of this primitive parser; they are provided by `cl-defun` in the prelude, which desugars to a plain `&rest` function whose body is a `cl-structuring-bind`.

Recursion as nested VMs

Recursion falls out of this mechanism with no special support. `(fact 5)` runs `fact`'s chunk on a VM; that chunk reaches `(fact 4)`, which `call_function` resolves to the same closure and runs on *another* nested VM; and so on down to `(fact 1)`, five VMs deep at the base, each with its own stack and its own scope binding its own `n`. As each returns, its VM completes and its value flows up to the caller's VM, unwinding the stack of nested VMs back to the top. This is exactly why the `functions.el` comment can say "calling it runs that chunk on a nested fusevm VM," and why a deeply recursive elisp program is a deeply nested stack of VM invocations. The fusevm engine is built to nest this way, and elisprs leans on that nesting for the entire user-function call graph — every `funcall`, every method dispatch, every recursive descent is one more nested VM running one more chunk, with the borrow discipline of Chapter 10 keeping each level's host access short and non-overlapping.

Chapter 67 – The Macro Expander and the Top-Level Loop, Traced

Macros run at compile time and return code, and getting the *ordering* right — a macro defined before the forms that use it — is what makes a file's macros available to its own later forms. This chapter traces the top-level evaluation loop and the three grades of expansion, because the form-by-form processing is the quiet machinery that makes the whole macro-and-prelude architecture work.

The form-by-form loop

The library's `eval_forms` reads *all* the top-level forms of a file first, then processes them one at a time: for each form it macro-expands, then lowers, then runs. The order matters because of a sequencing subtlety — a macro must be defined before the forms that use it are compiled. By compiling and running form by form, an earlier form's `defmacro` has installed the macro into the obarray (via `FSET` on the symbol's function cell) before a later form that uses the macro is expanded. Trace `(defmacro m (x)(list ,x ,x))` (m 5): the first form runs, setting `m`'s function cell to a macro closure; then the second form is expanded, finds `m` is a macro, runs it on the unevaluated 5, gets `(list 5 5)`, and compiles *that*. Had the file been processed as one unit, `m` would not yet exist when `(m 5)` was expanded — which is exactly why a `defmacro` and its first use cannot share a single-evaluation and live instead in an example script, the `constraint-cl-defun.el`'s header names.

splice_top_forms

One more wrinkle preserves Emacs's top-level behavior: a literal top-level `(progn ...)` is *spliced* into its subforms before the form-by-form processing, recursively, by `splice_top_forms`. The reason is that in Emacs a `defmacro` inside a top-level `progn` is in effect for the later subforms of that same `progn`. By flattening top-level prologs into the form sequence, `elisp` makes a macro defined in a `progn` visible to its `progn`-siblings, matching Emacs — so a `(progn (defmacro m ...) (m 5))` works as a single top-level form because the splice turns it into two.

The three grades

The runtime exposes three expansion operations, differing in how far they go. `macroexpand-1` expands the outermost form exactly one step. `macroexpand` expands the head repeatedly until it is no longer a macro call — a fixpoint on the outermost form — but does not descend into subforms. `macroexpand-all` expands fully and recursively, into every subform, and it is what the compiler driver runs before lowering. The recursive expander has one deliberate stop: it does not descend into `quote`, because quoted data is data, not code, and expanding inside `'(when x)` would corrupt a literal three-element list into an `if`. `(macroexpand '(push 1 lst))` is `(setq lst (cons 1 lst))` — one step revealing `push`'s expansion — and `(macroexpand '(my-when c x))` is `(if c (progn x))`. By the time the compiler sees a form, `macroexpand-all` has run, so only special forms and calls remain; every macro — `dolist`, `pcase`, `cl-loop`, the whole prelude vocabulary — is gone.

The prelude loads the same way

The prelude is loaded by the identical mechanism: form by form, in order, best-effort, so an earlier prelude `defmacro` is in effect before a later prelude form that uses it, and a single failing definition is reported and skipped rather than taking the runtime down. This is the discipline that lets `cl-loop` be written in terms of `while`, `pcase` in terms of `cond`, and the whole derived surface in terms of the primitives — each definition standing on the ones before it, the macro expander running over each in turn. The top-level loop is small, but it is the substrate the entire layered library is built on: read everything, splice the prologs, then expand-and-lower-and-run each form in order, so that the language can extend itself one definition at a time.

Chapter 68 – Characters, Unicode, and Counting in Characters

A thread runs through the reader, the strings, and the regexp engine that deserves to be pulled together: `elisprs` counts in characters, not bytes, and a character is an integer. This chapter consolidates the character model — how characters are represented, the escape and modifier vocabulary that names them, and the character-versus-byte discipline that the whole text surface maintains — because getting it right is what separates a working elisp from a plausible-looking one.

A character is an integer

In elisp a character is not a distinct type; it is an integer codepoint. `?A` is the integer 65, `?0` is 48, `?` (question mark space) is 32, and `?é` is the Unicode codepoint for `é`. Character operations are therefore integer operations: `(char-to-string ?A)` is "A", `(string-to-char "A")` is 65, and `(+ ?A 1)` is 66, the code for B. `(string-to-list "abc")` returns (97 98 99) — the list of codepoints — and `(append "ab" nil)` is (97 98), a string flattening to its character codes, which `tests/builtins.rs` asserts. This identity is why `aref` on a string returns an integer, and why a buffer's text is, at bottom, a vector of character integers.

The escape and modifier vocabulary

Because characters are written as `?`-prefixed literals, the reader carries a full escape vocabulary, shared with string literals. The single-letter escapes — `?\n`, `?\t`, `?\r`, `?\e`, `?\a`, `?\b`, `?\v`, `?\f`, `?\d` (delete, 127), `?\0` — map to their control codes, and the reader's test pins `(list ?\n ?\t ?\e ?\0)` to (10 9 27 0). The numeric forms are `?\x41` (hex), `?\101` (octal), `?é` and `?é` (four-hex Unicode), `?\U0001F600` (eight-hex), and `?N{U+1F600}` (codepoint by name). On top of these are the modifiers — control, meta, shift, hyper, super, alt — each a high bit set by the prefixes `\C-`, `\M-`, `\S-`, `\H-`, `\s-`, `\A-`, with meta at bit 27, control at bit 26, and so on down. The modifiers compose, so `?[C-\M-a` carries both, and control folds the base character so `?[C-a` and `?[C-A` are both 1. The whole keyboard-event vocabulary is integers with high bits set, which is why a character can carry a chord of modifiers and still be a single number `eq`-comparable as a key.

Counting in characters, everywhere

The discipline the whole text surface maintains is that lengths and positions are counted in *characters*, not bytes, even for multibyte text. `(length "héllö")` is 5, the character count, though the UTF-8 encoding is six bytes. `substring`, `aref`, `nth`, and the buffer position functions all index by character. This matters most at the regexp boundary, because the underlying `fancy-regexp` engine works in *bytes*, and a faithful elisp must report match positions in characters. `elisprs` bridges this: it converts the engine's byte spans to character spans before storing the match data, so `(progn (string-match "é" "aébé") (list (match-beginning 0) (match-end 0)))` is (1 2) — the `é` at character index 1, occupying one character position, though it is two bytes. The example `regexp.el` exists in part to pin exactly this, and it is the kind of fidelity detail that a port either gets right or quietly gets wrong, with the wrongness surfacing only on the first non-ASCII input.

The character predicates and conversions

The character layer has its own predicates and conversions. `characterp` tests for a valid codepoint; `char-or-string-p` for a character or string; `char-uppercase-p` for an uppercase character; and `char-equal` compares two characters, case-insensitively when `case-fold-search` is non-nil. `upcase` and `downcase` are polymorphic over a character or a string, so `(upcase ?a)` is `?A` (the integer 65). The conversions `char-to-string`, `string-to-char`, `string`, `string-to-list`, and `string-to-vector` move between characters, codes, and strings, and the display-width functions `char-width` and `string-width` of Chapter 50 measure how wide a character renders, which differs from its character count for CJK and combining marks. Taken together, the character model is small and consistent: a character is an integer, written with a rich escape syntax, counted in characters everywhere, and bridged to bytes only at the one place — the regexp engine — where the underlying machinery insists on bytes and `elisp` insists, correctly, on characters.

Chapter 69 – The Frontend Thesis, Re-viewed

The book began with a thesis and then spent sixty-eight chapters grounding it. This closing chapter reviews the argument as a whole, now that every piece has been seen, because the parts only become the thesis when they are held together. The claim was simple to state and took a book to earn: own the language, rent the engine, and connect them with a narrow protocol.

What elisprs owns

elisprs owns the personality of Emacs Lisp. It owns the reader, with its character literals and radix syntax and eager backquote. It owns the object heap — real cons cells with identity, interned symbols with two cells and a property list, vectors, hash tables, closures — living in an `ElispHost` arena. It owns the obarray and the Lisp-2, the lexical and dynamic binding routed by one decision, the twenty-two special forms, and the macro system. It owns the subr library, the three hundred-odd primitives written in Rust, and the enormous derived prelude written in elisp on top of them — `cl-lib`, `seq`, `map`, `subr-x`, `pcase`, `rx`, `cl-loop`, the object system, JSON, and ERT. These are the parts that make `.el` code mean what it means, and none of them could be delegated to a language-agnostic engine without teaching that engine about symbols and value cells and conditions, which would defeat the point of the engine being language-agnostic.

What elisprs rents

elisprs rents the execution substrate. It does not have a virtual machine, a JIT, an ahead-of-time compiler, or an on-disk code cache of its own; it lowers every form to a `fusevm::Chunk` and runs it on fusevm, the shared engine that already powers four sibling languages. A stack machine that executes `Add` and `Jump` and `LoadConst` is the same for every compiled language, and there was no reason for five languages to write it five times. So elisprs draws the line exactly there — the object heap and the semantics are bespoke, the execution is shared — and gets, for a version pin and a narrow protocol, a tiering Cranelift JIT, a persistent native-code cache, and an AOT backend it did not write.

The narrow protocol

The two sides connect through one interface, and its narrowness is the whole engineering achievement. A single value variant, `Value::Obj(u32)`, carries any elisp object through the VM as an opaque handle. A single opcode family, `Op::Extended`, calls back into elisprs for anything elisp-specific. Ten extension operations — `TRUTHY`, `CALL`, `GETVAR`, `SETVAR`, `FSET`, `SPECBIND`, `LETBIND`, `UNBIND`, `SCOPE_OPEN`, `MAKE_CLOSURE` — are the complete contract. Implement those ten callbacks and a value handle, and a language-agnostic VM runs a Lisp it knows nothing about: it carries cons cells on its stack, runs compiled bytecode in its dispatch loop, JITs hot loops to native code, and never learns what a cons cell is. The extension protocol is the hinge the whole architecture turns on, and its ten operations are small enough to list on a page.

What the prelude proves

The strongest evidence that the design is sound is the prelude. A standard library of genuine breadth — a Common Lisp compatibility layer, a generic sequence interface, an object system with multiple dispatch and method combination, a pattern matcher, a regexp notation, the loop facility, a JSON codec, and a test framework — is written entirely in elisp, on top of twenty-two special forms and a couple hundred primitives, expanding into the core with no compiler or VM support. If a library that large can be a library rather than a language change, the core is complete. Every worked example in the book is a different cross-section of that completeness, and every one runs byte for byte under both elisprs and GNU Emacs, which is the cross-check that turns the examples from illustrations into executable specifications.

The honest frontier

And the design is presented honestly, with its frontier marked rather than hidden. Three buffer-scoping special forms do not yet lower and fail loudly at compile time. Integers are `i64` with no bignum promotion. The AOT path's user-constant reification is a named gap. The buffer model is text plus point, without markers or overlays or surviving text properties, and `current-buffer` lags behind, documented as an expected failure inside the test suite itself. The hash table is a linear scan; `sxhash` is self-consistent but not Emacs-bit-compatible; the specstack unwinding across a non-local exit is the genuinely hard part. None of these is concealed; each is named in the source or the tests, which is the posture that lets the rest of the book be trusted. A frontend that refuses to miscompile is a frontend you can reason about.

The thesis, earned

So the thesis, stated once more now that it is earned: Emacs Lisp is a language, not an editor, and its semantics and its execution are separable. Keep the semantics — own them, faithfully, down to the corner cases that make `.el` code mean what it means — and throw away the bespoke execution, replacing it not with another interpreter but with a borrowed engine reached through ten callbacks and a handle. What you get is the editor's language running from a shell prompt: a `.el` file that reads, compiles, JITs, caches, and can freeze into a native binary, doing real work — files, processes, JSON, regexps, text — with no Emacs anywhere. That is elisprs, and it is what a frontend, done well, can be.

Chapter 10 – Sequence Polymorphism: One Vocabulary, Three Types

elisp has three sequence types — the list, the vector, and the string — and a great deal of the library treats them uniformly, so that one function works on all three. This chapter is the reference for how that polymorphism is built, because the trick is small, repeated everywhere in the prelude, and worth seeing once so that every `seq-` function's behavior on a vector or a string is no longer a surprise.

The three types and their predicates

A list is a chain of cons cells; a vector is a fixed-length array; a string is an array of character codes. The predicates that recognize them as sequences are `sequencep` (true for any of the three), `arrayp` (true for a vector or a string, the two random-access types), `listp` (a cons or nil), and `seqp` (the `seq` library's name for “is this a sequence”). The array operations `aref`, `length`, and `elt` work over both vectors and strings, indexing by position, while `length` is total over all three. So at the primitive level, the random-access surface already spans vectors and strings, and the list surface spans lists — the polymorphism the prelude adds is unifying *all three* under one vocabulary.

The coercion trick

The mechanism is one idiom, repeated across the prelude: `(append seq nil)`. Appending a sequence onto `nil` produces a *list* of its elements — copying a list, or flattening a vector or string into a list of its elements (character codes, for a string). Nearly every `seq-` and `cl-` function begins by coercing its argument this way, working over the resulting list, so a function written with list operations transparently accepts a vector or a string. `seq-filter`, `seq-reduce`, `cl-remove-if`, `cl-position` all open with this coercion. It is the single line that makes the generic sequence interface possible without writing three implementations of every function.

```
(append "ab" nil)      ; => (97 98)  - a string flattens to character codes
(append [1 2 3] nil)   ; => (1 2 3)  - a vector flattens to a list
(append '(1 2) nil)    ; => (1 2)    - a list is copied
```

Preserving the type

Coercing to a list is enough for functions that *return* a list — `seq-filter` returns a list regardless of input type. But some functions must preserve the input's type: `seq-uniq` of a string should arguably return a string, `seq-partition` of a vector should return vectors, and `remove` of a vector returns a vector — `(remove 3 [1 2 3])` is `[1 2]`, as `eval.rs` asserts. For these, the prelude detects the input type with `seq--type-of` at the start and converts the result back with `seq-into` at the end. `seq-partition`'s body shows the pattern: it records `(seq--type-of seq)`, builds chunks, and converts each chunk back to that type with `seq-into`, so a partitioned string yields strings and a partitioned vector yields vectors. `seq-into` itself is the explicit converter — `(seq-into '(1 2 3) 'vector)` builds a vector — and `cl--like` is `cl-lib`'s equivalent, re-typing a result to match the input. The discipline is consistent: coerce to a list to do the work, detect the type to restore it.

Where polymorphism stops

The unification is broad but not total, and the edges are the predictable ones. The destructive operations — `nreverse`, `nconc`, `delete` — make sense on lists (which can be relinked) but not on the fixed structure of a vector or the immutable backing of a string, so they are list operations. `mapcar` is generic over sequences, but `mapcan` (which concatenates with `nconc`) is list-oriented. And a function that returns its argument unchanged for effect, like `mapc`, returns whatever type it was given. The rule a reader can rely on: a `seq-` function accepts any sequence and returns a list unless its name implies a shape; a `cl-` sequence function does the same with the `cl--like` re-typing; and the destructive operations want lists. The polymorphism is a prelude achievement — one coercion idiom, one type-detection helper, one converter — layered over a primitive floor that already spans the array types, and it is what lets a program write `seq-map` over whatever sequence it happens to hold.

Chapter 71 – The Output Streams: `stdout`, `stderr`, and Capture

A script has two things to say — its *result* and its *chatter* — and `elisp` keeps them on separate file descriptors, which is a small design decision with large practical consequences. This chapter is the reference for where output goes, how it is captured, and why the split matters, because a script that mixes the two streams is a script you cannot pipe.

The two streams

The functions that emit a program's actual output — `princ`, `prin1`, `print`, `terpri` — go to *standard output*. The function that emits progress and diagnostic chatter — `message`, and the error reporters `error` and `user-error` on their way to the terminal — go to *standard error*. This is the deliberate split from Chapter 2: a script's real result lands on `stdout`, where a pipe or a redirect can capture it, while its `message` chatter lands on `stderr`, where it reaches the terminal without polluting the result. In Emacs, `message` writes to the echo area and the `*Messages*` buffer; in `elisp`, with no echo area, it writes a formatted line to `stderr`, which is exactly the right place for it. The consequence is that `elisp script.el > result.txt` captures the program's `princ` output and leaves its `message` progress visible on the terminal — the behavior every shell tool should have and many scripting setups get wrong.

`format` does not print

A point worth stating because it is a common confusion: `format` does not print anything. It *returns* a formatted string. `message` is the function that prints — it formats its arguments with the same engine and then writes the result to `stderr`. So `(format "...")` is for building a string you will use, and `(message "...")` is for emitting chatter; `(princ (format "..."))` is how you send a formatted string to `stdout`. The `-e` command-line mode is the other printer: it evaluates one expression and writes its *value*, in readable (`prin1`) form, to `stdout` — so a one-liner is a calculator whose answer you can pipe.

Capturing output into a string

Sometimes a program wants to capture what `princ` would have written rather than emit it, and `with-output-to-string` does exactly that. It is a prelude macro that brackets its body with the internal primitives `--push-output-capture--` and `--pop-output-capture--`, redirecting `princ/prin1/print` output into a capture buffer for the duration of the body and returning the captured string. This is how a program turns a printing routine into a string-producing one without rewriting it — wrap the call, get the text. The double-dash names mark these as internal hooks the prelude wraps in the friendlier macro, the seam between the Rust floor and the `elisp` surface.

Capture in the debugger

The output split earns its keep most visibly in the debug adapter. The debuggee runs in-process and writes its program output to fd 1 — but the DAP protocol *also* runs over fd 1 as JSON-RPC, so unredirected program output would corrupt the protocol stream. The fix, from Chapter 17, is file-descriptor surgery: the debugger saves the real stdout, creates a pipe, and uses `dup2` to point fd 1 at the pipe's write end, so the debuggee's `princ` output flows into the pipe instead of the terminal. A reader thread drains the pipe and wraps each chunk in a DAP `output` event, which the editor shows in its debug console, while the saved descriptor carries the clean JSON-RPC. And because `message` writes to `stderr` — *not* the captured stdout — the debuggee's chatter passes through untouched. That is the whole payoff of the two-stream design, concentrated in one place: program output and protocol output and diagnostic chatter, three things that would collide on one descriptor, kept cleanly apart because `elisprs` decided from the start which stream each belongs on.

Chapter 72 – The Special-Form Reference, Entry by Entry

The catalogue chapters gave the subrs and the prelude functions an entry-by-entry treatment; the twenty-two special forms — the irreducible core the compiler lowers directly — deserve the same. This chapter is that reference: each form with its signature, its meaning, the shape of its lowering (the full opcode account is Chapter 41), and an example. These are the forms that control evaluation rather than submit to it, and everything else in the language is a macro or a subr built on them.

The quoting forms

`(quote OBJECT)` returns its argument unevaluated; `'x` is its reader abbreviation. It lowers to a single `LoadConst`. It is the escape from evaluation — without it every symbol would be a variable reference and every list a call.

`(function OBJECT)` is `quote`'s Lisp-2 sibling, abbreviated `#'`. For a symbol it behaves like `quote`, loading the symbol so the call machinery resolves its function cell later; for a `(lambda ...)` argument it builds a closure, which is what makes `#'(lambda ...)` callable where a bare lambda form would be rejected.

`(lambda ARGLIST BODY...)` builds an anonymous function. The body is *compiled* into a sub-chunk, and the closure carries that chunk, the parsed parameters, and a capture of the current lexical environment. It lowers to a `MAKE_CLOSURE` over a template constant.

The sequencing forms

`(progn BODY...)` evaluates its forms in order and returns the last value; it lowers each form with a `Pop` between them. `(prog1 FIRST BODY...)` evaluates all its forms but returns the *first* — the “do this, then clean up, return the original” idiom — lowering with a `Pop` after each form past the first.

The conditionals

`(if COND THEN ELSE...)` evaluates `COND`, runs `THEN` if it is non-nil, else the `ELSE` forms as an implicit `progn`; it lowers to `TRUTHY` then a back-patched `JumpIfFalse`.

`(when COND BODY...)` runs `BODY` if `COND` is non-nil, returning `nil` otherwise; `(unless COND BODY...)` is the negation. In Emacs these are macros over `if`; elisprs makes them special forms with the same `TRUTHY`-then-jump shape, `unless` flipping to `JumpIfTrue`.

`(cond CLAUSES...)` tries each `(TEST BODY...)` clause and runs the first whose test is non-nil. A clause with a test but no body returns the test value when truthy — the `(cond (x) ...)` idiom. It lowers to a chain of test-and-jump sequences.

The connectives and the loop

`(and CONDITIONS...)` evaluates left to right, returning the first `nil` or the last value; `(or CONDITIONS...)` returns the first non-`nil` or `nil`. Both short-circuit, lowering by `Dup`-ing each value, testing it with `TRUTHY`, and jumping out early while leaving the deciding value on the stack — which is why `(and 1 2 3)` is 3 and `(or nil 5)` is 5.

`(while COND BODY...)` evaluates `BODY` while `COND` is non-`nil`, returning `nil`. It is the only primitive loop; every other looping construct is a macro over it. It lowers to a backward jump with a `TRUTHY`-guarded exit.

Assignment

`(setq SYM VAL SYM VAL...)` assigns to variables, its odd arguments names (unevaluated) and its even arguments values (evaluated). It respects the lexical-then-global search, lowering each pair to a `SETVAR`. It is the workhorse of imperative elisp.

The binding forms

`(let BINDINGS BODY...)` evaluates all its initializers in the surrounding scope, then binds them in parallel — so `(let ((a 1) (b a)) ...)` sees the *outer* `a` in `b`'s initializer. It lowers to a single wide `LETBIND`.

`(let* BINDINGS BODY...)` binds sequentially, each initializer seeing the previous bindings — so `(let* ((a 1) (b a)) ...)` makes `b` equal 1. It lowers by opening one scope and emitting a `SPECBIND` per binding.

The definition forms

`(defun NAME ARGLIST BODY...)` defines a function, building a closure for the body — capturing the defining lexical environment — and installing it in the symbol's function cell via `FSET`. `(defmacro NAME ARGLIST BODY...)` defines a macro the same way but with the `is_macro` flag and *no* environment capture, because a macro runs at expansion time.

`(defvar NAME &optional INIT DOC)` declares a special (dynamic) variable, and its crucial side effect is marking the symbol special at compile time so later `lets` of it bind dynamically. `(defconst NAME INIT &optional DOC)` is the same with the connotation of a constant.

The non-local exits

`(catch TAG BODY...)` establishes a labeled escape point; a `(throw TAG VALUE)` in its dynamic extent unwinds straight back to it, returning `VALUE`. `(unwind-protect BODY CLEANUP...)` runs `BODY` and guarantees `CLEANUP` runs afterward, on any exit — elisp's *try/finally*. `(condition-case VAR BODY HANDLERS...)` runs `BODY` and, if it signals, dispatches on the error's condition to a matching handler, binding `VAR` to the condition — elisp's *try/catch*. All three lower not to opcodes but to calls to internal intrinsics (`--catch--`, `--unwind--`, `--condition-case--`) with their subexpressions wrapped in thunks, so the host's `call_function` can implement the control logic without teaching the VM anything about elisp's error model.

The three that are recognized but refused

Finally, three forms are in the dispatch table but deliberately not lowered: `save-excursion`, `save-current-buffer`, and `save-restriction`. The compiler rejects them with a “buffer milestone” error rather than

miscompiling them — the loud-failure discipline, holding the line at the edge of the buffer object model. Twenty-two forms lowered, three refused, and everything else in the language a macro that expands into the twenty-two: that is the complete core, and this reference is its entry-by-entry map.

Chapter 13 — The Test Files, One by One

Chapters 46 and 47 read the test suite by topic; this chapter reads it by *file*, because each test file in `tests/` has a job, and knowing which file gates which behavior tells you where to look when something changes. The Rust integration tests all share one harness — `reset_host()`, `eval_str(src)`, `print(&v, true)` — so every assertion is an elisp form and its exact printed result, and together they are the most precise specification the language has. This is the map of that specification.

eval.rs: the behavioral core

`eval.rs` is the largest test file by far, the broad behavioral suite that exercises every corner of the language: arithmetic and contagion, cons mutation, the three equality grades, strings and substrings, sequences, control flow, closures and recursion, dynamic and lexical binding, the error data model, hash tables, structs, regexp with Unicode positions and backreferences, backquote, `pcase`, macros, `eval/read`, and `cl-loop`. It is where a new primitive's behavior is pinned and where a regression in lowering first shows. The assertions read like a specification — `(/ 7.0 2)` is `3.5`, `(eq (list 1 2) (list 1 2))` is `nil`, `(catch 'x (+ 1 (throw 'x 99)) 999)` is `99`, `(condition-case e (car 5) (error e))` is `(wrong-type-argument listp 5)` — each a single fact the CI asserts.

builtins.rs: the primitive edge cases

`builtins.rs` targets the subr layer's *edge cases*, the behaviors a casual implementation gets subtly wrong. Its comments name the traps: `mod` follows the sign of the divisor unlike `%`, so `(mod -7 3)` is `2`; `make-symbol` allocates a fresh symbol each call, so `(eq (make-symbol "g") (make-symbol "g"))` is `nil`; `sort` is stable and type-preserving, so `(sort (vector 5 3 1 4 2) #'<)` is `[1 2 3 4 5]`; zero-padding keeps the sign in front, so `(format "%05d" -42)` is `"-0042"`; and `append` flattens vectors and strings to character codes, so `(append "ab" nil)` is `(97 98)`. Each is the kind of detail that separates a faithful port from a plausible one.

library.rs and prelude.rs: the derived surface

`library.rs` and `prelude.rs` exercise the prelude — the elisp-on-elisp layer. `library.rs` checks the set and list operations and the convenience macros: `(seq-union '(1 2 3) '(3 4 5))` is `(1 2 3 4 5)`, `(flatten-list '(1 (2 (3 4)) 5))` is `(1 2 3 4 5)`, `(add-to-list 'al 'c)` prepends, `(cl-incf x 2)` works as a place macro, and `(with-demoted-errors "e: %s" (error "boom"))` is `nil` — the error-demoting macro swallowing a signal. `prelude.rs` checks the accessors and the generalized places: `(plist-get '(:a 1 :b 2) :b)` is `2`, `(seq-some (lambda (x) (and (> x 2) x)) '(1 2 3))` is `3`, `(cl-reduce (lambda (a b) (+ a b)) '(1 2 3 4))` is `10`, and the `setf` places `(setf (aref v 1) 7)` and `(setf (gethash 'k h) 42)` mutate a vector and a hash table. These two files are how the prelude's correctness is gated, definition by definition.

predicates_io.rs: types and output

`predicates_io.rs` covers the type predicates and the printing-and-match surface. `(list (atom 5) (atom '(1)) (atom nil))` is `(t nil t)` — `atom` is “not a cons,” and `nil` is an atom. `(prin1 'sym)` returns `sym`, the printed value distinct from its stdout side effect. The match-data round-trip `(set-match-data '(2 5))` then `(match-end 0)` is 5. `(seq-do ... '(1 2 3))` runs for effect and sums to 6. And `(pcase-let ((a 2) (b 3)) (+ a b))` is 5, the destructuring binder. It is the file that pins the predicate-and-print behaviors the larger suite assumes.

reader.rs, regexp.rs, and math.rs: the specialized engines

Three files gate the specialized engines. `reader.rs` checks the reader’s literal forms — the escape characters, the trailing-dot integer, contagion in a read expression, and backquote expansion — the syntax of Chapter 48. `regexp.rs` checks the Emacs-regexp bridge: POSIX classes like `[[:digit:]]`, the backslashed grouping, backreferences, and the character-position match data of Chapter 35. `math.rs` checks the numeric conversions and bit operations the value-parity tests leave out — the rounding directions `(floor -3.1)` is `-4` and `(truncate -3.7)` is `-3`, the float coercion `(float 5)` is `5.0`, the bitwise truth table `(logand 12 10)` is `8`, and the shifts `(ash -8 -1)` is `-4`. Each file is a focused gate on one engine’s fidelity.

ert.rs and examples.rs: the framework and the files

The last two files test the testing. `ert.rs` confirms the ERT framework’s own bookkeeping — that `should` explains a failure by reporting the subexpression value `((+ 1 2) . 3)`, that an XFAIL that fails counts zero, that an XFAIL that passes counts one (an unexpected success), and that a passing test alongside an XFAIL keeps the count at zero. This is the meta-level guarantee that the framework every example relies on is itself correct. `examples.rs` closes the loop by running the `.el` example files themselves as tests, so the files this book walks — `demo.el`, `stats.el`, `data-model.el`, `script-demo.el`, and the rest — are executed on every commit and asserted to exit cleanly. Between the topic-organized view of Chapters 46 and 47 and this file-organized view, the whole suite is accounted for: broad behavior in `eval.rs`, primitive edges in `builtins.rs`, the prelude in `library.rs` and `prelude.rs`, predicates and I/O in `predicates_io.rs`, the three engines in their named files, the framework in `ert.rs`, and the examples in `examples.rs`. Every behavior this book describes is, somewhere in that map, an assertion the CI runs — which is the grounding that lets the book claim fidelity rather than plausibility.

Chapter 14 – Generalized Variables: `gv`, `setf`, and First-Class Places

There is a small, deep idea buried in Emacs Lisp that most people meet only through its surface: `setf`. You write `(setf (car x) 9)` and the car of `x` becomes 9; you write `(setf (gethash 'k h) 5)` and the hash entry updates; you write `(setf (aref v 1) 8)` and the vector slot changes. What makes this work — that an access form like `(car x)` can be turned into the *store* it implies — is the generalized-variable machinery, `gv`, ported faithfully in `gv.el`. It is one of the more elegant corners of the language, and it is a good test of the frontend thesis, because a *place* is not a value the VM can hold; it is a compile-time transformation, and `elisp` has to perform that transformation with its own macro expander before a single opcode is emitted.

What a place actually is

The insight of generalized variables is that a form like `(car x)` names a *location*, not just a value, and every location can be described by two pieces of code: a getter that reads it and a setter that writes it. A place-expander is a higher-order function — it takes a “do” continuation and hands it a getter and a setter, so it takes the shape `(getter setter -> code) -> code`. The expander for `car` knows that its getter is `(car x)` and its setter is `(lambda (v) (setcar x v))`; the expander for `nth` knows that `(nth n l)` reads with `nth` and writes with `setcar` on the appropriate tail. `setf` does not have these rules baked in as a closed set. It consults a *registry*: each place’s head symbol carries its expander on the symbol’s `gv-expander` property, exactly where Chapter 5’s obarray and property system say it should live. `setf` special-cases the common places in its own `cond` — `car`, `cdr`, `nth`, `aref`, `gethash`, `alist-get` — and for anything its `cond` does not catch, it falls through to the registry. The `gv.el` example pins that fall-through with `setf-through-gv-registry`: a `gv-deref` place is reachable *only* via the registry, and `setf` finds it there.

Extending `setf` from user code

Because the rules live on symbol properties, a program can teach `setf` about a new place without touching `setf` itself. The example does exactly this at top level:

```
(gv-define-setter my-car (v x) `(setcar ,x ,v))
(gv-define-simple-setter my-set-value set 'fix)
```

`gv-define-setter` installs an expander that, given the new value `v` and the place’s argument `x`, produces the store code `(setcar x v)`; after it runs, `(setf (my-car l) 9)` expands to `(setcar l 9)` and mutates the list. `gv-define-simple-setter` is the common case made shorter: a place whose setter is just a function call. Its `set` argument names the store function and its `'fix` argument — `FIX-RETURN` — asks for a subtle but important guarantee: that the whole `setf` form evaluates to the *value assigned*, not to whatever the underlying store function happened to return. The test `gv-define-simple-setter-fix-return` pins it: `(setf (my-set-value s) 42)` returns 42, even though the underlying `set` returns its own thing. This is not cosmetic. `setf` is supposed to behave like assignment, and assignment yields the assigned value; `FIX-RETURN` is the machinery that keeps the promise when the store function does not cooperate.

The reason these definitions sit at top level in the example, and the reason the file’s own comment insists on it, is the same reason the macro chapters kept returning to: expanders run at *macroexpansion time*. `(setf (my-car l) 9)` is expanded when the form containing it is compiled, and that expansion consults the registry

as it stands *then*. If `gv-define-setter` ran inside the same top-level form as the `setf` that used it, the expander would not yet be installed when the `setf` was expanded. Top-level ordering is not style here; it is a correctness requirement, and it is one of the recurring shapes of `elisp`s code — anything that must be *visible to the compiler* has to be defined in an earlier top-level form than its first use.

First-class references: `gv-ref` and `gv-deref`

The last move `gv` makes is the surprising one. A place is a compile-time thing, but sometimes you want to pass “the car of `x`” to another function as a *value* — to capture a location now and read or write it later, elsewhere. `gv-ref` does this: it packages a place's getter and setter into a first-class object, and `gv-deref` reads it, with `(setf (gv-deref r) v)` writing back through it. The example captures `(car x)` as `r`, reads 1 through it, then writes 5 and watches `(car x)` become 5 — a genuine reference to a cons slot, carried as a value. It works on compound places too: `(gv-ref (nth 1 l))` captures the second element and writes through it. And, most tellingly, it works on a place whose identity is chosen at *runtime*: `(gv-ref (car (if c a b)))` captures the car of whichever list the condition selects, and writing through the reference mutates exactly that one. A reference is a getter/setter pair frozen at the moment `gv-ref` runs, and the `if` expander is what lets the selection happen inside that freeze.

That `gv-ref` is representable as an ordinary cons of two closures — the `setf-through-gv-registry` test builds one by hand, `(cons (lambda () (car cell)) (lambda (v) (setcar cell v)))`, and `setf` drives it through the registry fallback — is the whole thing in miniature. A generalized variable is not a language primitive the VM must understand. It is a pair of functions and a macro that knows how to assemble them, and `elisp`s, being a reader and a macro expander over its own heap, implements it in `elisp` on top of the same closures and symbol properties the rest of the language already has. `setf` never becomes an opcode; it becomes, by expansion, the store code the place implies, and that store code lowers to `fusevm` like anything else.

Chapter 75 — OClosures: A Closure With a Type and Slots

A closure is a function that carries an environment. An OClosure — an “open closure” — is a closure that carries, in addition, a *type* and a set of named, individually readable slots. It is Emacs’s answer to a recurring need: an object that is *also* callable, whose captured state you can inspect and sometimes mutate from the outside, and whose type participates in generic dispatch. `elisp` ports the machinery faithfully in `oclosure.el`, and it is worth a chapter because it braids together three subsystems the book has treated separately — the closure model of Chapter 6, the record and generic-function model of Chapter 22, and the symbol-property registries of Chapter 5 — into a single construct.

Defining a type, making an instance

An OClosure type is declared with `oclosure-define`, at top level, for the by-now-familiar reason: the class must exist when `oclosure-lambda` is macroexpanded. The example declares a family of them:

```
(oclosure-define foo a b)
(oclosure-define (point3 (:predicate point3-p)) x (y) (z :mutable t))
(oclosure-define (cpoint (:parent point3)) color)
(oclosure-define (counter) (n :mutable t))
```

The first is the plain case: a type `foo` with two immutable slots `a` and `b`. The second shows the full grammar — a `:predicate` naming the generated type test, and per-slot options, where `(z :mutable t)` marks `z` as writable while `x` and `y` stay read-only. The third declares inheritance: `cpoint` has `point3` as parent, so it gets `point3`’s slots first, in order, then its own `color`. An instance is built with `oclosure-lambda`, which looks like a `lambda` with its captured slots named up front:

```
(oclosure-lambda (foo (a 1) (b 2)) () (+ a b))
```

This is callable — `(funcall o)` returns 3 — and the slots are simultaneously in scope in the body, because they *are* the closure’s captured locals. That is the key identity the example pins with `oclosure-slots-are-closure-locals`: an instance built as `(foo (a 10) (b 5)) (k) (* k (+ a b))` and called with 3 returns 45. The slot values and the body’s free variables are the same storage, not two parallel copies. An OClosure is not a struct with a function stapled on; it is a closure whose captured environment has been given names the outside world can also reach.

Reading slots, mutating the mutable ones

Each slot generates an accessor named `TYPE--SLOT`: `foo--a`, `point3--z`, `cpoint--color`. Reading is total — every slot is readable from outside through its accessor, even the immutable ones, so `(foo--a o)` returns 1 and `(point3--z o)` returns 30. Writing is gated by the slot’s declaration. A `:mutable` slot can be set two ways, and the example exercises both: `(oclosure--set-slot-value o 'z 300)` writes by name, and `(setf (point3--z o) 999)` writes through the accessor as a generalized variable — the `gv` machinery of the previous chapter reaching into an OClosure’s slot. Crucially, the mutation is visible to the *body*, because the slot and the body local are one cell: after setting `z` to 300, calling the instance recomputes with the new value. And an immutable slot refuses: `(oclosure--set-slot-value o 'x 99)` on a slot that

was not marked `:mutable` signals `setting-constant`, the same condition Emacs raises when you try to `setcar` a literal — a read-only slot is genuinely read-only, enforced at store time.

Types, predicates, and the inheritance chain

The “type” in “open closure” is what earns the name. `(oclosure-type o)` returns `foo`; the same call on a plain `(lambda () 1)` returns `nil`, because a bare lambda is not an OClosure. The generated predicate is where inheritance shows its hand. `point3-p` and the internal `TYPE--internal-p` do not return a bare boolean; they return the *tail of the allparents list* from a `memq`, so a `cpoint` instance answers `(cpoint--internal-p c)` with `(cpoint point3 oclosure)` and a `point3` instance answers `(point3-p c)` with `(point3 oclosure)`. A `cpoint` satisfies the `point3` predicate — `(point3-p c)` is truthy — because `point3` is in its parent chain, but a plain `point3` fails `cpoint--internal-p`, because `cpoint` is not in *its* chain. Inheritance is a list-membership test over a precomputed ancestry, and the predicate returns the matching suffix rather than `t` so that callers can see *how* the type matched.

The example closes on the pattern that motivates the whole facility: `cl-generic`’s no-next-method closures. The declaration `(oclosure-define (cl--generic-nnm))` makes a *slotless* type whose only purpose is to be identifiable by `oclosure-type`. The generic dispatcher builds an instance of it and later asks, in effect, “is this the `nnm` closure?” with a bare `(eq (oclosure-type x) 'cl--generic-nnm)`. That is the entire reason OClosures exist in Emacs — `cl-generic` needed callables it could tag and later recognize — and the `oclosure-nnm-dispatch-pattern` test reproduces the exact shape the dispatcher relies on. Even the accessors are OClosures: `foo--a`’s function cell is itself an OClosure of type `oclosure-accessor` carrying `(accessor--type . foo)` and `(accessor--slot . a)`, so the reflection is total — you can ask an accessor which slot of which type it reaches. It is objects and closures fused into one thing, and `elisp` builds it, like `gv`, out of the records, closures, and symbol properties it already owns.

Chapter 76 – Customization Without a UI: `defcustom` and the Theme Machinery

Emacs's customization system exists to serve a graphical interface: the Customize buffers, the widgets, the “apply and save” buttons. `elisprs` has none of that — no buffers as windows, no widgets, no interactive loop. So it is a fair question why `defcustom`, `defgroup`, `defface`, and `custom-set-variables` are ported at all. The answer is concrete and unglamorous: init files. Every Emacs user's configuration is full of `defcustom` declarations and, at the bottom, a machine-generated `custom-set-variables` block. A faithful runtime that could not read those forms could not load real-world elisp, and before this port, loading any such file failed with “Symbol's function definition is void: custom-set-variables.” So `elisprs` ports the *declaration and persistence* half of `custom.el` — the part that stores symbol properties and writes variable defaults — and deliberately omits the UI half, which has nothing to attach to. The examples `defcustom-decl.el`, `custom-set-variables.el`, and `custom-set-default.el` are the regression gates on exactly that boundary.

What `defcustom` actually stores

Stripped of the interface, `defcustom` is `defvar` plus a handful of symbol properties. The example pins each one. `(defcustom dcd-x 5 "An integer option." :type 'integer :group 'dcd-g2)` binds `dcd-x` to 5 like a `defvar` would, and then records: `standard-value` becomes (5) — the *unevaluated* standard, wrapped in a list; `custom-type` becomes `integer`; the variable is marked special (`special-variable-p` is `t`); its documentation lands on `variable-documentation`. The `:group` does *not* land on the option — the option's own `custom-group` property stays `nil` — but the group's member list grows an entry `((dcd-x custom-variable))`. This asymmetry is exact Emacs behavior, and pinning it is the point: the association between an option and its group is stored on the group, as a typed member list, not on the option.

The `defvar`-like discipline is preserved down to its most important quirk. Like `defvar`, `defcustom` does *not* clobber an already-bound value — but it still records the standard. The test `defcustom-respects-existing-binding` sets `dcd-p` to 99, then declares it with a `defcustom` whose standard is 5, and checks both facts: `dcd-p` is still 99, and `standard-value` is (5). The value the user already chose wins; the standard is remembered so the customization system knows what “reset to default” would mean. Keyword arguments land where Emacs puts them: `:require` becomes `custom-requests`, `:options` becomes `custom-options`, and `:set` installs a setter that runs *at declaration time* — the example's `dcd-s`, declared with a doubling `:set`, ends up bound to 20, not 10. And the one genuine error the declaration syntax can raise is preserved: a keyword where the doc string belongs, `(defcustom bad 1 :type 'integer)`, signals “Doc string is missing,” because `custom.el` checks that the third argument is a string.

`custom-set-variables` and the fake “user” theme

The block a Customize session writes into an init file looks trivial — `(custom-set-variables '(inhibit-startup-screen t) '(tab-width 8))` — but the machinery it drives is not, because Emacs implements user customization as a *theme*. `custom-set-variables` delegates to `(custom-theme-set-variables 'user ...)`, and for each `(SYMBOL EXP ...)` entry it does three things: it records `EXP` unevaluated under

the symbol's `saved-value`; it registers the setting under the theme `user` via `custom-push-theme`, which threads through the symbol's `theme-value` and the `user` theme's `theme-settings`; and it calls the setter — the per-symbol `custom-set` property if present, else `custom-set-default` — but *only* if the variable already has a default or the `NOW` flag is set.

The theme bookkeeping is where the faithful port earns its keep, and the example pins the exact shapes. After `(custom-set-variables '(csv-a 8))` with `csv-a` previously `defvar`'d to 0, the symbol's `theme-value` is `((user 8) (changed 0))` — the new value under the `user` theme, and the *old* value stashed under a fake `changed` theme so that disabling the customization could restore it. That `changed` pseudo-theme is a real Emacs mechanism, not an approximation, and the test asserts it verbatim. The conditional-set logic is pinned too: `csv-unbound`, a symbol with no default, is *recorded* under `theme-value` but not *set* — it stays void as a value — because `custom.el` only funcalls the setter when the variable is already bound or `NOW` is supplied. Add the `NOW` flag, as in `(custom-set-variables '(csv-rogue 42 t))`, and the variable is force-bound to 42 and stamped with `force-value`. The per-symbol `custom-set` override is honored: install a logging setter on `csv-hooked`'s `custom-set` property and it, not the default, receives `(csv-hooked 99)`.

`custom-set-default`, the default setter, is a five-line function, and the example ports it to the letter — including the detail that it tail-calls the C subr `set-default-toplevel-value`, which returns `nil`, so `(custom-set-default 'v 7)` evaluates to `nil` even though it stored 7. It also honors the `custom-local-buffer` branch: bound to a buffer, the write goes through `with-current-buffer` and `set` rather than the global default, and the flag variable carries `permanent-local`. None of this touches a widget or a window. It is pure data — symbol properties, a theme registry, and a default write — which is exactly the part of the customization system that has meaning outside the editor, ported so that real init files load.

Chapter 77 — Keymaps as Data: The Editor's Binding Model, Without the Editor

A keymap is the most editor-shaped object in Emacs Lisp: it maps key sequences to commands, and its entire reason for existing is the interactive command loop that `elisp` does not have. So it is a genuinely interesting question what “keymaps without the editor” could even mean. The answer, pinned by `keymap.el`, is that a keymap is *two* things wearing one name. It is a piece of live editor state — the thing `use-local-map` installs and the command loop consults — and it is also a plain, inspectable *data structure* built by pure functions with no reference to any buffer or display. `elisp` ports the second entirely and declines the first, and the boundary between them turns out to be clean.

A keymap is a list

The data model is disarmingly literal. `(make-sparse-keymap)` returns `(keymap)` — the symbol `keymap` in a one-element list. Give it a menu name and you get `(keymap "m")`. Bind a key and the binding is consed on: after `(define-key m "a" 'foo)`, the keymap is `(keymap (97 . foo))`, where 97 is the character code of `a` and the binding is a dotted pair. `keymapp` is simply “is this a list whose head is `keymap`, or a symbol whose function cell is such a list.” Everything follows from that shape. `define-key` returns the definition it stored; redefining a key replaces the pair in place; `(define-key m "a" nil t)` — the `REMOVE` flag — strips the binding entirely, returning the keymap to `(keymap)`. `lookup-key` on an unbound key returns `nil`, and on a key sequence *longer* than any binding it returns the integer count of events it consumed before running out — `(lookup-key m "ab")` is 1 when only `a` is bound. These are not approximations of Emacs behavior; every one is annotated in the example with the exact `emacs -Q --batch` result it reproduces.

Multi-event sequences build the structure the editor's command loop walks one event at a time. `(define-key m "\C-xf" 'bar)` does not store a two-character key; it builds a *nested prefix keymap*: `(keymap (24 keymap (102 . bar)))` — event 24 (`C-x`) maps to an inner keymap in which 102 (`f`) maps to `bar`. Looking up the prefix alone, `(lookup-key m "\C-x")`, returns that inner keymap, and `keymapp` confirms it is one. This is precisely how a prefix key works in the editor — `C-x` selects a sub-keymap and the next event is looked up there — reproduced as pure list surgery. Vector keys carry non-character events like `[f5]`, and a meta-modified key expands into its two-event ESC form: `(define-key m (key-parse "M-a") 'foo)` produces `(keymap (27 keymap (97 . foo)))`, ESC then `a`, which is exactly how Emacs encodes Meta on a terminal.

Parents, composition, and the string API

Keymaps chain. `set-keymap-parent` splices a parent keymap onto the tail of a child, so a lookup that misses in the child falls through to the parent, and a child binding *shadows* a parent binding of the same key. The shared-tail representation is visible in the data: after parenting `m` to `p`, `m` is `(keymap (97 . ma) keymap (120 . px))` — the child's own binding, then the parent keymap inlined at the tail. `make-composed-keymap` builds the other combinator, a keymap that consults several maps in order, and `lookup-key` descends into each composed sub-keymap until one binds the key. The example even reconstructs the real preloaded maps —

`button-buffer-map` and `special-mode-map` — byte-for-byte and pins that `tabulated-list-mode-map`'s composed parent resolves `forward-button` through the nested button map and `revert-buffer` through the special-mode tail. That these library keymaps exist and have their exact Emacs shapes, with no editor running, is the whole point: they are data, and the data is faithful.

On top of the character-code primitives sits the modern string API from `keymap.el` — `keymap-set`, `keymap-lookup`, `define-keymap`, `defvar-keymap` — which speak human key descriptions like `"[C-c c]"` and translate through `key-parse` into the internal event vectors. `key-parse "[C-x C-f]"` is `[24 6]`; `key-parse "M-a"` is `[134217825]`, the meta bit set on character 97; `key-parse "<f5>"` is `[f5]`. `key-valid-p` gates which strings are acceptable — `"[C-c o]"` and `"RET"` yes, a raw control character `"\C-x"` no. `defvar-keymap` combines the variable declaration with the construction and supports `:doc`, `:parent`, and `:suppress`, so a real mode map like `special-mode-map` — defined in the Emacs source with `defvar-keymap ... :suppress t` — is reproduced with `q` bound to `quit-window` and self-insertion suppressed. `define-prefix-command` sets *both* a symbol's value and function cells to one shared keymap, which is why a prefix command's name is itself `keymapp`. `suppress-keymap` remaps `self-insert-command` to `undefined` and binds the digits to `digit-argument` — the machinery a read-only mode uses to turn ordinary keys inert.

Every one of these is a transformation on lists and vectors, and none of them touches a buffer, a window, or the command loop. The example is explicit about the line it does not cross: `use-local-map`, `current-local-map`, and `global-set-key` into a live global map are *not* covered, "because they need buffer-local/global state `elisp`s does not model." That is the exact seam the whole language sits on. The keymap as a data structure — constructed, parented, composed, looked up — is language, and `elisp`s has it in full. The keymap as installed editor state is editor, and `elisp`s, by design, does not.

Chapter 78 — First-Class Obarrays: Namespaces You Can Hold

Chapter 5 introduced the obarray as *the* symbol table — the single global namespace in which `intern` finds or creates the symbol for a name, the mechanism that makes two occurrences of `foo` in a program the same object. But the obarray is not only the global one. Emacs Lisp lets you make *more* of them, hold them as values, and intern into them independently, and `elisp` ports this in `obarrays.el`. A first-class obarray is a namespace you can carry in a variable, and it is a surprisingly powerful primitive: it is how you build a private symbol table that does not collide with the global one, which is what `intern`-based hashing, isolated completion tables, and hygienic symbol generation all quietly rely on.

Making one, and what makes it distinct

`(obarray-make)` — optionally with a size hint — returns a genuine obarray object, not a vector. This is a real Emacs 30 change the example is careful to pin: in older Emacs a bare vector could serve as an obarray, but Emacs 30 made obarrays a distinct type, so `(obarrayp (make-vector 3 0))` is now `nil` while `(obarrayp (obarray-make))` is `t`, and `(type-of ob)` is the symbol `obarray`. A fresh obarray prints as `#<obarray n=0>`, and the printed count tracks the symbols it holds — a nice observable that lets the tests assert membership without depending on symbol addresses. The size argument is vestigial (a capacity hint), but it is still validated as a whole number: `(obarray-make "x")` signals `(wrong-type-argument wholenump "x")` and `(obarray-make -1)` signals the same for `-1`. The global `obarray` variable is itself one of these objects, so the private and global tables are the same kind of thing, differing only in which one `intern` defaults to.

Isolation is the whole feature

The reason to make a private obarray is that it is a *separate namespace*, and the example pins isolation from every angle. Intern `"foo"` into a private `ob` and you get a symbol; intern `"foo"` into the same `ob` again and you get the *very same* symbol, `eq` to the first — interning is idempotent within a table. But that symbol is *not* `eq` to the global `foo` you get from `(intern "foo")` with no obarray argument. They share a name and nothing else. The private symbol carries its own value cell — `(set s 42)` then `(symbol-value s)` is 42 — with no effect on the global `foo`. `intern-soft` respects the boundary too: it finds a name present in the given obarray and returns `nil` for one that is absent, without ever creating it. This is the property that makes private obarrays useful for anything that generates or collects symbols: a macro that needs scratch symbols, a reader that wants a package-local namespace, a completion table built by interning candidate strings — all of them get a table whose contents cannot leak into, or be shadowed by, the global environment.

The rest of the surface is the symbol-table operations parameterized by which table they act on. `unintern` removes a name and returns `t` when it removed something, `nil` when the name was absent — `(unintern "a" ob)` is `t`, `(unintern "missing" ob)` is `nil`, and afterward `(intern-soft "a" ob)` is `nil`. `mapatoms` walks *exactly* the symbols of the obarray it is given: intern three names into a private table, `mapatoms` over that table, and you see those three and nothing else — no leakage from the enormous global obarray, and the count in the printed form matches. Order is unspecified, so the test sorts before comparing, which is itself a small fidelity detail: a faithful port must not promise an iteration order the real thing does not guarantee. First-class obarrays are, in the end, the same reader and heap machinery of Chapter 5 made plural — the

symbol table was never singular by necessity, only by default, and `elisprs` exposes the plurality as an ordinary value.

Chapter 79 — The Printer's Depth Guard: Circular Structure and PRINT_CIRCLE

The reader turns text into structure; the printer turns structure back into text. Chapters 23 and 34 covered the printer's normal behavior — how `prin1` quotes and `princ` does not, how `%S` and `%s` differ. This chapter is about the printer's one hard *limit*, ported in `print-circle-depth.el`, and it is a good study in what “faithful” means when the thing being matched is not a feature but a failure mode. Emacs's printer, when asked to print a structure that is nested too deeply, does not loop forever and does not silently truncate. It signals a specific error, at a specific depth, and a faithful runtime must signal the *same* error at the *same* depth — reproducing the boundary exactly, not merely staying safe near it.

A depth bound, not a length bound

Emacs's `print.c` defines a constant, `PRINT_CIRCLE`, equal to 200. With `print-circle nil` — the default — the printer aborts once nesting reaches that depth and signals (`error "Apparently circular structure being printed"`). The name reveals the intent: the guard exists to keep the printer from spinning forever on a genuinely circular structure, one that points back at itself, back when the printer had no other cycle detection. But the mechanism is cruder than true cycle detection — it is a plain *depth* counter — and the crucial thing to understand, and the thing the example spends most of its assertions pinning, is that the bound is on nesting **depth**, not on **length**. A flat list of a thousand elements prints without complaint; the printer descends into each element only one level. A structure nested two hundred levels deep in its car signals, because the printer has to recurse two hundred frames to reach the bottom.

The example builds the two structures with deliberate helpers. `pcd-nest-list` wraps a value in `list` *N* times, so *N* of 200 is a list nested 200 deep; `pcd-nest-vec` does the same with `vector`. And the boundary it asserts is *exact*: 199 deep prints fine — (`prin1-to-string (pcd-nest-list 199)`) returns a string — and 200 deep signals the circular-print error. Not “somewhere around 200,” but 199 succeeds and 200 fails, for both lists and vectors, because vectors count toward print depth identically to lists. The depth-not-length distinction gets its own test: a thousand-element flat list, built by consing, prints without error, proving the guard is counting recursion frames, not elements. To reproduce this a runtime must carry its own depth counter through the recursive printer and trip it at precisely 200, matching a constant buried in Emacs's C source — the kind of number that would be easy to get off by one and that only an oracle-checked test would catch.

Every entry point, the same guard

A subtlety that the example is careful to close is that the guard is not attached to one printing function; it lives in the shared recursive core that *all* of them call. So it must fire identically no matter how the printing was requested. The test `pcd-all-print-entry-points` takes one structure nested exactly 200 deep and drives it through four different doors: `prin1`, `princ`, (`format "%S" deep`), and (`format "%s" deep`). All four signal the same (`error "Apparently circular structure being printed"`). This matters because `%s` and `%S` differ in *how* they print — `%S` uses the read syntax, `%s` the display form — but they share the same recursive descent, so they share the same depth guard, and a faithful port cannot special-case one door

and forget another. The condition object itself is asserted as a whole — `(error "Apparently circular structure being printed")`, the symbol and the exact message string — because the message is part of the contract; code in the wild does `condition-case` on it and pattern-matches the text.

What makes this chapter representative of the whole project is that the behavior being ported is a *guard rail*, and guard rails are where careless reimplementations diverge. It would be easy to build a printer that recurses without a depth limit and blows the Rust stack on a deeply nested structure — a panic, an abort, the process gone. It would be almost as easy to add a limit but at the wrong depth, or with the wrong message, or firing from only some entry points. Faithfulness here is not “handle the case gracefully”; it is “fail at 200, from every door, with that exact sentence.” The example exists to hold *elisprs* to that standard, and it is the natural bridge to the last chapter, which is about the discipline of never failing in the *wrong* way.

Chapter 80 – Adversarial Robustness: The Frontend That Must Not Panic

There is a promise a faithful interpreter makes that is easy to state and hard to keep: on *any* input, it may signal an elisp error, and it may compute a value, but it must never *crash*. A crash — a Rust panic, a stack overflow, an aborted process — is not a behavior Emacs Lisp has a name for, and a runtime that crashes where Emacs would signal `wrong-type-argument` has broken the contract even though no data was corrupted, because a program that wrapped the call in `condition-case` expected to *catch* the failure and instead lost the whole process. The example `adversarial-panics.el` is the record of an adversarial hunt for exactly these divergences — malformed and extreme inputs that used to abort elisprs and now signal the precise condition Emacs 30.2 signals — and it is the right chapter to close the reference on, because it is where the frontend architecture's sharpest edge lives.

Why a frontend is exposed to this

The panic risk is not incidental to the frontend design; it is a direct consequence of it. elisprs is Rust lowering onto a Rust VM, and Rust's failure modes — integer overflow in debug, slice indexing past the end, formatter limits, unbounded recursion into a native stack — are *aborts*, not exceptions. Emacs, written in C with a longjmp-based signal system, has different failure modes and has spent forty years mapping extreme inputs onto elisp conditions. Every place where a Rust idiom's natural failure is a panic and Emacs's corresponding behavior is a signal is a place the frontend has to insert a translation, and the only way to find all of them is to attack the runtime with the inputs that trigger the underlying panics. The example pins three families the hunt turned up, each oracle-checked against `emacs -Q --batch`.

The three families the hunt closed

The first is the **formatter limit**. Rust's floating-point formatter carries a `u16` precision field, so a format directive asking for 65536 or more digits of precision — `(format "%.70000f" 1.5)` — used to panic with “Formatting argument out of range.” Emacs has no such limit: it prints the value's exact decimal expansion and pads past it with zeros. The fix reproduces that exactly, and the test pins the *lengths and contents*, not just the absence of a crash: `(format "%.70000f" 1.5)` is “1.5” followed by 69999 zeros, for a total length of 70002; `%e` pads the mantissa before the exponent; `%g` trims trailing zeros so the huge precision collapses back to “1.5”. It even pins the boundary just under the old cap, `(format "%.65535f" 1.0)` at length 65537, to prove the cap-and-pad path is correct at its edge. Matching Emacs here means not “don't crash” but “produce Emacs's exact string for a precision no sane program would request.”

The second family is **higher-order primitives called with too few arguments**. `funcall`, `mapcar`, `mapc`, `sort`, `maphash` — the re-entrant intrinsics of Chapter 10 — used to reach into an empty argument slice and index out of bounds, a panic, when called with fewer arguments than they need. Emacs signals `wrong-number-of-arguments` with the subr's name and the count given: `(funcall)` signals `(wrong-number-of-arguments funcall 0)`, `(mapcar #'car)` signals `(wrong-number-of-arguments mapcar 1)`, and so on through the family. These are exactly the subrs that Chapter 10 singled out as running *outside* the host borrow because they re-enter the evaluator, which is why they are also the ones most exposed to malformed arity — they are handled by name before the normal dispatch, and the by-name handling had to grow the same argument-count checks the normal path has. The third family is **apply's spread semantics**: `apply` spreads its *last*

argument, which must be a list, so `(apply '+)` with a single argument treats `+` itself as that last argument and signals `(wrong-type-argument listp +)` — matching Emacs — rather than indexing an empty list. `(apply '+ 5)` signals `(wrong-type-argument listp 5)`; well-formed applies like `(apply '+ 1 2 '(3 4))` still evaluate to 10, so the guard does not disturb the normal path.

The shape of every fix is the same, and it is the shape of the whole robustness discipline: find the input whose natural Rust behavior is a panic, determine the exact elisp condition Emacs signals for it, and insert the check that converts one into the other. The test then pins not just that a condition is raised but *which* condition, with its exact symbol and data, because “does not crash” is too weak a promise — a program catches `wrong-number-of-arguments` specifically, and getting the condition *type* wrong is its own divergence. This is the unglamorous underside of faithfulness. The narrative chapters admired an architecture that rents a VM and lowers a language onto it; this chapter is the reminder that the rented VM speaks Rust’s failure language, the hosted language speaks elisp’s, and every gap between the two is a panic waiting for an adversarial input to find it. The work of closing those gaps, one oracle-checked assertion at a time, is what turns “runs elisp” into “runs elisp faithfully” — which is the difference the entire book has been about.

Appendix A – The Special Forms

The twenty-two forms the compiler recognizes directly, with a one-line description of each. These are the irreducible core; everything else in the language is a macro or a subr built on top of them.

```
(quote X)           ; return X unevaluated ('X)
(function F)        ; the function F; closure for a lambda (#'F)
(lambda ARGS BODY...) ; an anonymous function (a closure)
(progn BODY...)     ; evaluate body in order, return the last value
(prog1 FIRST BODY...) ; evaluate all, return the first value
(if COND THEN ELSE...) ; conditional; ELSE is an implicit progn
(when COND BODY...)  ; BODY if COND is non-nil, else nil
(unless COND BODY...) ; BODY if COND is nil, else nil
(cond CLAUSE...)     ; first clause whose test is non-nil wins
(and FORM...)        ; left-to-right; first nil, or the last value
(or FORM...)         ; left-to-right; first non-nil, or nil
(while COND BODY...) ; loop while COND is non-nil; returns nil
(setq SYM VAL ...)   ; assign to variables (lexical, then global)
(let (BINDINGS) BODY...) ; bind all inits in the outer scope, in parallel
(let* (BINDINGS) BODY...) ; bind sequentially; each init sees the prior ones
(defun NAME ARGS BODY...) ; define a function (captures lexical env)
(defmacro NAME ARGS BODY...) ; define a macro (no env capture)
(defvar NAME [VALUE]) ; declare a special (dynamic) variable
(defconst NAME VALUE) ; declare a special constant
(catch TAG BODY...) ; establish a catch point for throw
(unwind-protect BODY CLEANUP...) ; run BODY, then CLEANUP unconditionally
(condition-case VAR BODY HANDLERS...) ; run BODY, dispatch errors to HANDLERS
```

The three buffer-scoping forms `save-excursion`, `save-current-buffer`, and `save-restriction` are recognized but *not yet lowered*; the compiler rejects them at compile time with a “buffer milestone” message rather than miscompiling them. `throw` is a subr, not a special form (its arguments are evaluated), and pairs with the `catch` special form.

Appendix B — Glossary

AOT (ahead-of-time) compilation. Emitting a native object file or standalone executable from a `.el` program, via `--aot/--aot-exe`, through fusevm's AOT backend. The lowering and linking work; user-program constant reification is a named limitation.

Arena. The `ElispHost`'s `Vec<Obj>` of heap objects. A `Value::Obj(u32)` handle is an index into it. There is no garbage collector; objects live for the run.

Backquote. The ``` templating syntax, expanded *eagerly at read time* into `cons/append/vconcat/quote` calls, which is what makes `pcase` backquote patterns recognizable.

Banker's rounding. `round`'s rule of rounding a tie to the nearest even integer, so `(round 2.5)` is `2` and `(round 0.5)` is `0`. Matches Emacs; avoids the upward bias of round-half-up when summing.

Borrow discipline. The rule that re-entrant intrinsics (`mapcar`, `apply`, the non-local-exit thunks) run *outside* the thread-local host borrow, threaded through `call_function`, so a callee's own `with_host` does not deadlock on an already-held borrow.

Builtin count. The arena length right after `install` finishes registering the subr library (over three hundred subrs). Everything at or above this index is user-or-prelude data; the heap image and the schema fingerprint key off this boundary.

Bytecode cache. The rkyv shard at `~/.elisp/ris/scripts.rkyv` holding per-form `fusevm::Chunks` and a clean heap image, keyed by path, mtime, and a schema fingerprint, so a second run skips read/expand/compile and the prelude rebuild.

Chunk (`fusevm::Chunk`). The unit of compiled code: opcodes, a constant pool, and a names table. The artifact the compiler produces and fusevm runs.

Clean heap image. The heap image with symbol value cells reset to the post-prelude baseline (before the user file ran), so re-running cached chunks reproduces the original execution exactly rather than re-applying global mutations.

Constant pool. The part of a chunk holding its literal constants, including `Value::Obj` handles baked in at compile time — the reason a chunk needs a heap image to run against.

Closure. An `Obj::Closure`: parsed parameters, a compiled chunk body, an `is_macro` flag, and a captured lexical environment. A macro is a closure with `is_macro: true`.

Coercion idiom. The prelude's `(append seq nil)`, which flattens any sequence (list, vector, or string) to a list of elements. It is what makes one `seq-/cl-` function work on all three sequence types.

Cons cell. An `Obj::Cons(Value, Value)`: `car` and `cdr`, each an arbitrary value. Has identity; `setcar/setcdr` mutate it in place. A non-nil, non-cons `cdr` is a dotted pair.

Contagion (numeric). elisp's rule that a float anywhere in an arithmetic expression makes the result a float. Implemented by an `isf` flag OR-ed across operands; `(+ 1.5 2)` is `3.5`.

DAP / debug adapter. `elisp --dap`, a Debug Adapter Protocol server giving form-level breakpoints, a live `Globals` scope over the value cells, and an `evaluate` that runs against the real host. Captures the debuggee's stdout via `pipe/dup2` to keep the JSON-RPC clean.

Determinism. The property that the append-only arena allocates the same objects in the same order every run, so handles are stable. It is what makes the heap image — and thus the bytecode cache and AOT — correct.

Dynamic binding. A binding visible to everything that runs during its dynamic extent, implemented with a specstack of saved value cells. Used for variables declared special by `defvar/defconst`.

Extension handler. The callback `elisprs` registers on every `fusevm` VM. When the VM hits an `Op::Extended/Op::ExtendedWide` it does not understand, it calls this handler, which performs the elisp-specific operation against the host heap.

fancy-regex. The matching engine the Cargo manifest pins. It adds a backtracking fallback for backreferences (`\1..\9`) while keeping the linear `regex` crate's fast path for patterns that do not need it.

ERT. Emacs Lisp Regression Testing — the test framework every example file uses, built in the prelude from `condition-case`, `signal`, and the condition hierarchy. `should` explains a failure by reporting the subexpression values.

Fast path (native arithmetic). The compiler's lowering of un-redefined `+`, `-`, `*`, `1+`, `1-`, and the two-argument comparisons to native `fusevm` opcodes instead of a host `CALL`, so hot numeric loops JIT. `/`, `%`, and `mod` stay on the host for signaling fidelity.

Frame stack. Per-call bookkeeping of `(saved lexical env, specstack depth at entry)`. On a non-local exit out of an inner `let`, `unwind_scopes_to` uses it to recover the caller's binding state so leaked scopes do not corrupt it.

fusevm. The shared, language-agnostic bytecode VM with a three-tier Cranelift JIT and an AOT backend, used by five languages. `elisprs` has no VM of its own; it lowers to `fusevm` and reuses its engine.

Generic function. A function with several type-specialized methods, defined by `cl-defgeneric/cl-defmethod`, dispatched at call time on argument types by specificity. Stored in `cl--generic-table`; entirely prelude elisp.

gv (generalized variables). The machinery behind `setf`: the place-expander that inverts an accessor into its mutator, so `(setf (gethash k h) v)` becomes `(puthash k v h)` and a struct accessor becomes an `aset`. Every place macro expands through it.

Heap image. A serialized snapshot of the user-and-prelude arena objects, embedded in a cache entry or AOT object so that baked-in `Value::Obj` handles resolve correctly on reload.

Immediate. A value `fusevm` understands natively — integer, float, string, boolean, nil (`Value::Undef`) — as opposed to a compound object reached by handle.

indirect-function. The operation that follows a function-cell alias chain (a symbol whose function cell holds another symbol) to the ultimate callable. The resolver follows up to 64 hops before signaling “function indirection too deep.”

Intern. To look a name up in the obarray, returning the existing symbol or allocating and recording a new one — so two reads of a name yield the same object, which is what makes `eq` work on symbols. `make-symbol` deliberately does not intern.

JIT (three-tier). `fusevm`'s Cranelift just-in-time compiler, rented via the `jit-disk-cache` feature, which trace-compiles hot loops (especially native-arithmetic ones) to machine code and persists that code in an on-disk cache keyed by chunk hash.

LSP / language server. `elisp --lsp`, a read-only Language Server Protocol server offering completion, hover, signature help, and document symbols from a static corpus — the same corpus `gen-docs` renders to `docs/reference.html`.

Lexical binding. A binding visible only in the textual region where it is written, implemented with a chain of `Scope` frames that a closure captures. The default for non-special variables.

Lisp-2. A Lisp in which a symbol has two independent cells — a value cell and a function cell — so a name means different things in function and argument position. Emacs Lisp is a Lisp-2; `#'foo` reaches the function cell.

map library. A prelude library giving one generic keyed interface — `map-elt`, `map-keys`, `map-values`, `map-into` — over alists, plists, hash tables, and arrays, dispatching on the backing type at run time.

Method combination. The CLOS protocol `cl-defmethod` implements: `:before`, `:after`, and `:around` qualified methods running in standard order around the primary, with `cl-call-next-method` chaining the applicable primaries by specificity.

Nested VM. A fresh `fusevm:VM` run for a closure's body via `run_closure`. Recursion is recursion in nested VMs — `(fact 5)` is five nested VM invocations deep — each with its own stack and scope.

Obarray. The table of interned symbols, keyed by name. Interning the same name twice returns the same symbol object, which is what makes `eq` work on symbols.

Prelude. The large body of elisp in `src/prelude.rs`, loaded form by form on top of the primitives, that defines `cl-lib`, `seq`, `subr-x`, `pcase`, `cl-loop`, `cl-defstruct`, JSON, `rx`, `map`, and ERT — the derived standard library.

Output streams. The two-descriptor split: `princ/prinl/print` write program output to stdout, while `message` and error chatter go to stderr — so a script's result can be piped while its progress stays on the terminal.

pcase. A prelude macro for structural dispatch. Its clause compiler turns each pattern into a `(TESTS . BINDS)` pair and assembles a `cond`; the `cond` (not `catch/throw`) shape is deliberate, to expand cleanly inside a macro-produced `defun`.

Place. A form naming a settable location — a variable, a `car`, an `aref`, a `gethash`, a struct accessor. `setf` and the place macros (`cl-incf`, `push`, ...) read-modify-write one by inverting the accessor into its mutator.

Property list. A symbol's third store, a flat key-value list reached by `get/put`. The condition hierarchy (`error-conditions`) and the struct system both ride on it.

Reader. The recursive-descent parser (`src/reader.rs`) that turns text into heap objects directly — cons cells and symbols — rather than a private AST, expanding backquote eagerly at read time. Reachable at run time as `read/read-from-string`.

Record (struct). A vector tagged in slot zero with a `cl-struct-NAME` symbol, the representation of a `cl-defstruct` instance. Prints as `#s(NAME ...)`, which the reader reads back.

rx. A prelude macro that compiles an S-expression regular expression to an Emacs regexp string at expansion time, folding all-single-character alternatives into a character class as Emacs does.

Schema fingerprint. The hash of the builtin layout (count and ordered names of all subrs) and the prelude source, folded into the bytecode cache key so any change to the primitives or the prelude automatically invalidates stale chunks.

Sequence. Any of the three uniformly-operable container types — list, vector, string. The `seq` library treats them generically via the coercion idiom and `seq--type-of/seq-into` for type preservation.

setf. The generalized-assignment macro that inverts an accessor place into its mutator. The foundation under every place macro; defined twice in the prelude (a simple stub early, the full expander late).

Special form. One of the twenty-two forms the compiler lowers directly because it controls evaluation rather than submitting to it (`if`, `let`, `quote`, ...).

Specstack. The stack of `(symbol, previous-value)` entries that implements dynamic binding; pushed on `bind`, popped and restored on `unbind`, including on non-local exit.

subr-x. The prelude's modern convenience library: the string trim/pad/replace helpers and the binding-and-threading macros (`if-let`, `when-let`, `thread-first`, `thread-last`, `named-let`).

Subr. A primitive function implemented in Rust, registered by `install`, holding a name, arity bounds, and a function pointer. The floor of the library.

sxhash. Structural hashing, in the family `sxhash-equal/sxhash-eq/sxhash-eql`. Hash values are not bit-compatible with GNU Emacs but are internally self-consistent — `equal` objects hash equally within a bounded depth, which is what hash tables need.

Truthiness. elisp's rule that only `nil` is false. Because it differs from fusevm's, the compiler emits a `TRUTHY` extension op before every conditional jump to convert the value.

TRUTHY (op 0). The extension op the compiler emits before every conditional jump, converting an elisp value to a VM boolean under the “only nil is false” rule, because elisp truthiness differs from fusevm's. The conversion is `el_truthy`.

Value::Obj(u32). The fusevm value variant elisprs uses as an opaque handle into its object arena. The one door through which the entire elisp object model rides a language-agnostic VM.

Wide op. An `Op::ExtendedWide(id, n)` carrying a full-width count rather than a byte argument — used for `LETBIND` and `UNBIND`, whose binding count can exceed 255. Dispatched to a separate wide handler from the narrow extension ops.

Appendix C — The Reference Shelf

The full reference (<docs/reference.html>, generated from the language-server corpus by the `gen-docs` binary) organizes the language into the chapters below. This appendix is a map of that shelf — what lives where — not a full enumeration; reach for the reference itself when you need the exact signature and behavior of a specific entry.

Special Forms. The twenty-two compiler-recognized forms of Appendix A.

Arithmetic & Numbers. The arithmetic and comparison operators (`+` `-` `*` `/` `%` `mod` `1+` `1-` `=` `/=` `<` `>` `<=` `>=`), float and rounding operations (`floor` `ceiling` `round` `truncate` `float` `abs`), transcendentals (`sqrt` `exp` `log` `sin` `cos` `tan` `asin` `acos` `atan` `expt`), bit operations (`logand` `logior` `logxor` `lognot` `ash` `lsh` `logcount`), and `random`, `string-to-number`. The numeric helpers `max` `min` `gcd` `lcm` `isqrt` and the predicates `evenp` `oddp` `zerop` come from the prelude.

Predicates & Type Tests. The equality trio (`eq` `eql` `equal`), the negations (`null` `not`), and the type tests (`consp` `listp` `atom` `symbolp` `stringp` `numberp` `integerp` `floatp` `vectorp` `functionp`), plus `type-of`, `recordp`, `cl-struct-p`.

Cons, Lists & Sequences. The cons and list primitives (`cons` `car` `cdr` `setcar` `setcdr` `list` `append` `reverse` `length` `nth`) and the large derived list and sequence surface from the prelude: the `c[ad]+r` family, `nthcdr` `last` `number-sequence` `take` `butlast` `flatten-tree`, the membership and alist functions (`memq` `member` `assq` `assoc` `rassoc` `alist-get`), and the whole `seq-*` library.

Symbols, Cells & Binding. The Lisp-2 machinery (`symbol-name` `intern` `make-symbol` `set` `symbol-value` `boundp` `makunbound` `fset` `fboundp` `symbol-function` `special-variable-p`), property lists (`get` `put` `symbol-plist`), and `func-arity`.

Strings & Characters. The string primitives (`concat` `substring` `split-string` `upcase` `downcase` `number-to-string` `string-to-number` `string-prefix-p` `string-suffix-p` `char-to-string` `string-to-char` `make-string`) and the `subr-x` derived helpers (`string-trim` `string-join` `string-pad` `string-replace` `string-blank-p` `string-reverse`).

I/O, Print & Format. The printers (`princ` `prinl` `print` `terpri` `prinl-to-string`), `message`, `format` and `format-time-string`, `read` and `read-from-string`, and the file, time, environment, and subprocess primitives (`write-region` `insert-file-contents` `file-exists-p` `current-time` `shell-command-to-string` `call-process` `process-lines`).

Control & Functional. The functional primitives intercepted as re-entrant intrinsics (`funcall` `apply` `mapcar` `mapc` `sort` `maphash` `eval`), the macroexpand family, the signalling subrs (`signal` `error` `user-error` `throw`), and the large derived control surface from the prelude — `dolist` `dotimes` `push` `pop` `when-let` `if-let` `thread-first` `thread-last` `cl-loop` `pcase` `cl-defun` `cl-defstruct` `cl-defmethod` and the rest.

Association Lists & Hash Tables. The hash-table primitives (`make-hash-table` `gethash` `puthash` `remhash` `clrhash` `hash-table-count` `hash-table-keys` `hash-table-values`) and the alist and plist functions and the `map-*` generic interface from the prelude.

Other. The remainder — crypto and encoding (`sha1` `md5` `secure-hash` `base64-encode-string` `url-hexify-string`), the buffer and point surface (`insert` `point` `goto-char` `buffer-string` `re-search-forward` `looking-at` `replace-match` `forward-line`), the regexp surface (`string-match` `match-string` `match-beginning` `replace-regexp-in-string` `regexp-quote` `rx`), the JSON encoder and

parser, and the ERT testing surface (`ert-deftest should should-not should-error ert-run-tests-batch-and-exit`).

The in-depth library chapters of the fifth movement map directly onto this shelf, and the correspondence is the intended way to use the two documents together. When the reference's *Arithmetic & Numbers* entry leaves you wanting the *why* — why `/` is not on the fast path, why `(round 2.5)` is `2`, why a float contaminates a chain — Chapter 19 is the companion. *Cons, Lists & Sequences* pairs with Chapter 20, *Association Lists & Hash Tables* with Chapter 21, the struct-and-generic surface of *Control & Functional* with Chapter 22, the printer and equality of *I/O, Print & Format* and *Predicates* with Chapter 23, the file/time/process/crypto entries of *I/O and Other* with Chapter 24, and the `pcase/rx/JSON` and `cl-loop` entries of *Control & Functional* with Chapters 25 and 26. The reference tells you the signature; the chapter tells you the mechanism and the edge.

The reference enumerates each of these entries with its exact signature and a one-line description, generated from the same corpus the language server serves on hover. This book was the *why*; the reference is the *what*. Read this one in a chair; keep that one by the keyboard.

Appendix D – A Compact Index of the Standard Library

This appendix is a flat index of the names the catalogue chapters (28 through 40) walked in prose — the primitive subrs and the principal prelude functions, grouped by family. It is for the moment when you remember a capability but not the exact spelling. The primitives are from the language-server corpus (the 213 subrs alongside the 22 special forms); the prelude families are from [src/prelude.rs](#). This is an index, not the reference: for a precise signature and behavior, follow the name to its catalogue chapter or to [docs/reference.html](#).

Primitive subrs, by family

Arithmetic and comparison. `+`, `-`, `*`, `/`, `%`, `mod`, `1+`, `1-`, `=`, `/=`, `<`, `>`, `<=`, `>=`. (Chapter 28.)

Rounding and float. `floor`, `ceiling`, `round`, `truncate`, `float`, `fround`, `ffloor`, `fceiling`, `ftruncate`, `abs`. (Chapter 28.)

Bitwise and shift. `logand`, `logior`, `logxor`, `lognot`, `ash`, `lsh`, `logcount`, `logb`. (Chapter 28.)

Transcendental and IEEE-754. `expt`, `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `ldexp`, `frexp`, `copysign`, `isnan`, `random`. (Chapter 28.)

Number conversion. `number-to-string`, `string-to-number`. (Chapters 28, 32.)

Equality and negation. `eq`, `eq1`, `equal`, `null`, `not`, `zerop`. (Chapter 29.)

Type predicates. `consp`, `listp`, `atom`, `symbolp`, `stringp`, `numberp`, `integerp`, `floatp`, `vectorp`, `functionp`, `char-or-string-p`, `subrp`, `macro`, `special-form-p`, `recordp`, `cl-struct-p`, `char-upcase-p`, `special-variable-p`, `char-equal`, `type-of`. (Chapter 29.)

Cons and list. `cons`, `car`, `cdr`, `setcar`, `setcdr`, `list`, `append`, `reverse`, `length`, `nth`. (Chapter 30.)

Symbols and binding. `symbol-name`, `intern`, `make-symbol`, `intern-soft`, `set`, `symbol-value`, `boundp`, `makunbound`, `fset`, `fboundp`, `symbol-function`, `indirect-function`, `func-arity`, `subr-arity`. (Chapter 31.)

Evaluation. `eval`, `funcall`, `apply`, `identity`. (Chapter 31.)

Strings. `concat`, `substring`, `split-string`, `make-string`, `string`, `string-to-list`, `string-to-vector`, `char-to-string`, `string-to-char`, `string-search`, `string-prefix-p`, `string-suffix-p`, `string-empty-p`, `string-join`, `upcase`, `downcase`, `compare-strings`, `string-distance`. (Chapter 32.)

Format and message. `format`, `message`. (Chapter 32.)

Vectors. `vector`, `make-vector`, `aref`, `aset`, `fillarray`, `vconcat`. (Chapter 33.)

Hash tables. `make-hash-table`, `gethash`, `puthash`, `remhash`, `clhash`, `hash-table-count`, `hash-table-test`, `hash-table-p`, `hash-table-keys`, `hash-table-values`, `copy-hash-table`. (Chapter 33.)

Printing and reading. `princ`, `prin1`, `print`, `terpri`, `prin1-to-string`, `read`, `read-from-string`. (Chapter 34.)

Regexp and match data. `string-match`, `string-match-p`, `match-beginning`, `match-end`, `match-string`, `match-data`, `set-match-data`, `save-match-data`, `replace-regexp-in-string`, `regexp-quote`. (Chapter 35.)

Buffer extent and content. `insert`, `insert-char`, `buffer-string`, `buffer-substring`, `buffer-substring-no-properties`, `buffer-size`, `erase-buffer`, `delete-region`, `delete-char`. (Chapter 36.)

Point and position. `point`, `point-min`, `point-max`, `goto-char`, `char-after`, `char-before`, `bolp`, `eolp`, `bobp`, `eobp`, `line-beginning-position`, `line-end-position`, `pos-bol`, `pos-eol`, `current-column`, `line-number-at-pos`, `count-lines`. (Chapter 36.)

Motion and search. `forward-char`, `backward-char`, `forward-line`, `beginning-of-line`, `end-of-line`, `forward-word`, `backward-word`, `skip-chars-forward`, `skip-chars-backward`, `search-forward`, `search-backward`, `re-search-forward`, `re-search-backward`, `looking-at`, `looking-at-p`, `replace-match`. (Chapter 36.)

Files. `file-exists-p`, `file-directory-p`, `file-regular-p`, `file-readable-p`, `file-writable-p`, `file-symlink-p`, `insert-file-contents`, `write-region`, `delete-file`, `make-directory`, `rename-file`, `copy-file`. (Chapter 37.)

Processes and environment. `shell-command-to-string`, `call-process`, `process-lines`, `getenv`, `setenv`. (Chapter 37.)

Time. `current-time`, `current-time-string`, `float-time`, `format-time-string`, `decode-time`, `encode-time`. (Chapter 37.)

Crypto and encoding. `sha1`, `md5`, `secure-hash`, `base64-encode-string`, `base64-decode-string`, `base64url-encode-string`, `base64url-decode-string`, `url-hexify-string`, `url-unhex-string`, `sxhash`, `sxhash-equal`, `sxhash-eq`, `sxhash-eql`. (Chapter 37.)

Control and error. `throw`, `error`, `user-error`, `signal`. (Chapter 38.) The higher-order `mapcar`, `mapc`, `sort`, and `maphash` are intercepted intrinsics. (Chapter 38.)

Internal primitives. `--current-directory--`, `--directory-files--`, `--buffer-push--`, `--buffer-pop--`, `--push-output-capture--`, `--pop-output-capture--` — the double-dash hooks the prelude wraps. (Chapter 12.)

Prelude families, by section

The prelude defines, under twenty-five `;;;` section banners, the derived surface. The principal families and a representative sampling of each:

c[ad]+r and numeric helpers. `caar`, `cadr`, `cddr`, `caddr`, `cadddr` ...; `max`, `min`, `gcd`, `lcm`, `isqrt`, `evenp`, `oddp`, `natnump`, `plusp`, `minusp`. (Chapters 19, 30.)

List construction and access. `nthcdr`, `last`, `make-list`, `number-sequence`, `elt`, `take`, `butlast`, `nbutlast`, `flatten-tree`, `flatten-list`, `copy-tree`, `copy-sequence`, `ensure-list`, `mapcan`, `delete-dups`, `nconc`, `nreverse`. (Chapters 30, 51.)

Membership and alist/plist. `memq`, `member`, `memql`, `assq`, `assoc`, `rassq`, `rassoc`, `alist-get`, `assoc-default`, `plist-get`, `plist-put`, `plist-member`, `assq-delete-all`, `add-to-list`. (Chapters 30, 51.)

seq. `seq-map`, `seq-filter`, `seq-remove`, `seq-reduce`, `seq-find`, `seq-some`, `seq-every-p`, `seq-count`, `seq-do`, `seq-take`, `seq-drop`, `seq-subseq`, `seq-uniq`, `seq-min`, `seq-max`, `seq-position`, `seq-into`, `seq-difference`, `seq-intersection`, `seq-union`, `seq-partition`, `seq-group-by`, `seq-map-indexed`, `seq-keep`, `seq-mapcat`, `seq-mapn`, `seq-let`. (Chapters 30, 40.)

cl-lib sequences and ordinals. `cl-remove-if`, `cl-remove-if-not`, `cl-find`, `cl-find-if`, `cl-position`, `cl-count`, `cl-reduce`, `cl-mapcar`, `cl-subseq`, `cl-remove-duplicates`, `cl-sort`, `cl-some`,

`cl-every`, `cl-union`, `cl-intersection`, `cl-set-difference`, `cl-member`, `cl-assoc`, `cl-first` ... `cl-tenth`, `cl-rest`. (Chapter 39.)

cl-lib place and control macros. `cl-incf`, `cl-decf`, `cl-pushnew`, `cl-rotatef`, `cl-shiftf`, `cl-callf`, `cl-case`, `cl-typecase`, `cl-block`, `cl-return-from`, `cl-flet`, `cl-labels`, `cl-macrolet`, `cl-structuring-bind`, `cl-defun`, `cl-defmacro`, `cl-assert`, `cl-check-type`, `cl-do`. (Chapter 39.)

cl-lib numbers. `cl-floor`, `cl-ceiling`, `cl-truncate`, `cl-round`, `cl-mod`, `cl-rem`, `cl-gcd`, `cl-lcm`, `cl-isqrt`, `cl-signum`, `cl-evenp`, `cl-oddp`, `cl-parse-integer`. (Chapters 19, 39.)

subr-x strings. `string-trim`, `string-trim-left`, `string-trim-right`, `string-blank-p`, `string-remove-prefix`, `string-remove-suffix`, `string-pad`, `string-replace`, `string-reverse`, `string-chop-newline`, `string-lines`, `string-clean-whitespace`. (Chapters 32, 40, 51.)

subr-x and binding macros. `if-let`, `when-let`, `if-let*`, `when-let*`, `and-let*`, `named-let`, `letrec`, `dlet`, `thread-first`, `thread-last`, `cl-flet`, `cl-letf`. (Chapter 40.)

map. `map-elt`, `map-keys`, `map-values`, `map-pairs`, `map-do`, `map-apply`, `map-filter`, `map-remove`, `map-some`, `map-every-p`, `map-length`, `map-empty-p`, `map-into`, `map-insert`, `map-put!`, `map-delete`, `map-merge`, `map-merge-with`, `map-nested-elt`. (Chapters 21, 40.)

Object system. `cl-defstruct`, `cl-defgeneric`, `cl-defmethod`, `cl-call-next-method`, `cl-next-method-p`, `cl-typep`, `cl-deftype`, `record`, `make-record`. (Chapters 22, 59.)

Control macros (non-cl). `prog2`, `incf`, `decf`, `push`, `pop`, `setf`, `dolist`, `dotimes`, `while-let`, `ignore-errors`, `ignore-error`, `with-demoted-errors`, `with-output-to-string`, `with-temp-buffer`. (Chapters 7, 8, 14.)

pcase and rx. `pcase`, `pcase-let`, `pcase-let*`, `pcase-exhaustive`, `pcase-dolist`, `rx`, `rx-to-string`. (Chapters 25, 56.)

cl-loop. `cl-loop` and its parser helpers `cl-loop--kw`, `cl-loop--clause-p`, `cl-loop--accum`. (Chapters 26, 55.)

JSON. `json-encode`, `json-read-from-string`, `json-parse-string`, `json-serialize`, with the dynamic variables `json-object-type`, `json-array-type`, `json-key-type`, `json-null`, `json-false`. (Chapter 25.)

Pure path manipulation. `file-name-directory`, `file-name-nondirectory`, `file-name-extension`, `file-name-sans-extension`, `file-name-base`, `file-name-as-directory`, `directory-file-name`, `file-name-concat`, `file-name-absolute-p`, `file-name-split`, `expand-file-name`, `file-relative-name`, `directory-files`. (Chapters 37, 40.)

Region and word. `upcase-region`, `downcase-region`, `capitalize-region`, `subst-char-in-region`, `upcase-word`, `downcase-word`, `capitalize-word`, `transpose-chars`, `kill-line`, `newline`, `count-words`, `how-many`, `following-char`, `preceding-char`, `buffer-hash`. (Chapter 51.)

Time, key, completion, display. `time-add`, `time-subtract`, `time-less-p`, `time-convert`, `format-seconds`, the `decoded-time-*` accessors; `kbd`, `key-description`; `all-completions`, `try-completion`, `test-completion`; `string-width`, `char-width`, `truncate-string-to-width`, `pp`, `pp-to-string`. (Chapter 50.)

Conditions and properties. `define-error` and the seeded conditions (`error`, `user-error`, `arith-error`, `wrong-type-argument`, `void-variable`, `void-function`, `args-out-of-range`, `end-of-file`, ...); `get`, `put`, `symbol-plist`, `setplist`, `function-get`, `function-put`. (Chapters 14, 49.)

Module system. `provide`, `require`, `featurep`, over the `features` list (`emacs`, `cl-lib`, `cl-macs`, `cl-seq`, `cl-extra`, `seq`, `subr-x`, `map`, `rx`, `pcase`, `gv`, `ert`). (Chapter 39.)

ERT. `should`, `should-not`, `should-error`, `skip-unless`, `ert-deftest`, `ert-run-tests-batch`, `ert-run-tests-batch-and-exit`. (Chapters 14, 54.)

This index is deliberately partial — the prelude defines several hundred functions, and the corpus 213 subrs — but it covers the families a program reaches for, and each entry points at the chapter that explains it. The complete, signature-by-signature enumeration is [docs/reference.html](#); this index is the bridge from “what was that called” to the page that has the answer.